

ESAME:

Domande teoriche + esercizi (Progettazione, 2 sql, Algebra, Esercizi vari)

MODULO 2

# Sommario

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>1. Introduzione alle basi di dati e ai DBMS</b>                    | <b>4</b>  |
| 1.1. Gestione dei dati: introduzione                                  | 4         |
| 1.2. Modelli dei dati                                                 | 5         |
| 1.3. Utilizzo dei DBMS                                                | 6         |
| <b>2. Modello relazionale e algebra relazionale</b>                   | <b>8</b>  |
| 2.1. Modello relazionale                                              | 8         |
| 2.1.1. Introduzione al concetto di relazione                          | 8         |
| 2.1.2. Introduzione al modello relazionale                            | 8         |
| 2.1.3. Vincoli di integrità                                           | 9         |
| 2.2. Algebra relazionale                                              | 10        |
| 2.2.1. Introduzione all'algebra relazionale                           | 10        |
| 2.2.2. Selezione e proiezione                                         | 11        |
| 2.2.3. Unione, differenza, intersezione                               | 11        |
| 2.2.4. Prodotto cartesiano, join                                      | 12        |
| 2.2.5. Equivalenza di espressioni algebriche                          | 13        |
| 2.2.6. Idiomi di interrogazione                                       | 14        |
| 2.2.6.1. Esiste / per ogni, minimo / massimo                          | 14        |
| 2.2.6.2. Idiomi di interrogazione: cardinalità e copertura            | 14        |
| 2.3. Esercizi di algebra relazionale                                  | 15        |
| 2.3.1. Parte I                                                        | 15        |
| 2.3.2. Parte II                                                       | 16        |
| <b>3. Linguaggio SQL</b>                                              | <b>18</b> |
| 3.1. DDL                                                              | 18        |
| 3.1.1. Domini                                                         | 18        |
| 3.1.2. Schemi, tabelle e vincoli                                      | 18        |
| 3.2. DML: interrogazioni semplici                                     | 20        |
| 3.2.1. Select, from, where                                            | 20        |
| 3.2.2. Join e variabili                                               | 21        |
| 3.3. DML: interrogazioni complesse                                    | 21        |
| 3.3.1. Ordinamenti, aggregazioni, raggruppamenti                      | 21        |
| 3.3.2. Interrogazioni insiemistiche e nidificate                      | 22        |
| 3.3.3. Funzioni scalari e condizionali                                | 23        |
| 3.4. DML per le modifiche: inserimenti, cancellazioni e aggiornamenti | 24        |
| 3.5. Aspetti evoluti del DDL                                          | 25        |
| 3.5.1. Vincoli di integrità generici                                  | 25        |
| 3.5.2. Viste                                                          | 25        |
| 3.5.3. Autorizzazioni e controllo dell'accesso                        | 26        |
| 3.6. Esercizi di SQL                                                  | 27        |
| 3.6.1. Parte I                                                        | 27        |
| 3.6.2. Parte II                                                       | 29        |
| <b>4. Progettazione di basi di dati</b>                               | <b>31</b> |
| 4.1. Metodologie di progettazione                                     | 31        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 4.1.1. Ciclo di vita dei sistemi informativi              | 31        |
| 4.1.2. Raccolta e analisi dei requisiti                   | 32        |
| 4.1.2. Fasi della progettazione                           | 33        |
| 4.2. Modello entità-relazione                             | 34        |
| 4.2.1. Entità, relazioni, attributi                       | 34        |
| 4.2.2. Cardinalità                                        | 35        |
| 4.2.3. Identificatori e gerarchie                         | 36        |
| 4.2.4. Documentazione di schemi                           | 37        |
| 4.3. Progettazione concettuale                            | 38        |
| 4.3.1. Strategie di progetto                              | 38        |
| 4.3.1.1. Top-down                                         | 38        |
| 4.3.1.2. Bottom-up                                        | 38        |
| 4.3.1.3. Inside-out                                       | 39        |
| 4.3.1.4. Mista                                            | 39        |
| 4.3.2. Qualità di schemi concettuali e strumenti          | 39        |
| 4.4. Progettazione logica                                 | 40        |
| 4.4.1. Ristrutturazione di schemi ER                      | 41        |
| 4.4.1.1. Analisi di ridondanze                            | 41        |
| 4.4.1.2. Eliminazione delle gerarchie                     | 41        |
| 4.4.1.3. Partizionamenti e accorpamenti                   | 42        |
| 4.4.1.4. Scelta degli identificatori primari              | 43        |
| 4.4.2. Traduzione verso il modello logico                 | 44        |
| 4.4.3. Documentazione, verifica, progettazione fisica     | 45        |
| 4.5. Esercizi di progettazione concettuale e logica       | 46        |
| 4.5.1. Parte I                                            | 46        |
| 4.5.1.1. Progettazione concettuale                        | 46        |
| 4.5.1.2. Progettazione logica                             | 47        |
| 4.5.2. Parte II                                           | 48        |
| 4.5.2.1. Progettazione concettuale                        | 48        |
| 4.5.2.2. Progettazione logica                             | 49        |
| 4.5.3. Parte III                                          | 49        |
| 4.5.3.1. Progettazione concettuale                        | 49        |
| 4.5.3.2. Progettazione logica                             | 50        |
| <b>5. Organizzazione fisica delle basi di dati</b>        | <b>51</b> |
| 5.1. Moduli per accesso ai dati e gestione della memoria  | 51        |
| 5.2. Organizzazione di file                               | 52        |
| 5.2.1. Strutture sequenziali                              | 52        |
| 5.2.2. Strutture hash based                               | 53        |
| 5.2.3. Gestione delle tuple nelle pagine                  | 54        |
| 5.2.4. Strutture ad albero                                | 55        |
| 5.3. Progettazione fisica e gestione delle interrogazioni | 57        |
| <b>6. Gestione delle transazioni</b>                      | <b>60</b> |
| 6.1. Transazioni                                          | 60        |
| 6.2. Controllo di affidabilità                            | 61        |
| 6.2.1. Gestore dell'affidabilità e organizzazione del log | 61        |
| 6.2.2. Esecuzione di transazioni e scritture del log      | 63        |

|                                                        |           |
|--------------------------------------------------------|-----------|
| 6.2.3. Gestione dei guasti                             | 63        |
| 6.3. Controllo di concorrenza                          | 65        |
| 6.3.1. Anomalie delle transazioni concorrenti          | 65        |
| 6.3.2. Schedule e serializzabilità                     | 66        |
| 6.3.3. View-serializzabilità                           | 66        |
| 6.3.4. Conflict-serializzabilità                       | 67        |
| 6.3.5. Locking a due fasi                              | 67        |
| 6.3.6. Timestamp                                       | 69        |
| 6.3.7. Meccanismi per la gestione dei lock             | 70        |
| 6.3.8. Deadlock                                        | 71        |
| <b>7. Architetture distribuite</b>                     | <b>74</b> |
| 7.1. Architettura client-server                        | 74        |
| 7.2. Basi di dati distribuite                          | 75        |
| 7.2.1. Introduzione alle basi di dati distribuite      | 75        |
| 7.2.2. Frammentazione e allocazione dei dati           | 76        |
| 7.2.3. Accesso ai dati distribuiti                     | 77        |
| 7.3. Tecnologia della basi di dati distribuite         | 77        |
| 7.3.1. Ottimizzazione e atomicità                      | 78        |
| 7.3.2. Controllo di concorrenza                        | 78        |
| 7.3.3. Protocollo di commit a due fasi                 | 79        |
| <b>8. Dati semistrutturati: XML</b>                    | <b>82</b> |
| 8.1. Introduzione a XML                                | 82        |
| 8.2. DTD e XML Schema                                  | 83        |
| 8.3. Xquery                                            | 85        |
| <b>9. Basi di dati attive</b>                          | <b>87</b> |
| 9.1. Caratteristiche evolutive delle regole            | 88        |
| 9.2. Proprietà delle regole                            | 88        |
| 9.3. Applicazioni ed esempi                            | 88        |
| <b>10. Basi di dati per il supporto alle decisioni</b> | <b>89</b> |
| 10.1. OLTP e OLAP                                      | 89        |
| 10.2. Data warehouse                                   | 90        |
| 10.3. Rappresentazione multidimensionale dei dati      | 92        |
| 10.4. Realizzazione di data warehouse                  | 92        |
| 10.5. Data mining                                      | 93        |

# 1. Introduzione alle basi di dati e ai DBMS

## 1.1. Gestione dei dati: introduzione

### **Sistema organizzativo**

Insieme di **risorse** e **regole** per lo svolgimento coordinato delle attività, al fine di raggiungere degli scopi dell'organizzazione. Nello svolgimento di ogni attività sono essenziali la **disponibilità di informazioni** e la **capacità di gestirle** in modo efficace.

### **Sistema informativo**

Componente del sistema organizzativo per la:

- raccolta e acquisizione delle informazioni;
- archiviazione delle informazioni;
- elaborazione delle informazioni;
- distribuzione, scambio e condivisione delle informazioni.

### **Sistema informatico**

**Parte del sistema informativo** che **gestisce le informazioni** grazie a tecnologie informatiche e le rappresenta per mezzo di dati (la cui interpretazione fornisce informazione).

### **Informazione**

notizia, dato o elemento che consente di avere conoscenza di fatti, situazioni o modi di essere

### **Dato**

elemento di informazione costituito da simboli che devono essere elaborati

### **Base di dati**

**Collezione di dati** utilizzati per rappresentare le informazioni di interesse per il sistema informativo.

### **Data Base Management System (DBMS)**

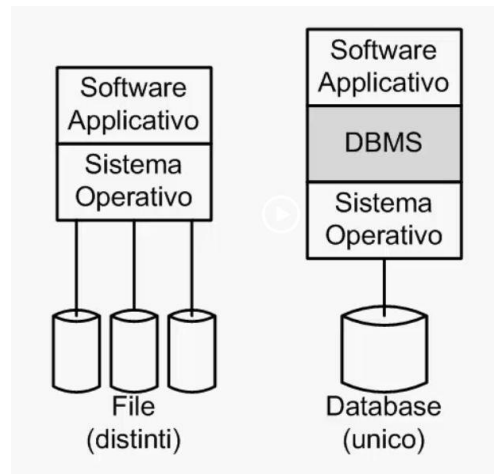
Sistema **software** per la gestione di **collezioni di dati**:

- **grandi** (di dimensioni molto maggiori della memoria centrale);
- **condivisi** (utilizzate da più utenti e applicazioni, anche contemporaneamente);
- **persistenti** (con un tempo di vita indipendente dalle singole esecuzioni dei programmi che li usano).

Questi sistemi sono in grado di garantire:

- **affidabilità** (resistenza a malfunzionamenti hardware e software);
- **privatezza** dei dati (per mezzo di accesso selettivo);
- **efficienza** (utilizzando al meglio le risorse in termini di spazio e tempo);
- **efficacia** (rendendo produttive le attività dei suoi utilizzatori).

I DBMS **estendono le funzioni del file system** e utilizzano i file per la memorizzazione dei dati, **operando al di sopra del file system** (**non** prendono il suo posto)



## 1.2. Modelli dei dati

### Dati nei DBMS

Esistono descrizioni e rappresentazioni dei dati a diversi livelli: i **programmi e gli utenti** fanno riferimento alla **struttura a livello più alto (che usa un modello dei dati)**, **senza preoccuparsi** della **rappresentazione a basso livello (file system)**.

### Modello dei dati

Insieme di **concetti e costrutti** utilizzati per **organizzare i dati**. Fornisce **meccanismi di strutturazione** (o costruttori di tipo) che permettono di **definire nuovi tipi sulla base di tipi predefiniti (elementari)**. Il modello più diffuso è il **modello relazionale**: basato sul costruttore **relazione (tabella)** che permette di definire insiemi di record **omogenei**. Altri modelli di dati sono:

- **modello gerarchico** (basato su strutture ad albero);
- **modello reticolare** (basato sui grafi);
- **modello a oggetti** (estende il modello relazionale con il paradigma della programmazione ad oggetti)

### Modelli logici e concettuali

Esistono due classi di modelli:

- i **modelli logici** sono **utilizzati nei DBMS per l'organizzazione dei dati**; ad essi fanno riferimento i programmi e sono **indipendenti dalle strutture fisiche**;
- i **modelli concettuali** sono **utilizzati nelle fasi preliminari della progettazione della base di dati**; rappresentano i concetti del mondo reale senza dipendere da una particolare organizzazione; infatti, **sono indipendenti sia dalle strutture fisiche, sia dai modelli logici**.

### Schemi e istanze

In ogni base di dati esistono:

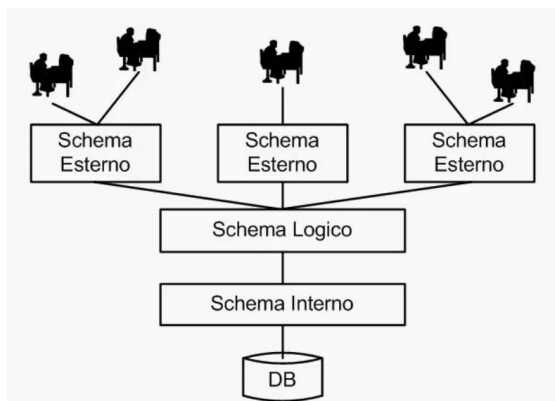
- **schema** (o **componente intensionale**): descrive la **struttura dei dati** ed è **invariante nel tempo (raro che cambi)**;
- **istanza** (o **stato** o **componente estensionale**): è costituita dai **valori dei dati** ed è **variabile nel tempo**.

In riferimento al modello relazionale, lo **schema** consiste **nell'intestazione della relazione (nome e attributi)**, mentre l'**istanza** è costituita **dall'insieme delle righe (o record) della relazione**.

### Livelli di astrazione dei DBMS

Esiste un'architettura standardizzata per i DBMS, denominata ANSI/SPARC, che fa riferimento a tre livelli, ognuno con un proprio schema:

- **schema logico**: descrizione dell'intera base di dati per mezzo del **modello logico**
- **schema interno**: rappresentazione dello **schema logico** per mezzo delle **strutture fisiche** di memorizzazione (quindi del **file system**)
- **schema esterno**: descrizione, per mezzo del modello logico, di una **porzione della base di dati** di interesse (riflette la **vista sui dati di un particolare utente o gruppo di utenti**)



## Viste

Nei sistemi più moderni il **livello esterno non è esplicitamente presente** ma è possibile **definire relazioni derivate**, dette anche **viste**, definite sulle relazioni memorizzate nella base di dati.

### CORSI

| Corso        | Docente | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

### AULE

| Nome  | Piano |
|-------|-------|
| Alfa  | Terra |
| Beta  | Terra |
| Gamma | Primo |
| Delta | Primo |

### LEZIONI

| Corso        | Aula  | Piano |
|--------------|-------|-------|
| Architetture | Alfa  | Terra |
| Basi di dati | Beta  | Terra |
| Linguaggi    | Gamma | Primo |
| Sistemi      | Beta  | Terra |

## Indipendenza dei dati

L'**articolazione in livelli** dei DBMS e l'**accesso ai dati mediante il livello esterno** garantisce l'**indipendenza dei dati**, che si divide in:

- **indipendenza fisica** (*il livello logico e quello esterno sono indipendenti dal livello fisico*, perciò utenti e applicazioni interagiscono con il DBMS in modo indipendente dalla struttura fisica dei dati; in questo modo, modifiche alla struttura fisica non influiscono sul livello logico);
- **indipendenza logica** (*il livello esterno è indipendente dal livello logico*, perciò è possibile aggiungere o modificare uno schema esterno senza influire sullo schema logico, così come è possibile modificare lo schema logico mantenendo inalterati gli schemi esterni visibili agli utenti).

## 1.3. Utilizzo dei DBMS

### Linguaggi per le basi di dati

Ne esistono due categorie principali:

- **DDL (Data Definition Language)**: sono linguaggi per la **definizione dei dati**, che permettono di descriverne gli schemi logici e fisici e le relative autorizzazioni di accesso;
- **DML (Data Manipulation Language)**: sono linguaggi per la **manipolazione dei dati**, che permettono di definire, interrogare e aggiornare le istanze delle basi di dati.

Alcuni linguaggi, come **SQL**, offrono entrambe le funzionalità.

I linguaggi possono anche essere classificati in:

- **linguaggi testuali interattivi** (ad esempio SQL): gli utenti possono **scrivere direttamente i comandi** che sono poi interpretati dal DBMS;
- **comandi** (come quelli dei linguaggi interattivi) immersi in un **linguaggio ospite** (ad esempio C o Cobol);
- **comandi** (come quelli dei linguaggi interattivi) immersi in un **linguaggio ad hoc** (spesso con funzionalità specifiche, ad esempio per la creazione di grafici): estendono i linguaggi interattivi con costrutti tipici dei linguaggi di programmazione;
- **interfacce amichevoli** (es. Access): permettono le interrogazioni senza l'uso di un linguaggio testuale

### Categorie di utenti

Gli utenti che si interfacciano al DBMS si dividono in:

- **amministratori della base di dati (DBA)**: responsabili della progettazione, del controllo e dell'amministrazione della base di dati;
- **progettisti e programmatori delle applicazioni**: definiscono e realizzano i **programmi** che **accedono alla base di dati**;
- **utenti**:
  - **finali** (terminalisti): utilizzano le cosiddette **transazioni**, ovvero **programmi che svolgono attività predefinite**, realizzati dai progettisti e programmatori;
  - **casuali**: utilizzano direttamente i **linguaggi interattivi** per accedere alla base di dati in modo non

predefinito (perciò un accesso di tipo casuale).

### Vantaggi dei DBMS

L'utilizzo dei DBMS comporta diversi vantaggi rispetto ai file system:

- i dati sono considerati come risorsa comune a disposizione di tutte le componenti del sistema, con le opportune forme di controllo;
- fornisce un modello unificato del mondo reale di interesse, utilizzabile dalle applicazioni;
- permette una gestione centralizzata dei dati con possibilità di standardizzazione, garantendo quindi interoperabilità e fornendo economia di scala;
- diminuisce le ridondanze e le potenziali inconsistenze dei dati;
- garantisce l'indipendenza dei dati, favorendo lo sviluppo e la gestione delle applicazioni.

### Svantaggi dei DBMS

I DBMS sono prodotti costosi e complessi e il loro utilizzo comporta notevoli **investimenti diretti e indiretti**. Inoltre, dal momento che questi forniscono in forma integrata una serie di servizi non scorporabili, si ha una riduzione delle prestazioni del sistema dovuta ai **servizi non necessari**.

## 2. Modello relazionale e algebra relazionale

### 2.1. Modello relazionale

#### 2.1.1. Introduzione al concetto di relazione

##### Modello relazionale

Favorisce l'indipendenza dei dati ed è caratterizzato da un alto livello di astrazione; si basa sul concetto matematico di relazione e sulla rappresentazione delle relazioni per mezzo di tabelle.

##### Relazione

Esistono **tre diverse accezioni** del concetto di relazione:

- relazione matematica (secondo la teoria degli insiemi);
- relazione secondo il modello relazionale (concetto di tabella);
- relazione come costruito del modello concettuale Entità-Relazione utilizzato nella progettazione delle basi di dati..

Prodotto cartesiano → insieme di tutte le n-uple ordinate (possibili combinazioni degli insiemi di partenza)

Relazione Matematica → (su un insieme,  $D_1, D_2, \dots, D_n$ ) è un sottoinsieme del prodotto cartesiano

Data la **relazione matematica** su  $D_1, D_2, \dots, D_n$ :

- **Domini** della relazione:  $D_1, D_2, \dots, D_n$
- **Grado** della relazione: il numero di domini n su cui è definita
- **Cardinalità** della relazione: numero di n-uple che ne fanno parte (*cardinalità dell'insieme relazione*)

(di solito cardinalità finite su insiemi eventualmente infiniti)

### Relazione matematica: esempio

$$D_1 = \{a, b\} = \begin{array}{|c|} \hline a \\ \hline b \\ \hline \end{array} \quad D_2 = \{x, y, z\} = \begin{array}{|c|} \hline x \\ \hline y \\ \hline z \\ \hline \end{array}$$

Prodotto cartesiano  $c = D_1 \times D_2 =$

- $\text{grado}(c)=2$ ;  $\text{card}(c)=|c|=6$

|   |   |
|---|---|
| a | x |
| a | y |
| a | z |
| b | x |
| b | y |
| b | z |

Un esempio di relazione  $r \subseteq D_1 \times D_2 =$

- $\text{grado}(r)=2$ ;  $\text{card}(r)=|r|=3$

|   |   |
|---|---|
| a | y |
| a | z |
| b | x |

Una **relazione** sui **domini  $D_1, D_2, \dots, D_n$**  è un **insieme di n-uple ordinate**  $d_1, d_2, \dots, d_n$  tali che  $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$

### Tabelle e relazioni

Una tabella rappresenta una relazione se soddisfa le seguenti proprietà:

- le **intestazioni** delle colonne sono **diverse** tra loro;
- i **valori** di ogni colonna sono **omogenei** tra loro;
- le **righe** sono **diverse** tra loro.

Inoltre, in una tabella che rappresenta una relazione, **l'ordinamento delle righe e delle colonne è irrilevante**

## 2.1.2. Introduzione al modello relazionale

### Definizioni

Schema di relazione  $R(X)$ :

- $R$  è il nome della relazione;
- $X = \{A_1, \dots, A_n\}$  è l'insieme degli attributi.

Schema di base di dati  $R = \{R_1(X_1), \dots, R_n(X_n)\}$ :

- $\{R_1(X_1), \dots, R_n(X_n)\}$  è l'insieme di schemi di relazioni con nomi diversi.

Relazione (istanza)  $r$  su uno schema  $R(X)$ :

- insieme  $r$  di tuple su  $X$ .

Base di dati (istanza)  $r$  su uno schema  $R$ :

- insieme di relazioni  $r = \{r_1, \dots, r_n\}$  (ciascuna relazione  $r_i$  è definita sul proprio schema  $R_i(X_i)$ ).

### Notazioni

- Attributi:  $A, B, A', A_1, \dots$
- Insiemi di attributi:  $X, Y, Z, X', X_1, \dots$
- Nomi di relazione (nello schema):  $R, S, R', R_1, \dots$
- Relazione (istanza):  $r, s, r', r_1, \dots$
- Schema di base di dati:  $R, S, R', R_1, \dots$
- Base di dati (istanza):  $r, s, r', r_1, \dots$

## Modello relazionale basato su valori

I riferimenti fra dati di relazioni diverse sono rappresentati per mezzo di valori dei domini che compaiono nelle tuple; quindi, per mezzo dei valori che gli attributi assumono, e **non per mezzo di puntatori (perlomeno a livello logico), come succede nei linguaggi di programmazione**; questo modello presenta perciò alcuni vantaggi:

- rappresenta solo ciò che è rilevante dal punto di vista dell'applicazione (infatti l'utilizzo dei puntatori è un aspetto realizzativo a basso livello);
- fornisce indipendenza della rappresentazione logica rispetto a quella fisica (indipendenza fisica dei dati);
- favorisce portabilità dei dati (tutta l'informazione è nei valori);
- le associazioni fra i valori sono bidirezionali (a differenza dei puntatori).

## Strutture nidificate

Il modello relazionale **ammette però solo valori scalari**, ovvero **permette ad ogni attributo di ogni tupla di assumere un solo valore** e non permette l'assegnazione di insiemi di valori o altre strutture nidificate. Comunque, è possibile rappresentare informazioni strutturate organizzando propriamente i dati.

### Strutture nidificate: esempio (1)

| "DA MARIO"       |           |       |
|------------------|-----------|-------|
| Ricevuta n. 2453 |           |       |
| del 6/7/2005     |           |       |
| 3                | Coperti   | 6.00  |
| 2                | Antipasti | 15.00 |
| 3                | Primi     | 27.00 |
| 2                | Secondi   | 42.00 |
| 3                | Caffè     | 6.00  |
| 1                | Acqua     | 3.00  |
| Totale           |           | 99.00 |

| "DA MARIO"       |         |        |
|------------------|---------|--------|
| Ricevuta n. 2564 |         |        |
| del 8/7/2005     |         |        |
| 2                | Coperti | 4.00   |
| 2                | Primi   | 27.00  |
| 2                | Secondi | 42.00  |
| 2                | Dessert | 12.00  |
| 2                | Caffè   | 4.00   |
| 1                | Vino    | 25.00  |
| 1                | Acqua   | 3.00   |
| Totale           |         | 117.00 |

#### RICEVUTE

| Num  | Data   | Totale |
|------|--------|--------|
| 2453 | 6/7/05 | 99.00  |
| 2564 | 8/7/05 | 117.00 |

### Strutture nidificate: esempio (2)

| DETTAGLIO | Numero | Q.ta | Descr.    | Importo |
|-----------|--------|------|-----------|---------|
|           | 2453   | 3    | coperti   | 6.00    |
|           | 2453   | 2    | antipasti | 15.00   |
|           | 2453   | 3    | primi     | 27.00   |
|           | 2453   | 2    | secondi   | 42.00   |
|           | 2453   | 3    | caffè     | 6.00    |
|           | 2453   | 1    | acqua     | 3.00    |
|           | 2564   | 2    | coperti   | 4.00    |
|           | 2564   | 2    | primi     | 27.00   |
|           | 2564   | 2    | secondi   | 42.00   |
|           | 2564   | 2    | dessert   | 12.00   |
|           | 2564   | 2    | caffè     | 4.00    |
|           | 2564   | 1    | vino      | 25.00   |
|           | 2564   | 1    | acqua     | 3.00    |

La rappresentazione data per le ricevute è **corretta se**:

- l'ordine delle righe è irrilevante
- non ci possono essere 2 righe ripetute (es. non posso aggiungere altra acqua dopo che l'avevo già inserita (stesso ordine))

(se non vengono soddisfatte queste condizioni, devo distinguere le singole righe nelle ricevute)

## Informazioni incomplete e valori nulli

All'interno di una tabella, un valore non specificato può assumere diverse semantiche:

- il valore è sconosciuto (il valore c'è ma non lo conosco)
- il valore è inesistente (il valore non esiste proprio)
- senza informazione (valore sconosciuto o inesistente, non so se sia uno o l'altro)

Nel modello relazionale, l'assenza di un valore del dominio rispetto a un attributo di una certa tupla viene indicato con il valore nullo (NULL), che denota l'assenza di un valore del dominio (senza informazione). A livello di definizione delle relazioni si pongono delle **restrizioni sulla presenza di valori nulli**.

## Valori nulli: esempi

### Troppi valori nulli!

#### STUDENTI

| Matr.  | Cognome | Nome  | Data-nasc  |
|--------|---------|-------|------------|
| 276545 | Rossi   | Maria | NULL       |
| NULL   | Neri    | Anna  | 23/04/1972 |
| NULL   | Verdi   | Fabio | 12/02/1972 |

#### ESAMI

| Studente | Voto | Corso |
|----------|------|-------|
| 276545   | 28   | 01    |
| NULL     | 27   | NULL  |
| 200768   | 24   | NULL  |

#### CORSI

| Codice | Titolo  | Docente |
|--------|---------|---------|
| 01     | Analisi | Giani   |
| 03     | Chimica | NULL    |
| NULL   | Chimica | Belli   |

## 2.1.3. Vincoli di integrità

### Chiavi

È un insieme di attributi utilizzato per identificare univocamente le tuple all'interno di una relazione:

- un insieme di attributi  $K$  è detto **superchiave** di una relazione  $r$  se  $r$  non contiene due tuple distinte  $t_1$  e  $t_2$  con  $t_1[K] = t_2[K]$ ;
- un insieme di attributi  $K$  è detto **chiave di una relazione  $r$**  se è una **superchiave** ed è **minimale** (ovvero non esiste un'altra superchiave  $K'$  contenuta in  $K$ ).

Ogni relazione  $R(X)$  ha sempre almeno una superchiave (e quindi almeno una chiave) costituita dall'insieme di tutti i suoi attributi, e questo garantisce che tutti i valori della base di dati siano accessibili e identificabili, oltre a permettere di ricostruire le corrispondenze (quindi le associazioni) tra i dati contenuti in relazioni diverse.

È necessario limitare la presenza di valori nulli nelle chiavi; perciò, in ogni relazione è identificata una chiave primaria (che può essere anche un attributo aggiunto apposta per l'identificazione) sui cui attributi non sono permessi valori nulli.

### Vincoli di integrità

Esistono istanze di basi di dati che, pur essendo sintatticamente corrette, **non rappresentano valori corretti per l'applicazione**. I vincoli di integrità sono proprietà che devono essere soddisfatte dalle istanze che rappresentano informazioni corrette per l'applicazione, e possono essere visti come dei predicati che associano ad ogni istanza il valore vero, se il **vincolo è soddisfatto**, o falso altrimenti.

Ogni schema di base di dati è associato ad un insieme di vincoli, e le istanze delle basi di dati sono corrette solo se soddisfano tutti i vincoli.

I vincoli di integrità possono essere classificati in:

- vincoli intra-relazionali** (si applicano a **singole relazioni**), a loro volta classificabili in:
  - vincoli di tupla**: esprime condizioni sui valori di ciascuna tupla, indipendentemente dalle altre, ed è sostanzialmente costituita da un'espressione booleana di atomi che confrontano valori di attributo o espressioni aritmetiche su attributi;  
 $ESAMI(Studente, Voto, Lode, Corso) \text{ NOT } (Lode = 'lode') \text{ OR } (Voto = 30)$
  - vincoli sui valori (o di dominio)**: esprime condizioni sui valori di singoli attributi all'interno di una tupla, indipendentemente dagli altri, ed anch'esso è costituito da un'espressione booleana di atomi che confrontano i valori degli attributi con delle costanti;  
 $ESAMI(Studente, Voto, Lode, Corso) (Voto \geq 18) \text{ AND } (Voto \leq 30); (Lode = 'lode') \text{ OR } (Lode = NULL)$
  - vincoli di chiave**: definiti rispetto a un insieme di attributi che identificano univocamente le tuple all'interno di una relazione;
- vincoli inter-relazionali** (coinvolgono **più relazioni**): il più noto è il **vincolo di integrità referenziale**, in cui ogni insieme di attributi di una relazione (**chiave esterna**) ammette soltanto i valori assunti dagli attributi di un'altra relazione (**referenza**).

## Chiavi: esempio

| Matricola | Cognome | Nome  | Nascita  | Corso       |
|-----------|---------|-------|----------|-------------|
| 4328      | Rossi   | Luigi | 29/04/59 | Sicurezza   |
| 6328      | Rossi   | Dario | 29/04/59 | Sicurezza   |
| 4766      | Rossi   | Luca  | 01/05/61 | Informatica |
| 4846      | Neri    | Luca  | 01/05/61 | Matematica  |
| 5536      | Neri    | Luca  | 05/03/58 | Sicurezza   |

## Chiave primaria: esempi

Notazione:

- per distinguere la chiave primaria sottolineiamo i nomi degli attributi che ne fanno parte

### Esempio

#### chiavi:

- Matricola
- Cognome, Nome, Nascita
- Nome, Corso

#### superchiavi:

- qualsiasi superinsieme che contiene le chiavi

### Esempi

- STUDENTI (Matricola, Nome, Cognome)
- SEMINARI (Codice, Relatore, Titolo, data)
- RICEVUTE (Num, Data, Totale)
- DETTAGLIO2(Numero, Riga, Q.ta, Descr, Importo)

## Vincoli di integrità: esempi

Esistono istanze di basi di dati che, pur sintatticamente corrette, non rappresentano valori corretti per l'applicazione

| STUDENTI | Matr.         | Cognome | Nome  | Data-nasc  |
|----------|---------------|---------|-------|------------|
|          | 200768        | Verdi   | Fabio | 12/02/1972 |
|          | <u>937653</u> | Rossi   | Luca  | 10/10/1971 |
|          | <u>937653</u> | Bruni   | Mario | 11/02/1971 |

### ESAMI

| Studente | Voto      | Lode | Corso         |
|----------|-----------|------|---------------|
| 200768   | <u>36</u> |      | <del>05</del> |
| 937653   | 28        | Lode | 02            |
| 276745   | 25        |      | 01            |

### CORSI

| Cod | Titolo  | Docente |
|-----|---------|---------|
| 01  | Analisi | Giani   |
| 02  | Chimica | Melli   |
| 03  | Chimica | Belli   |

## Integrità referenziale: esempio (2)

R=INFRAZIONI

S=AUTO

X'=Prov,Num

Y'=Provincia,Numero

#### Chiave esterna:

- Prov, Num  
in INFRAZIONI

#### Referenzia:

- Provincia, Numero  
in AUTO

In **rosso** le violazioni al vincolo

### INFRAZIONI

| Codice | Data     | Prov | Num    |
|--------|----------|------|--------|
| 987554 | 12/10/05 | FI   | 4E5432 |
| 630876 | 15/10/05 | MI   | 2F7653 |
| 726375 | 15/10/05 | FI   | 4G7686 |
| 123435 | 11/10/05 | MO   | 7D6563 |

### AUTO

| Provincia | Numero | Proprietario   |
|-----------|--------|----------------|
| FI        | 4E5432 | Rossi Mario    |
| MI        | 9H4467 | Verdi Paolo    |
| FI        | 7D6563 | Gialli Sara    |
| MO        | 4G7686 | Bianchi Silvia |

## 2.2. Algebra relazionale

### 2.2.1. Introduzione all'algebra relazionale

#### Classificazione dei linguaggi

I linguaggi utilizzati per interrogare le basi di dati possono essere classificati in:

- **formali o programmatici**
- **procedurali o dichiarativi**

Infatti, ad esempio:

- **l'algebra relazionale** è un linguaggio formale e procedurale;
- **SQL** è un linguaggio programmatico e dichiarativo.

#### Operatori dell'algebra relazionale

L'algebra relazionale è composta da **5 operatori di base** che insieme danno l'intero potere espressivo del linguaggio

- **operatori unari**:
  - **selezione**: seleziona un insieme di **tuple**;
  - **proiezione**: seleziona un insieme di **colonne**;
- **operatori binari**:
  - **unione**: insieme di **tuple** che appartengono a **due relazioni**;
  - **differenza**: insieme di **tuple** che appartengono ad **una relazione e non all'altra**;
  - **prodotto cartesiano**: **combinazione di tuple di due relazioni**.

Esistono poi **due operatori derivati**, entrambi binari, che possono cioè essere espressi tramite gli operatori di base:

- **intersezione**: seleziona le **tuple** che appartengono a **due relazioni**;
- **join** (*naturale, theta-join, equi-join*): **selezione** su un **prodotto cartesiano**.

Infine, ci sono degli **operatori aggiunti**, ovvero che non appartengono propriamente all'algebra:

- **ridenominazione**: permette di **rinominare un attributo**;
- **assegnamento**: dà un **nome** alla **relazione risultante** da un'operazione.

#### Ridenominazione

Scrittura:  $\rho_{B_1, B_2, \dots, B_n \leftarrow A_1, A_2, \dots, A_n} r$ , dove:

- $r$ : relazione;
- $A_1, A_2, \dots, A_n$ : sottoinsieme di **attributi** che appartengono ad  $r$ ;
- $B_1, B_2, \dots, B_n$ : **nuovi nomi** da dare **agli attributi**  $A_1, A_2, \dots, A_n$ .

**Risultato**: **relazione senza nome** (e questo *vale per tutti gli operatori dell'algebra relazionale*) **uguale ad  $r$**  *tranne per il nome degli attributi modificati*, quindi:

- **schema**:  $r - \{A_1, A_2, \dots, A_n\} \cup \{B_1, B_2, \dots, B_n\}$
- **grado**:  $\text{grado}(r)$
- **cardinalità**:  $\text{card}(r)$

#### Assegnamento

Scrittura:  $\text{risultato} := \text{espressione algebrica}$ , dove:

- $\text{risultato}$ : **nome** da dare alla **relazione risultato dell'espressione**.

## 2.2.2. Selezione e proiezione

### Selezione

Scrittura:  $\sigma_{condizione} r$ , dove:

- $r$ : relazione;
- **condizione**: espressione booleana di condizioni atomiche del tipo  $A \theta B$  o  $A \theta c$ , dove:
  - $\theta$ : operatore di confronto ( $=, \neq, <, >, \leq, \geq$ );
  - $A$  e  $B$ : attributi di  $r$  su cui l'operatore di confronto abbia senso;
  - $c$ : costante compatibile con il dominio di  $A$ .

**Risultato: relazione senza nome che contiene tutte le tuple di  $r$  che soddisfano la condizione**, quindi:

- schema: schema di  $r$
- grado:  $grado(r)$
- cardinalità:  $\leq card(r)$

### Proiezione

Scrittura:  $\Pi_Y r$ , dove:

- $r$ : relazione;
- $Y$ : sottoinsieme degli attributi di  $r$ ;

**Risultato: relazione senza nome che contiene la restrizione delle tuple di  $r$  agli attributi  $Y$** , quindi:

- schema:  $Y$ ;
- grado:  $|Y|$ ;
- cardinalità:  $\leq card(r)$ , ovvero:
  - uguale se  $Y$  è superchiave;
  - altrimenti può essere minore, perché **i duplicati vengono eliminati**.

## 2.2.3. Unione, differenza, intersezione

### Unione

Scrittura:  $r_1 \cup r_2$ , dove:

- $r_1$  e  $r_2$ : relazioni sullo stesso insieme di attributi.

**Risultato: relazione senza nome che contiene tutte le tuple che appartengono a  $r_1$  e/o a  $r_2$  (le tuple duplicate compaiono soltanto una volta)**, perciò:

- schema: schema di  $r_1$  = schema di  $r_2$
- grado:  $grado(r_1) = grado(r_2)$
- cardinalità:  $\geq \max(card(r_1), card(r_2))$  e  $\leq card(r_1) + card(r_2)$

### Differenza

Scrittura:  $r_1 - r_2$ , dove:

- $r_1$  e  $r_2$ : relazioni sullo stesso insieme di attributi.

**Risultato: relazione senza nome che contiene tutte le tuple che appartengono ad  $r_1$  ma non ad  $r_2$** , quindi:

- schema: schema di  $r_1$  = schema di  $r_2$ ;
- grado:  $grado(r_1) = grado(r_2)$ ;
- cardinalità:  $\geq 0$  e  $\leq card(r_1)$ .

## Intersezione

Scrittura:  $r_1 \cap r_2 \equiv r_1 - (r_1 - r_2)$ , dove:

- $r_1$  e  $r_2$ : relazioni sullo stesso insieme di attributi.

**Risultato:** relazione senza nome che contiene tutte le tuple che appartengono sia ad  $r_1$  sia ad  $r_2$ , quindi:

- schema: schema di  $r_1$  = schema di  $r_2$  ;
- grado:  $\text{grado}(r_1) = \text{grado}(r_2)$  ;
- cardinalità:  $\geq 0$  e  $\leq \min(\text{card}(r_1), \text{card}(r_2))$  .

## 2.2.4. Prodotto cartesiano, join

### Prodotto cartesiano

Scrittura:  $r_1 \times r_2$ , dove:

- $r_1$  e  $r_2$ : relazioni su insiemi di attributi disgiunti.

**Risultato:** relazione senza nome che contiene tutte le tuple ottenute concatenando tuple di  $r_1$  con tuple di  $r_2$ , quindi:

- schema: schema di  $r_1$  concatenato allo schema di  $r_2$  ;
- grado:  $\text{grado}(r_1) + \text{grado}(r_2)$  ;
- cardinalità:  $\text{card}(r_1) * \text{card}(r_2)$  .

### Theta join

Scrittura:  $r_1 \bowtie_{\text{condizione}} r_2 \equiv \sigma_{\text{condizione}} (r_1 \times r_2)$ , dove:

- $r_1$  e  $r_2$ : relazioni su insiemi di attributi disgiunti;
- condizione: espressione booleana di condizioni atomiche del tipo  $A \theta B$ , dove;
  - $A$ : attributo di  $r_1$  ;
  - $B$ : attributo di  $r_2$  ;
  - $\theta$ : operatore di confronto ( $=, \neq, <, >, \leq, \geq$ ) che abbia senso per  $A$  e  $B$  ;

**Risultato:** relazione senza nome che contiene le tuple del prodotto cartesiano tra  $r_1$  ed  $r_2$  che soddisfano la condizione, quindi:

- schema: schema di  $r_1$  concatenato allo schema di  $r_2$  (se ho attr. con nome uguale non si può fare)
- grado:  $\text{grado}(r_1) + \text{grado}(r_2)$  ;
- cardinalità:  $\geq 0$  e  $\leq \text{card}(r_1) * \text{card}(r_2)$  .

### Equi join

È un caso particolare di Theta join in cui la condizione è una congiunzione ( $\wedge$ ) di atomi di uguaglianza ( $A = B$ ).

### Natural join

Scrittura:  $r_1 \Join r_2$ , dove:

- $r_1$  e  $r_2$ : relazioni con un insieme di attributi in comune:
  - $X_1$ : attributi di  $r_1$  ;
  - $X_2$ : attributi di  $r_2$  ;
  - $X_{1,2}$ :  $X_1 \cap X_2$  .

**Risultato:** relazione senza nome che contiene le tuple ottenute combinando le tuple delle due relazioni sugli attributi con nome uguale (eliminando le colonne doppie), quindi:

- schema: schema di  $r_1$  concatenato allo schema di  $r_2$  (ma gli attributi con nome uguale compaiono solo una volta);
- grado:  $\text{grado}(r_1) + \text{grado}(r_2) - |X_{1,2}|$  ;
- cardinalità:  $\geq 0$  ,  $\leq \max(\text{card}(r_1), \text{card}(r_2))$  se  $X_{1,2}$  è chiave per almeno una delle relazioni,  $= \text{card}(r_1) * \text{card}(r_2)$  se  $X_{1,2}$  è insieme vuoto (ovvero le relazioni non hanno attributi in comune).

**Con vincolo di integrità  $\rightarrow$  card. nel join sarà sempre uguale alla card della tabella che viene**

| Operation         | Notation                                                  | Purpose                                                                                                                                      |
|-------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| SELECT            | $\sigma_{\langle \text{selection condition} \rangle}(R)$  | Selects all tuples that satisfy the selection condition from a relation R                                                                    |
| PROJECT           | $\pi_{\langle \text{attribute list} \rangle}(R)$          | New relation with subset of attributes of R and removes duplicate tuples                                                                     |
| THETA_JOIN        | $R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$ | All combinations of tuples from $R_1$ and $R_2$ that satisfy the join condition                                                              |
| EQUIJOIN          | $R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$ | Theta join with only equality join comparisons                                                                                               |
| NATURAL JOIN      | $R_1 *_{\langle \text{join condition} \rangle} R_2$       | Equijoin except join attributes of $R_2$ are not included in the resulting relation                                                          |
| UNION             | $R_1 \cup R_2$                                            | Relation that includes all tuples in $R_1$ or $R_2$                                                                                          |
| INTERSECTION      | $R_1 \cap R_2$                                            | Relation that includes all tuples in both $R_1$ and $R_2$                                                                                    |
| DIFFERENCE        | $R_1 - R_2$                                               | Relation that includes all tuples in $R_1$ that are not in $R_2$                                                                             |
| CARTESIAN PRODUCT | $R_1 \times R_2$                                          | Relation with attributes of $R_1$ and $R_2$ and includes tuples with all possible combinations of tuples of $R_1$ and $R_2$                  |
| DIVISION          | $R_1(Z) \div R_2(Y)$                                      | Relation that includes all tuples $t[X]$ in $R_1(Z)$ that appear in $R_1$ in combination with every tuple from $R_2(Y)$ where $Z = X \cup Y$ |

**Ridenominazione: esempio**

**Assegnamento: esempio**

**CORSI**

| Corso        | Docente | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

**CORSI**

| Corso        | Docente | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

$\rho_{\text{Insegnam, Prof} \leftarrow \text{Corso, Docente}} \text{CORSI}$

$\text{INSEGNAMENTI} := \rho_{\text{Insegnam, Prof} \leftarrow \text{Corso, Docente}} \text{CORSI}$

| Insegnam     | Prof    | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

**INSEGNAMENTI**

| Insegnam     | Prof    | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

**Selezione: esempi**

**Proiezione: esempi**

**CORSI**

| Corso        | Docente | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

**CORSI**

| Corso        | Docente | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

$\sigma_{\text{Aula} = \text{'Beta'}} \text{CORSI}$

| Corso        | Docente | Aula |
|--------------|---------|------|
| Basi di dati | Neri    | Beta |
| Sistemi      | Verdi   | Beta |

$\Pi_{\text{Corso, Docente}} \text{CORSI}$

| Corso        | Docente |
|--------------|---------|
| Architetture | Rossi   |
| Basi di dati | Neri    |
| Linguaggi    | Bianchi |
| Sistemi      | Verdi   |

$\Pi_{\text{Aula}} \text{CORSI}$

| Aula  |
|-------|
| Alfa  |
| Beta  |
| Gamma |

$\sigma_{\text{Aula} = \text{'Gamma'}} \vee \text{Corso} = \text{'Sistemi'}} \text{CORSI}$

| Corso     | Docente | Aula  |
|-----------|---------|-------|
| Linguaggi | Bianchi | Gamma |
| Sistemi   | Verdi   | Beta  |

## Selezione, proiezione: esempio

CORSI

| Corso        | Docente | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

$\Pi_{\text{Docente}} (\sigma_{\text{Aula} = \text{'Beta'}} \text{CORSI})$

| Docente |
|---------|
| Neri    |
| Verdi   |

## Selezione, proiezione, assegnamento: es.

CORSI

| Corso        | Docente | Aula  |
|--------------|---------|-------|
| Architetture | Rossi   | Alfa  |
| Basi di dati | Neri    | Beta  |
| Linguaggi    | Bianchi | Gamma |
| Sistemi      | Verdi   | Beta  |

$\text{BETA} := \sigma_{\text{Aula} = \text{'Beta'}} \text{CORSI}$

BETA

| Corso        | Docente | Aula |
|--------------|---------|------|
| Basi di dati | Neri    | Beta |
| Sistemi      | Verdi   | Beta |

$\Pi_{\text{Docente}} \text{BETA}$

| Docente |
|---------|
| Neri    |
| Verdi   |

## Join (theta): esempio (1)

CORSI

| Corso        | Docente | Aula |
|--------------|---------|------|
| Architetture | Rossi   | Alfa |
| Basi di dati | Neri    | Beta |

AULE

| Nome | Piano |
|------|-------|
| Alfa | Terra |
| Beta | Primo |

$\text{CORSI} \bowtie_{\text{Aula} = \text{Nome}} \text{AULE} = \text{AULE} \bowtie_{\text{Nome} = \text{Aula}} \text{CORSI}$

| Corso        | Docente | Aula | Nome | Piano |
|--------------|---------|------|------|-------|
| Architetture | Rossi   | Alfa | Alfa | Terra |
| Basi di dati | Neri    | Beta | Beta | Primo |

## Join (theta): esempio (2)

STUDENTI

| Matr  | Cognome |
|-------|---------|
| 11111 | Bianchi |
| 22222 | Rossi   |
| 33333 | Verdi   |

ESAMI

| Stud  | Esame |
|-------|-------|
| 11111 | CS101 |
| 22222 | CS102 |
| 33333 | CS101 |

CORSI

| Codice | Titolo   |
|--------|----------|
| CS101  | Basidati |
| CS102  | Sistemi  |

$\text{STUDENTI} \bowtie_{\text{Matr} = \text{Stud}} \text{ESAMI} \bowtie_{\text{Esame} = \text{Codice}} \text{CORSI}$

| Matr  | Cognome | Stud  | Esame | Codice | Titolo   |
|-------|---------|-------|-------|--------|----------|
| 11111 | Bianchi | 11111 | CS101 | CS101  | Basidati |
| 22222 | Rossi   | 22222 | CS102 | CS102  | Sistemi  |
| 33333 | Verdi   | 33333 | CS101 | CS101  | Basidati |

## Join naturale: esempio (2)

STUDENTI

| Matr  | Cognome |
|-------|---------|
| 11111 | Bianchi |
| 22222 | Rossi   |
| 33333 | Verdi   |

ESAMI

| Matr  | Codice |
|-------|--------|
| 11111 | CS101  |
| 22222 | CS102  |
| 33333 | CS101  |

CORSI

| Codice | Titolo   |
|--------|----------|
| CS101  | Basidati |
| CS102  | Sistemi  |

$\text{STUDENTI} \bowtie \text{ESAMI} \bowtie \text{CORSI}$

| Matr  | Cognome | Codice | Titolo   |
|-------|---------|--------|----------|
| 11111 | Bianchi | CS101  | Basidati |
| 22222 | Rossi   | CS102  | Sistemi  |
| 33333 | Verdi   | CS101  | Basidati |

## Join naturale: esempio (1)

CORSI

| Corso        | Docente | Aula |
|--------------|---------|------|
| Architetture | Rossi   | Alfa |
| Basi di dati | Neri    | Beta |

AULE

| Aula | Piano |
|------|-------|
| Alfa | Terra |
| Beta | Primo |

$\text{CORSI} \bowtie \text{AULE} = \text{AULE} \bowtie \text{CORSI}$

| Corso        | Docente | Aula | Piano |
|--------------|---------|------|-------|
| Architetture | Rossi   | Alfa | Terra |
| Basi di dati | Neri    | Beta | Primo |

## 2.2.5. Equivalenza di espressioni algebriche

L'algebra relazionale è un linguaggio procedurale, che prevede quindi di specificare sia le operazioni da eseguire sia l'ordine in cui eseguirle. Questo ordine avrà un impatto sul costo dell'esecuzione, perciò è importante scrivere le espressioni in modo tale da ottimizzare i costi, ovvero limitando la dimensione dei risultati intermedi.

Nel caso del linguaggio SQL, che invece è un linguaggio dichiarativo, le query vengono tradotte in espressioni algebriche dal DBMS che le ottimizza automaticamente.

### Equivalenza di espressioni algebriche

Due espressioni algebriche  $E_1$  ed  $E_2$  si dicono equivalenti se producono lo stesso risultato sulle relazioni a cui sono applicate, indipendentemente dalle specifiche istanze. Occorre quindi trasformare un'espressione iniziale in un'altra equivalente ma meno costosa per mezzo di una trasformazione di equivalenza.

### Trasformazioni di equivalenza

- **Atomizzazione delle espressioni:**  $\sigma_{C_1 \wedge C_2}(E) \equiv \sigma_{C_1}(\sigma_{C_2}(E))$ , dove:
  - $E$ : espressione;
  - $C_1$  e  $C_2$ : condizioni;Se ho l'AND le condiz. sono commutative  $\rightarrow \sigma_{C_1 \wedge C_2}(E) \equiv \sigma_{C_2}(\sigma_{C_1}(E))$
- **Idempotenza delle proiezioni:**  $\Pi_X(E) \equiv \Pi_X(\Pi_{X,Y}(E))$ , dove:
  - $E$ : espressione;
  - $X$  e  $Y$ : insiemi di attributi;
- **Anticipazione della selezione sul join:**  $\sigma_C(E_1 \bowtie E_2) \equiv E_1 \bowtie \sigma_C(E_2)$ , dove:
  - $E_1$  ed  $E_2$ : espressioni;
  - $C$ : condizione che fa riferimento solo ad attributi di  $E_2$ ;
- **Anticipazione della proiezione sul join:**  $\Pi_{X_1, Y_2}(E_1 \bowtie E_2) \equiv E_1 \bowtie \Pi_{Y_2}(E_2)$ , dove:
  - $E_1$  ed  $E_2$ : espressioni;
  - $X_1$  ed  $X_2$ : attributi di  $E_1$  ed  $E_2$ ;
  - $Y_2 \supseteq (X_1 \cap X_2)$ : insieme di attributi che include tutti gli attributi di  $E_2$  coinvolti nel join;
- **Combinando l'idempotenza delle proiezioni con l'anticipazione della proiezione sul join**, si ottiene una nuova trasformazione di equivalenza:  $\Pi_Y(E_1 \bowtie_C E_2) \equiv \Pi_Y(\Pi_{Y_1}(E_1) \bowtie_C \Pi_{Y_2}(E_2))$ , dove:
  - $E_1$  ed  $E_2$ : espressioni;
  - $X_1$  ed  $X_2$ : attributi di  $E_1$  ed  $E_2$ ;
  - $J_1$  e  $J_2$ : attributi di  $E_1$  ed  $E_2$  coinvolti nella join;
  - $Y_1 = (X_1 \cap Y) \cup J_1$ : insieme di attributi di  $E_1$  coinvolti nella join o appartenenti al risultato;
  - $Y_2 = (X_2 \cap Y) \cup J_2$ : insieme di attributi di  $E_2$  coinvolti nella join o appartenenti al risultato;
- **Inglobamento della selezione in un prodotto cartesiano (definizione di join):**  $\sigma_C(E_1 \times E_2) \equiv E_1 \bowtie_C E_2$ , dove:
  - $E_1$  ed  $E_2$ : espressioni;
  - $C$ : condizione che fa riferimento ad attributi di  $E_1$  ed  $E_2$ ;Sarebbe la definizione di join  $\rightarrow$  **faccio il cartesiano e poi la selezione** è come fare il join
- **Distributività della selezione rispetto all'unione:**  $\sigma_C(E_1 \cup E_2) \equiv \sigma_C(E_1) \cup \sigma_C(E_2)$ , dove:
  - $E_1$  ed  $E_2$ : espressioni;
  - $C$ : condizione;
- **Distributività della selezione rispetto alla differenza:**  $\sigma_C(E_1 - E_2) \equiv \sigma_C(E_1) - \sigma_C(E_2)$ , dove:
  - $E_1$  ed  $E_2$ : espressioni;

- $C$  : condizione;

- **Distributività della proiezione rispetto all'unione:**  $\Pi_X (E_1 \cup E_2) \equiv \Pi_X (E_1) \cup \Pi_X (E_2)$  , dove:

- $E_1$  ed  $E_2$  : espressioni;
- $X$  : attributi in  $E_1$  ed  $E_2$  ;

**Non** vale la *distributività della proiezione* rispetto alla  **differenza**  e all'**intersezione**

Perché se proietto (butto via dei campi che mi fanno da chiave) dopo potrei fare un'intersezione senza un senso logico. Ad esempio, se proietto nome e cognome (togliendo C.F.) quando vado a fare l'intersezione di due persone con nomi uguali non ho più la chiave per distinguerli e quindi le tratterei allo stesso modo di avere più campi ma della stessa persona.

**Se la proiezione non contiene più l'attributo chiave, i valori saranno valutati secondo gli attributi della proiezione (in sto esempio nome e cognome), quindi persone con nome o cognome uguale saranno valutate come la stessa persona.**

- **Distributività del join rispetto all'unione:**  $E \bowtie (E_1 \cup E_2) \equiv (E \bowtie E_1) \cup (E \bowtie E_2)$  , dove:

- $E, E_1$  ed  $E_2$  : espressioni;

- **Corrispondenza tra operatori insiemistici e la selezione:**

- $\sigma_{C_1 \vee C_2} (E) \equiv \sigma_{C_1} (E) \cup \sigma_{C_2} (E)$  ;
- $\sigma_{C_1 \wedge C_2} (E) \equiv \sigma_{C_1} (E) \cap \sigma_{C_2} (E) \equiv \sigma_{C_1} (E) \bowtie \sigma_{C_2} (E)$  (*in questo caso join con schemi uguali*)
- $\sigma_{C_1 \wedge \neg C_2} (E) \equiv \sigma_{C_1} (E) - \sigma_{C_2} (E)$  ;

- **Associatività e commutatività degli operatori binari:**

- $E_1 \text{ op } (E_2 \text{ op } E_3) \equiv (E_1 \text{ op } E_2) \text{ op } E_3$  ;
- $E_1 \text{ op } E_2 \equiv E_2 \text{ op } E_1$  , dove  $\text{op} \in \{\cup, \cap, \times, \bowtie\}$

## Alcune formule utili

- $r \bowtie r \equiv r$

- $r \cup r \equiv r$

- $r \cap r \equiv r$

- $r - r \equiv \emptyset$

- $r \bowtie \sigma_C r \equiv \sigma_C r$

- $r \cup \sigma_C r \equiv r$

- $r \cap \sigma_C r \equiv \sigma_C r$

- $r - \sigma_C r \equiv \sigma_{\neg C} r$

- $r \bowtie \emptyset \equiv \emptyset$

- $r \cup \emptyset \equiv r$

- $r \cap \emptyset \equiv \emptyset$

- $r - \emptyset \equiv r$

- $\emptyset - r \equiv \emptyset$

- $r \times \emptyset \equiv \emptyset$

- $\sigma_C \emptyset \equiv \emptyset$

- $\Pi_X \emptyset \equiv \emptyset$

## 2.2.6. Idiomi di interrogazione

### 2.2.6.1. Esiste / per ogni, minimo / massimo

#### Esiste / per ogni

Data la relazione  $r(A, B)$  :

- trovare gli  $A$  per cui esiste un  $B$  che soddisfa la condizione  $p(B) : \Pi_A (\sigma_{p(B)} (r))$

REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare gli studenti che hanno preso più di 26 in almeno un esame

$\Pi_{\text{Matr}}(\sigma_{\text{Voto} > 26} \text{REGISTRI})$

| Matr  |
|-------|
| 11111 |
| 22222 |
| 33333 |

- trovare gli  $A$  per i quali tutti i  $B$  soddisfano la condizione  $p(B) : \Pi_A (r) - \Pi_A (\sigma_{\neg p(B)} (r))$

REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare gli studenti che hanno preso più di 26 in tutti gli esami

$\Pi_{\text{Matr}}(\text{REGISTRI}) - \Pi_{\text{Matr}}(\sigma_{\text{Voto} \leq 26} \text{REGISTRI})$

| Matr  |
|-------|
| 11111 |
| 33333 |

#### Minimo e massimo assoluto

Data la relazione  $r(A, B)$  :

- trovare il minimo valore di  $B$  :  $val := \Pi_B (r) , val - \Pi_B (val \bowtie_{B > B'} (\rho_{B' \leftarrow B} (val)))$

L'idea è di **trovare tutti i valori di B tranne il più piccolo** facendo una join tra la tabella B e una sua copia B' solo se i valori sono **maggiori**. Quindi per forza **il più piccolo non joinerà con nessuno perché non ci sono valori più piccoli di lui**, essendo il minimo. Una volta fatto questo **proietto** (elimino doppioni) e poi **sottraggo** tutti i valori tranne il min dalla tabella B così da trovare effettivamente il minimo (simile al ragionamento del per ogni)

REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare il voto minimo

$VOTI := \rho_{V \leftarrow \text{Voto}}(\Pi_{\text{Voto}}(\text{REGISTRI}))$   
 $VOTI - \Pi_V(VOTI \bowtie_{V > V'} (\rho_{V' \leftarrow V} VOTI))$

| V  |
|----|
| 25 |

- trovare il massimo valore di  $B$  :  $val := \Pi_B (r) , val - \Pi_B (val \bowtie_{B < B'} (\rho_{B' \leftarrow B} (val)))$

Stessa cosa di prima solo che prendo tutti i valori tranne il max, proietto e poi sottraggo dal tot.

REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare la data più recente

$DATE := \rho_{D \leftarrow \text{Data}}(\Pi_{\text{Data}}(\text{REGISTRI}))$   
 $DATE - \Pi_D(DATE \bowtie_{D < D'} (\rho_{D' \leftarrow D} DATE))$

| D        |
|----------|
| 21-11-05 |

## Minimo e massimo relativo

Data la relazione  $r(A, B)$  : (negli esercizi  $r$  potrebbe essere anche su un altro dominio  $\rightarrow$  devo proiettare)

- trovare, per ogni  $A$ , il minimo valore di  $B$  :  $r - \Pi_{A, B} (r \bowtie_{A=A' \wedge B > B'} (\rho_{A', B' \leftarrow A, B} (r)))$
- trovare, per ogni  $A$ , il massimo valore di  $B$  :  $r - \Pi_{A, B} (r \bowtie_{A=A' \wedge B < B'} (\rho_{A', B' \leftarrow A, B} (r)))$ .

### REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare per ogni studente il voto minimo

$VOTI := \rho_{M, V \leftarrow Matr, Voto} (\Pi_{Matr, Voto} (REGISTRI))$   
 $VOTI - \Pi_{M, V} (VOTI \bowtie_{M=M' \wedge V > V'} (\rho_{M', V' \leftarrow M, V} (VOTI)))$

| M     | V  |
|-------|----|
| 11111 | 28 |
| 22222 | 25 |
| 33333 | 30 |

### REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare per ogni corso la data più recente

$ESAMI := \rho_{C, D \leftarrow Corso, Data} (\Pi_{Corso, Data} (REGISTRI))$   
 $ESAMI - \Pi_{C, D} (ESAMI \bowtie_{C=C' \wedge D < D'} (\rho_{C', D' \leftarrow C, D} (ESAMI)))$

| C        | D        |
|----------|----------|
| Basidati | 21-11-05 |
| Sistemi  | 15-06-05 |

## Rispondente a minimo e massimo

Data la relazione  $r(A, B)$  :

- trovare gli  $A$  per cui  $B$  ha il valore minimo :  $\Pi_A (r - \Pi_{A, B} (r \bowtie_{B > B'} (\rho_{A', B' \leftarrow A, B} (r))))$

Stiamo facendo una cosa simile al minimo assoluto ma ci stiamo portando dietro anche la A perché dopo ci serve per trovare, appunto, gli A per cui B ha il valore minimo.

### REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare gli studenti che hanno preso il voto più basso

$VOTI := \rho_{M, V \leftarrow Matr, Voto} (\Pi_{Matr, Voto} (REGISTRI))$   
 $\Pi_M (VOTI - \Pi_{M, V} (VOTI \bowtie_{V > V'} (\rho_{M', V' \leftarrow M, V} (VOTI))))$

| M     |
|-------|
| 22222 |

- trovare gli  $A$  per cui  $B$  ha il valore massimo :  $\Pi_A (r - \Pi_{A, B} (r \bowtie_{B < B'} (\rho_{A', B' \leftarrow A, B} (r))))$

Stiamo facendo una cosa simile al massimo assoluto ma ci stiamo portando dietro anche la A perché dopo ci serve per trovare, appunto, gli A per cui B ha il valore massimo.

### REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare il corso con la data più recente

$ESAMI := \rho_{C, D \leftarrow Corso, Data} (\Pi_{Corso, Data} (REGISTRI))$   
 $\Pi_C (ESAMI - \Pi_{C, D} (ESAMI \bowtie_{D < D'} (\rho_{C', D' \leftarrow C, D} (ESAMI))))$

| C        |
|----------|
| Basidati |

## 2.2.6.2. Idiomi di interrogazione: cardinalità e copertura

### Cardinalità

Data la relazione  $r(A, B)$ , trovare gli  $A$  che sono associati a :

- almeno due  $B : \Pi_A (r \bowtie_{A=A' \wedge B \neq B'} (\rho_{A', B' \leftarrow A, B} (r)))$

#### REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare gli studenti che hanno fatto almeno due esami

ESAMI :=  $\rho_{M, C \leftarrow \text{Matr}, \text{Corso}} (\Pi_{\text{Matr}, \text{Corso}} \text{REGISTRI})$   
 $\Pi_M (\text{ESAMI} \bowtie_{M=M' \wedge C \neq C'} (\rho_{M', C' \leftarrow M, C} (\text{ESAMI})))$

| M     |
|-------|
| 11111 |
| 22222 |

- al più un  $B : \Pi_A (r) - \Pi_A (r \bowtie_{A=A' \wedge B \neq B'} (\rho_{A', B' \leftarrow A, B} (r)))$

#### REGISTRI

| Matr  | Corso    | Data     | Voto |
|-------|----------|----------|------|
| 11111 | Basidati | 20-01-05 | 28   |
| 11111 | Sistemi  | 15-06-05 | 30   |
| 22222 | Basidati | 21-11-05 | 27   |
| 22222 | Sistemi  | 15-03-05 | 25   |
| 33333 | Basidati | 16-05-05 | 30   |

Trovare gli studenti che hanno fatto al più un esame:

R1 :=  $\rho_{M, C \leftarrow \text{Matr}, \text{Corso}} (\Pi_{\text{Matr}, \text{Corso}} \text{REGISTRI})$   
ESAME :=  $\Pi_M (R1 \bowtie_{M=M' \wedge C \neq C'} (\rho_{M', C' \leftarrow M, C} (R1)))$   
 $(\Pi_M R1) - \text{ESAME}$

| M     |
|-------|
| 33333 |

- almeno tre  $B : \Pi_A (\sigma_{A=A' \wedge A=A'' \wedge B \neq B' \wedge B \neq B'' \wedge B' \neq B''} (r \times \rho_{A', B' \leftarrow A, B} (r) \times \rho_{A'', B'' \leftarrow A, B} (r)))$

*La terza disuguaglianza la scrivo lo stesso perché la transitività non vale per le disuguaglianze*

In questo caso faccio due duplicati della tabella perché devo avere altre due corrispondenze oltre a quella originale, quindi 3 duplicati. In questo caso facciamo una selezione su un triplo prodotto cartesiano, che è la stessa cosa di fare un **triplo join in cascata** ->  $r \text{ join } r' \text{ join } r''$

- almeno  $n$   $B$  : allo stesso modo, ma con  $n - 1$  prodotti cartesiani;
- al più  $n$   $B : \Pi_A (r) - "A \text{ associati ad almeno } n + 1 B"$  ;
- esattamente  $n$   $B : "A \text{ associati ad almeno } n B" - "A \text{ associati ad almeno } n + 1 B"$  .

### Copertura

Date le relazioni  $r(A, B)$  e  $s(B)$  :

- trovare gli  $A$  che sono associati in  $r$  a tutti i valori di  $B$  in  $s : \Pi_A (r) - \Pi_A ((\Pi_A (r) \times s) - r)$

L'idea è di **trovare tutte le coppie possibili (studente <-> esame)** e sottrarci il mio insieme  $r$  di partenza. Poi, avendo proiettato su  $A$  sottraggo il valore appena trovato alla proiezione su  $A$  del mio insieme originale

| ESAMI |       | CORSI |          |
|-------|-------|-------|----------|
| Matr  | Esame | Esame | Titolo   |
| 11111 | CS101 | CS101 | Basidati |
| 22222 | CS102 | CS102 | Sistemi  |
| 11111 | CS102 |       |          |

Trovare la matricola degli studenti che hanno fatto tutti gli esami

CO :=  $\Pi_{\text{Esame}} (\text{CORSI})$   
 $\Pi_{\text{Matr}} (\text{ESAMI}) - \Pi_{\text{Matr}} ((\Pi_{\text{Matr}} (\text{ESAMI}) \times \text{CO}) - \text{ESAMI})$

| Matr  |
|-------|
| 11111 |

## 2.3. Esercizi di algebra relazionale

### 2.3.1. Parte I

#### Relazioni

CELLULARE (Codice, CFUtente, Modello, Marca, Colore)

ABBONAMENTO (Numero, CFUtente, Operatore, Tariffa)

UTENTE (CF, Nome, Cognome, Città)

#### Esercizi

- Interrogazione 1:

- Richiesta: determinare il codice dei cellulari di marca Nokia di colore rosso oppure nero;

- Soluzione:

- $\Pi_{\text{Codice}} (\sigma_{(\text{Colore} = \text{'rosso'} \vee \text{Colore} = \text{'nero'}) \wedge \text{Marca} = \text{'Nokia'}} \text{CELLULARE})$

- Interrogazione 2:

- Richiesta: determinare il codice dei cellulari degli utenti residenti a Milano;

- Soluzione:

- $UT\_MIL := \Pi_{CF} (\sigma_{\text{Città} = \text{'Milano'}} \text{UTENTE})$

- $\Pi_{\text{Codice}} (UT\_MIL \bowtie_{CF = CF_{\text{Utente}}} \text{CELLULARE})$

- 

Per ottimizzare sarebbe meglio fare il join sulla proiezione codice, cfutente di cellulare al posto di solo cellulare però per l'esame va bene. Però è sbagliato **non selezionare subito milano** ma selezionarlo alla fine

- Interrogazione 3:

- Richiesta: determinare la marca dei cellulari degli utenti serviti da Tim nella città di Milano;

- Soluzione:

- $UT\_TIM := \Pi_{CF_{\text{Utente}}} (\sigma_{\text{Operatore} = \text{'Tim'}} \text{ABBONAMENTO})$

- $UT\_MIL := \rho_{CF_{\text{Utente}} \leftarrow CF} (\Pi_{CF_{\text{Utente}}} (\sigma_{\text{Città} = \text{'Milano'}} \text{UTENTE}))$

- $\Pi_{\text{Marca}} ((UT\_TIM \cap UT\_MIL) \bowtie \text{CELLULARE})$

Fare il **join naturale** tra UT\_TIM e UT\_MIL va bene ma costa di più dell'intersezione.

- Interrogazione 4:

- Richiesta: determinare gli operatori che servono tutti gli utenti della città di Milano;

- Soluzione:

- $UT\_MIL := \rho_{CF_{\text{Utente}} \leftarrow CF} (\Pi_{CF_{\text{Utente}}} (\sigma_{\text{Città} = \text{'Milano'}} \text{UTENTE}))$

- $OP := \Pi_{\text{Operatore}} \text{ABBONAMENTO}$

- $TUTTI := UT\_MIL \times OP$

- $ABB\_MIL := \Pi_{CF_{\text{Utente}}, \text{Operatore}} (UT\_MIL \bowtie \text{ABBONAMENTO})$

- $\Pi_{\text{Operatore}} \text{ABB\_MIL} - \Pi_{\text{Operatore}} (TUTTI - \text{ABB\_MIL})$

Usiamo **l'idioma della copertura** -> mi chiede tutti

- Interrogazione 5:

- Richiesta: determinare la tariffa minima applicata da Tim;

- Soluzione:

- $TAR\_TIM := \Pi_{\text{Tariffa}} (\sigma_{\text{Operatore} = \text{'Tim'}} \text{ABBONAMENTO}) ;$

- $TAR\_TIM - \Pi_{\text{Tariffa}} (TAR\_TIM \bowtie_{\text{Tariffa} > \text{Tariffa}} (\rho_{\text{Tariffa} \leftarrow \text{Tariffa}} \text{ABBONAMENTO})) ;$

Qua semplicemente applichiamo **l'idioma del minimo** -> join escludendo il minimo e poi sottraggo dall'ins di partenza  
Ricorda sempre di fare una copia esatta della tabella per poter fare il join e eliminare il minimo (accoppio solo i valori maggiori degli altri valori nella tabella)









- Interrogazione 6:
  - Richiesta: determinare gli utenti (CF, Nome; Cognome e Città) per i quali tutti i numeri di telefono loro intestati hanno tariffa maggiore di 100;
  - Soluzione:
    - $UT\_MIN := \Pi_{CFUtente} (\sigma_{Tariffa \leq 100} ABBONAMENTO) ;$
    - $UT\_RIS := \Pi_{CFUtente} ABBONAMENTO - UT\_MIN ;$
    - $\Pi_{CF, Nome, Cognome, Città} (UT\_RIS \bowtie_{CFUtente = CF} UTENTE)$

Qua ci chiedi gli utenti per i quali **tutti i numeri di telefono intestati hanno una tariffa maggiore di 100**. Quindi, in modo simile all'esercizio degli ingredienti biologici del gelato, dobbiamo prima trovare gli utenti che hanno **almeno una tariffa minore di 100**, sottrarre il valore appena trovato alla **tabella totale** così da trovare **tutti i numeri di telefono con tariffe maggiori di 100** (UT\_RIS). **Non posso selezionare direttamente maggiore di 100 perché troverei gli utenti con almeno una tariffa maggiore di 100** (Attenzione: non va bene perché **io voglio quelli con TUTTI** gli abbonamenti >100)

In UT\_RIS è **sbagliato pescare da UTENTI al posto di ABBONAMENTO** perché altrimenti **andrei a trovare anche gli utenti che non hanno l'abbonamento**. (sarebbe stato giusto se la traccia avesse chiesto gli utenti con abbonamenti con tariffa minore di 100)

È **anche importante proiettare su CFUtente nella var UT\_MIN** perché se no nel passaggio dopo andremmo semplicemente a **trovare gli utenti con abbonamenti > di 100**, cadendo nell'errore di prima in cui selezionavamo utenti con almeno un abbonamento maggiore di 100 e non tutti >100

## Relazioni

RADIO (Codice, Nome, Frequenza, Luogo)

PROGRAMMA (Codice, Nome, Conduttore, FasciaOraria, Durata, Tipo, CodiceRadio)

## Esercizi

- Interrogazione 1:
  - Richiesta: determinare i nomi delle radio che trasmettono programmi non musicali nella fascia del **mattino e della sera** ma non nella fascia del pomeriggio;
  - Soluzione:
    - $MAT := \Pi_{CodiceRadio} (\sigma_{Tipo \neq 'musicale' \wedge FasciaOraria = 'mattina'} PROGRAMMA) ;$
    - $SER := \Pi_{CodiceRadio} (\sigma_{Tipo \neq 'musicale' \wedge FasciaOraria = 'sera'} PROGRAMMA)$
    - $POM := \Pi_{CodiceRadio} (\sigma_{Tipo \neq 'musicale' \wedge FasciaOraria = 'pomeriggio'} PROGRAMMA)$
    - $COD\_RIS := (MAT \cap SER) - POM$
    - $\Pi_{Nome} (RADIO \bowtie_{Codice = CodiceRadio} COD\_RIS)$
- Interrogazione 2:
  - Richiesta: determinare per ogni radio il codice del programma di durata minima;
  - Soluzione:
    - $PROG := \Pi_{Codice, Durata, CodiceRadio} PROGRAMMA$
    - $PROG' := \rho_{C', D', CR' \leftarrow Codice, Durata, CodiceRadio} PROG$
    - $\Pi_{Codice, CodiceRadio} PROG - \Pi_{Codice, CodiceRadio} (PROG \bowtie_{CodiceRadio = CR' \wedge Durata > D'} PROG')$

Qui ci chiede **per ogni radio il cod del programma di durata minima**. Usiamo **l'idioma del minimo rispondente**. Faccio la mia **tabella mirror**, stando sempre attento a **rinominare i campi** (nel join devono essere diversi). Faccio il join come quando devo trovare il min (durata > D', quindi escludo il minimo) ma allo stesso tempo devo anche selezionare **per ogni radio** (quindi CodiceRadio=CR'). Faccio la proiezione sul join, trovando appunto tutte le tuple tranne quelle con "per ogni codice il programma con durata minore". Poi semplicemente sottraggo dal totale (**ATTENZIONE:** essendo minimo rispondente devo portarmi dietro entrambi i campi "codice, codice radio").

• Interrogazione 3:

- Richiesta: determinare i nomi delle radio che trasmettono **almeno due** programmi non musicali;
- Soluzione:
  - $NO\_MUS := \Pi_{Codice, CodiceRadio} (\sigma_{Tipo \neq 'musicale'} PROGRAMMA)$
  - $NO\_MUS' := \rho_{C'}; CR' \leftarrow Codice, CodiceRadio \ NO\_MUS$
  - $COD\_RIS := \Pi_{CodiceRadio} (NO\_MUS \bowtie_{CodiceRadio = CR' \wedge Codice \neq C'} NO\_MUS')$
  - $\Pi_{Nome} (RADIO \bowtie_{Codice = CodiceRadio} COD\_RIS)$

• Interrogazione 4:

- Richiesta: determinare i nomi delle radio che **non hanno alcun programma** musicale;
- Soluzione:
  - $MUS := \Pi_{CodiceRadio} (\sigma_{Tipo = 'musicale'} PROGRAMMA)$
  - $NO\_MUS := \Pi_{Codice} RADIO - \rho_{Codice \leftarrow CodiceRadio} MUS$
  - $\Pi_{Nome} (RADIO \bowtie NO\_MUS)$

Trovo le **radio musicali** → proiezione su codice radio su selezione tipo=musicale. Poi **sottraggo da radio** (sempre proiettando su codice) **le radio musicali**, trovando così le **radio che non hanno programmi musicali**. Alla fine per trovare il nome faccio un join naturale tra radio e le radio che non hanno programmi musicali e proietto su nome.

• Interrogazione 5:

- Richiesta: determinare i nomi delle radio che hanno programmi musicali **condotti da DjX**;
- Soluzione:
  - $COD\_RIS := \Pi_{CodiceRadio} (\sigma_{Tipo = 'musicale' \wedge Conduttore = 'DjX'} PROGRAMMA)$
  - $\Pi_{Nome} (RADIO \bowtie_{Codice = CodiceRadio} COD\_RIS)$

Avrei anche potuto rinominare codiceRadio nella prima espressione così da avere i **campi codice da confrontare con lo stesso nome** e quindi sotto lasciare solo un **join naturale**.

• Interrogazione 6:

- Richiesta: determinare i nomi delle radio che trasmettono **programmi musicali o quiz**;
- Soluzione:
  - $COD\_RIS := \Pi_{CodiceRadio} (\sigma_{Tipo = 'musicale' \vee Tipo = 'quiz'} PROGRAMMA)$
  - $\Pi_{Nome} (RADIO \bowtie_{Codice = CodiceRadio} COD\_RIS)$

Qua è come prima: faccio una **selezione** (musicale o quiz) e poi faccio il **join** per andare a trovare il campo nome

• Interrogazione 7:

- Richiesta: determinare i nomi delle radio che trasmettono **programmi musicali e quiz**;
- Soluzione:
  - $MUS := \Pi_{CodiceRadio} (\sigma_{Tipo = 'musicale'} PROGRAMMA) ;$
  - $QUIZ := \Pi_{CodiceRadio} (\sigma_{Tipo = 'quiz'} PROGRAMMA) ;$
  - $COD\_RIS := MUS \cap QUIZ ;$
  - $\Pi_{Nome} (RADIO \bowtie_{Codice = CodiceRadio} COD\_RIS) ;$

Qua il ragionamento che devo fare è trovare prima le radio che trasmettono prog musicali , poi quelle che trasmettono quiz e , infine, farne l'intersezione (**intersezione fatta su codice radio**). Infine, come sempre, per **trovare il nome** devo joinare radio con l'insieme di radio che trasmettono sia programmi musicali che quiz.

**Attenzione:** è sbagliato mettere l'AND → tipo=musicale AND tipo=quiz perché mi darebbe valore vuoto perché stiamo **lavorando sullo stesso attributo**. Se invece lavoro su **attributi diversi** allora **ha senso fare l'AND**

## 3. Linguaggio SQL

### 3.1. DDL

#### 3.1.1. Domini

##### Domini

Permettono di specificare i valori ammissibili per gli attributi, e nel linguaggio SQL ne esistono di due tipi:

- **elementari**: predefiniti dallo standard o built-in nello specifico DBMS, che includono:
  - caratteri;
  - tipi numerici (esatti e approssimati);
  - data e ora (date, time e timestamp);
  - intervalli temporali;
  - boolean;
  - blob (binary large object);
  - clob (character large object);
- **definiti dall'utente**: sulla base di altri domini, possono essere specificati con la seguente sintassi:  
`create domain NomeDominio as TipoDato [ValoreDefault] [Vincoli]`, dove:
  - *NomeDominio* : nome da dare al dominio;
  - *TipoDato* : **dominio** su cui costruire;
  - *ValoreDefault* : da assumere se nulla è specificato, a sua volta specificato con la seguente sintassi:  
`default <ValoreGenerico | user | null >`, dove:
    - *ValoreGenerico* : qualsiasi valore compatibile con il dominio;
    - *user* : login dell'utente che effettua il comando;
    - *null* : valore nullo (polimorfo);
  - *Vincoli* : condizioni che i valori del dominio devono rispettare.

#### 3.1.2. Schemi, tabelle e vincoli

##### Schemi

Gli schemi possono essere specificati con la seguente sintassi:

`create schema [NomeSchema] [[authorization] Autorizzazione] {DefElementoSchema}`, dove:

- *NomeSchema* : nome dello schema;
- *Autorizzazione* : nome dell'**utente proprietario**;
- *DefElementoSchema* : definizione di elementi nello schema (domini, tabelle, indici, asserzioni, viste, privilegi, ...).

##### Tabelle

Le tabelle possono essere specificate con la seguente sintassi:

`create table NomeTabella (NomeAttributo Dominio [ValoreDefault] [Vincoli] {, NomeAttributo Dominio`

`[ValoreDefault] [Vincoli]} {AltriVincoli})`, dove i **vincoli possono essere**:

- **intra-relazionali** (**coinvolgono una relazione** e devono essere verificati da ogni istanza della relazione):
  - **not null** : su un singolo attributo;
  - **unique** : permette la definizione di chiavi candidate:
    - per un solo attributo: `unique` dopo il dominio;
    - per più attributi: `unique (Attributo {, Attributo})` associata alla tabella (alla fine)

- **primary key** : definisce la chiave primaria (implica unique e not null );
- **check** : specifica vincoli;

- Ogni coppia (Nome, Cognome) identifica univocamente una tupla

```
Nome character(20) not null
Cognome character(20) not null,
unique (Nome,Cognome)
```

```
create table Studenti
( Matr      char(6)      primary key,
  Cognome   varchar(30) not null,
  Nome      varchar(30) not null,
  CdL       char(5)      not null)
```

- Il nome o il cognome singolarmente identificano univocamente una tupla

```
Nome character(20) not null unique
Cognome character(20) not null unique
```

```
create table Corsi
( Codice char(6)      primary key,
  Titolo  varchar(30) not null unique,
  Cognome varchar(20),
  Nome    varchar(20))
```

- **inter-relazionali** (coinvolgono più relazioni e devono essere verificati da ogni istanza delle relazioni):

- **check** : vincoli generici;
- **references e foreign key** : **integrità referenziale**:
  - per un solo attributo: references dopo il dominio;
  - per più attributi (o anche uno): foreign key (*Attributo {, Attributo}*) references ...

```
create table Docenti
( Cognome varchar(20),
  Nome    varchar(20),
  Dip     varchar(30),
  primary key (Cognome,Nome))
```

```
create table Corsi
( Codice char(6)      primary key,
  Titolo  varchar(30) not null unique,
  Cognome varchar(20),
  Nome    varchar(20),
  foreign key (Cognome,Nome)
    references Docenti(Cognome,Nome))
```

```
create table Esami
( Matr      char(6)      references
  Studenti(Mat),
  Cod-corso char(6)      references
  Corsi(Codice),
  Data      date         not null,
  Voto      smallint     not null,
  primary key (Matr,Cod-corso))
```

## Integrità referenziale

I vincoli di integrità referenziale possono essere **violati** a seguito di:

- operazioni sulla **tabella interna** (che riferenzia): **inserimento o modifica di tuple** che **non hanno corrispondenza** in una **tabella esterna**, nel qual caso il sistema rifiuta l'operazione;
- operazioni sulla **tabella esterna** (referenziata): **cancellazione o modifica di tuple** che sono correttamente **referenziate** da qualche tabella (in SQL specifichiamo una **politica di reazione**)

|               |        |                        |
|---------------|--------|------------------------|
| CANCELLAZIONE | DELETE | STUDENTI               |
| INSERIMENTO   | INSERT | ISCRIZIONE             |
| MODIFICA      | UPDATE | STUDENTI<br>ISCRIZIONE |

## Politiche di reazione

Nella definizione di vincoli di integrità referenziale è anche possibile **associare delle politiche di reazione** nel caso in cui questi non siano soddisfatti, con la seguente sintassi:

on <delete|update> <cascade|set null|set default|no action> , dove:

- **cascade** : **propaga** la modifica o cancellazione;
- **set null** : imposta a **null** l'**attributo** che rimarrebbe **pendente**;
- **set default** : imposta al **valore di default** l'**attributo** che rimarrebbe **pendente**;
- **no action** : **impedisce la modifica o cancellazione** (**ATTENZIONE**: non vuol dire "non fa niente")

```
create table Esami
( Matr      char(6)  references
  Studenti(Matr)
  on delete cascade
  on update cascade,
  Cod-corso char(6)  references
  Corsi(Codice)
  on delete no action
  on update no action,
  Data      date      not null,
  Voto      smallint not null,
  primary key (Matr,Cod-corso))
```

```
STUDENTI(Matr,Cognome,Nome,CdL)
ESAMI(Matr,Cod-corso,Data,Voto)
```

Cosa succede agli esami di uno studente se si cancella uno studente?

- cascade: si cancellano
- set null: si pone a null la matricola (violerebbe non nullità di chiave)
- set default: si pone al valore di default la matricola
- no action: si impedisce la cancellazione dello studente

```
STUDENTI(Matr,Cognome,Nome,CdL)
ESAMI(Matr,Cod-corso,Data,Voto)
```

Cosa succede agli esami di uno studente se si modifica la matricola dello studente (nella tabella Studenti)?

- cascade: si modifica la matricola degli esami
- set null: si pone a null la matricola (violerebbe non nullità di chiave)
- set default: si pone al valore di default la matricola
- no action: si impedisce la modifica dello studente

## Modifiche agli schemi

Alcuni comandi permettono di **modificare** definizioni di elementi dello schema precedentemente introdotti:

- **alter** : per **modificare domini e schemi di tabelle** (se si aggiunge un nuovo vincolo questo deve essere soddisfatti dai dati già presenti), con le seguenti sintassi:
  - alter domain *NomeDominio* <set default *ValoreDefault*|drop default|addconstraint *DefVincolo*|drop constraint *NomeVincolo*> ;
  - alter table *NomeTabella* <alter column *NomeAttributo* <set default *ValoreDefault*|drop default>|add constraint *DefVincolo*|drop constraint *NomeVincolo*|add column *DefAttributo*|drop column *NomeAttributo*>
- **drop** : per **rimuovere gli elementi dallo schema della base di dati**, con la seguente sintassi: drop <schema|domain|table|view|assertion> *NomeElemento* [restrict|cascade] , dove:
  - **restrict** : indica che il comando non deve essere eseguito in presenza di oggetti non vuoti o utilizzati (è il default);
  - **cascade** : indica che se si rimuove un oggetto non vuoto o referenziato, tutti quelli che contiene o a cui si riferiscono vengono rimossi (se si rimuove un dominio, gli attributi che l'utilizzavano vengono riferiti al dominio elementare su cui il primo era stato definito).

### Esempi

```
alter domain PrezzoQuotidiani
  set default 1.20
alter table Esami
  add column Sessione smallint
```

## Cataloghi relazionali

I cataloghi sono i “**dizionari dei dati**”, ossia la **descrizione della struttura degli elementi della base di dati**. Sono basati su una **struttura relazionale** (sono quindi **riflessivi**) e si dividono in due livelli:

- **Definition Schema**: tabelle che contengono la descrizione di tutte le strutture della base di dati;
- **Information Schema**: composto da viste sul Definition\_Schema.

## 3.2. DML: interrogazioni semplici

### 3.2.1. Select, from, where

Ricordiamo che **SQL è un linguaggio dichiarativo**, infatti **l'utente specifica l'informazione che vuole ritrovare, non la procedura necessaria per estrarla**. Le interrogazioni vengono tradotte dall'ottimizzatore all'interno del DBMS, così che il programmatore possa concentrarsi solamente sulla leggibilità delle interrogazioni e non debba preoccupare della loro efficienza.

#### Interrogazioni semplici

Le interrogazioni più semplici hanno la seguente sintassi:

```
select ListaAttributi
from ListaTabelle
[where Condizione]
```

Questa interrogazione restituisce la *ListaAttributi* del prodotto cartesiano di *ListaTabelle* per le tuple che soddisfano la *Condizione*, ed equivale, a meno di duplicati, a:

$\Pi_{\text{ListaAttributi}} (\sigma_{\text{Condizione}} T_1 \times T_2 \times \dots \times T_n)$  dove  $T_1 \times T_2 \times \dots \times T_n$  sono le tabelle in *ListaTabelle*.

Una forma più dettagliata di questa interrogazione è la seguente (che prevede l'utilizzo di alias da dare ad attributi o tabelle):

```
select AttrEspr [[as] Alias] {, AttrEspr [[as] Alias]}
from Tabella [[as] Alias] {, Tabella [[as] Alias]}
[where Condizione]
```

```
select Cognome, Nome, CdL as
CorsoDiLaurea
from Studenti
where Matr='123456'
```

#### Clausola select

*AttrEspr* :

- può essere il carattere speciale **\***, che include **tutti gli attributi**;
- può essere una **formula**;
- se preceduta da **distinct** elimina i duplicati (altrimenti **mantenuti da SQL**, in quanto il valore di default è **all**).

#### Clausola from

*ListaTabelle* è la lista delle tabelle da unire in prodotto cartesiano, ognuna delle quali può essere ridenominata. Inoltre, nella clausola from, è possibile specificare il join di tabelle (*from S join E on S.Matr = E.Matr*)

#### Clausola where

*Condizione* :

- è un'espressione **booleana di predicati semplici**, come in algebra relazionale;
- permette l'utilizzo di alcuni predicati aggiuntivi, come:
  - **like** (confronto di stringhe) `Corsi.Titolo like 'Basi%' Matr like '12_45_'`
  - **between** (confronto di date) `Esami.Data between 1-1-2005 and 31-12-2005`

In SQL-2 vi è una logica a tre valori → il confronto con NULL restituisce **unknown** (verifichiamo con `attr is [not] null`)

### 3.2.2. Join e variabili

#### Join

In SQL è possibile **rappresentare i join esplicitandoli nella clausola from** :

```
select AttrEspr [[as] Alias] {, AttrEspr [[as] Alias]}
```

```
from Tabella [[as] Alias] { TipoJoin join AltraTabella [[as] Alias] on CondizioneJoin}
```

```
[where Condizione]
```

```
select S.Matr, Cognome, Nome
from Studenti as S join Esami as E
on S.Matr=E.Matr
where Voto=30
```

uguale a →

```
select S.Matr, Cognome, Nome
from Studenti as S, Esami as E
where S.Matr=E.Matr and Voto=30
```

dove *TipoJoin* può essere:

- inner (ovvero theta join, default);
- right [outer];
- left [outer];
- full [outer];
- natural (raramente implementato).

#### Outer join

Gli outer join permettono di **mantenere le tuple di una tabella anche se non hanno nessuna corrispondenza nell'altra** (concatenandole con valori nulli). In particolare:

- right [outer] mantiene le **tuple della tabella di destra**;
- left [outer] mantiene le **tuple della tabella di sinistra**;
- full [outer] mantiene le **tuple di entrambe le tabelle**.

#### Esempio

STUDENTI (Matr, Cognome, Nome, CdL)  
ESAMI (Matr, Cod-corso, Data, Voto)  
CORSI (Codice, Titolo, Docente)

seleziona tutti gli studenti con l'eventuale elenco degli esami (cod-corso e voto)

```
select S.*, Cod-corso, Voto
from Studenti as S leftjoin Esami as E
on S.Matr=E.Matr
```

seleziona tutti i corsi (cod e titolo) con l'eventuale elenco delle matricole degli studenti che hanno fatto l'esame

```
select Codice, Titolo, Matr
from Esami rightjoin Corsi on Cod-corso=Codice
```

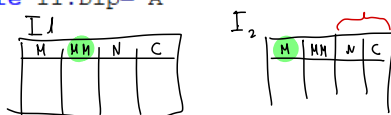
CORSI LEFT JOIN ESAMI

#### Uso di variabili

SQL permette di **specificare alias per le tabelle da interrogare**, che **possono essere utilizzate come variabili**, permettendo di dare nomi più semplici o più familiari alle tabelle, di distinguere nelle query diverse occorrenze di una stessa tabella che partecipa con più istanze, oppure di riferirsi a tabelle esterne nel caso di query nidificate.

**Esempio:** elenca gli impiegati del dip A indicando anche Cognome e Nome del loro manager (se esiste)

```
select I1.*, I2.Cognome, I2.Nome
from Impiegati as I1 leftjoin Impiegati as I2
on I1.Matr-manager=I2.Matr
where I1.Dip='A'
```



## 3.3. DML: interrogazioni complesse

### 3.3.1. Ordinamenti, aggregazioni, raggruppamenti

#### Ordinamento

In alcune interrogazioni è necessario che vengano **restituite le tuple secondo un certo ordine**. In SQL questo è possibile utilizzando la seguente sintassi:

```
order by AttrOrdinamento [asc|desc] {, AttrOrdinamento [asc|desc]}
```

che ordina le righe in ordine **ascendente** (default) o **discendente** degli attributi specificati.

#### Funzioni aggregate

Sono un'importante estensione rispetto all'algebra, in quanto **permettono di esprimere e di valutare funzioni che nell'algebra relazionale non esprimibili**. SQL offre cinque operatori aggregati:

- count (cardinalità);
- sum (sommatoria);
- max (massimo); min (minimo); avg (media).

#### Count

```
count (<*[distinct|all] ListaAttributi>)
```

restituisce il **numero di righe (\*) o di valori degli attributi in ListaAttributi**, in particolare:

- distinct considera i valori diversi  
**calcola il numero di studenti che hanno fatto almeno un esame**  
`select count(distinct Matr) from Esami`
- all considera i valori non nulli (default)

#### Sum, max, min e avg

```
<sum|max|min|avg> ([distinct|all] AttrEspr)
```

calcola il risultato dell'operazione su *AttrEspr*, in particolare:

- distinct considera valori diversi, usando una sola volta ogni valore (utile per sum e avg);
- all considera tutti i valori non nulli (default).

Esempio di **errore**: `select Matr, max(Voto)` → non va bene, la **target list deve essere omogenea**

#### Raggruppamento

Si possono **applicare operatori aggregati a sottoinsiemi di righe**, con le seguenti clausole:

- `group by ListaAttributi` (raggruppamento di attributi);
- `having Condizione` (selezione dei gruppi).

Bisogna fare particolare attenzione però, perché **nelle interrogazioni con group by**, la **target list non può contenere attributi che non siano elencati nella clausola group by**.

**Esempio**: calcola per ogni studente la media dei voti degli esami nel 2005

```
select Matr, avg(Voto) as MediaVoti from Esami where Data between 1-1-2005 and 31-12-2005 group by Matr
```

**Esempio**: calcola, per ogni studente che ha fatto almeno tre esami, la media dei voti

```
select Matr, avg(Voto) as MediaVoti from Esami group by Matr having count(*) >= 3
```

**Se è una condizione sulle tuple la metto nel where, se è una condizione "aggregata" va nel having**

### 3.3.2. Interrogazioni insiemistiche e nidificate

#### Interrogazioni insiemistiche

La sintassi:

**SelectSQL** {<union|intersect|except> [all] **SelectSQL**}

combina il risultato di due interrogazioni, in particolare:

- **union** opera un'unione;
- **intersect** opera un'intersezione;
- **except** opera una differenza.

Le interrogazioni **eliminano i duplicati di default**, a meno che non venga specificato di mantenerli con **all**.

Gli operandi devono avere un **numero di attributi uguale** e con **domini compatibili**. La corrispondenza tra gli attributi si basa **sulla posizione**, non sul nome, e usualmente il risultato prende i nomi degli attributi del primo operando.

STUDENTI(Matricola,Cognome,Nome,CdL)  
ESAMI(Matricola,Cod-corso,Data,Voto)

STUDENTI(Matricola,Cognome,Nome,CdL)  
ESAMI(Matricola,Cod-corso,Data,Voto)  
CORSI(Codice,Titolo,Docente)

- trova le matricole degli studenti iscritti a SSRionline che non hanno fatto neanche un esame

```
select Matr
from Studenti
where CdL='SSRionline'
except
select Matr
from Esami
```

- trova le matricole degli studenti iscritti a SSRionline che hanno fatto Basidati

```
select Matr
from Studenti
where CdL='SSRionline'
intersect
select Matr
from Esami join Corsi on Cod-corso=Codice
where Titolo='Basidati'
```

#### Interrogazioni nidificate

Utilizzano nella clausola **where** dei predicati che:

- **confrontano un attributo** (o un'espressione sugli attributi) **con il risultato di una query SQL**
- **usano il quantificatore esistenziale sul risultato di una query**

L'interrogazione che compare **nella clausola where** è chiamata **interrogazione nidificata**.

Il confronto di un attributo (o un'espressione sugli attributi) con il risultato di una query SQL ha la seguente sintassi:

**ValScalare** <Operator [any|all][not] in> **SelectSQL**

dove:

- **ValScalare** : valore scalare o un attributo con valore scalare
- **Operator** : operatore qualsiasi tra =, <>, <, <=, >, >=
- **any** : il predicato è vero se almeno una tupla restituita da **SelectSQL** soddisfa il confronto
- **all** : il predicato è vero se tutte le tuple restituite da **SelectSQL** soddisfano il confronto
- **in** : equivalente a = any
- **not in** : equivalente a <> all

Per utilizzare un quantificatore esistenziale sul risultato di una query, invece, si usa la seguente sintassi:

[not] exists **SelectStarSQL**

dove **SelectStarSQL** è una query nella forma **select \***.

Questo tipo di interrogazioni nidificate sono spesso utilizzate con un **passaggio di variabili** tra le interrogazioni.

**IN vs EXISTS** → **IN** --> un attributo appartiene a una **lista di cose** (select che mi ritorna una lista di cose)  
in mi valuta l'appartenenza all'insieme

**EXISTS** mi controlla se la query innestata mi contiene qualcosa o meno

STUDENTI(Matr,Cognome,Nome,CdL)  
 ESAMI(Matr,Cod-corso,Data,Voto)

- trova cognome e nome degli studenti che hanno preso almeno un 28 o un 30

```
select Cognome, Nome
from Studenti
where Matr = any (select Matr
                  from Esami
                  where Voto=28 or Voto=30)
```

= any poteva anche essere scritto come in

STUDENTI(Matr,Cognome,Nome,CdL)  
 ESAMI(Matr,Cod-corso,Data,Voto)

- trova cognome e nome degli studenti che non hanno mai preso 30

```
select Cognome, Nome
from Studenti
where Matr <= all (select Matr
                  from Esami
                  where Voto=30)
```

<> all poteva anche essere scritto come not in

STUDENTI(Matr,Cognome,Nome,CdL)  
 ESAMI(Matr,Cod-corso,Data,Voto)  
 CORSI(Codice,Titolo,Docente)

- trova cognome e nome degli studenti che hanno fatto almeno un esame con il docente Rossi

```
select Cognome, Nome
from Studenti
where Matr in
  (select Matr
   from Esami
   where Cod-corso in
     (select Codice
      from Corsi
      where Docente='Rossi'))
```

STUDENTI(Matr,Cognome,Nome,CdL)

- trova tutti gli studenti che hanno degli omonimi

```
select *
from Studenti as S1
where (Nome,Cognome) in
  (select Nome,Cognome
   from Studenti as S2
   where S1.Matr <> S2.Matr)
```

- trova tutti gli studenti che non hanno degli omonimi  
 - come sopra ma con not in

ESAMI(Matr,Cod-corso,Data,Voto)

- trova la matricola degli studenti che hanno preso il voto più alto

```
select Matr
from Esami
where Voto = (select max(Voto)
              from Esami)
```

oppure

```
select Matr
from Esami
where Voto >= all (select Voto
                  from Esami)
```

ESAMI(Matr,Cod-corso,Data,Voto)

- trova la matricola degli studenti che hanno fatto più di un esame in uno stesso giorno

```
select Matr
from Esami as E1
where exists
  (select *
   from Esami as E2
   where E1.Matr = E2.Matr and
         E1.Data = E2.Data and
         E1.Cod-corso <> E2.Cod-corso)
```

STUDENTI(Matr,Cognome,Nome,CdL)

- trova tutti gli studenti che hanno degli omonimi (soluzione alternativa)

```
select *
from Studenti as S1
where exists
  (select *
   from Studenti as S2
   where S1.Cognome = S2.Cognome and
         S1.Nome = S2.Nome and
         S1.Matr <> S2.Matr)
```

- trova tutti gli studenti che non hanno degli omonimi  
 - come sopra ma con not exists

Per le **interrogazioni nidificate** ci sono alcuni **limiti** che vanno rispettati:

- non possono contenere operatori insiemistici**, ma possono essere applicate solo **a livello esterno**;
- bisogna considerare le **regole di visibilità** delle variabili:  
 o una variabile è visibile solo all'**interno** dell'interrogazione in cui è definita e dentro le sue eventuali nidificazioni;  
 o se il nome di una variabile è omissso si assume il riferimento alla variabile più vicina.

### 3.3.3. Funzioni scalari e condizionali

#### Funzioni scalari

SQL mette a disposizione diverse funzioni scalari utilizzabili all'interno delle interrogazioni:

- funzioni previste dallo standard:
  - temporali (ad esempio `current_date`, `current_time`);
  - per la manipolazione di stringhe (ad esempio `char_length`);
  - per la conversione di domini ( `cast` );
  - condizionali;
- funzioni offerte da diverse implementazioni:
  - per la formattazione dell'output;
  - matematiche (ad esempio `sqrt` );
  - per l'accesso a servizi del SO.

#### Funzioni condizionali

Non estendono il potere espressivo del linguaggio, ma permettono di realizzare comandi in modo più compatto. Le funzioni sono tre:

- `coalesce` : ammette come argomento una sequenza di espressioni e restituisce il primo valore non nullo;
- `nullif` : richiede come argomento un'espressione e un valore costante, e restituisce:
  - `null` se l'espressione valuta pari al valore costante;
  - il valore dell'espressione, altrimenti;
- `case` : permette di specificare espressioni condizionali che generano i valori sulla base di generici predicati SQL, permettendo di scrivere con una sola istruzione un'operazione che considera valori diversi. La sua sintassi è la seguente:

```
case when Condizione then Espressione      {when Condizione then Espressione}  
else Espressione end
```

#### ESAMI(Matr,Cod-corso,Data,Voto)

- trova gli esami convertendo i voti in fasce di punteggio (A,B,C)

```
select Matr,Cod-corso,  
       case  
         when Voto < 24 then 'C'  
         when Voto >= 24 and Voto < 27 then 'B'  
         else 'A'  
       end as Fascia  
from Esami
```

## 3.4. DML per le modifiche: inserimenti, cancellazioni e aggiornamenti

### Comandi di modifica

I comandi di modifica operano su un insieme di tuple e prevedono istruzioni per:

- l'inserimento ( `insert` );
- la cancellazione ( `delete` );
- la modifica dei valori degli attributi ( `update` ).

#### Inserimento

Il comando per l'inserimento ha la seguente sintassi:

```
insert into NomeTabella [(ListaAttributi)] <values (ListaValori)| SelectSQL>
```

dove:

- *ListaAttributi* specifica la lista degli attributi da inserire (quelli non specificati vengono impostati a null);
- *ListaValori* specifica i valori a cui impostare gli attributi;
- *SelectSQL* specifica la query che restituisce i valori da inserire. La

corrispondenza tra gli attributi e i valori da inserire è posizionale.

#### Cancellazione

Il comando per la cancellazione ha la seguente sintassi:

```
delete from NomeTabella [where Condizione]
```

dove:

- *Condizione* si usa in modo analogo alle interrogazioni;
- in caso venga **omessa la clausola *where***, vengono **cancellate tutte le tuple della tabella** (rimane così la tabella vuota, quindi questa **operazione è differente dalla *drop***);
- se è presente un vincolo di integrità referenziale con politica ***cascade***, può provocare cancellazioni in altre tabelle.

#### Aggiornamento

Il comando per l'aggiornamento ha la seguente sintassi:

```
update NomeTabella set
```

```
Attributo = <Espressione| SelectSQL|null|default>
```

```
{, Attributo = <Espressione| SelectSQL|null|default>}
```

```
[where Condizione]
```

dove:

- *Condizione* si usa in modo analogo alle interrogazioni;
- in caso venga **omessa la clausola *where***, vengono **modificate tutte le tuple della tabella**.

- aumenta di un punto (se possibile) il voto di tutti gli studenti che hanno fatto più di tre esami

```
update Esami
set Voto = Voto+1
where Voto < 30 and Matr in (select Matr
                             from Esami
                             group by Matr
                             having count(*)>3)
```

aumenta (se possibile) di un punto i voti di Basidati (codice 'BD'), diminuisci (se possibile) di un punto i voti di Sistemi (codice 'SO')

```
update Esami
set Voto =
case
when (Cod-corso='BD' and Voto<30) then
  Voto+1
when (Cod-corso='SO' and Voto>18) then
  Voto-1
else Voto
end
```

## 3.5. Aspetti evoluti del DDL

### 3.5.1. Vincoli di integrità generici

I vincoli di integrità generici possono essere associati:

- a una singola tabella o attributo, con la clausola: `check (Condizione)` ;
- all'intera base di dati, con la sintassi: `create assertion NomeAsserzione check (Condizione)`

```
create table Corsi
(   Codice   char(6)           primary key,
    check (Codice like 'F%'),
    Titolo   varchar(30) not null unique,
    Cod-docente varchar(20)
    check (3 > (select count(*)
                from Corsi as C
                where Cod-docente = C.Cod-docente))
)
```

deve esserci almeno un esame registrato per ogni studente

Definisce CORSI(Codice,Titolo,Cod-docente) con vincoli:

- il codice dei corsi deve cominciare per F
- ogni docente deve avere non più di due corsi

```
create assertion almeno-un-esame
check (0 = select count(*)
        from Studenti
        where Matr not in (select Matr
                           from Esami))
```

#### Controllo dei vincoli

I vincoli definiti tramite clausola `check` o `assertion` possono essere controllati in modo:

- **immediato** ( `immediate` ), ovvero a seguito di ogni singola operazione (la loro violazione annulla l'operazione, con un comportamento detto rollback parziale);
- **differito** ( `deferred` ), ovvero al termine delle transazioni (la loro violazione annulla l'intera transazione, con un comportamento detto rollback).

Tutti i **vincoli predefiniti** ( `not null` , `unique` , `primary key` , `foreign key` ) sono **controllati in modo immediato**, mentre è possibile cambiare il tipo di controllo associato ai vincoli definiti tramite `check` o `assertion` (che di default è immediato), nel seguente modo:

```
set constraint NomeVincolo [immediate|deferred]
```

### 3.5.2. Viste

Le viste sono **tabelle virtuali** definite come **interrogazioni su altre tabelle** (di base o virtuali). Possono essere specificate con la seguente sintassi:

```
create view NomeVista [(ListaAttributi)] as SelectSQL [with [local|cascaded] check option]
```

dove:

- `check option` specifica che le **righe della vista possono essere aggiornate** e che **dopo gli aggiornamenti le righe devono continuare ad appartenere alla vista**, ovvero che le **modifiche non violino la condizione di selezione che permette alle tuple di appartenere alla vista** (altrimenti la modifica non è permessa);
- nel caso in cui la vista sia definita in termini di altre viste:
  - l'opzione `local` indica che il controllo specificato con la `check option` va **eseguito solo sulla vista**;
  - l'opzione `cascaded` (di default) indica che il controllo deve essere **propagato anche sulle viste su cui è definita**.

Infatti, è possibile definire viste in termini di altre viste, con l'unico vincolo di **non mutua dipendenza** tra le viste.

```
create view Esami05 as
select *
from Esami
where Data between 1-1-2005 and 31-12-2005
```

### **Viste su Viste**

Vista: ESAMI05(Matricola,Cod-corso)

```
create view EsamiSSRI as
select Esami.Matricola,Cod-corso
from Studenti join Esami
on Studenti.Matricola=Esami.Matricola
where CodL='SSRI'
```

```
create view StudentiAttivi(Matricola) as
select Matricola
from Esami05
```

### **Viste e interrogazioni**

Le viste sono particolarmente utili per formulare **interrogazioni che combinano e nidificano diversi operatori aggregati** (che altrimenti non possono essere eseguiti in cascata) e/o che fanno un uso sofisticato dell'operatore di **unione**.

- trova il CdL con il maggior numero di studenti

```
select CdL
from Studenti
group by CdL
having count(*) >= all (select count(*)
                        from Studenti
                        group by CdL)
```

non è riconosciuta da tutti gli interpreti SQL

- non tutti gli interpreti permettono interrogazione nidificate nella clausola having

- trova il CdL con il maggior numero di studenti

```
create view NumStudenti(CdL,numero) as
select CdL,count(*)
from Studenti
group by CdL
```

Num Sto

| CdL  | numero |
|------|--------|
| SSRI | 30     |
| BIO  | 50     |

```
select CdL
from NumStudenti
where numero = (select max(numero)
                from NumStudenti)
```

- trova il numero medio di esami per ogni studente

#### **Query scorretta!**

```
select avg(count(*))
from Esami
group by Matricola
```

non si possono combinare in cascata operatori aggregati

- trova il numero medio di esami per ogni studente

```
create view NumEsami(Matricola,numero) as
select Matricola,count(*)
from Esami
group by Matricola
```

```
select avg(numero)
from NumEsami
```

### **Viste e aggiornamenti**

Non tutte le viste sono modificabili, perché ***viste definite su più tabelle possono avere ambiguità negli aggiornamenti***. Infatti, la maggior parte dei sistemi commerciali ammette **modifiche solo a viste definite su una singola tabella**, mentre ***altri sistemi*** richiedono anche che le **viste** contengano anche la **chiave primaria** della tabella su cui sono definite.

Aggiornamenti su una vista si propagano alla tabella su cui la vista è definita

#### **Esempio**

ESAMI(Matricola,Cod-corso,Data,Voto)

```
create view Esami05 as
select *
from Esami
where Data between 1-1-2005 and 31-12-2005
```

```
update Esami05
set Voto = Voto + 1
where Voto < 25
```

si propaga sulla tabella Esami

```
create view ListaDocCorsi(Docente,Titolo) as
select Docente, Titolo
from Corsi join Assegn on Codice=Codice-corso
```

```
update ListaDocCorsi
set Titolo = 'Basidati'
where Docente = 'Rossi'
```

#### **Modifica ambigua!**

- il corso ha cambiato titolo?
- Rossi ha cambiato corso?

### 3.5.3. Autorizzazioni e controllo dell'accesso

#### Autorizzazioni in SQL

SQL supporta una politica di tipo **discrezionale**, in cui ogni elemento dello schema può essere protetto, in particolare:

- politica **discrezionale chiusa**: **solo le azioni esplicitamente permesse sono autorizzate**;
- con **ownership**: il **creatore di un oggetto** ne è il **proprietario** e riceve sull'oggetto **tutti i privilegi** (per quanto riguarda le viste, i permessi di un soggetto su un oggetto sono ristretti a quelli che aveva sulle tabelle su cui la vista si basa).

Ogni autorizzazione è caratterizzata da:

- l'**oggetto** al quale l'autorizzazione è riferita;
- l'**azione** alla quale l'autorizzazione è riferita;
- il **soggetto** al quale viene concesso il privilegio, che può essere un utente, un ruolo o una procedura;
- l'**utente** che ha concesso il privilegio;
- il **flag** che indica se vale o meno la `grant option`.

Esiste inoltre un utente predefinito `_system`, che rappresenta **l'amministratore di sistema** e ha **pieno accesso a tutte le risorse**.

#### Azioni in SQL

Le azioni che possono comparire nelle autorizzazioni su tabelle sono:

- `select` (tabella, attributi);
- `insert` (tabella, attributi);
- `update` (tabella, attributi);
- `delete` (tabella);
- `trigger` (tabella);
- `references` (tabella, attributi);
- `all privileges` (li riassume tutti);

Esistono inoltre altri **privilegi per i domini** (ad es `usage`) e **per le procedure** (ad es `execute`). In particolare, il privilegio di `references`, permette che **venga fatto riferimento alla risorsa nell'ambito della definizione dello schema di una tabella**, il che è utile perché le **politiche di reazione dei vincoli di integrità referenziale possono avere effetti sulle tabelle esterne**.

#### Grant

Il proprietario di un oggetto può concedere ad altri autorizzazioni per privilegi sull'oggetto:

```
grant <ListaPrivilegi>|all privileges  
on Risorsa
```

```
to ListaRiceventi [with grant option]
```

dove la `grant option` permette agli **utenti riceventi** i **privilegi di garantire a loro volta i privilegi su quella risorsa ad altri utenti** (con o senza `grant option`).

#### Revoke

L'utente che ha concesso un'autorizzazione può **toglierla** (`revoke`). Se l'**utente** a cui il **privilegio è revocato** aveva a sua volta **concesso il privilegio ad altri** e non ha altra autorizzazione per il privilegio con la `grant option`, le **autorizzazioni da lui concesse rimarrebbero pendenti**. Perciò esistono due opzioni di revoca:

- `restrict`: **impedisce la revoca** se ci sono **autorizzazioni** che rimarrebbero **pendenti**;
- `cascade`: **propaga la revoca** (cancellando ricorsivamente le **autorizzazioni pendenti**).

La sintassi del comando di revoca è la seguente:

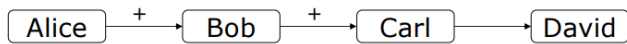
```
revoke <ListaPrivilegi>|all privileges  
on Risorsa
```

from *ListaRiceventi* [restrict|cascade]

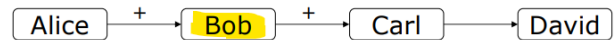
Alice proprietario di STUDENTI

- Alice: `grant select on Studenti to Bob with grant option`
- Bob: `grant select on Studenti to Carl with grant option`
- Carl: `grant select on Studenti to David`

Grafo di autorizzazioni:



### Esempi



- Alice: `revoke select on Studenti from Bob restrict`  
**non viene eseguita**
- Alice: `revoke select on Studenti from Bob cascade`  
cancella anche le autorizzazioni di Carl e David

### Autorizzazioni su viste

SQL non permette di restringere le autorizzazioni sulla base di condizioni sul valore dei dati, ma è possibile **ottenere autorizzazioni ristrette sui dati utilizzando le viste:**

- si **definisce la vista** che specifica le **condizioni**;
- si **concedono i privilegi sulla vista**.

L'esercizio del **privilegio sulla vista non implica** il **privilegio sulle tabelle su cui si basa**.

STUDENTI(Matr,Cognome,Nome,CdL)

```
create view Studentissri as
select *
from Studenti
where CdL = 'SSRI'
```

```
grant select on Studentissri to Ellen
```

### Ruoli

Un ruolo rappresenta un **insieme di privilegi**: attivando un ruolo un utente è abilitato ad esercitare i privilegi associati ad esso. Il ruolo è **diverso** dal concetto di gruppo, che invece è un **insieme di utenti** con un **insieme di privilegi esercitabili da ogni membro del gruppo** (**ruolo è dinamico** (insieme di privilegi che prendo e lascio quando mi serve) invece il **gruppo è statico**)

### Grant e revoke di ruoli

La gestione dei ruoli è simile alla gestione delle risorse; perciò, il creatore di un ruolo può concedere ad altri il ruolo (abilitandoli così ad attivare il ruolo) utilizzando eventualmente la `admin option` (equivalente alla `grant option` delle autorizzazioni). La sintassi per la concessione di ruoli è la seguente:

```
grant ListaRuoli
to ListaRiceventi [with admin option]
```

mentre la sintassi per la revoca dei ruoli (con eventuale propagazione ricorsiva, come nel caso delle autorizzazioni) è la seguente:

```
revoke [admin option] ListaRuoli
from ListaRiceventi [restrict|cascade]
```

## 3.6. Esercizi di SQL

### 3.6.1. Parte I

#### Relazioni

CELLULARE (Codice, CFUtente, Modello, **Marca**, Colore)

ABBONAMENTO (Numero, CFUtente, **Operatore**, Tariffa)

UTENTE (CF, Nome, Cognome, Città)

#### Esercizi

- Interrogazione 1:

- Richiesta: determinare la tariffa minima applicata dall'operatore Vodafone;

- Soluzione:

- ```
select min(Tariffa)
from Abbonamento
where Operatore = 'Vodafone'
```

- Interrogazione 2:

- Richiesta: determinare nome e cognome degli utenti che possiedono un cellulare Nokia ed hanno un abbonamento Tim;

- Soluzioni equivalenti:

- ```
select Nome, Cognome
from Utente, Abbonamento as A, Cellulare as C
where CF = A.CFUtente and CF = C.CFUtente
and Operatore = 'Tim' and Marca = 'Nokia'
```

OK

- ```
select Nome, Cognome
from Utente
join Abbonamento as A on CF = A.CFUtente
join Cellulare as C on CF = C.CFUtente
where Operatore = 'Tim' and Marca = 'Nokia'
```
- ```
select Nome, Cognome from Utente where CF in
(Select CFUtente from Abbonamento where
Operatore = 'Tim') and CF in (Select
CFUtente from Cellulare where Marca='Nokia')
```

Sono soluzioni equivalenti io ho pensato alla terza ma il join è più pulito

- Interrogazione 3:

- Richiesta: determinare codice fiscale, cognome e nome degli utenti che possiedono più di tre numeri telefonici;

- Soluzione:

- ```
select CF, Cognome, Nome
from Utente
join Abbonamento on CF = CFUtente
group by CF, Cognome, Nome
having count(Numero) > 3
```

In questo caso facciamo join tra ut e abb usando la clausola having perché abbiamo una clausola su una funzione aggregata. Occhio a **non dimenticare il group by** perché voglio vedere **per ogni utente** e selezionare solo chi ha + di 3 numeri – in **group by** raggruppo anche per *Cognome* e *Nome* perché **dopo vado a farci una selezione !!!** “in select posso mettere solo attributi che rientrano nel group by” à lo riconosce lo stesso però meglio farlo

- Interrogazione 4:

- Richiesta: determinare, per ciascun operatore, il numero di abbonamenti e di persone servite;
- Soluzione:
  - `select Operatore, count(Numero), count(distinct CFUtente)`  
from Abbonamento  
group by Operatore

Qui dobbiamo **per forza fare un group by** e: facciamo un **count su Numero** (andava bene anche CFUtente) per trovare il **numero di abbonamenti** (ogni utente può avere anche più abbonamenti) mentre un **count su distinct CFUtente** per trovare il **numero di utenti** (distinct elimina duplicati). Sempre **per ogni operatore** perché prima ho fatto il **group by**

- Interrogazione 5:

- Richiesta: determinare gli operatori che hanno l'esclusiva per almeno una città;
- Soluzioni equivalenti:
  - `select distinct A1.Operatore`  
from Abbonamento as A1  
join Utente as U1 on CFUtente = CF  
where not exists (  
    **select \***  
    from Abbonamento as A2  
    join Utente as U2 on CFUtente = CF  
    where U1.Città = U2.Città and A1.Operatore <> A2.Operatore  
)
  - `select distinct A1.Operatore`  
from Abbonamento as A1  
join Utente as U1 on CFUtente = CF  
where U1.Città not in (  
    select U2.Città  
    from Abbonamento as A2  
    join Utente as U2 on CFUtente = CF  
    where A1.Operatore <> A2.Operatore  
)
  - `select distinct Operatore`  
from Abbonamento as A1 join Utente as U1 on CFUtente = CF  
where Città in  
    (select Città  
    from Abbonamento as A2 join Utente as U2 on CFUtente = CF  
    group by Città having count(distinct Operatore)=1)

La prima soluzione fa una cosa simile al mirror che facevamo nell'algebra relaz : Prende tutti gli operatori (distinct) dal join che non esistono in: **cerca tuple nel join abb,ut che abbiano la stessa città ma abbonamento diverso**. In sostanza vede se ci sono operatori diversi nella stessa città → (select \* trova qualcosa) allora con il not exist prima non li prendo.

La seconda soluzione è simile alla prima ma verifico che la città non sia in una di quelle nelle quali ci sono operatori diversi, dopodiché vado a selezionare di queste l'operatore

La terza soluzione è quella che ho pensato ed è una query nidificata. Vado innanzitutto a trovare tutte le città in cui è presente solo un operatore (group by città having count(distinct operatore)=1) e poi seleziono gli operatori che hanno la città rientrante nella query innestata di prima. (occhio con le query innestate a mettere l'attributo giusto nella cl IN)

Nella terza è sbagliato mettere where A1.operatore in (select A2.operatore ...

Attenzione anche a non fare il group by di operatore al posto di città e poi selezionare gli operatori (nella query nidificata) perché andrei a trovare gli operatori che servono una città sola

### 3.6.2. Parte II

#### Relazioni

RADIO (Codice, Nome, Frequenza, Luogo)

PROGRAMMA (Codice, Nome, Conduttore, FasciaOraria, Durata, Tipo, CodiceRadio)

#### Esercizi

- Interrogazione 1:

- Richiesta: determinare il codice delle radio per cui il numero totale di ore di programmi musicali è minore o uguale a 8 (si noti che le radio ritornate devono avere almeno un programma musicale);

- Soluzione:

- ```
select CodiceRadio
from Programma
where Tipo = 'musicale'
group by CodiceRadio
having sum(Durata) <= 8
```

Posso tranquillamente lavorare solo su una tabella visto che gli attributi che mi interessano sono tutti lì. Vado a selezionare il codice radio dei programmi musicali che hanno meno di 8 ore)

- Interrogazione 2:

- Richiesta: determinare il nome e la frequenza delle radio che trasmettono in un luogo dove non trasmette alcuna altra radio;

- Soluzioni equivalenti:

- ```
select R1.Nome, R1.frequenza
from Radio as R1
where R1.Luogo not in (
    select R2.Luogo
    from Radio as R2
    where R1.Codice <> R2.Codice
)
■ select R1.Nome, R1.frequenza
from Radio as R1
where not exists (
    select *
    from Radio as R2
    where R1.Luogo = R2.Luogo and R1.Codice <> R2.Codice
)
```

Nella prima soluzione vado a vedere quei luoghi che non appartengono alla selezione di quei luoghi che hanno più di una radio (codice<>codice). Poi come risultato selezione nome e frequenza

Nella seconda soluzione faccio circa la cosa di prima ma uso not exist. Selezionando nella query innestata tutto quello che ha lo stesso luogo ma codice diverso. Quindi escludo i luoghi che hanno più radio.

Si poteva anche fare una query annidata con group by su luogo e having count(codice)=1

- Interrogazione 3:

- Richiesta: determinare il nome del conduttore che conduce esclusivamente programmi di tipo musicale;
- Soluzioni equivalenti:
  - ```
select distinct Conduttore
from Programma
where Tipo = 'musicale'
except
select Conduttore
from Programma
where Tipo <> 'musicale'
```
  - ```
select distinct Conduttore
from Programma
where Tipo = 'musicale'
and Conduttore not in (
    select Conduttore
    from Programma
    where Tipo <> 'musicale'
)
```
  - ```
select distinct Conduttore
from Programma as P1
where P1.Tipo = 'musicale'
and not exists (
    select *
    from Programma as P2
    where P1.Coduttore = P2.Coduttore and Tipo <> 'musicale'
)
```

Nella prima soluzione vado a vedere i conduttori che hanno il tipo musicale (non basta) e escludo da questi quelli che hanno anche programmi non di tipo musicale à in questo modo distinguo bene da tutti e almeno uno, quest'ultimo è diverso dalla consegna

Nella seconda soluzione facciamo la stessa cosa della prima ma usiamo not in e and (confronto, al posto dell'except che fa la differenza)

Possiamo anche usare il count ma evitare l'errore che ho fatto io: devo contare in un group by i conduttori che hanno solo programmi di un tipo (count = 1) e poi **DOPO selezionare** di questi conduttori chi ha **solo programmi musicali** (in questo caso è necessario fare una **query nidificata**, non posso fare tutto in una unica à SQL legge prima il where dell'having)

- Interrogazione 4:

- Richiesta: determinare il nome delle radio che trasmettono il maggior numero di programmi;
- Soluzione:
  - ```
create view NumProgrammi (NomeRadio, NumProg) as
select R.Nome, count(*)
from Radio as R
join Programma as P on R.Codice = P.CodiceRadio
group by P.CodiceRadio, R.Nome
```
  - ```
select NomeRadio
from NumProgrammi
where NumProg = (
    select max(NumProg)
    from NumProgrammi
)
```

Si poteva anche non creare la view e fare due query annidate

### Esercizio M3\_E1 scheda sql

Date le relazioni R(A,C) e S(A,C) contenenti le stesse 2 tuple, scrivere in corrispondenza di ogni query la relazione ritornata dalla query. (Indicare sia il nome delle colonne sia le tuple contenute nella relazione risultato).

SELECT COUNT(\*) AS B FROM R GROUP BY A

*B 1 1 → group by A gruppetti costituiti da una tupla sola (ho il group by)*

SELECT COUNT(DISTINCT A) AS B FROM R

*B 2 → qui non ho il group by e il distinct sulla chiave non fa nulla quindi conta semplicemente le tuple*

SELECT COUNT(\*) AS B FROM R UNION SELECT COUNT(\*) AS B FROM S

*B 2 → la prima query mi da B 2 la seconda mi da B 2, l'unione elimina i duplicati quindi ne tengo una*

SELECT COUNT(\*) AS B FROM R UNION ALL SELECT COUNT(\*) AS B FROM S

*B 2 2 → come prima faccio l'unione tra le due cose ma siccome ho ALL mantengo i duplicati*

SELECT COUNT(\*) AS B FROM R JOIN S ON R.A=S.A

*B 2 → se faccio il join su tabelle uguali mi rimane la stessa tabella. Poi se conto è 2*

### Esercizio M3\_E2 scheda sql

GELATO (Id, Gusto, Calorie)

RICETTA (IdGelato, IdIngrediente, Quantita)

INGREDIENTE (Id, Nome, Tipo, Biologico, PaeseProv, PrezzoKg)

Determinare il paese produttore dell'ingrediente utilizzato nel maggior numero di gelati (o ingredienti nel caso siano più di uno con lo stesso numero maggiore di gelati)

~~Select PaeseProv~~

~~From ricetta join ingrediente on idIngrediente=id~~

~~Group by idingrediente~~

~~Having max(count(idgelato))~~

**Non posso mettere having max e max count (funzioni aggregate)**

**Nella select devo avere solo attributi presenti nella group by (quindi anche solo un sottinsieme degli attributi presenti nel group by)**

```
CREATE VIEW NumGelati AS
SELECT IdIngrediente, COUNT(IdGelato) AS Num
FROM Ricetta
GROUP BY IdIngrediente

SELECT DISTINCT PaeseProv
FROM NumGelati JOIN Ingrediente ON IdIngrediente=Id
WHERE Num >= ALL (SELECT Num FROM NumGelati)
```

```
SELECT PaeseProv
FROM Ricetta JOIN Ingrediente ON IdIngrediente=Id
GROUP BY IdIngrediente, PaeseProv
HAVING COUNT(*) >= ALL (
    SELECT COUNT(*)
    FROM Ricetta
    GROUP BY IdIngrediente)
```

Nella prima soluzione possiamo anche evitare di creare una vista ma ci semplifica la vita (sol prof)

La seconda soluzione si avvicina di più a quella sbagliata che avevo scritto prima: conto tutto raggruppando per IdIngrediente e poi usando having >= ALL per trovare quello che ha il maggior numero di ingredienti

### Esercizio M3\_E3 scheda sql

GELATO (Id, Gusto, Calorie)

RICETTA(IdGelato, IdIngrediente, Quantita)

INGREDIENTE(Id, Nome, Tipo, Biologico, PaeseProv, PrezzoKg)

Determinare i gelati per cui tutti gli ingredienti vengono dallo stesso paese di provenienza

SELECT IdGelato

FROM Ricetta JOIN Ingrediente ON IdIngrediente = Id

GROUP BY IdGelato

HAVING COUNT(DISTINCT PaeseProv) = 1

Soluzione della prof con il not exist lunghissima non proponibile

### Esercizio M3\_E4 scheda sql

OCCHIALE(IdOcchiale, IdMontatura, IdLenteSx, IdLenteDx, Sconto, IdCliente)

MONTATURA(IdMont, Nome, Marca, Colore, Materiale, Prezzo)

LENTE(IdLente, Forma, Correzione, DaSole, Prezzo)

Determinare, per ciascun cliente, il numero di occhiali acquistati e la percentuale media di sconto ricevuta.

Select idcliente, count(idocchiale), avg(sconto)

from occhiale

Group by idcliente

### Esercizio M3\_E5 scheda sql

CLIENTE(CF, Cognome, Nome, Nazione, Email)

PRODOTTO(Codice, Nome, Descrizione, Prezzo)

ORDINE(Id, CodProdotto, CFCliente, Data, MetPag, TipoConsegna, DataSpedizione, DataRicezione)

Determinare il codice fiscale dei clienti che non hanno mai ri-acquistato lo stesso prodotto

**Soluzione 0 (mia)** → **SBAGLIATA** → sto trovando i clienti che hanno fatto almeno un doppio ordine

Select CFCliente

From ordine

Group by CFCliente, CodProdotto

Having count(CodProdotto) = 1

```
SELECT CFCliente
FROM ORDINE
WHERE CFCliente NOT IN (
    SELECT CFCliente
    FROM ORDINE
    GROUP BY CFCliente, CodProdotto
    HAVING COUNT(*) > 1
)
```

**Questa ha più senso. Ricorda gli insiemi**

### Soluzione 1

Select distinct cfcliente

From ordine *//interrogiamo ordine perché se prendo CF di cliente prendo anche quelli che non hanno fatto ordini*

Where cfcliente not in(

    Select d.cfcliente

    From ordine as o join ordine as o1

    On o.cfcliente=o1.cfcliente

    And o.codprodotto = o1.codprodotto

    And o.id <> o1.id)

*Vado a joinare le tuple che si riferiscono allo stesso cliente e prodotto ma con acquisto diverso. Poi prendo tutti quelli che non rientrano in questo insieme → tutti quei clienti che non hanno mai fatto più ordini per lo stesso prodotto*

### Soluzione 2

Select distinct cfcliente

From ordine

Where cfcliente not in(

    select cfcliente *//questa parte dentro va a cercare quelli che hanno comprato un prodotto almeno 2 volte*

    From ordine as o

    Where exist( *//dove il risultato della query qua dentro ha effettivamente qualcosa*

        Select \*

        From ordine as o1

        Where o.cfcliente =o1.cfcliente

        And o.codprodotto = o1.codprodotto

        And o.id <> o1.id

    )

)

### Soluzione 3

Select distinct cfcliente

From ordine as o

Where 1=all (select count(\*) *//controllo che la query qui dentro mi restituisca 1*

    From ordine as o1 *//quindi prendo tutti quelli che mi danno 1*

    Where o1.cfcliente = o.cfcliente *//ovvero quelli che non hanno più ordini per lo stesso prodotto*

    Group by o1.codprodotto)

#### Soluzione 4 (img sopra)

Select distinct cfcliente

From ordine

Where cf cliente not in( *//nella sottoquery trovo tutti i clienti che hanno fatto più ordini dello stesso prodotto*

Select cfcliente *//poi li tolgo con la clausola not in*

From ordine

Group by cfcliente, codprodotto

Having count(\*) > 1

) *//se avessi messo in e poi having count(\*)=1 sarebbe sbagliato perché prendo **ALMENO UNO***

#### Soluzione 5 → SBAGLIATA

Select distinct cfcliente *//tutti i clienti per cui esiste almeno un prodotto che loro hanno comperato una volta sola*

From ordine as o

Where codprodotto not in(

Select o1.codprodotto

From ordine as o1

Where o.cfcliente = o1.cfcliente

And o.id <> o1.id)

#### Esercizio ROBOT

Determinare il CF dei clienti che hanno acquistato tutti i robot della serie AX500

**Query robot**

Si considerino i seguenti schemi relazionali

ROBOT(Codice, Modello, Serie, Colore, Funzione, Prezzo)  
CLIENTE(CF, Cognome, Nome, Tel, DataNascita, Città)  
ACQUISTO(CFCliente, CodRobot, DataAcq, ModPagamento)

Scrivere in SQL la seguente interrogazione:

sql) Determinare il CF dei clienti che hanno acquistato tutti i robot della serie AX500.

*clienti che hanno acquistato almeno un robot AX500*

```
SELECT DISTINCT A.CFCliente
FROM Acquisto A JOIN Robot R ON A.CodRobot=R.Codice
WHERE R.Serie="AX500"
AND NOT EXISTS (SELECT
FROM Robot R1
WHERE R1.Serie="AX500"
AND NOT EXISTS (SELECT *
FROM Acquisto A1
WHERE A1.CodRobot=R1.Codice
AND A1.CFCliente=A.CFCliente))
```

*(A1, R1)*

ROBOT(Codice, Modello, Serie, Colore, Funzione, Prezzo)  
CLIENTE(CF, Cognome, Nome, Tel, DataNascita, Città)  
ACQUISTO(CFCliente, CodRobot, DataAcq, ModPagamento)

Scrivere in SQL la seguente interrogazione:

sql) Determinare il CF dei clienti che hanno acquistato tutti i robot della serie AX500.

```
SELECT CFCliente
FROM Acquisto JOIN Robot ON CodRobot=Codice
WHERE R.Serie="AX500"
GROUP BY CFCliente
HAVING COUNT(DISTINCT CodRobot) = (SELECT COUNT(*)
FROM Robot
WHERE R.Serie="AX500")
```

## 4. Progettazione di basi di dati

### 4.1. Metodologie di progettazione

#### 4.1.1. Ciclo di vita dei sistemi informativi

La base di dati è una **componente del sistema informativo** e la sua **progettazione** ed utilizzo si inseriscono nel ciclo di vita del sistema informativo, riassumibile in:

- **studio di fattibilità**
- **raccolta e analisi dei requisiti**
- **progettazione**
- **implementazione**
- **validazione e collaudo**
- **funzionamento**

In realtà questo processo **non è quasi mai sequenziale**, infatti spesso è necessario, durante una fase, rivedere le decisioni prese durante la fase precedente, mentre a volte si include una fase di prototipizzazione (ovvero una realizzazione rapida per la sperimentazione delle funzionalità) che può portare a revisioni dei requisiti e modifica del progetto.

#### **Studio di fattibilità**

In questa fase si definisce, in maniera per quanto possibile precisa, i **costi delle varie alternative possibili** e si stabiliscono le **priorità di realizzazione** delle varie componenti del sistema. In questa fase, i costi da considerare sono:

- **costi diretti (budget)**
- **costi indiretti:**
  - impegno del **personale**
  - **inefficienze temporanee** dovute al cambio di sistema e di modalità di lavoro

#### **Raccolta e analisi dei requisiti**

Questa fase si occupa di **individuare e studiare le proprietà e le funzionalità** che il sistema deve avere:

- necessità di **interazione con gli utenti** del sistema;
- produce una **descrizione completa** (ma informale) di:
  - **dati coinvolti**;
  - **operazioni sui dati**;
- stabilisce i **requisiti hardware e software** che il sistema dovrà avere.

#### **Progettazione**

È la fase in cui vengono **formalizzati i requisiti** *espressi informalmente nella fase precedente* per mezzo di **modelli specifici**:

- **progettazione dei dati**: individua la struttura e l'organizzazione che i dati dovranno avere
- **progettazione delle applicazioni**: definisce le caratteristiche delle operazioni sui dati e progetta il software applicativo che le implementa

#### **Implementazione**

Realizza il **sistema informativo** secondo la struttura e le caratteristiche definite nella fase precedente

- si definiscono le componenti hardware e software di base e di sviluppo da acquistare sul mercato
- viene definita e popolata la base di dati
- si sviluppano i programmi applicativi

## Validazione e collaudo

Verifica il corretto funzionamento e la qualità del sistema realizzato, ovvero:

- la correttezza dei dati;
- il corretto funzionamento delle applicazioni;
- i tempi di risposta nelle varie condizioni operative;
- la sicurezza dei dati a fronte di violazioni in termini di integrità e confidenzialità;
- la tolleranza del sistema ai guasti.

## Funzionamento

Più che una fase, è la “vita” del sistema una volta che è stato reso operativo ed esegue i compiti per i quali è stato realizzato. Se non ci sono malfunzionamenti o revisioni sui requisiti, richiede solo operazioni di gestione e manutenzione:

- revisione periodica delle funzionalità;
- gestione di contratti per:
  - manutenzione e aggiornamento dell’hardware;
  - manutenzione correttiva ed evolutiva del software.

### 4.1.2. Raccolta e analisi dei requisiti

La raccolta dei requisiti è la completa individuazione di:

- problemi che l’applicazione dovrà risolvere;
- caratteristiche che l’applicazione dovrà avere, che possono essere:
  - statiche (dati)
  - dinamiche (operazioni sui dati)

I requisiti vengono raccolti in specifiche generalmente espresse in linguaggio naturale, quindi spesso ambigue e disorganizzate. Perciò la fase di analisi consiste nel **valutare le specifiche, chiarendole ed organizzandole eliminando le ambiguità**.

## Raccolta dei requisiti

I requisiti possono provenire da **fonti diverse**:

- utenti dell’applicazione, che possono fornire le informazioni sui dati e le operazioni necessarie, per mezzo di interviste o documentazione che hanno preparato;
- documentazione esistente, che ha attinenza con l’applicazione (ad esempio moduli, regolamenti interne normative);
- realizzazioni preesistenti, ovvero applicazioni che devono essere sostituite da quella da sviluppare o che interagiscono con essa.

Nella raccolta dei requisiti, l’interazione che gli utenti del sistema informativo gioca un ruolo molto importante. Spesso però utenti differenti danno informazioni contrastanti (in genere utenti di alto livello hanno una migliore visione di insieme, ma meno precisa, mentre utenti di basso livello hanno una visione parziale del sistema, ma molto più dettagliata), perciò si rende necessario:

- fare verifiche di consistenza e comprensione delle specifiche che si stanno raccogliendo;
- individuare gli aspetti essenziali rispetto a quelli marginali e procedere per raffinamenti successivi.

## Analisi dei requisiti

Le specifiche dei requisiti sono generalmente espresse in linguaggio naturale e possono quindi essere inesatte o ambigue. Nella fase di analisi le specifiche dei requisiti vengono controllate ed opportunamente corrette in modo tale da ottenere una **maggior precisione e una minore ambiguità**. Si utilizzano a tale scopo delle **regole** generali:

- scelta del corretto livello di astrazione, evitando **termini troppo specifici o troppo generici**;
- standardizzazione della struttura delle fasi, utilizzando uno **stile sintattico omogeneo**;

- rimozione delle ambiguità, evitando **frasi contorte** e fornendo **definizioni semplici e chiare**;
- individuazione e correzione di sinonimi e omonimi, sostituendo i **sinonimi con uno stesso termine** e utilizzando **termini diversi per gli omonimi**;
- esplicitazione del riferimento tra termini, eliminando **ambiguità** in assenza di contesto;
- costruzione di un glossario dei termini, elencando **per ogni termine una breve descrizione del significato**, unitamente a possibili sinonimi e legami con altri termini.

### Requisiti sulle operazioni

Le fasi di **raccolta e analisi dei requisiti** devono riguardare anche le **operazioni** che dovranno essere effettuate sui dati e la loro frequenza di utilizzo, il che avrà un impatto sull'organizzazione di dati e operazioni in termini di **efficienza**. La fase di **progettazione logica**, che realizza un DBMS a partire dal progetto della base di dati, prende in particolare considerazione quale sarà il carico applicativo sulla tipologia di operazioni che dovranno essere eseguite, per cercare di **ottimizzare** le operazioni previste.

## 4.1.2. Fasi della progettazione

### Metodologia di progettazione

La metodologia di progettazione consiste in:

- decomposizione dell'attività di progetto in passi successivi;
- strategie da seguire nei vari passi e criteri di scelta in caso di alternative;
- modelli di riferimento per descrivere i dati di ingresso e uscita delle varie fasi;e

deve garantire le seguenti proprietà:

- generalità rispetto alle applicazioni e ai sistemi interessati;
- qualità del prodotto in termini di correttezza, completezza ed efficienza delle risorse impiegate;
- facilità d'uso delle strategie e dei modelli di riferimento.

### Progettazione di basi di dati

La progettazione di basi di dati è composta da tre fasi, da eseguire in cascata:

- progettazione **concettuale**
- progettazione **logica**
- progettazione **fisica**

### Progettazione concettuale

È indipendente da qualsiasi codifica dei dati dei sistemi di gestione dei dati (DBMS). Traduce i requisiti informali in termini di una rappresentazione formale e completa, ma indipendente dai criteri di rappresentazione utilizzati nei DBMS. La rappresentazione formale fa riferimento ad un modello concettuale dei dati. Il risultato di questa fase è uno schema concettuale che:

- rappresenta il contenuto informativo;
- non si preoccupa della modalità di codifica dei DBMS o dell'efficienza dei programmi

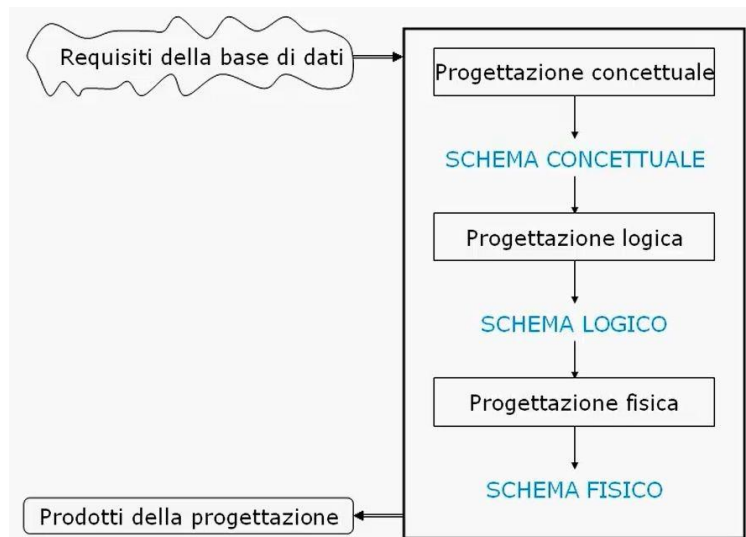
### Progettazione logica

Dipende dalla categoria del DBMS (**modello logico**) ma **non dallo specifico DBMS utilizzato**. Traduce lo schema concettuale in termini del modello di rappresentazione dei dati utilizzato dal tipo di DBMS a disposizione. La rappresentazione dei dati per il DBMS fa riferimento ad un modello logico dei dati. Il risultato di questa fase è uno schema logico che:

- descrive i dati per la rappresentazione al DBMS
- tiene in considerazione l'ottimizzazione delle operazioni che saranno effettuate sui dati
- è indipendente dai dettagli fisici

## Progettazione fisica

Dipende dallo specifico DBMS utilizzato. Completa lo schema logico con la specifica dei parametri fisici di memorizzazione dei dati. Fa riferimento ad un modello fisico dei dati e si basa sui criteri dell'organizzazione fisica dello specifico DBMS utilizzato. Il risultato di questa fase è uno schema fisico.



## Utilizzo dei requisiti nelle fasi

L'input fornito alla progettazione dalla fase di analisi dei requisiti contiene:

- specifiche sui dati, che riguardano il contenuto della base di dati;
- specifiche sulle operazioni, che riguardano l'uso che gli utenti e le applicazioni fanno della base di dati. In

particolare:

- la fase di progettazione concettuale fa uso delle specifiche sui dati utilizzando le specifiche sulle operazioni solo per controllare che lo schema sia completo e contenga tutte le informazioni per effettuare le operazioni previste;
- la fase di progettazione logica fa uso dello schema concettuale in ingresso che riassume le specifiche sui dati, mentre fa uso delle specifiche sulle operazioni insieme alle previsioni sul carico applicativo per ottenere uno schema logico che renda le operazioni eseguibili in modo efficiente;
- la fase di progettazione fisica, infine, fa uso dello schema logico e delle specifiche sulle operazioni per ottimizzare le prestazioni del sistema.

## Prodotti della progettazione

I risultati della fase di progettazione sono:

- schema fisico: è la base di dati da utilizzare;
- schema logico: fornisce una descrizione concreta del contenuto della base di dati, utile come riferimento per le operazioni di interrogazione e aggiornamento;
- schema concettuale: fornisce una rappresentazione ad alto livello, utile a scopo documentativo.

## 4.2. Modello entità-relazione

### 4.2.1. Entità, relazioni, attributi

#### Modello entità-relazione

È un modello concettuale (utilizzato nella fase di progettazione concettuale della base di dati) che offre una serie di costrutti per la definizione di schemi concettuali che descrivono l'organizzazione e la struttura delle occorrenze (o istanze) dei dati. Ogni costrutto ha una rappresentazione grafica, quindi è possibile definire lo schema E-R mediante un diagramma grafico (che favorisce un'immediata interpretazione) con aggiunte di frasi di specifica e di vincolo.

#### Entità

Rappresenta una classe di oggetti con caratteristiche comuni ed esistenza "autonoma" dal punto di vista dell'applicazione di interesse. Gli oggetti possono corrispondere a concetti materiali o immateriali e le occorrenze delle entità sono istanze degli oggetti.

Ogni entità ha un nome che la identifica univocamente nello schema ed è rappresentata graficamente da un rettangolo con il nome al suo interno.

#### Relazione

Rappresenta legami logici, significativi per l'applicazione di interesse, tra due o più entità. Ogni occorrenza della relazione è una n-upla costituita da occorrenze delle entità (una per ognuna delle n entità) e tra le occorrenze non possono esserci n-uple ripetute (*esempio studente-esame-corso*).

Ogni relazione (o associazione) ha un nome che la identifica univocamente nello schema ed è rappresentata graficamente da un rombo con il nome della relazione all'interno e con le linee che connettono la relazione con le entità coinvolte. Inoltre possono esserci delle etichette sulle linee che specificano il ruolo delle occorrenze nella relazione (e queste sono necessarie nel caso una relazione non sia simmetrica).

Possono esistere più relazioni che collegano le stesse entità e una relazione può anche coinvolgere occorrenze della stessa entità (autorelazione o relazione ricorsiva).

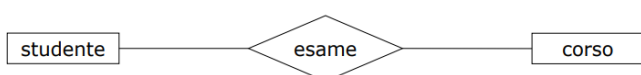
La relazione nel modello E-R corrisponde ad una relazione matematica tra le occorrenze delle entità coinvolte ed è un sottoinsieme del prodotto cartesiano delle entità coinvolte.

#### Attributo

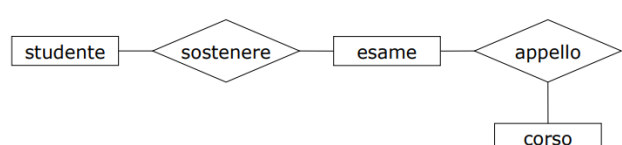
Descrivono proprietà elementari di entità o di relazioni che sono di interesse ai fini dell'applicazione. Un attributo associa a ciascuna occorrenza di entità (o relazione) un valore appartenente ad un insieme, detto dominio, di valori ammissibili per l'attributo.

Ogni attributo ha un nome che lo identifica all'interno di ogni entità/relazione ed è rappresentato graficamente da un pallino collegato alla entità/relazione a cui si riferisce (i domini non vengono riportati nello schema, ma sono generalmente descritti nella documentazione associata).

In alcuni casi occorre rappresentare attributi composti, che raggruppano cioè al loro interno altri attributi che presentano affinità nel loro significato o uso. Per rappresentarli graficamente si usa un cerchio, con all'interno il nome dell'attributo composto, collegato alla entità/relazione a cui si riferisce, da cui partono i pallini ai relativi sotto attributi.



È corretto se ci può essere un solo esame per ogni coppia *studente, corso*. Altrimenti:



## 4.2.2. Cardinalità

### Cardinalità delle relazioni

Vengono specificate per ciascuna partecipazione di entità a una relazione e **descrivono il numero minimo e massimo di occorrenze di relazione a cui un'occorrenza delle entità può partecipare**. Ogni occorrenza dell'entità potrà essere legata ad un numero di occorrenza dell'altra entità compreso tra il minimo ed il massimo specificati.

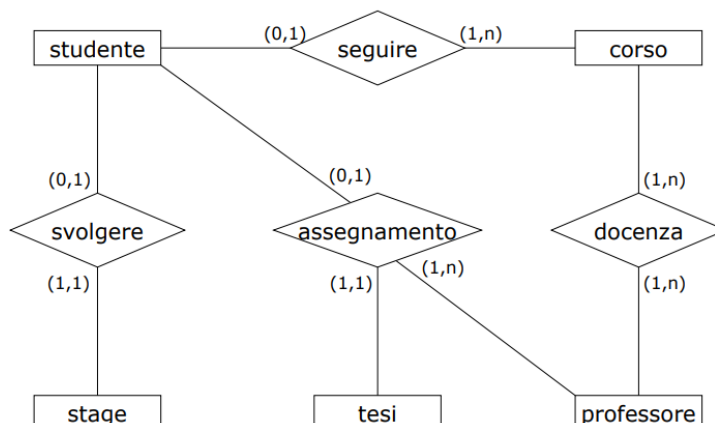
Ogni cardinalità è specificata da una coppia di valori (minimo, massimo) e viene **rappresentata graficamente dalla coppia di valori** messa come **etichetta sulla linea che collega l'entità alla relazione**. **Qualsiasi coppia di numeri interi non negativi può essere specificata come cardinalità delle relazioni**, con il **solo vincolo che la cardinalità minima sia minore o uguale a quella massima (può anche essere utilizzato il valore n, che genericamente indica un qualsiasi numero maggiore o uguale a 1)**.

Le relazioni binarie possono essere classificate sulla base delle cardinalità massime specificate per le due entità coinvolte (uno a uno, uno a molti, molti a molti)

**Card MIN (0-1) → Card MAX (1-n)**

### Cardinalità generalmente utilizzate:

- (0,1) partecipazione opzionale, al più una volta
- (0,n) partecipazione opzionale, n volte
- (1,1) partecipazione obbligatoria, al più una volta
- (1,n) partecipazione obbligatoria, n volte



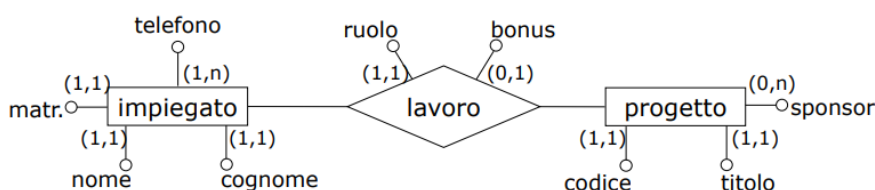
### Cardinalità degli attributi

Vengono specificate per ciascun attributo di entità/relazione e **descrivono il numero minimo e massimo di valori associati ad ogni occorrenza di entità/relazione**.

Qualsiasi coppia di numeri interi non negativi può essere specificata come cardinalità di un attributo, con il solo vincolo che la cardinalità minima sia minore o uguale a quella massima (anche in questo caso può essere utilizzato il valore n).

### Cardinalità generalmente utilizzate:

- (0,1) proprietà opzionale, al più un valore
- (0,n) proprietà opzionale, può avere un insieme di valori
- (1,1) proprietà obbligatoria, al più un valore (esattamente un valore), chiamata proprietà scalare può essere omessa nello schema
- (1,n) proprietà obbligatoria, può avere un insieme di valori



### 4.2.3. Identificatori e gerarchie

#### Identificatori delle entità

Vengono **specificati per ciascuna entità** e descrivono i concetti (attributi e/o entità) dello schema che permettono di **identificare univocamente le occorrenze delle entità**.

In molti casi uno o più attributi di un'entità **possono funzionare da identificatore per l'entità** (**identificatore interno**), ma in altri casi per identificare le occorrenze di un'entità (**entità debole**) **non bastano attributi interni**, ma serve la sua **relazione con un'altra entità** (**identificatore esterno**).

#### Identificatori interni

Quando uno o più attributi (scalari) **identificano univocamente le occorrenze delle entità**.

Vengono rappresentati graficamente come segue:

- **pallino pieno**, se l'identificatore è un solo attributo;
- **linea** che "taglia" le linee degli attributi nell'insieme e che si **conclude con un pallino pieno**, se l'identificatore è un'insieme di attributi.

Ogni entità deve avere **almeno un identificatore**, ma **può averne più di uno**, e in questo caso gli attributi coinvolti in alcune identificazioni (tranne uno) possono essere opzionali.

#### Identificatori esterni

Quando le occorrenze delle entità sono identificate da zero o più attributi (scalari) e dalla loro partecipazione in una o più relazioni in cui l'entità partecipa con cardinalità (1, 1). Vengono rappresentati graficamente con una **linea** che "taglia" le linee degli attributi interni all'entità debole che partecipano all'identificazione e la relazione inclusa nell'identificazione e che termina con un **pallino pieno**.

Un'identificazione esterna può coinvolgere una relazione con un'entità anch'essa identificata esternamente (e così via in modo ricorsivo) **purché non vengano generati cicli di identificazioni esterne**.

#### Generalizzazioni e specializzazioni

Rappresentano **legami logici (gerarchie)** tra un'entità **E (padre)** e una o più entità **E<sub>1</sub>, ..., E<sub>n</sub> (figlie)**. **E è generalizzazione di E<sub>1</sub>, ..., E<sub>n</sub>, le quali sono specializzazione di E**.

Ogni occorrenza di un'entità figlia è anche un'occorrenza dell'entità padre. Ogni proprietà dell'entità padre (attributi, identificatori, relazioni e altre generalizzazioni) è anche proprietà delle figlie.

Queste gerarchie sono rappresentate graficamente con frecche che congiungono le entità figlie all'entità padre. Le proprietà ereditate non vanno rappresentate esplicitamente nelle figlie.

#### Legami gerarchici

Le gerarchie possono essere classificate sulla base di due proprietà ortogonali:

- **totale o parziale**
- **esclusiva o sovrapposta**

In particolare:

- **gerarchia totale**: ogni occorrenza dell'entità padre è occorrenza di almeno una delle entità figlie (ovvero non esiste occorrenza dell'entità padre che non appartenga ad almeno una delle figlie)  
→ ES. padre: studenti --- figlie: I anno, II anno, III anno, fuoricorso
- **gerarchia parziale**: ci possono essere occorrenza dell'entità padre che non appartengono ad alcuna delle figlie;  
→ ES. padre: persona --- figlie: lavoratore, studente
- **gerarchia esclusiva**: ogni occorrenza dell'entità padre è occorrenza di al più una entità figlia  
→ ES. padre: studente --- figlie: I anno, II anno, III anno, fuoricorso
- **gerarchia sovrapposta**: un'occorrenza delle entità padre può appartenere a più di una entità figlia.  
→ ES. padre: persona --- figlie: lavoratore, studente

Combinando queste quattro proprietà con cui si classificano le gerarchie si ottengono quattro possibili casi:

- **totale ed esclusiva**: ogni occorrenza dell'entità padre appartiene esattamente ad una figlia;
- **totale e sovrapposta**: ogni occorrenza dell'entità padre appartiene ad almeno una figlia;
- **parziale ed esclusiva**: ogni occorrenza dell'entità padre appartiene ad al più una figlia;
- **parziale e sovrapposta**: ogni occorrenza dell'entità padre può appartenere a nessuna, una o più figlie. Le

proprietà di generalizzazione e specializzazione vengono rappresentate graficamente mediante un'etichetta (coppia) sulla freccia che congiunge le entità figlie all'entità padre.

#### 4.2.4. Documentazione di schemi

Lo schema E-R da solo non è sufficiente a rappresentare tutti gli aspetti di un'applicazione, quindi è necessario corredare ogni schema E-R con una documentazione di supporto per:

- facilitare l'interpretazione dello schema stesso;
- descrivere le proprietà dei dati che non possono essere espresse con costrutti del modello E-R.

##### Limitazioni degli schemi E-R

- Il nome dei concetti può non essere sufficiente a comprenderne il significato;
- può non essere possibile rappresentare tutti gli attributi senza inficiare la leggibilità dello schema;
- alcune proprietà (vincoli) non sono rappresentabili con i costrutti del modello E-R.

##### Regole aziendali

Uno degli strumenti più utilizzati per la descrizione di proprietà non rappresentabili con modelli concettuali è quello delle regole aziendali (business rules), che descrivono informazioni che definiscono o vincolano aspetti di un'applicazione:

- descrizione di un concetto: definizione precisa di un'entità, relazione o attributo;
- vincolo d'integrità: restrizioni sul valore dei dati (che sia esprimibile o meno nel modello E-R);
- derivazione: concetti che sono ottenuti, per inferenza o calcolo aritmetico, da altri oggetti dello schema. Le

regole aziendali possono essere espresse con frasi in linguaggio naturale o con metodi formali:

- per le descrizioni dei concetti si fa in genere ricorso al linguaggio naturale;
- per i vincoli d'integrità e le derivazioni esistono diverse proposte di sintassi formali, ma non esiste uno standard.

##### Regole per i vincoli d'integrità

Possono essere espresse sotto forma di asserzioni, cioè affermazioni che devono essere sempre verificate nella base di dati, che siano:

- atomiche: per chiarezza, non possono essere decomposte in frasi che contengono altre asserzioni;
- dichiarative: devono enunciare la proprietà, non il modo per soddisfarla (la reazione ad una possibile violazione è infatti un problema realizzativo, non pertinente alla rappresentazione concettuale).

##### Asserzioni per i vincoli di integrità

Possibile struttura: *<concetto> deve/non deve <espressione su concetti> .*

I concetti citati possono corrispondere ad oggetti citati nello schema o ad oggetti derivabili da essi.

##### Regole di derivazione

Possono essere espresse specificando le operazioni che permettono di ottenere il concetto derivato. Possibile struttura: *<concetto> si ottiene <operazione su concetti> .*

Anche in questo caso i concetti citati possono corrispondere ad oggetti citati nello schema o ad oggetti derivabili da essi.

## Implementazione di regole

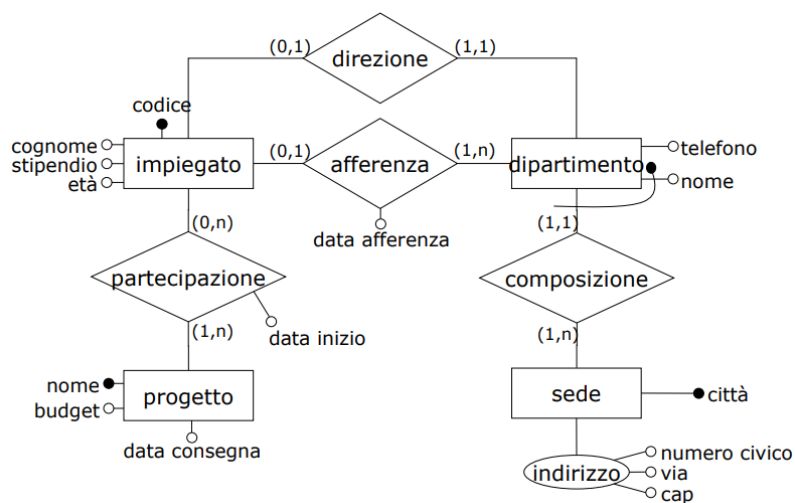
Nella traduzione dello schema concettuale in una base di dati, le regole aziendali non descrittive (asserzioni per integrità e derivazioni) devono essere codificate per garantire la consistenza dei dati rispetto alle proprietà stabilite dalle regole. Per farlo esistono diversi approcci:

- clausole del linguaggio SQL;
- regole attive (trigger);
- opportune procedure scritte in qualche linguaggio di programmazione.

## Descrizione di concetti

Può essere prodotta facendo uso di un dizionario dei dati composto da due tabelle:

- descrizione delle entità, con attributi:
  - nome;
  - definizione informale;
  - elenco degli attributi (con eventuale descrizione);
  - identificatore;
- descrizione delle relazioni, con attributi:
  - nome;
  - definizione informale;
  - elenco degli attributi (con eventuale descrizione);
  - elenco delle entità coinvolte insieme alla loro cardinalità di partecipazione.



| Nome Entità  | Descrizione                                          | Attributi                           | Identificatore |
|--------------|------------------------------------------------------|-------------------------------------|----------------|
| impiegato    | Impiegato che lavora nella azienda.                  | codice, cognome, stipendio, età     | codice         |
| progetto     | Progetti aziendali sui quali lavorano gli impiegati. | nome, budget, data consegna         | nome           |
| dipartimento | Dipartimenti delle sedi dell' azienda.               | telefono, nome                      | nome, sede     |
| sede         | Sede dell' azienda in una certa città.               | città, indirizzo (numero, via, CAP) | città          |

| Nome Relazione | Descrizione                                           | Entità Coinvolte                      | Attributi      |
|----------------|-------------------------------------------------------|---------------------------------------|----------------|
| direzione      | Associa un dipartimento al suo direttore.             | impiegato (0,1)<br>dipartimento (1,1) |                |
| afferenza      | Associa un impiegato al suo dipartimento.             | impiegato (0,1)<br>dipartimento (1,n) | data afferenza |
| partecipazione | Associa agli impiegati i progetti sui quali lavorano. | impiegato (0,1)<br>progetto (1,n)     | data inizio    |
| composizione   | Associa una sede ai dipartimenti di cui è composta.   | dipartimento (1,1)<br>sede (1,n)      |                |

| Regole di Vincolo     |                                                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| (RV1)                 | Il direttore di un dipartimento deve afferire a tale dipartimento.                                                  |
| (RV2)                 | Un impiegato non deve avere uno stipendio maggiore del direttore del dipartimento al quale afferisce.               |
| (RV3)                 | Un dipartimento con sede a Roma deve essere diretto da un impiegato con più di dieci anni di anzianità.             |
| (RV4)                 | Un impiegato che non afferisce a alcun dipartimento non deve partecipare a alcun progetto.                          |
| Regole di Derivazione |                                                                                                                     |
| (RD1)                 | Il budget di un progetto si ottiene moltiplicando per 3 la somma degli stipendi degli impiegati che vi partecipano. |

## 4.3. Progettazione concettuale

### 4.3.1. Strategie di progetto

#### 4.3.1.1. Top-down

A partire dalle specifiche si produce uno schema iniziale con pochi concetti molto astratti. Lo schema viene via via raffinato aumentando il dettaglio dei vari concetti: i raffinamenti prevedono l'uso di **trasformazioni elementari (primitive)** che operano sul singolo concetto per descriverlo con maggior dettagli. Per raffinamenti successivi si arriva quindi allo schema finale.

#### Primitive di trasformazione

Le primitive utilizzate in questa strategia possono essere riassunte nelle seguenti primitive principali:

- **da entità a relazione tra entità**: un'entità viene sostituita da due entità collegate tra loro
- **da entità a generalizzazione**: un'entità è scomposta in sotto-entità distinte;
- **da relazione a insieme di relazioni**: una relazione viene sostituita da due relazioni
- **da relazione a entità con relazioni**: una relazione viene sostituita con un'entità (costituisce un oggetto con esistenza autonoma)
- **introduzione di attributi su entità**
- **introduzione di attributi su relazioni**

#### Vantaggi e svantaggi

Tutti gli aspetti presenti nello schema finali sono presenti, in linea di principio, ad ogni livello di raffinamento. Un **vantaggio** è che il **progettista può descrivere inizialmente tutte le specifiche dei dati, trascurando i dettagli**, per poi entrare nel merito di un concetto alla volta. Lo **svantaggio** però è che questo è **possibile solo quando si possiede sin dall'inizio una visione globale e astratta** di tutte le componenti del sistema, il che generalmente non è possibile, soprattutto per quanto riguarda applicazioni complesse.

#### 4.3.1.2. Bottom-up

Le specifiche iniziali sono suddivise in sottocomponenti sempre più piccole fino a quando queste descrivono un frammento elementare dell'applicazione. Le varie componenti vengono rappresentate tramite schemi concettuali (prevede l'uso di **trasformazioni elementari**, o **primitive**, che introducono nuovi concetti).

I vari sottoschemi vengono quindi fusi fino a giungere, attraverso la completa integrazione delle componenti, allo schema concettuale finale.

#### Primitive di trasformazione

Le primitive utilizzate in questa strategia possono essere riassunte nelle seguenti primitive principali:

- **generazione di entità**: si individua, nelle specifiche, una classe di oggetti con proprietà comuni;
- **generazione di relazione**: si individua, nelle specifiche, un legame logico tra entità;
- **generazione di generalizzazione**: si individua, nelle specifiche, un legame di generalizzazione tra entità;
- **aggregazione di attributi su entità**: a partire da un insieme di attributi si individua un'entità che li aggrega;
- **aggregazione di attributi su relazioni**: a partire da un insieme di attributi si individua una relazione che li aggrega.

#### Vantaggi e svantaggi

Le primitive introducono nello schema nuovi concetti per descrivere aspetti della realtà di interesse non ancora rappresentati. Il **vantaggio** di questa strategia è la sua capacità di decomporre il problema in componenti più

semplici, il cui progetto può essere affidato a progettisti diversi (molto utile nel caso di collaborazione nello sviluppo di applicazioni complesse). Gli svantaggi consistono nelle operazioni di integrazione di schemi concettuali differenti (spesso difficoltose) e nel fatto che i progettisti così avranno solo una visione parziale delle componenti del sistema.

#### 4.3.1.3. Inside-out

È un caso particolare della strategia bottom-up, e consiste nell'individuare inizialmente alcuni concetti e poi, a partire da questi, procedere "a macchia d'olio", rappresentando i concetti più vicini a quelli iniziali e procedendo attraverso una navigazione delle specifiche.

#### Vantaggi e svantaggi

Il vantaggio principale di questa strategia è che, a rispetto al bottom-up, non richiede passi di integrazione. Gli svantaggi invece consistono nella necessità di esaminare di volta in volta tutte le specifiche per individuare i concetti non ancora rappresentati e nel fatto che non permette di procedere per livelli di astrazione.

#### 4.3.1.4. Mista

Questa strategia **combina top-down e bottom-up** e consiste quindi nel **suddividere i requisiti in componentiseparate** (come nel bottom-up) **definendo uno "schema scheletro"** contenente, a livello astratto, i **concetti principali dell'applicazione** (come nel top-down).

#### Vantaggi

Questa strategia unisce i vantaggi delle due strategie su cui si basa, e ne riduce gli svantaggi, in quanto la divisione in sottocomponenti permette di affrontare applicazioni complesse, mentre l'utilizzo dello schema scheletro fornisce una visione unitaria e favorisce le fasi di integrazione degli schemi sviluppati separatamente. Questa strategia è effettivamente l'unica utilizzabile nei casi pratici di una certa complessità, dove spesso è necessario cominciare la progettazione quando non tutti i dati sono disponibili.

### 4.3.2. Qualità di schemi concettuali e strumenti

La qualità dello schema concettuale ne definisce la bontà rispetto alle proprietà di correttezza, completezza, leggibilità e minimalità.

#### Correttezza

Lo schema concettuale deve utilizzare propriamente i costrutti messi a disposizione, si parla quindi di:

- correttezza sintattica: i costrutti devono essere utilizzati solo secondo l'uso previsto;
- correttezza semantica: l'uso dei costrutti deve rispettare la loro definizione.

#### Completezza

Lo schema concettuale deve rappresentare tutti i dati di interesse e permettere di eseguire tutte le operazioni a partire dai dati espressi nello schema, perciò tutte le specifiche sui dati devono essere rappresentate da qualche concetto nello schema e tutti i concetti coinvolti in un'operazione presente nelle specifiche devono essere raggiungibili "navigando" attraverso lo schema.

#### Leggibilità

Lo schema concettuale deve rappresentare i requisiti in maniera naturale e facilmente comprensibile, occorre quindi rendere lo schema autoesplicativo scegliendo in modo opportuno i nomi dei concetti e mantenendo il più possibile semplice e pulito il diagramma, ad esempio:

- disponendo i costrutti scegliendo come elementi centrali quelli con più relazioni;
- tracciando solo linee perpendicolari e minimizzando le intersezioni;

- disponendo le entità padri di generalizzazioni al di sopra delle loro figlie.

### Minimalità

Lo schema concettuale dovrebbe rappresentare ogni specifica sui dati una sola volta, in modo da evitare ogni ridondanza: esistono ridondanze quando un concetto può essere derivato da altri (tipicamente corrisponde ad un ciclo nello schema in cui è possibile rimuovere una relazione senza perdere informazioni). Tuttavia, non sempre le ridondanze sono indesiderate, in quanto a volte possono essere mantenute per facilitare l'accesso ai dati.

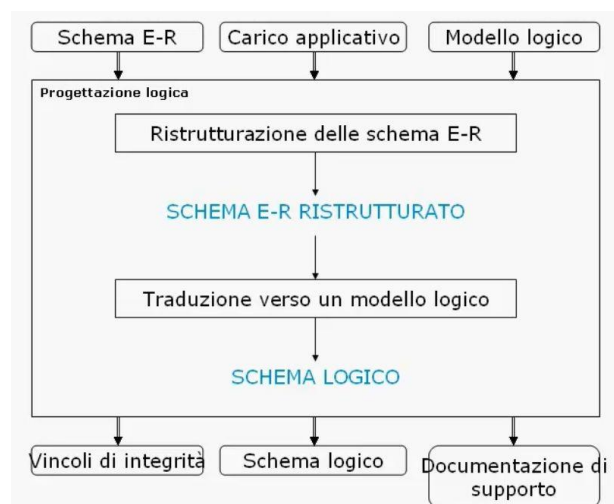
### Strumenti CASE

Esistono diversi pacchetti applicativi per il progetto e lo sviluppo di basi di dati, che offrono generalmente:

- interfaccia grafica per manipolare schemi E-R;
- dizionario dei dati che memorizza informazioni sui vari concetti dello schema;
- strumenti integrati che eseguono, automaticamente o tramite interazione con l'utente, compiti della progettazione (come, ad esempio, analisi di qualità e produzione automatica del codice per il DBMS).

## 4.4. Progettazione logica

Traduce lo schema concettuale in uno schema logico in grado di descrivere in maniera corretta ed efficiente tutte le informazioni prodotte dalla progettazione concettuale (schema E-R e vincoli).



#### 4.4.1. Ristrutturazione di schemi ER

Questa fase è necessaria per **ottimizzare e semplificare lo schema E-R** (rendendo più efficienti le operazioni previste e traducendo i concetti non rappresentabili nel modello logico) ed è **indipendente dal modello logico** scelto.

##### 4.4.1.1. Analisi di ridondanze

Si ha una **ridondanza** quando nello schema viene **espresso esplicitamente un concetto che può essere derivato da altri concetti presenti nello schema**. Si può avere ridondanza di attributi e di relazioni, infatti:

- alcuni attributi possono essere derivabili da:
  - altri attributi della stessa entità o relazione;
  - altri attributi di altre entità o relazioni;
  - operazioni di conteggio di occorrenze;
- alcune relazioni possono essere derivabili dalla composizione di altre relazioni già presenti nello schema (la presenza di cicli nello schema può indicare la presenza di una ridondanza).

##### Vantaggi e svantaggi

Le ridondanze **dovrebbero essere possibilmente evitate**, ma la loro presenza *può comportare dei vantaggi*, permettono una riduzione degli accessi e delle operazioni necessari per computare il dato derivato. D'altro canto, le ridondanze hanno lo svantaggio di occupare **maggiore memoria** (anche se spesso non è rilevante) e di richiedere **operazioni aggiuntive per l'aggiornamento del dato derivato**. Perciò, la decisione di mantenere o eliminare una ridondanza dipende dai costi delle operazioni e dallo spazio di memoria occupato con o senza ridondanze (specificati dal carico applicativo)

##### 4.4.1.2. Eliminazione delle gerarchie

Mentre per le ridondanze si può operare una scelta, **le gerarchie devono essere eliminate** perché **non sono rappresentabili direttamente nei DBMS**. Esistono tre approcci per la loro eliminazione:

- accorpamento delle figlie nel padre (collasso verso l'alto)
- accorpamento del padre nelle figlie (collasso verso il basso)
- mantenimento delle entità (delle entità non cancello nulla ma *sostituisco il legame gerarchico con delle relazioni*)

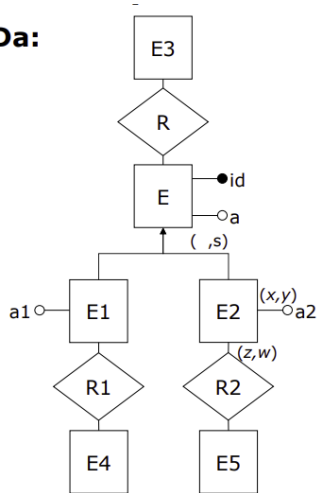
##### Accorpamento delle figlie nel padre

Le entità figlie vengono eliminate e i loro attributi e le loro relazioni vengono trasferiti nel padre, diventando **opzionali**. Uno o più attributi (chiamati selettori) vengono aggiunti al padre per indicare per ogni istanza a quale/i tra le  $N$  figlie appartiene l'istanza, ma va fatta una distinzione in base al tipo di gerarchia in esame:

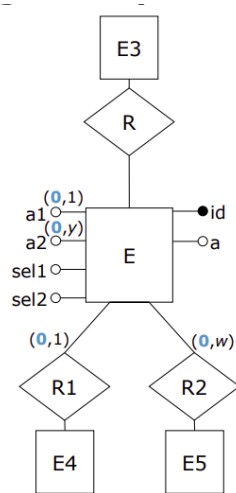
- esclusiva: un selettore:
  - totale: dominio del selettore di  $N$  possibili valori;
  - parziale: dominio del selettore di  $N + 1$  possibili valori;
- sovrapposta:  $N$  selettori con dominio *vero / falso* :
  - totale: almeno un selettore avrà valore *vero* ;
  - parziale: i selettori potrebbero tutti avere valore *falso* .

**Cardinalità minima va a 0 (opzionali), qualunque essa sia, la max resta uguale**

Da:



A:



### Vantaggi e svantaggi

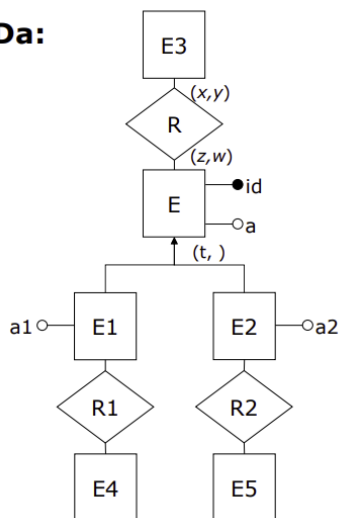
Questa strategia è conveniente quando le operazioni non fanno molta distinzione tra le occorrenze e gli attributi del padre o delle figlie, perché si accede così ad una sola entità anziché a due o più attraverso la relazione. Lo svantaggio invece è che tutti gli attributi e le relazioni che sono obbligatori per le figlie diventano opzionali per il padre, con la possibilità di avere valori nulli.

### Accorpamento del padre nelle figlie

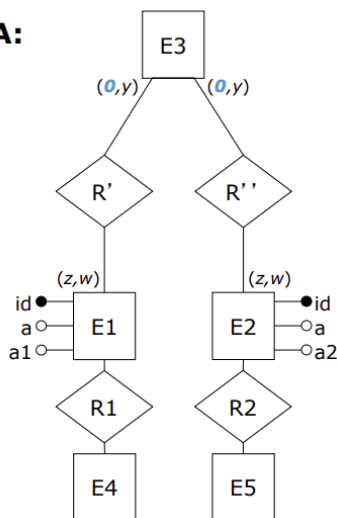
L'entità padre viene eliminata e i suoi attributi e le sue relazioni vengono trasferiti in ciascuna delle figlie; le entità che erano in relazione con il padre ora sono in relazione opzionale con le figlie. Nel caso in cui la gerarchia sia parziale, però, questa strategia non è possibile, a meno che non si aggiunga un'entità figlia aggiuntiva (che raggruppa tutte le istanze che non appartengono ad alcuna figlia).

**Cardinalità va a 0 (quale??)**

Da:



A:



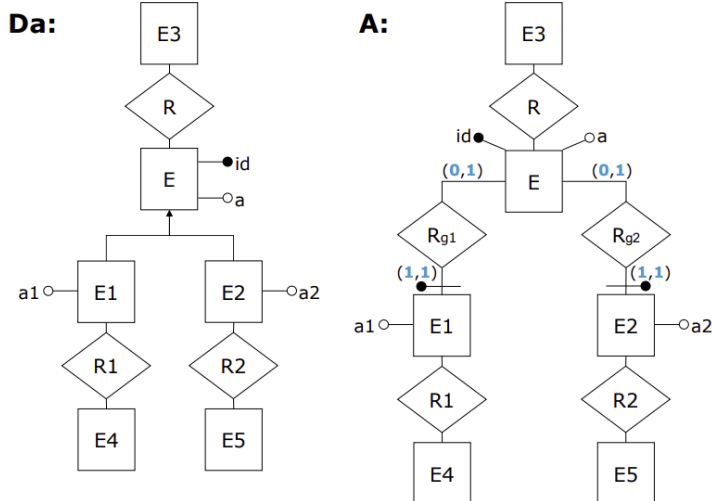
### Vantaggi e svantaggi

Questa strategia è conveniente in presenza di molti attributi di specializzazione (il collasso verso l'alto porterebbe a molti valori nulli) e per le operazioni che accedono separatamente alle figlie. Gli svantaggi sono la difficoltà di accorpamento in caso di **gerarchia parziale** e la presenza di ridondanze in caso di **gerarchia sovrapposta**.

### Mantenimento delle entità

Il legame gerarchico viene sostituito con relazioni tra il padre e ciascuna delle figlie, che sono identificate esternamente tramite la relazione con il padre (sono quindi **entità deboli**). Nello schema occorre aggiungere i vincoli che derivano dal tipo di gerarchia:

- esclusiva: ogni istanza del padre **può** partecipare alla relazione con **una sola** delle figlie;
- totale: ogni istanza del padre **deve** partecipare alla relazione con **almeno una** delle figlie.



### Vantaggi e svantaggi

Il vantaggio di questa strategia è che è **sempre possibile** ed è **conveniente** in presenza di **operazioni che si riferiscono solo alle occorrenze e agli attributi delle figlie**. Gli **svantaggi** del mantenimento delle entità consistono invece in una maggiore quantità di accessi per le **operazioni che accedono agli attributi del padre e delle figlie** e nella necessità di mantenere i vincoli introdotti (**esercizio esame**: toglie le gerarchie, con eventuali vincoli → **errore più grave**: tradurre in E1,E2 in relazione ternaria con E3. **SBAGLIATO**: quella relazione viene specificata per **ciascuna** delle figlie)

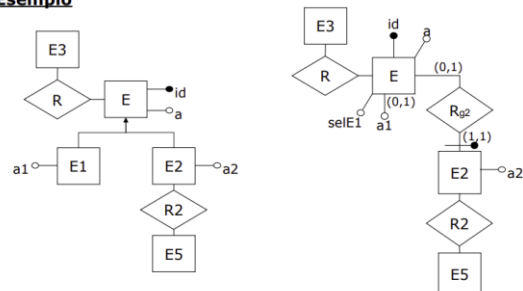
### Scelta della strategia

La scelta tra le varie alternative dipende dalle **valutazioni sull'occupazione di memoria e sul costo delle operazioni nei diversi casi**. Il semplice conteggio degli accessi e delle occorrenze non è sufficiente, in quanto è **necessario considerare anche la dimensione dei domini e la quantità di dati recuperabili con un'operazione di accesso a memoria secondaria**.

È comunque possibile adottare strategie diverse all'interno di ogni gerarchia dello schema, ottenendo così delle **ristrutturazioni ibride**

**Sono possibili ristrutturazioni ibride (che combinano più di una alternativa)**

#### Esempio



### 4.4.1.3. Partizionamenti e accorpamenti

I partizionamenti e gli accorpamenti sono utili per due motivi:

- per **aumentare l'efficienza** delle operazioni:
  - partizionando entità o relazioni (dividendo attributi di una stessa entità o relazione che sono acceduti da operazioni diverse);
  - accorpando entità o relazioni (unendo in una stessa entità o relazione attributi che sono acceduti spesso insieme);
- per **tradurre concetti non supportati** dal modello logico:
  - eliminando degli attributi multi-valore;

- eliminando degli attributi composti.

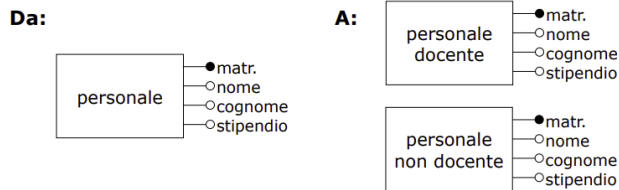
## Partizionamento di entità

Ne esistono di due tipi:

- **partizionamento orizzontale**: un'entità  $E$  viene sostituita con due entità  $E_1$  e  $E_2$  che si dividono le istanze di  $E$  (equivale a sostituire  $E$  con sue figlie);



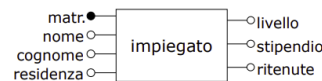
### Esempio



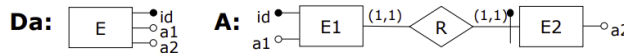
- **partizionamento verticale**: un'entità  $E$  viene sostituita con due entità  $E_1$  e  $E_2$  che si dividono gli attributi o relazioni di  $E$  e che sono in relazione (una delle entità è identificata esternamente da quella che mantiene l'identificatore)

### Esempio

**Da:**



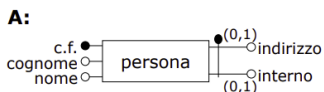
**A:**



## Accorpamento di entità

**L'accorpamento è l'operazione inversa rispetto al partizionamento** e permette di prendere due entità  $E_1$  e  $E_2$  legate da una relazione (uno a uno o uno a molti) e di sostituirle con un'entità  $E$  che ha gli attributi e le relazioni di entrambe.

### Esempio

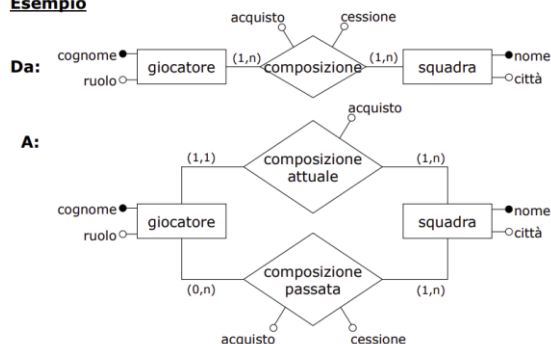


## Partizionamento e accorpamento di relazioni

Il partizionamento e l'accorpamento possono essere effettuati anche sulle relazioni:

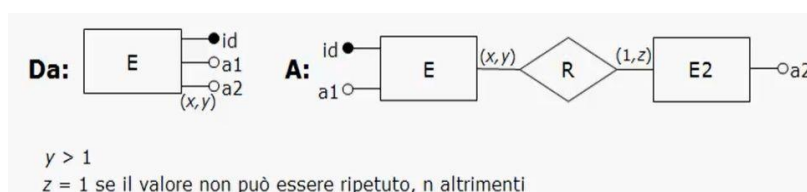
- **partizionamento**: una relazione  $R$  viene sostituita con due relazioni  $R_1$  e  $R_2$  tra le stesse entità per separare le occorrenze di  $R$  accedute sempre separatamente;
- **accorpamento**: due relazioni  $R_1$  e  $R_2$  tra le stesse entità (che si riferiscono a due aspetti del medesimo concetto e le cui occorrenze siano sempre accedute contemporaneamente) vengono sostituite con una sola relazione  $R$

### Esempio

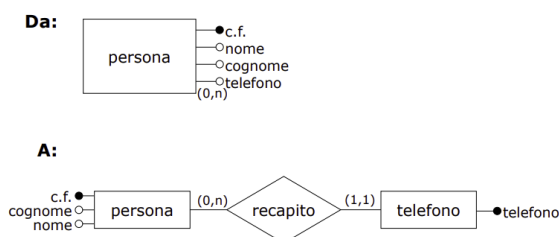


## Eliminazione di attributi multivalore

Un'altra ragione per cui occorre ristrutturare lo schema consiste nella **necessità di eliminare** quei **concetti che non possono essere rappresentati dal modello logico**, come ad esempio gli **attributi multivalore**. Per eliminarli, si crea una **nuova entità** che contiene l'**attributo multivalore** e che sia **collegata all'entità originale** da una relazione (la cardinalità della relazione dipende dalla cardinalità originale e dal fatto che i valori dell'attributo possano comparire più di una volta).



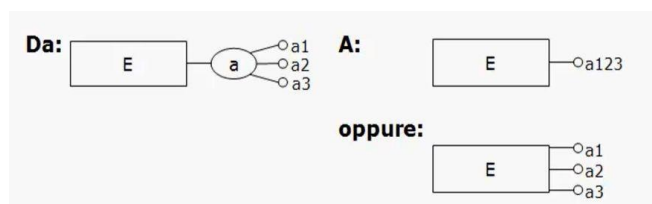
### Esempio



## Eliminazione di attributi composti

Nemmeno gli attributi composti sono supportati dal modello relazionale. Le **soluzioni** che si possono adottare per eliminarli sono due:

- si **elimina la suddivisione** in componenti e si considera il composto come un attributo semplice;
- si **elimina l'attributo composto** e si considerano le sue componenti come attributi semplici.



### 4.4.1.4. Scelta degli identificatori primari

Per ogni entità deve essere selezionato un **identificatore primario**:

- se l'entità **non ha** identificatori, occorre individuarne uno tra i suoi attributi, oppure inserirne uno;
- se l'entità **ha** più identificatori (**chiavi candidate**) bisogna decidere quale verrà utilizzato come **chiave primaria**.

In caso di **più identificatori candidati**, è preferibile scegliere come primario un identificatore:

- utilizzato in molte operazioni per accedere all'entità;
- con pochi attributi (nel senso di pochi attributi che compongono la chiave)
- con solo attributi interni;

mentre non è possibile scegliere come primari quegli identificatori con attributi che possono avere valori nulli. Gli identificatori candidati che non sono stati selezionati come chiavi primarie, potranno costituire (in sede di progettazione fisica) possibili indici secondari.

#### 4.4.2. Traduzione verso il modello logico

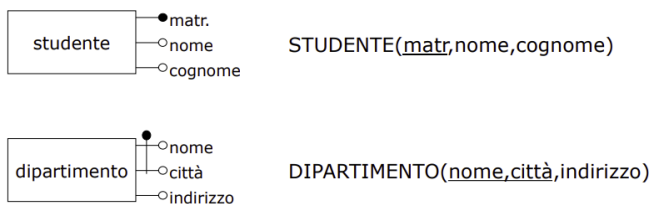
Questa fase fa riferimento allo specifico modello logico scelto e può includere un'ulteriore **ottimizzazione delle operazioni** che verranno eseguite basata sulle caratteristiche del modello logico. Si traduce lo schema concettuale ristrutturato in uno schema logico che:

- fa riferimento allo specifico modello logico scelto;
- può includere un'ulteriore ottimizzazione basata sulle caratteristiche del modello logico. Utilizziamo come modello logico il modello relazionale e per distinguere la relazione del modello E-R dalla relazione del modello relazionale, ci riferiamo a quest'ultima come tabella.

##### Traduzione di entità

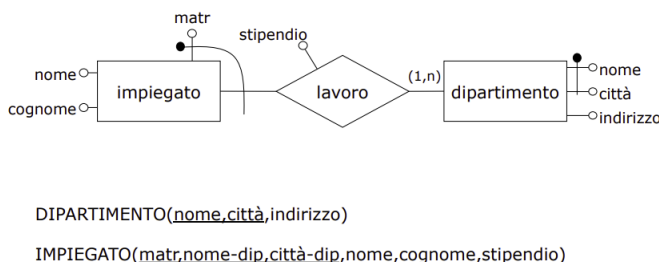
Ogni entità forte  $E$  è tradotta in una tabella  $R$  con:

- attributi (colonne): gli attributi di  $E$  ;
- chiave primaria: l'identificatore primario di  $E$  ;
- vincoli: derivanti da vincoli sull'entità (per esempio cardinalità di attributi o vincoli di non nullità su attributi obbligatori)



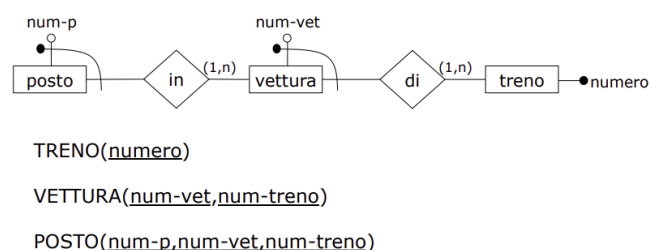
Mentre ogni entità debole  $E$  identificata esternamente da  $E'$  è tradotta in una tabella  $R$  con:

- attributi: gli attributi di  $E$  più l'identificatore di  $E'$  più gli attributi della relazione che identifica  $E$  esternamente;
- chiave primaria: l'identificatore primario di  $E$ , quindi gli attributi interni ad  $E$  unitamente all'identificatore di  $E'$
- vincoli:
  - integrità referenziale sull'identificatore di  $E'$  ;
  - altri vincoli (per esempio non nullità su attributi obbligatori)
  -



Per catene di entità deboli il processo di traduzione deve seguire il percorso a partire dall'entità forte

##### Esempio



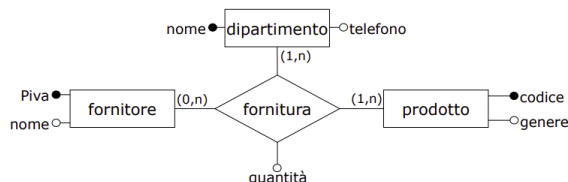
## Traduzione di relazioni (standard)

Ogni relazione  $R$  tra le entità  $E_1, E_2, \dots, E_n$  è tradotta in una tabella  $R$  con:

- attributi: gli attributi di  $R$  più gli identificatori di  $E_1, E_2, \dots, E_n$
- chiave primaria: l'unione  $K$  degli identificatori di  $E_1, E_2, \dots, E_n$ , a meno che  $K$  non sia superchiave (ilche dipende dalla cardinalità della relazione o da eventuali dipendenze), in tal caso si utilizza un sottoinsieme di  $K$
- vincoli:
  - integrità referenziale sugli identificatori di  $E_1, E_2, \dots, E_n$
  - altri vincoli (per esempio non nullità su attributi obbligatori)

Si noti come le relazioni che legano le entità deboli a quelle forti non vengono tradotte (perché sono già state tradotte nel processo di traduzione delle entità deboli)

### Esempio



DIPARTIMENTO(nome,telefono)

FORNITORE(Piva,nome)

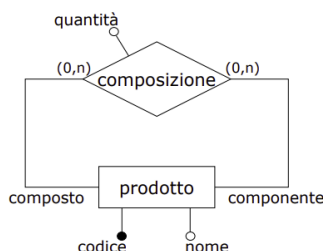
PRODOTTO(codice,genere)

FORNITURA(nome-dip,Piva-forn,cod-prodotto,quantità)

## Casi particolari

- Nel caso di autorelazioni (o relazioni ricorsive) la tabella che descrive la relazione contiene due volte l'identificatore dell'entità, che deve quindi essere necessariamente ridenominata;
- nel caso di relazioni con cardinalità uno a molti, la chiave è l'identificatore dell'entità che partecipa con cardinalità massima uno;
- nel caso di relazioni con cardinalità uno a uno la chiave è l'identificatore di una delle entità

### Esempio



PRODOTTO(codice,nome)

COMPOSIZIONE(composto,componente,quantità)

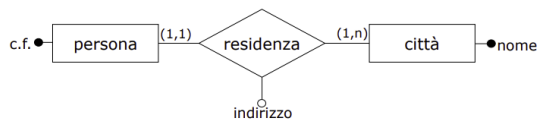
## Altri tipi di traduzione

La traduzione standard è sempre possibile ed è l'unica possibile anche nel caso di relazioni con cardinalità molti a molti. Negli altri casi, invece, sono possibili traduzioni alternative che fondono in una tabella la relazione e una o entrambe le entità. I vantaggi di queste traduzioni sono il minor numero di tabelle e di operazioni di join per navigare la relazione, ma lo svantaggio principale è che si penalizzano le operazioni che consultano soltanto gli attributi di un'entità che è stata fusa.

## Traduzione di relazioni (1:n)

La relazione può essere fusa nell'entità che partecipa con cardinalità massima uno (se la cardinalità minima è zero, gli attributi inglobati dalla relazione assumeranno valori nulli per le istanze che non partecipano alla relazione)

## Esempio

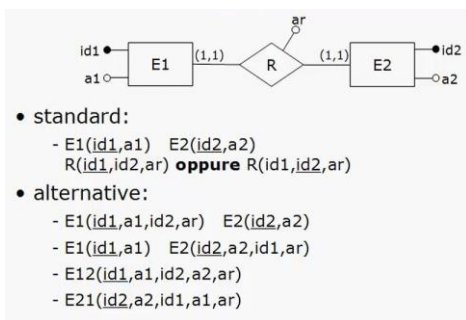


PERSONA(c.f.)  
CITTÀ(nome)  
RESIDENZA(c.f., città, indirizzo)

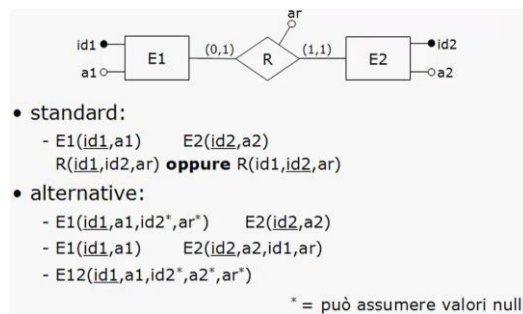
## Traduzione di relazioni (1:1)

La relazione può essere fusa, distinguendo tre casi a seconda delle cardinalità minime con cui le entità  $E_1$  ed  $E_2$  partecipano alla relazione:

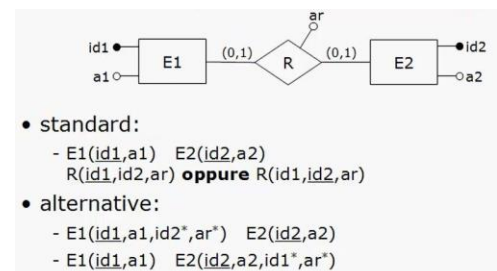
### Entrambe uno



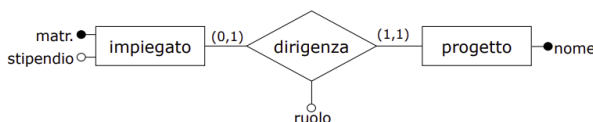
### Una uno e l'altra zero



### Entrambe zero



## Esempio



IMPIEGATO(matr., stipendio)  
PROGETTO(nome)  
DIRIGENZA(matr., nome-prog, ruolo)  
oppure  
DIRIGENZA(matr., nome-prog, ruolo)

- standard:
  - PERSONA(c.f., nome)
  - MATR(c.f., lui, c.f. lei, data) oppure MATR(c.f. lui, c.f. lei, data)
- alternativa:
  - PERSONA(c.f., nome, c.f. altro\*, data\*)

\* = può assumere valori nulli

## 4.4.3. Documentazione, verifica, progettazione fisica

### Documentazione di schemi logici

I risultati della progettazione logica sono:

- lo schema logico;
- la documentazione associata:
  - ereditata e adattata dallo schema concettuale;
  - introdotta per descrivere nuovi vincoli (come per esempio l'integrità referenziale, introdotto in fase di traduzione verso il modello logico).

Lo schema e i vincoli di integrità referenziale possono essere rappresentati con una notazione grafica, ma non ne esiste una standard. Una possibile notazione grafica consiste di rappresentare:

- schemi di relazione come rettangoli;
- chiave in grassetto;
- possibilità di valori nulli con asterisco;
- integrità referenziale con frecce (con partenza evidenziata da un rettangolo e con eventuale etichetta).

### Verifica di schemi

Lo schema prodotto dalla progettazione logica deve essere verificato rispetto a dei criteri di qualità. Alcune proprietà (dette forme normali) certificano la qualità dello schema, permettendo ad esempio di stabilire che la base di dati sia priva di ridondanze (che potrebbero comportare delle anomalie di comportamento e inficiare l'integrità dei dati). Se la progettazione logica è stata fatta bene, lo schema sarà in forma normale (quindi la fase di verifica certifica che lo schema sia in forma normale), altrimenti si opera un processo di normalizzazione.

### Strumenti CASE per la progettazione logica

Supportano la fase di progettazione logica operando una traduzione automatica dallo schema ristrutturato al modello relazionale, anche se la fase di ristrutturazione deve essere fatta manualmente (questo perché i sistemi automatici non sono in grado di valutare le diverse alternative).

### Progettazione fisica

Una volta ottenuto lo schema logico della base di dati unitamente alla documentazione di supporto associata si passa alla fase di progettazione fisica, che produce infatti lo schema fisico della base di dati.

Questa fase ha come:

- ingresso:
  - schema logico;
  - caratteristiche del DBMS scelto;
  - previsioni sul carico applicativo;
- uscita: schema fisico della base di dati, che include:
  - definizioni di relazioni con DDL del DBMS scelto;
  - strutture fisiche di accesso con relativi parametri;
  - indici per l'ottimizzazione dell'accesso a particolari attributi.

### Indici

Sono definiti su attributi e si traducono in speciali strutture di accesso che garantiscono accesso diretto e ottimizzano l'accesso agli attributi su cui sono definiti. Gli indici vengono spesso individuati in modo empirico con una fase di regolazione (tuning) che può introdurre nuovi indici per migliorare le prestazioni.

## 4.5. Esercizi di progettazione concettuale e logica

### 4.5.1. Parte I

#### 4.5.1.1. Progettazione concettuale

##### Requisiti

L'associazione *Ed È Subito Sera* (EESS) organizzatrice di eventi culturali legati alla poesia contemporanea vuole realizzare un'applicazione base di dati.

Scopo primario dell'associazione EESS è quello di organizzare eventi culturali in tutta Italia. Tipicamente, gli eventi sono caratterizzati da uno slogan, si svolgono in qualche città italiana, hanno una certa durata e hanno una quota di partecipazione.

Gli eventi possono essere sovvenzionati da enti di cui si conosce il nome, una breve descrizione della loro attività, l'indirizzo e l'importo versato per la sponsorizzazione. Si noti che uno stesso ente può sponsorizzare più eventi e che l'importo versato può variare.

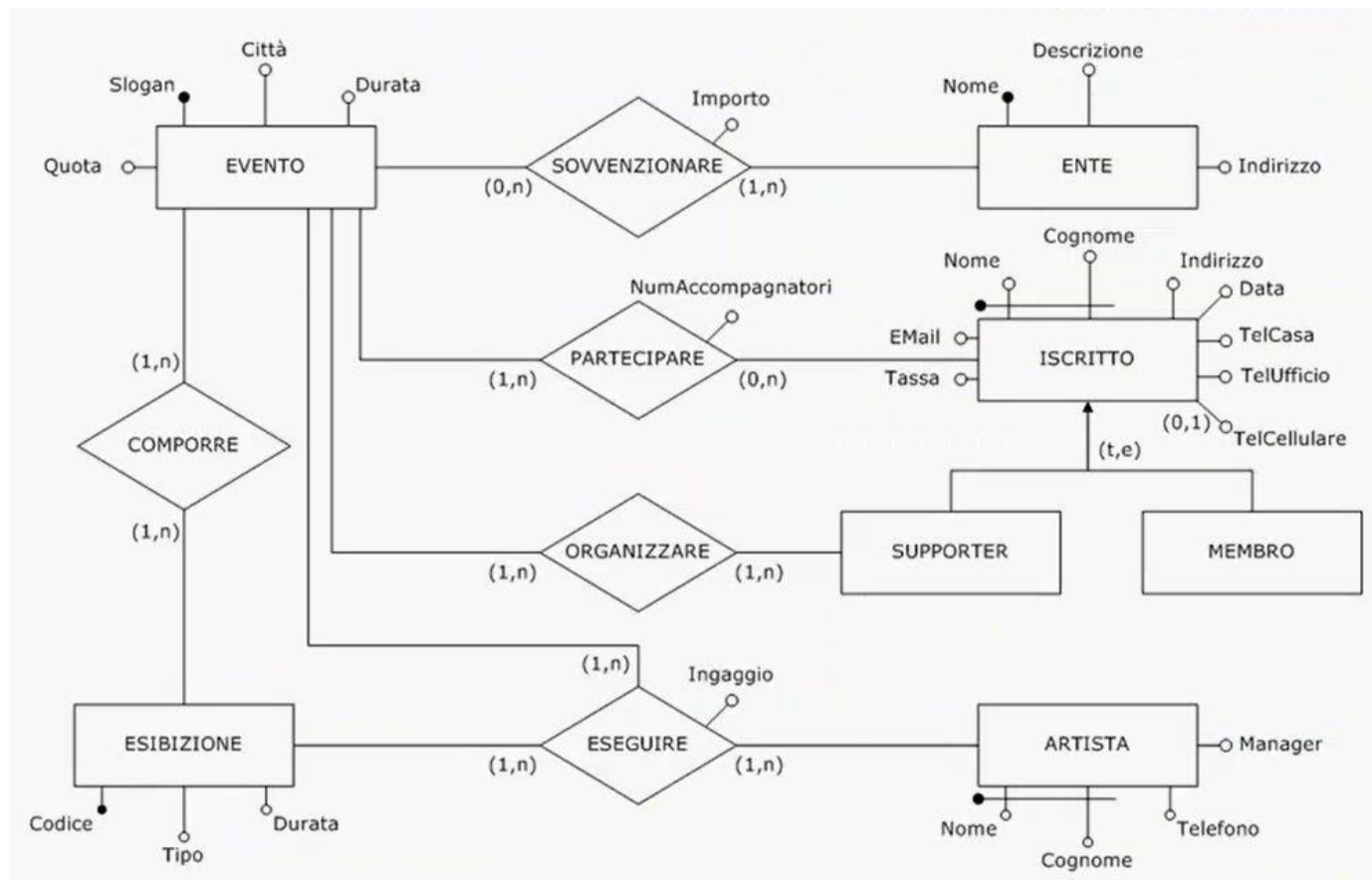
Gli eventi sono in generale costituiti da una o più esibizioni, eseguite da uno o più artisti (per ognuno dei quali si conosce nome e cognome, nome del manager, numero di telefono di contatto) e sono caratterizzati dalla tipologia e dalla durata. Si noti che la stessa esibizione può aver luogo in più eventi.

EESS deve inoltre tenere traccia dell'importo versato per l'ingaggio degli artisti, importo che varia a seconda dell'esibizione alla cui esecuzione l'artista partecipa e dall'evento in cui si svolgerà l'esibizione stessa.

Per gli iscritti all'associazione si conoscono nome, cognome, indirizzo e numeri di telefono (casa, ufficio ed eventualmente cellulare), indirizzo e-mail, data di iscrizione e tassa di iscrizione.

Gli iscritti si suddividono in due classi: i membri e i supporter. Questi ultimi partecipano all'organizzazione degli eventi che l'associazione pubblica. Tutti gli iscritti possono partecipare gratuitamente agli eventi mentre gli eventuali accompagnatori devono versare la quota di partecipazione prevista per l'evento (è necessario tenere traccia solo del numero di persone che accompagnano ciascun iscritto ad ogni evento).

##### Schema E-R



#### 4.5.1.2. Progettazione logica

##### Ristrutturazione dello schema

Nello schema non sono presenti ridondanze delle relazioni (non ci sono cicli nella relazioni) né degli attributi. Supponiamo anche di non dover eseguire operazioni di partizionamento o accorpamento per la semplicità dello schema. Gli identificatori primari sono già stati individuati durante la fase di progettazione concettuale. Occorre solamente eliminare le generalizzazioni: in questo schema c'è solamente una gerarchia di tipo totale ed esclusivo, e le entità figlie non hanno proprietà specifiche, se non una sola relazione a cui una delle figlie partecipa e l'altra no. Quindi è opportuno tradurre la gerarchia con l'eliminazione delle figlie, ovvero con un collasso verso l'alto, per poi specificare nuovamente la relazione e la sua cardinalità e aggiungere un selettore all'entità padre.

##### Schema logico

*EVENTO (Slogan, Città, Durata, Quota)*

*ESIBIZIONE (Codice, Tipo, Durata)*

*ARTISTA (Nome, Cognome, Manager, Telefono)*

*ENTE (Nome, Descrizione, Indirizzo)*

*ISCRITTO (Nome, Cognome, Indirizzo, Email, TelCasa, TelUfficio, TelCellulare, Data, Tassa, Selettore)*

*COMPORRE (SloganEvento, CodiceEsibizione)*

*ESEGUIRE (SloganEvento, CodiceEsibizione, NomeArtista, CognomeArtista, Ingaggio)*

*SOVVENZIONARE (SloganEvento, NomeEnte, Importo)*

*PARTECIPARE (SloganEvento, NomeIscritto, CognomeIscritto, NumAccompagnatori)*

*ORGANIZZARE (SloganEvento, NomeIscritto, CognomeIscritto)*

#### 4.5.2. Parte II

##### 4.5.2.1. Progettazione concettuale

##### Requisiti

La ditta *Sempre Vivo* (SV), specializzata nella realizzazione di manichini, busti vetrina, torsi ed accessori, vuole realizzare un'applicazione base di dati.

I prodotti SV sono principalmente manichini ed accessori. Il catalogo dell'azienda riporta, per ciascun prodotto, un codice, il materiale di cui è composto, le dimensioni ed il prezzo. Per gli accessori vengono inoltre riportati i colori disponibili e, nel caso di accessori venduti ma non prodotti da SV, il nome del produttore.

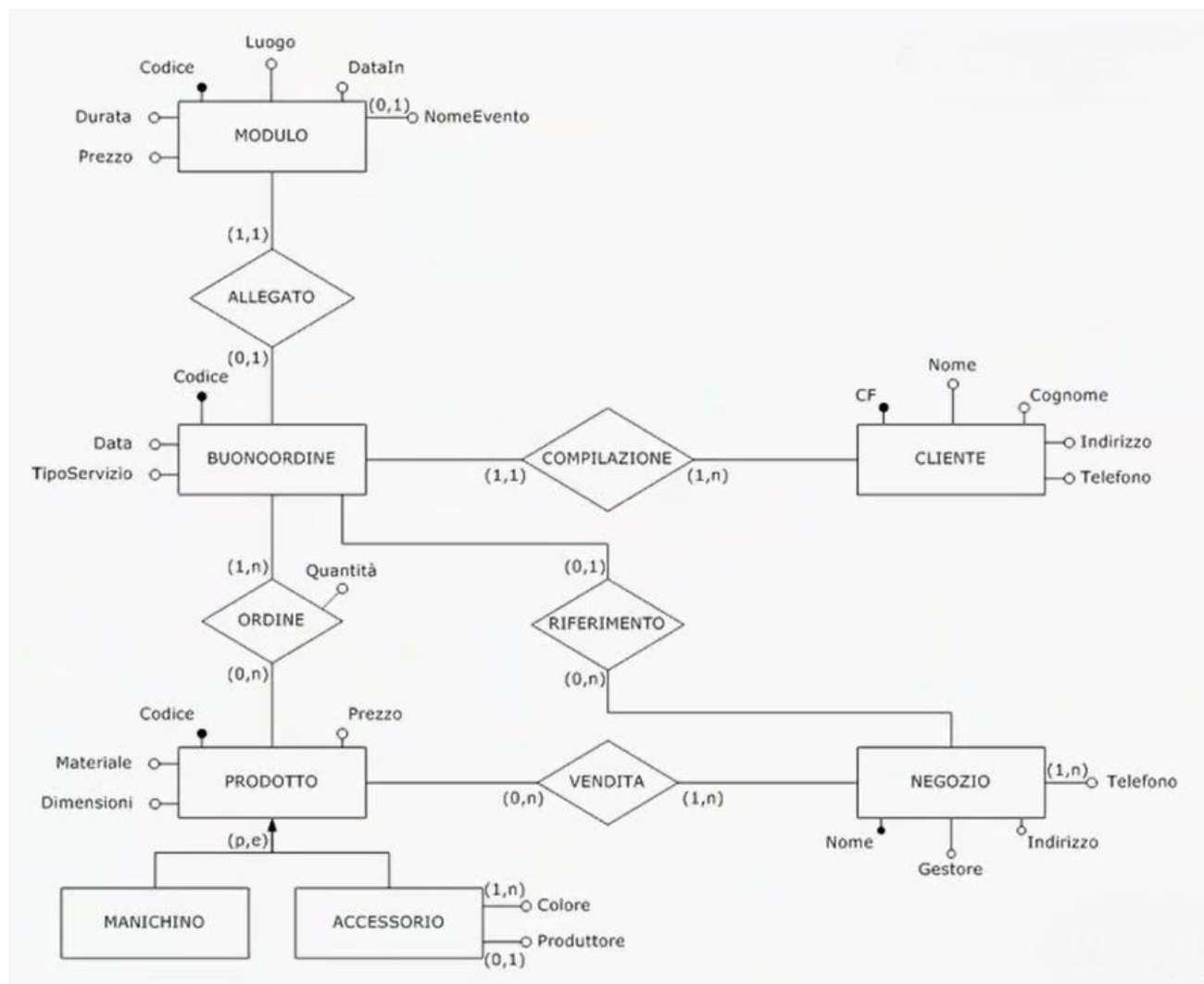
SV è inoltre proprietaria di un certo numero di negozi dislocati su tutto il territorio nazionale. Di ciascun negozio si conosce il nome, le informazioni di contatto (che comprendono il nome del gestore, l'indirizzo ed i numeri di telefono) ed i prodotti venduti.

L'azienda, oltre a vendere i propri prodotti tramite i negozi, fornisce anche direttamente un servizio di noleggio e allestimento per fiere, manifestazioni e vetrine.

I clienti dell'azienda o dei negozi (caratterizzati dalle usuali informazioni anagrafiche) devono compilare un buono d'ordine sul quale viene riportata la data di compilazione, l'eventuale codice del negozio presso cui viene compilato il buono, il tipo di servizio richiesto ed i prodotti richiesti con la relativa quantità.

Ad ogni buono d'ordine può essere associato un modulo sul quale si indicano il luogo presso il quale devono essere portati prodotti, la data di inizio del servizio e la sua durata, il prezzo totale del servizio ed eventualmente il nome dell'evento per cui si richiede il servizio.

## Schema E-R



#### 4.5.2.2. Progettazione logica

##### Ristrutturazione dello schema

Nello schema non ci sono attributi ridondanti, ma c'è un ciclo nelle relazioni, quindi occorre verificare che non ci sia una ridondanza provando ad eliminare una relazione senza che si perdano informazioni, tuttavia questo non è possibile, quindi non c'è ridondanza. Per eliminare la gerarchia, optiamo ancora per un collasso verso l'alto, specificando nuovamente gli attributi delle figlie e le loro cardinalità e aggiungendo un selettore al padre. Ora resta da risolvere il problema degli attributi multivalore, aggiungendo un'entità ed una relazione per per gli attributi *Colore* e *Telefono*.

##### Schema logico

*PRODOTTO* (Codice, *Materiale*, *Dimensioni*, *Prezzo*, *Produttore*, *Selettore*)

*NEGOZIO* (Nome, *Gestore*, *Indirizzo*)

*CLIENTE* (CF, *Nome*, *Cognome*, *Indirizzo*, *Telefono*)

*BUONOORDINE* (Codice, *Data*, *TipoServizio*, *CFCliente*, *NomeNegozio*)

*MODULO* (Codice, *Luogo*, *DataIn*, *Durata*, *Prezzo*, *NomeEvento*, *CodiceBuono*)

*COLORE* (CodiceProdotto, CodiceColore)

*TELEFONO* (NumTel, *NomeNegozio*)

*VENDITA* (CodiceProdotto, *NomeNegozio*)

*ORDINE* (CodiceBuono, CodiceProdotto, *Quantità*)

#### 4.5.3. Parte III

##### 4.5.3.1. Progettazione concettuale

##### Requisiti

I proprietari della cascina *La Piccola Luna* hanno deciso di progettare una base di dati in grado di gestire efficientemente tutte le informazioni relative alla cascina ed allo svolgimento delle loro attività.

La cascina dispone di più appezzamenti di terreno ognuno dei quali è suddiviso in più lotti. Ciascun appezzamento è caratterizzato da un nome, dal tipo di terreno e da una posizione geografica. Per i lotti, identificati da un codice univoco nell'ambito del relativo appezzamento, si conosce la dimensione, la data in cui si è effettuata l'ultima semina all'interno del lotto ed il numero medio di mesi che devono trascorrere prima di poter effettuare il raccolto.

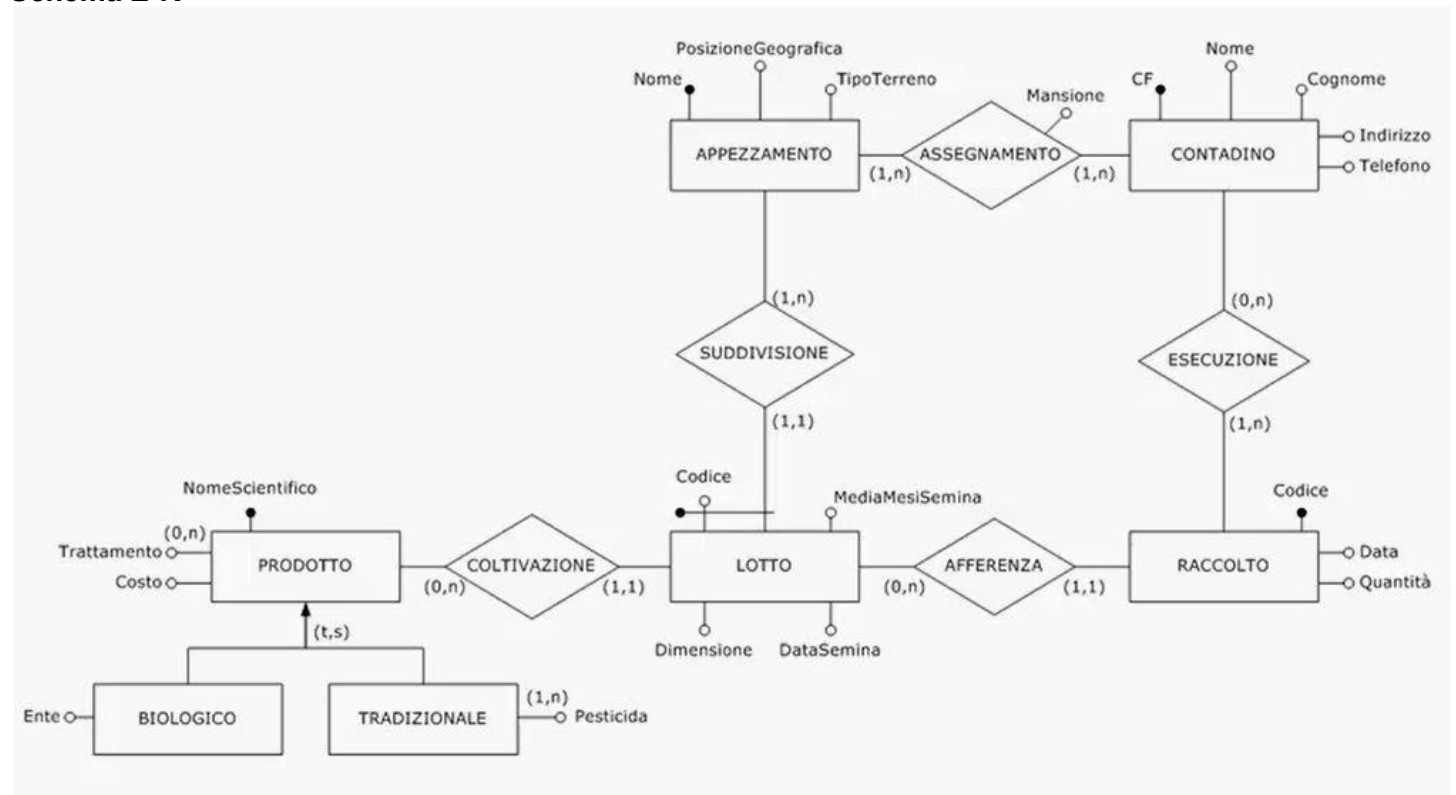
Ogni lotto viene sempre utilizzato per la coltivazione di un ben preciso prodotto per il quale è noto il nome scientifico, la lista di trattamenti a cui il prodotto deve essere sottoposto ed il costo (per chilo) delle relative sementi.

I prodotti coltivati presso la cascina si suddividono in due categorie principali: i prodotti biologici, per i quali non è ammesso l'uso dei pesticidi, ed i prodotti coltivati con tecniche tradizionali. Si osservi che queste due tipologie non sono mutuamente esclusive (per uno stesso prodotto è possibile la presenza contemporanea di coltivazioni biologiche e non). Per i prodotti biologici si deve tener traccia del nome dell'ente che attesterà la qualità del prodotto, mentre per quelli coltivati con tecniche tradizionali d deve specificare la lista dei pesticidi che possono essere utilizzati.

I contadini che lavorano nella cascina (per i quali si conoscono le usuali informazioni anagrafiche come nome, cognome, indirizzo e telefono) possono essere assegnati ad uno o più appezzamenti di terreno dove possono svolgere mansioni diverse.

Si vogliono inoltre rilevare tutte le informazioni relative ai singoli raccolti. Ogni raccolto è caratterizzato da una data, dalla quantità di prodotto raccolto, dai contadini che hanno effettuato il raccolto e il lotto coinvolto.

## Schema E-R



### 4.5.3.2. Progettazione logica

#### Ristrutturazione dello schema

In questo schema gli attributi non sono ridondanti, ma è presente un ciclo, quindi occorre verificare che non ci sia ridondanza: si tenta di rimuovere una relazione e si controlla che così facendo non si perdono informazioni (il che non è possibile, quindi non ci sono ridondanze). Occorre eliminare la gerarchia presente, che essendo sovrapposta non andrebbe risolta con l'eliminazione del padre, quindi anche questa volta utilizziamo un collasso verso l'alto. Infine, si rimuovono i due attributi multivalore, sostituendoli con un'entità e una relazione.

#### Schema logico

**APPEZZAMENTO** (Nome, TipoTerreno, PosizioneGeografica)

**CONTADINO** (CF, Nome, Cognome, Indirizzo, Telefono)

**LOTTO** (NomeAppezzamento, Codice, Dimensione, DataSemina, MediaMesiSemina, NomeProdotto)

**PRODOTTI** (NomeScientifico, Costo, Ente, Biologico, Tradizionale)

**RACCOLTO** (Codice, Data, Quantità, NomeAppezzamento, CodiceLotto)

**ASSEGNAZIONE** (NomeAppezzamento, CFContadino, Mansione)

**ESECUZIONE** (CFContadino, CodiceRaccolto)

**TRATTAMENTO** (NomeProdotto, CodiceTrattamento)

**PESTICIDA** (NomeProdotto, CodicePesticida)

## 5. Organizzazione fisica delle basi di dati

### 5.1. Moduli per accesso ai dati e gestione della memoria

#### Organizzazione fisica dei dati

Si è visto in precedenza come l'architettura della base di dati sia strutturata su più livelli, tra cui il **livello logico** (a cui fanno riferimento gli utenti e le operazioni SQL) e il **livello fisico** (su cui avvengono gli accessi in risposta alle traduzioni di definizioni e interrogazioni da parte del DBMS), che sono indipendenti tra loro (indipendenza fisica).

#### Moduli DBMS per l'accesso ai dati

L'esecuzione delle interrogazioni coinvolge diversi moduli in cui è strutturato il DBMS:

- gestore (o ottimizzatore) delle interrogazioni: traduce le interrogazioni in operazioni su strutture fisiche (in modo ottimizzato)
- gestore dei metodi d'accesso: traduce le operazioni su strutture fisiche in operazioni di lettura e scrittura su memoria secondaria
- gestore del buffer: media le operazioni sulla memoria secondaria mantenendo temporaneamente i dati in memoria principale
- gestore della memoria secondaria: esegue le operazioni di lettura e scrittura su memoria secondaria

#### Vantaggi e svantaggi della memoria secondaria

La memoria secondaria, a differenza di quella principale, è di grandi dimensioni e garantisce la persistenza dei dati. D'altro canto, la memoria secondaria non è direttamente utilizzabile dai programmi (i cui dati devono comunque essere trasferiti in memoria principale) ed è organizzata a blocchi di dimensione fissa, permettendo quindi solo operazioni a livello di un intero blocco (e non l'accesso ai singoli bit). Inoltre, la memoria secondaria ha un costo molto maggiore (in termini di tempo d'accesso ai dati) rispetto a quella principale, che dipende anche dall'ordine di esecuzione degli accessi (pos. Testina, latenza, accessi)

#### Buffer

Il buffer è una zona di memoria centrale gestita dal DBMS in modo condiviso con tutte le applicazioni che permette di evitare di ripetere accessi in memoria secondaria ed è organizzato in pagine pari ad un numero intero di blocchi. Il gestore del buffer si occupa del caricamento e scaricamento delle pagine tra la memoria principale e quella secondaria.

#### Gestore del buffer

Per gestire i dati in memoria principale e il loro caricamento e scaricamento in memoria secondaria, il gestore del buffer utilizza delle informazioni di gestione, costituite da:

- direttorio che descrive i contenuti del buffer e la corrispondenza tra pagine e blocchi fisici;
- variabili di stato per ogni pagina, che consistono a loro volta in:
  - contatore che indica il numero di applicazioni che utilizzano la pagina;
  - bit di stato che indicano se la pagina è stata modificata.

Il gestore del buffer, inoltre, ottimizza gli accessi ai dati, in quanto le letture e scritture sono effettuate su pagine del buffer e le scritture su memoria secondaria sono differite.

#### Primitive del gestore del buffer

Le operazioni primitive permesse dal gestore del buffer sono:

- fix: per caricare una pagina nel buffer, quando non presente (incrementa il contatore relativo all'utilizzo della pagina);
- unfix: per terminare l'utilizzo di una pagina (decrementa quindi il contatore di utilizzo);

- **setDirty**: per indicare che la pagina è stata modificata (va quindi a modificare il bit di stato);
- **force**: trasferisce in modo sincrono una *pagina del buffer* su memoria secondaria;
- **flush**: trasferisce in modo asincrono le *pagine non utilizzate* su memoria secondaria.

### Primitiva fix

Quando viene eseguita l'operazione *fix*:

- se la **pagina è nel buffer**, ne restituisce l'indirizzo;
- se la **pagina non è nel buffer**:
  - se **ci sono pagine libere**: legge la pagina da memoria secondaria e la carica nel buffer, restituendone poi l'indirizzo;
  - se **non ci sono pagine libere**: seleziona una pagina vittima e la **scrive in memoria secondaria(flush)** per poi leggere la pagina richiesta e caricarla nel buffer, restituendone poi l'indirizzo.

Per quanto riguarda la **scelta della pagina vittima**, esistono due politiche possibili:

- **steal(rom)**: permette al gestore del buffer di **scegliere come vittima una pagina correntemente utilizzata** (mettendo quindi **in attesa la transazione che la stava utilizzando**);
- **no-steal**: **esclude la possibilità permessa dalla politica *steal*** ed è generalmente utilizzata dai DBMS commerciali.

### Politiche del gestore del buffer

Per scegliere **quali pagine mantenere in memoria e quali scaricare** in caso sia necessario fare spazio, il gestore del buffer utilizza delle **politiche di gestione** molto simili a quelle usate dai sistemi operativi:

- **principio di località**: secondo cui i dati referenziati di recente hanno **maggiore probabilità** di essere referenziati in futuro;
- **legge empirica**: secondo cui il 20% dei dati è tipicamente utilizzato dall'80% delle applicazioni.

### DBMS e filesystem

Il **DBMS opera al di sopra del sistema operativo**, che offre il filesystem per organizzare e gestire l'accesso ai dati, ma la separazione tra i due non è sempre netta, in quanto il **DBMS replica alcune funzionalità del SO** con lo scopo di **garantire maggiore efficienza e affidabilità**. Il DBMS infatti:

- utilizza il SO per creare e cancellare file e per eseguire operazioni sui blocchi;
- crea una propria astrazione dei file per garantire efficienza e transazionalità;
- gestisce i blocchi dei file allocati come se fossero un unico grande spazio di memoria e vi costruisce le proprie strutture fisiche;
- fornisce metodi di accesso (moduli software) per accedere alle diverse strutture.

## 5.2. Organizzazione di file

L'organizzazione fisica regola la **disposizione delle tuple all'interno del file**. Il DBMS fa riferimento tre tipi di strutture fisiche, per ognuna delle quali ci sono dei metodi per accedere alle tuple:

- **sequenziale**: tuple disposte in sequenza;
- **accesso calcolato (hash-based)**: posizione delle tuple determinata da un algoritmo di calcolo;
- **a indici (index-based)**: organizzazione ad albero.

## 5.2.1. Strutture sequenziali

Esistono tre possibili organizzazioni:

- **seriale** (entry-sequenced / heap): la sequenza delle tuple è indotta dal loro **ordine di inserimento**;
- **ad array**: le tuple sono **disposte in un array** e la loro posizione dipende dal **valore di uno o più campi indice**;
- **sequenziale ordinata**: la **sequenza delle tuple dipende dal valore**, in *ciascuna tupla*, di un **campo di ordinamento** (detto **campo chiave**, che **non è necessariamente la chiave primaria**).

### Struttura seriale

**Vantaggi**: ottimale per le operazioni di scrittura (che avviene alla fine del file) e lettura sequenziale. **Svantaggi**:

- le cancellazioni possono lasciare spazio inutilizzato
- le modifiche che creano aumento delle dimensioni della tupla **non** possono essere **gestite "in loco"**
- l'accesso ai singoli dati richiede la scansione dell'intero file

### Struttura sequenziale ad array

Le tuple sono **disposte in un array** e la loro **posizione dipende dal valore di uno o più campi indice**.

L'array è composto da  $n$  blocchi contigui, ognuno di  $m$  slot disponibili per le tuple, per un totale di  $n \cdot m$ . Ogni tupla ha un indice  $i$  ed è **posizionata nell' $i$ -esima cella dell'array**.

Il **caricamento iniziale** avviene utilizzando gli indici ottenuti incrementando un contatore, mentre per l'accesso basato sull'indice si hanno diverse primitive:

- **lettura della tupla** con un dato indice (**read-in**);
- **modifica della tupla** con un dato indice (**update-in**);
- **cancellazione della tupla** con un dato indice (**delete-in**);
- **inserimento della tupla**:
  - in un dato indice (**insert-at**);
  - nella posizione libera più vicina (**insert-near**);
  - alla fine dell'array (**insert-at-end**).

**Vantaggi**: accesso **diretto** (sulla base della chiave);

**Svantaggi**:

- possibile solo quando le tuple hanno **lunghezza fissa**;
- le **cancellazioni lasciano celle vuote**;
- si può applicare solo quando la **chiave assume valori consecutivi**.

Questa struttura è **raramente utilizzata dai DBMS** a causa dei vincoli di applicabilità.

### Struttura sequenziale ordinata

**Vantaggi**: le **operazioni** basate sul **valore della chiave** sono **efficienti**;

**Svantaggi**: gli **inserimenti e le modifiche** richiedono la **riorganizzazione delle tuple** già presenti.

Questo tipo di strutture sono state storicamente utilizzate su dispositivi sequenziali a nastro: i **dati erano posizionati in un file principale**, le modifiche erano raccolte in file differenziali e i file venivano periodicamente fusi tra loro con un'operazione di **merge**.

Oggi i **merge periodici sono inaccettabili per i DBMS**, perciò, per evitare di dover riordinare le tuple, si può:

- lasciare un certo **numero di slot liberi** al tempo del caricamento (permettendo un riordinamento locale);
- inserire **nuovi blocchi a seguito delle saturazioni** (con lo svantaggio di perdere la continuità dei blocchi);
- integrare il **file ordinato con un file di overflow** (per cui i blocchi dei file di overflow sono collegati a formare una **catena di overflow** e le **ricerche sequenziali** devono essere **alternate ad un'analisi dei blocchi di overflow**).

## 5.2.2. Strutture hash based

La posizione delle tuple è determinata da un algoritmo di calcolo applicato alla chiave. Questa struttura si basa sull'allocazione di  $B$  bucket (blocchi) spesso adiacenti al file; la locazione fisica di una tupla dipende da un **campo chiave** (composto da un numero arbitrario di attributi di una tabella) e il metodo di accesso utilizza un algoritmo di hash che, applicato alla chiave, ritorna un valore compreso tra  $0$  e  $B - 1$ .

### Funzione di hash

L'implementazione dell'algoritmo consiste in due parti:

- **folding**: trasforma il **valore chiave in un intero positivo**, distribuito uniformemente su un intervallo;
- **hashing**: trasforma l'intero positivo in un numero compreso tra  $0$  e  $B - 1$ .

Questo algoritmo **funziona meglio quando il file è più grande del necessario** (se  $T$  è il numero di tuple attese ed  $F$  è il numero medio di tuple per pagina, una **buona scelta di  $B$  è  $T / (0.8 \cdot F)$**  (**si utilizza così solo l'80% dello spazio disponibile**)).

### Problema delle collisioni

Essendo la **funzione di hash non iniettiva**, c'è il rischio che si **verifichino collisioni**, che possono essere risolte **internamente** (**eseguendo nuovamente l'hashing della chiave** e trovando così un bucket alternativo) o **esternamente** (aggiungendo ad un bucket saturo una **catena di overflow**).

Quest'ultima soluzione **aggiunge il costo computazionale di dover scandire la catena per la ricerca di una chiave**, e la lunghezza media della catena di overflow aumenta con l'aumentare del coefficiente di riempimento del file e diminuisce con l'aumentare della dimensione del bucket.

### Vantaggi e svantaggi

Le strutture hash-based hanno il **vantaggio** di assicurare un **accesso associativo** ed **efficiente** ai dati, ma comportano alcuni **svantaggi** come:

- problemi di **collisione**;
- necessità di **ristrutturazione in caso di modifica della dimensione del file**;
- **ottimizzazione solo degli accessi basati sul campo chiave** (per gli altri accessi si opera ancora una scansione sequenziale);
- **inefficienza delle operazioni** su intervalli di valori.

## 5.2.3. Gestione delle tuple nelle pagine

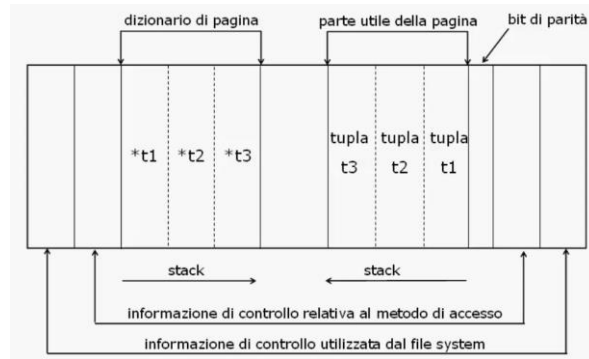
Ogni **pagina sequenziale o hash-based** ha (assumendo un blocco per pagina):

- una **parte iniziale (block header)** e una **parte finale (block trailer)** contenenti l'informazione di controllo usata dal filesystem;
- una **parte iniziale (page header)** e una **parte finale (page trailer)** contenenti l'informazione di controllo del metodo di accesso.

Oltre a queste informazioni di controllo, una pagina ha anche:

- un **dizionario di pagina** che contiene i **puntatori ad ogni dato elementare** contenuto nella pagina;
- una **parte utile** che contiene i **dati** (generalmente il dizionario e la parte utile crescono come stack contrapposti);

- un **bit di parità** per verificare l'integrità dell'informazione contenuta nella pagina.



## Primitive

Le primitive offerte dal gestore delle pagine sono le seguenti:

- **inserimento e aggiornamento di una tupla** (può richiedere una **riorganizzazione della pagina** se **non c'è spazio contiguo** sufficiente per gestire i byte aggiuntivi introdotti);
- **cancellazione di una tupla** (spesso implementata marcando la tupla come invalida);
- **accesso ad una specifica tupla** (in base alla chiave in caso di accesso associativo, o in base all'offset);
- **accesso ad un campo di una specifica tupla** (identificato in base all'offset e alla lunghezza del campo stesso, dopo aver identificato la tupla).

## 5.2.4. Strutture ad albero

In questa struttura, **ogni nodo dell'albero coincide con una pagina o blocco** a livello di filesystem o di gestore del buffer. Le **relazioni tra i nodi** sono stabilite da **puntatori** e per ogni albero può essere specificato un **fan-out**  $F$ , ovvero un **numero massimo di puntatori** (discendenti) ammessi **per ogni nodo** dell'albero (per gli **alberi binari**  $F = 2$ ).

Per quanto riguarda gli alberi paginati, si distingue tra **alberi B** e **alberi B+**.

### Alberi B

Ogni loro nodo contiene:

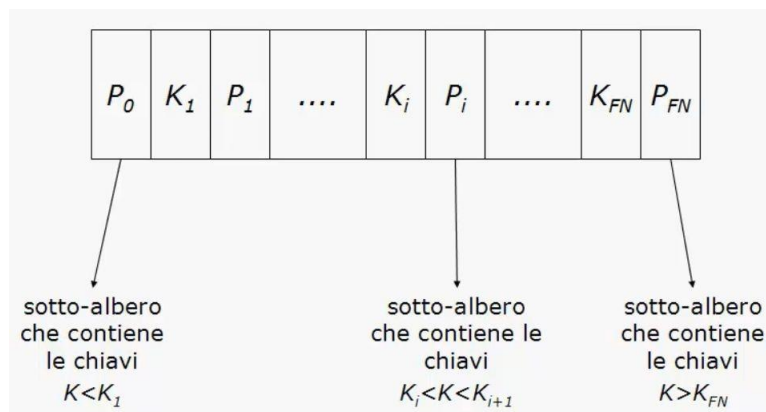
- $F_N \leq F - 1$  **valori** (ordinati secondo la chiave);
- $F_N + 1$  **puntatori**;

Ogni valore con chiave  $K_j$ , dove  $1 \leq j \leq F_N$ , è **seguito da un puntatore**  $P_j$

Per quanto riguarda i puntatori nei nodi non foglia:

- $P_0$  indirizza il sottoalbero con **chiavi minori di**  $K_1$ ;
- $P_{F_N}$  indirizza il sottoalbero con **chiavi maggiori di**  $K_{F_N}$ ;
- $P_j < K < K_{j+1}$  indirizza il sottoalbero con **chiavi**  $K$ , con  $K_j < K < K_{j+1}$ .

I **nodi foglia**, invece, sono tutti allo **stesso livello** e hanno **puntatore a null**.

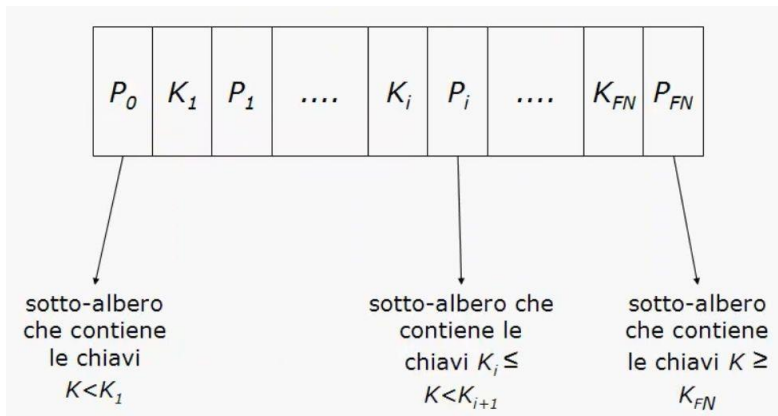


## Alberi B+

Questi alberi sono come i precedenti, con la differenza che per ogni nodo  $N$ :

- $P_{FN}$  indirizza il sottoalbero con **chiavi maggiori o uguali di  $K_{FN}$** ;
- $P_j$ , dove  $0 < j < F_N$ , indirizza il sottoalbero con **chiave  $K$ , con  $K_j \leq K < K_{j+1}$** .

Quindi, mentre negli alberi B possono essere o nei nodi intermedi o nelle foglie, **negli alberi B+ tutti i valorisono nelle foglie**; inoltre, i **nodi foglia sono collegati tra loro** da una catena sulla base della chiave.



## Valori dei nodi

Per memorizzare la tupla vera e propria, il **valore dei nodi può essere composto** da:

- **chiave e tupla** (indice primario): insieme ad ogni chiave è memorizzata la tupla associata alla chiave (key-sequenced);
- **chiave e puntatore** (indice secondario): insieme alla chiave è memorizzato un **puntatore al blocco della base di dati che contiene la tupla associata alla chiave**; le tuple possono essere allocate per mezzo di altri meccanismi primari (come la struttura sequenziale o hash-based).

## Utilizzo degli alberi B e B+

Questi alberi sono spesso **utilizzati come indice secondario** (indicizzando quindi le **informazioni già organizzate in un file**, anche con l'utilizzo di altre strutture). Forniscono un **modo efficiente di accedere alle tuple in dipendenza di uno o più chiavi** e ogni file può avere un solo indice primario e più indici secondari. Gli indici, anche primari, non possono essere implementati con struttura sequenziale o hash (dove la posizione non dipende dall'indice).

## Operazioni sugli alberi B+

Gli **alberi B+ sono quelli di maggior interesse per le basi di dati**, in quanto permettono di **percorrere in ordine le chiavi** e di eseguire efficientemente le interrogazioni su intervalli di valori.

Le operazioni possibili su questi alberi sono:

- **ricerca** di un certo valore chiave;
- **inserimento** di un valore chiave;
- **cancellazione** di un valore chiave;
- **modifica** di un valore chiave (cancellazione seguita da inserimento).

## Operazione di ricerca

La **ricerca** di un certo valore  $V$  avviene:

- partendo dalla **radice**, ad ogni nodo non foglia  $N$ :
  - se  $V < K_1$  si segue il puntatore  $P_0$ ;
  - se  $V \geq K_{FN}$  si segue il puntatore  $P_{FN}$ ;
  - altrimenti si segue il puntatore  $P_j$

se si arriva ad un nodo foglia: Tale che  $K_j \leq V < K_{j+1}$  ;

- se l'indice è primario si leggono le tuple;
- se l'indice è secondario si segue il puntatore alle tuple.

Gli alberi B+ **ottimizzano l'accesso a intervalli**, in quanto **basta trovare il primo valore dell'intervallo** per poi percorrere la catena tra le foglie

### Operazione di inserimento

L'inserimento di un certo valore  $V$  avviene:

- partendo dalla radice, si procede cercando  $V$  ;
- se si arriva ad un nodo foglia  $N$  :
  - se **non è pieno** si **inserisce il valore nella foglia**;
  - se **è pieno** (quindi ha un numero di **valori uguale al fan-out dell'albero meno uno**), occorre eseguire un'operazione detta **split and promotion**.

### Split and promotion

Questa operazione consiste in:

- supporre di **inserire il valore** e di **ordinare i valori** (incluso quello nuovo);
- sia  $K_i$  la chiave in posizione  $\lceil F/2 \rceil$

si crea un nodo  $N'$  e si **trasportano i valori con  $K > K_i$**  (o  $K \geq N$  se è foglia) in  $N'$

- si **inserisce il valore  $K_i$**  seguito dal **puntatore a  $N'$**  nel padre di  $N$ 
  - se il **padre di  $N$  è pieno**, si **ripete l'operazione di split and promotion** (che **può propagarsi fine alla radice**, aumentando in tal caso **l'altezza dell'albero**).

### Operazione di cancellamento

La **cancellazione** di un certo valore  $V$  avviene:

- cercando  $V$  fino ad arrivare ad un nodo foglia  $N$
- cancellando  $V$  :

◦ se  $V$  era il primo valore ( $K_1$ ) si sostituisce il nuovo  $K_1$  (ex  $K_2$ ) a  $V$  nel suo ascendente **se  $N$  è troppo vuoto** (ad esempio ha meno di  $\lceil F/2 \rceil$  valori):

- se possibile, si **bilancia** con un nodo adiacente;
- altrimenti si esegue un **merge**.

Il motivo per cui è necessario **impedire che i nodi siano troppo vuoti**, è che al caso limite si arriverebbe ad **alberi con un solo valore per nodo**, **perdendo così tutti i vantaggi degli alberi paginati**.

### Merge

Questa operazione consiste in:

- **unire  $N$  ad un nodo adiacente  $N'$**  creando un **unico nodo** (con i valori di entrambi più il valore contenuto nel padre che separa i puntatori dei due nodi);
- **cancellare il valore** (e puntatore alla sua destra) che separa i puntatori dei due nodi nel padre;
- se il **padre è troppo vuoto**, si esegue **un'operazione di bilanciamento o di merge** (che può anche in questo caso propagarsi fino alla radice, diminuendo in tal caso l'altezza dell'albero)

### Vantaggi degli alberi B+

I **vantaggi** forniti da questi alberi, che vengono infatti usati frequentemente dai DBMS relazioni, consistono in:

- fornire **accesso associativo** (basato sul valore di una **chiave**) senza restrizioni sulla posizione fisica delle tuple
- essere **efficienti** anche nelle **interrogazioni di intervallo**;

- consentire di **mantenere l'albero ben bilanciato** attraverso l'utilizzo attento delle operazioni di **split** e **merge** (se necessario, si può ri-bilanciare l'albero in modo esplicito)

### 5.3. Progettazione fisica e gestione delle interrogazioni

La fase della **progettazione fisica** prende in **input lo schema logico della base di dati**, le caratteristiche del DBMS scelto, le previsioni sul carico applicativo e altre informazioni (come, ad esempio, le dimensioni iniziali dei file e la loro possibilità di espansione). Il **risultato** di questa fase è lo **schema fisico della base di dati**, che descrive le **strutture fisiche di accesso** con i relativi parametri e nel quale è particolarmente importante l'individuazione degli **indici** (che permettono di ottimizzare l'accesso ai dati).

Gli **indici sono definiti sugli attributi delle tabelle** e si traducono in **speciali strutture di accesso** che garantiscono **accesso diretto** e quindi **ottimizzano l'accesso** agli attributi su cui sono definiti.

Perciò la **progettazione fisica** deve **identificare gli indici** da definire su ogni relazione:

- **indice primario**: tipicamente uno con struttura key-sequenced, che fornisce ricerca sequenziale (generalmente definiti su attributi *unique* come la chiave primaria della tabella);
- **indici secondari**: tipicamente **sono molti** e sono definiti con **alberi B+** (quindi con accesso indiretto):
  - su attributi spesso **coinvolti in predicati** delle interrogazioni;
  - su attributi per i quali spesso è richiesto **ordinamento in uscita**.

#### Individuazione degli indici

L'individuazione degli indici è spesso fatta in modo empirico con una fase di regolazione (detta tuning) che può **introdurre nuovi indici rispetto a quelli già selezionati** per migliorare le prestazioni, oppure **rimuoverli**. È utile controllare come gli indici sono utilizzati dalle query utilizzando il comando `show plan`.

#### Definizione degli indici in SQL

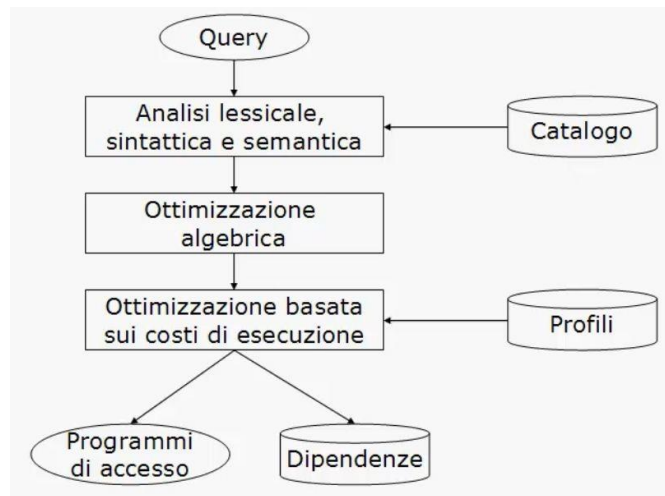
I comandi nei sistemi relazionali per la creazione e la cancellazione di indici non sono parte dello standard SQL ma la loro **sintassi è abbastanza simile** in tutti i DBMS:

- **creazione**: `create [unique] index IndexName on TableName (AttributeList) ;`
- **eliminazione**: `drop index IndexName .`

#### Gestione delle interrogazioni

Il DBMS ha il compito di **tradurre le query SQL in operazioni sulle strutture dati** sottostanti; perciò, il modulo di gestione delle interrogazioni riceve in **input** un'interrogazione e produce in **output** un programma di accesso informato interno, passando attraverso le seguenti fasi:

- **analisi lessicale, sintattica e semantica** per verificare che la query sia ben formata e soddisfi i vincoli;
- **traduzione in forma algebrica** interna e operazioni di **ottimizzazione algebrica**;
- **ottimizzazione** basata sui **costi di esecuzione** (che dipende dai metodi di accesso e dai profili delle tabelle);
- **traduzione** delle operazioni in termini dei metodi di accesso forniti dal DBMS.



### Profili delle tabelle

Sono **informazioni quantitative** sulle tabelle quali:

- cardinalità di ogni tabella;
- dimensione in byte di ogni tupla;
- dimensione in byte di ogni attributo;
- numero di valori distinti di ogni attributo;
- valore minimo e massimo di ogni attributo.

Queste informazioni sono **calcolate periodicamente** attivando appropriate primitive di sistema e sono **memorizzate nel dizionario dei dati**.

### Compilazione delle interrogazioni

Esistono due approcci possibili:

- **compile-and-store**: l'interrogazione è compilata una volta e riutilizzata più volte (il codice interno è memorizzato nella base di dati, insieme ad una indicazione delle dipendenza del codice sulle specifiche versioni delle tabelle e degli indici della base di dati, quando queste vengono modificate la compilazione è invalidata e ripetuta);
- **compile-and-go**: l'esecuzione è immediata e l'interrogazione compilata non viene memorizzata.

## 6. Gestione delle transazioni

### 6.1. Transazioni

La transazione rappresenta un'**unità elementare di lavoro** svolta da un'applicazione, alla quale si associano particolari caratteristiche di **correttezza**, **robustezza** e **isolamento** che devono essere garantite. Viene detto **sistema transazionale** un sistema che mette a disposizione un **meccanismo per la definizione e l'esecuzione di transazioni** da parte di più applicazioni anche concorrenti.

## Sintassi della transazione

Da un punto di vista sintattico, la transazione è una procedura racchiusa da due comandi:

- begin transaction (bot);
- end transaction (eot).

All'interno di una transazione, uno dei seguenti due comandi è eseguito esattamente una volta:

- commit work (commit): per **terminare con successo**;
- rollback work (abort): per fare un **undo** della procedura.

Per essere **ben formata**, inoltre, una transazione **non contiene operazioni di accesso successive all'esecuzione** del comando di *commit* o di *rollback*.

```
begin transaction
  x := x-10
  y := y+10
  z := z-y
  if z < 50
    then commit work
    else rollback
end transaction
```

```
begin transaction
  update ContoCorrente
  set Saldo = Saldo - 100
  where NumConto = '123456'
  update ContoCorrente
  set Saldo = Saldo + 100
  where NumConto = '123457'
  commit work
end transaction
```

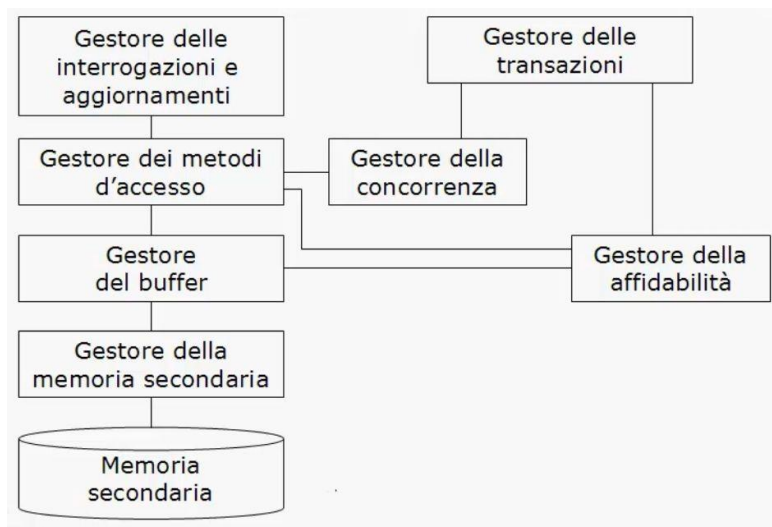
## Proprietà delle transazioni

Le **transazioni devono soddisfare le proprietà ACID**, che sono controllate e garantite dai moduli del DBMS:

- **Atomicità** (garantita dal **gestore dell'affidabilità**): una transazione è un'**unità atomica di lavoro**; perciò, **non può lasciare la base di dati in uno stato intermedio** (un guasto o errore che avviene **prima del commit** causa un **UNDO**, mentre un guasto/err **dopo il commit** può richiedere un **REDO**);
- **Consistenza** (garantita dal **compilatore del DDL**): l'esecuzione di una transazione **non deve violare i vincoli di integrità definiti sulla base di dati**, e il controllo sul mantenimento della consistenza può essere:
  - **immediato**: durante la transazione (**l'operazione che causa la violazione è rifiutata**);
  - **differito**: alla fine della transazione (**se dei vincoli sono violati, l'intera transazione è rifiutata**; questa tecnica **permette di avere transazioni i cui stati intermedi violano i vincoli**, ma sono comunque **accettate se alla fine li rispettano**);
- **Isolamento** (garantita dal **gestore della concorrenza**): l'esecuzione di una transazione deve essere **indipendente da quella di tutte le altre transazioni concorrenti** (in altre parole, **l'esecuzione concorrente di un insieme di transazione** deve portare allo **stesso risultato di un'esecuzione sequenziale** arbitraria delle stesse transazioni).
- **Durabilità** (garantita dal **gestore dell'affidabilità**): gli **effetti** di una transazione che ha eseguito il commit **non devono essere persi** (il **sistema deve garantire la persistenza dei dati** anche in caso di malfunzionamenti e guasti).

## Ruolo dei moduli di sistema nelle transazioni

- Il gestore delle **transazioni coordina** le attività connesse con le transazioni attraverso l'esecuzione delle operazioni di **begin**, **commit**, e **rollback**;
- il gestore dell'**affidabilità** garantisce **atomicità e persistenza** interagendo con:
  - il gestore dei **metodi di accesso** (per tenere traccia delle operazioni richieste);
  - il gestore del **buffer** (per richiedere, se necessario, scritture sui dischi);
- il gestore della **concorrenza** garantisce l'**isolamento** filtrando ed eventualmente **pianificando gli accessi** (richiesti dal gestore degli accessi).



## 6.2. Controllo di affidabilità

### 6.2.1. Gestore dell'affidabilità e organizzazione del log

#### Gestore dell'affidabilità

Il gestore dell'affidabilità assicura **atomicità** e **durabilità** della transazione ed è il modulo responsabile per:

- l'esecuzione dei **comandi transazionali**:
  - `begin transaction (B);`
  - `commit (C);`
  - `rollback (o abort, A);`
- **ripristino** dopo i malfunzionamenti:
  - ripresa a caldo;
  - ripresa a freddo.

#### Memoria stabile

Il corretto funzionamento del gestore dell'affidabilità è basato sull'utilizzo della cosiddetta memoria stabile. Questa **memoria resistente ai guasti** è un'**astrazione**: **nessuna memoria può avere probabilità nulle di guasto**, ma la replicazione delle informazioni e i protocolli di scrittura robusti possono avvicinarsi a questa probabilità. La memoria stabile è fisicamente organizzata in modi diversi a seconda delle criticità dell'applicazione e un suo guasto è considerato impossibile.

#### Log

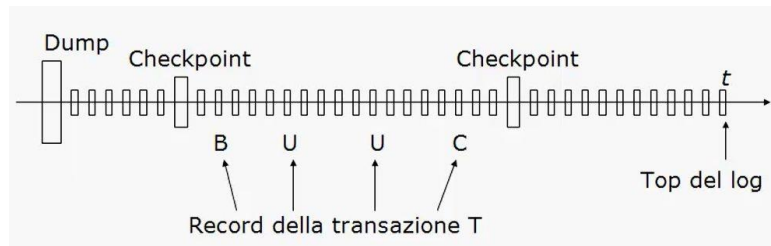
Per poter operare, il gestore dell'affidabilità mantiene un **log**, ovvero un **file sequenziale che registra**, in ordine cronologico, le **azioni seguite dalle varie transazioni**. È scritto su **memoria stabile** (archivio permanente) e rende **possibili le operazioni di undo e di redo**.

#### Organizzazione del log

Nel log ci sono due tipi di record possibili: (T → transazione, O → oggetto)

- record di **transazione**:
  - `begin : B(T) ;`
  - `insert : I(T, O, AS) ;`
  - `delete : D(T, O, BS) ;`
  - `update : U(T, O, BS, AS) ;`
  - `commit : C(T) ;`
  - `abort : A(T) ;`

- record di **sistema** (azioni compiute dal gestore dell'affidabilità):
  - dump (raro);
  - checkpoint (più frequente).



I record di transazione contengono, per le **operazioni di insert, delete e update**, le informazioni:

- before state (BS)**, ovvero lo stato dell'oggetto **prima** dell'operazione;
- after state (AS)**, ovvero lo stato dell'oggetto **dopo** l'operazione.

Proprio **queste due informazioni** rendono possibili le operazioni di **undo** e di **redo** delle operazioni.

### Operazione di **undo** (disfare azioni perché operazione non è andata a buon fine)

Per disfare un'azione:

- update:  $\text{undo}(U(T, O, BS, AS))$  copia il valore *BS* nell'oggetto *O*;
- delete:  $\text{undo}(D(T, O, BS))$  ripristina l'oggetto *O* con valore *BS*;
- insert:  $\text{undo}(I(T, O, AS))$  cancella l'oggetto *O*.

### Operazione di **redo** (rifare azione per ottenere il risultato voluto???)

Per rifare un'azione:

- update:  $\text{redo}(U(T, O, BS, AS))$  copia il valore *AS* nell'oggetto *O*;
- delete:  $\text{redo}(D(T, O, BS))$  cancella l'oggetto *O*;
- insert:  $\text{redo}(I(T, O, AS))$  ripristina l'oggetto *O* con valore *AS*.

### Proprietà di idempotenza

Le operazioni di *undo* e di *redo* godono della **proprietà di idempotenza**, in quanto un numero arbitrario di operazioni *undo/redo* della stessa azione è equivalente all'esecuzione dell'operazione **una sola volta**.

L'idempotenza garantisce quindi la correttezza anche in caso di ripetizione delle operazioni.

### Checkpoint

È un'operazione di sistema svolta dal gestore dell'affidabilità con il coordinamento del gestore del buffer, che **viene eseguita periodicamente e registra sul log le transazioni attive, aggiorna la memoria secondaria rispetto a tutte le transazioni completate**.

L'esecuzione dell'operazione di checkpoint avviene come segue:

- si **sospende** l'accettazione di operazioni di **scrittura, commit o abort** da parte delle transazioni;
- si trasferiscono (**flush**) su memoria secondaria tutte le **pagine "sporche" del buffer** relative a transazioni che hanno già effettuato il commit;
- si scrive in modo sincrono (**force**) nel log un **record di checkpoint**  $CK(T_1, \dots, T_n)$  che contiene gli **identificatori di tutte le transazioni attive**;
- si **riprende** l'accettazione delle operazioni da parte delle transazioni.

### Dump

È una **copia completa della base di dati memorizzata su memoria stabile (backup)** ed è tipicamente effettuata su nastro quando il sistema non è operativo (quindi in **mutua esclusione** con tutte le altre transazioni). Alla conclusione del dump viene **scritto nel log un record di dump** che segnala la creazione di un backup ad un certo istante di tempo e che identifica la copia.

## Scrittura dei record di log

Deve obbedire a due regole:

- **write ahead log (WAL)**, richiede che la parte **before state** dei record di log sia **scritta nel log prima di effettuare la corrispondente operazione** sulla base di dati: ciò **consente l'undo** delle scritture effettuate da transazioni che non hanno fatto commit e il ripristino in caso di guasto dopo aver effettuato un'operazione sulla base di dati (se il log non venisse scritto prima, infatti, il **valore precedente verrebbe perso**);
- **commit-precedenza**, richiede che la parte **after state** dei record di log sia **scritta nel log prima di effettuare il commit**: ciò **consente il redo** delle scritture già decise da transazioni che hanno fatto commit, ma le cui pagine modificate non sono ancora state scritte dal gestore del buffer su memoria secondaria.

## Record di commit

Viene **scritto in modo sincrono (force) nel record di log dalle transazioni che scelgono di terminare con successo**, ciò permette:

- in caso di **guasto prima del commit**, di fare un **undo delle azioni effettuate** e ripristinare lo stato iniziale della base di dati;
- in caso di **guasto dopo il commit**, di fare un **redo delle azioni effettuate** per ricostruire lo stato finale della transazione.

## Record di abort

Definisce la **scelta di abortire**, prodotta dal sistema o dalle transazioni, **anche se in realtà non incide sulle decisioni del gestore dell'affidabilità** (in quanto questo considera come **andate a buon fine solo le transazioni che hanno fatto un commit**). Perciò **questo record è meno critico rispetto a quello di commit**, ed è quindi scritto in modo asincrono nel buffer che contiene il blocco corrente del log (e viene successivamente riscritto sul log in modo asincrono, con un flush).

## Scrittura congiunta

Si distinguono tre schemi, **a seconda che le modifiche sulla base di dati da parte di una transazione siano effettuate sulla memoria secondaria**:

- **prima del commit**: **non richiede** operazioni di **redo**;
- **dopo il commit**: **non richiede** operazioni di **undo**;
- **alcune prima ed alcune dopo il commit**: richiede sia **operazioni di redo** sia di **undo** ed è quello più comunemente utilizzato perché consente al gestore del buffer di **ottimizzare le operazioni di flush indipendentemente dal gestore dell'affidabilità**.

## 6.2.3. Gestione dei guasti

I guasti possono essere classificati in due categorie:

- **guasti di sistema**: dovuti a **bug del software** o ad interruzioni del funzionamento dei dispositivi (**SW**)
- **guasti di dispositivo**: ovvero **guasti ai dispositivi di gestione della memoria di massa (HW)**

### Guasti di sistema

In questo caso, il **contenuto della memoria centrale viene perso** (e così tutti i buffer) ma il contenuto della **memoria secondaria no**. In questo caso, quindi, il gestore dell'affidabilità può ripristinare la situazione attraverso la cosiddetta **ripresa a caldo**.

### Guasto di dispositivo

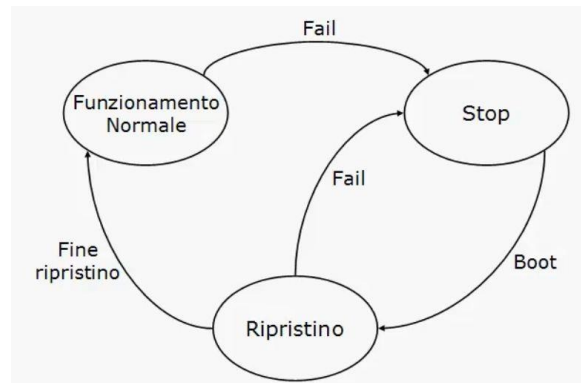
Questi **guasti sono più dannosi**, in quanto anche in questo caso il **contenuto della memoria centrale viene perso** (e così tutti i buffer) ma **anche il contenuto della memoria secondaria** viene perso; tuttavia, il **contenuto della memoria stabile è ancora presente** (astrazione, per definizione). In questo caso, quindi, il gestore dell'affidabilità può ripristinare la situazione attraverso la cosiddetta **ripresa a freddo**.

## Gestione dei guasti

Il gestore dell'affidabilità gestisce i guasti attenendosi al **modello fail-stop**, il quale prevede che, **se il sistema individua un guasto**:

- si forza l'arresto completo delle transazioni;
- si forza il corretto funzionamento del sistema operativo (boot);
- si effettua una ripresa a caldo o freddo a seconda del tipo di guasto.

Alla fine della ripresa, il sistema è nuovamente utilizzabile (ovviamente i buffer sono vuoti).



## Classificazione di transazioni

Rispetto ad un guasto del quale il gestore dell'affidabilità deve fare un ripristino, si individuano **due classi di transazioni**:

- **committed** (le cui azioni devono essere eseguite nuovamente);
- **non committed** (le cui azioni devono essere disfatte)

## Ripresa a caldo

La ripresa a caldo è composta da due fasi:

### 1. determinare l'insieme delle transazioni da rifare e da disfare:

- percorrendo il log all'indietro fino al record di **checkpoint più recente**;
- inizializzando gli insiemi di UNDO e di REDO:
  - $UNDO := \text{transazioni nel checkpoint}$  //le transazioni attive (nel CK) le metto nell'UNDO
  - $REDO := \emptyset$ ;
- percorrendo il log in avanti:
  - per ogni  $B(T)$  :
    - $UNDO := UNDO + \{T\}$

//per ogni begin che incontro, metto le transazioni come "possibilmente da rifare": stanno ancora lavorando

- per ogni  $C(T)$  :
  - $UNDO := UNDO - \{T\}$
  - $REDO := REDO + \{T\}$

//se però, proseguendo, dovessi trovare un commit, vuol dire che la transazione ha finito e quindi la tolgo dall'UNDO e la metto nel REDO

### 2. ripristinare la situazione:

- percorrendo il log all'indietro ed eseguendo, per ogni  $A$  delle transazioni in  $UNDO$ ,  $undo(A)$  ;
- arrivando fino alla prima azione della transazione più vecchia in  $UNDO \cup REDO$  ;
- percorrendo il log in avanti ed eseguendo, per ogni  $A$  delle transazioni in  $REDO$ ,  $redo(A)$  .

L'operazione di ripresa a caldo da parte del gestore dell'affidabilità garantisce:

- atomicità: tutte le transazioni in corso al momento del guasto lasciano la base di dati nello stato finale o iniziale;
- persistenza: tutte le pagine delle transazioni in corso sono scritte su memoria secondaria.

### ESEMPIO:

|       | UNDO     | REDO  |
|-------|----------|-------|
| CK    | T1,T3,T4 |       |
| C(T4) | T1,T3    | T4    |
| B(T5) | T1,T3,T5 | T4    |
| C(T1) | T3,T5    | T4,T1 |

**INIZIALIZZO UNDO E REDO:** Osservo il CK-> le T attive sono T1,T3,T4 e quindi le metto nell'UNDO. Poi osservo il C(T4) -> T4 la sposto dall'UNDO al REDO. Guardo B(T5) -> inserisco T5 nell'UNDO. Guardo C(T1) -> sposto T1 dall'UNDO al REDO.

**Alla fine: UNDO: T3,T5 e REDO: T1,T4**

Posso procedere ad eseguire le rispettive azioni per undo (in ordine anti-cronologico delle operazioni) e redo (in ordine cronologico delle operazioni)

### Ripresa a caldo: esempio (2)

B(T1), B(T2), I(T2,O1,A1), B(T3), I(T3,O2,A2), D(T1,O3,B3), B(T4),  
U(T3,O2,B4,A4), I(T4,O4,A5), U(T4,O2,B6,A6), C(T2),  
CK(T1,T3,T4), C(T4), B(T5), D(T5,O4,B7), U(T1,O2,B8,A8), A(T3),  
C(T1), guasto

- UNDO {T3,T5}

| record         | azione            |
|----------------|-------------------|
| D(T5,O4,B7)    | insert O4, O4:=B7 |
| U(T3,O2,B4,A4) | O2:=B4            |
| I(T3,O2,A2)    | delete O2         |

### Ripresa a caldo: esempio (3)

B(T1), B(T2), I(T2,O1,A1), B(T3), I(T3,O2,A2), D(T1,O3,B3), B(T4),  
U(T3,O2,B4,A4), I(T4,O4,A5), U(T4,O2,B6,A6), C(T2),  
CK(T1,T3,T4), C(T4), B(T5), D(T5,O4,B7), U(T1,O2,B8,A8), A(T3),  
C(T1), guasto

- REDO {T1,T4}

| record         | azione            |
|----------------|-------------------|
| D(T1,O3,B3)    | delete O3         |
| I(T4,O4,A5)    | insert O4, O4:=A5 |
| U(T4,O2,B6,A6) | O2:=A6            |
| U(T1,O2,B8,A8) | O2:=A8            |

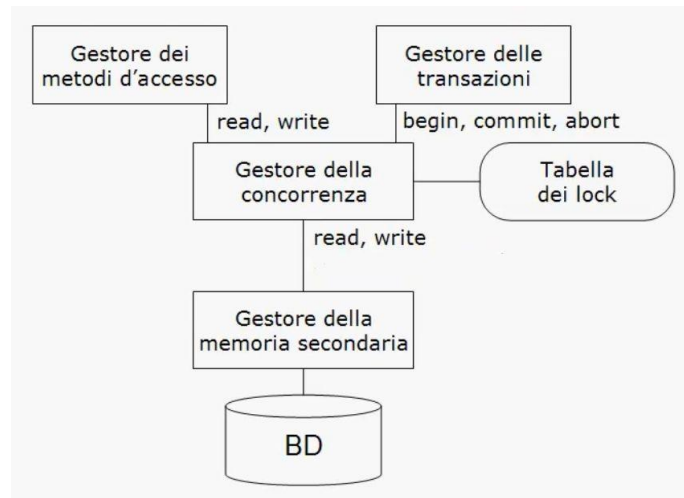
### Ripresa a freddo

Anche la ripresa a freddo è composta da due parti:

1. **ripristinare le base di dati come al momento del guasto:**
  - accedendo al dump e ricopiando selettivamente tutte le parti deteriorate della base di dati;
  - accedendo al record di dump più recente registrato nel log;
  - percorrendo il log in avanti applicando, relativamente alla parte deteriorata, sia le azioni della base di dati sia le azioni di commit o abort;
2. eseguire la **ripresa a caldo**.

## 6.3. Controllo di concorrenza

Permette l'esecuzione concorrente di più transazioni ed è cruciale in sistemi informativi con alto carico (ad esempio i sistemi bancari) in quanto consente un uso efficiente del DBMS massimizzando il numero di transazioni servite e minimizzando i tempi di risposta. Il carico applicativo nei sistemi concorrenti viene misurato in transazioni per secondo (tps). Perciò il **controllo delle concorrenze** ha il compito di **mediare le richieste** di accesso ai dati, decidendo se autorizzarle o meno, e stabilendone l'ordine di esecuzione (da parte di un modulo chiamato **scheduler**).



### 6.3.1. Anomalie delle transazioni concorrenti

L'esecuzione **concorrente di più transazioni** può portare a **problemi di correttezza e anomalie**, ovvero:

- **perdita di aggiornamenti**: consiste nella **perdita delle modifiche** di una transazione **perché sovrascritte** da una transazione concorrente

| Transazione $t_1$                                         | Transazione $t_2$                                         |
|-----------------------------------------------------------|-----------------------------------------------------------|
| bot<br>$r_1(x)$<br>$x := x + 1$<br><br>$w_1(x)$<br>commit | bot<br>$r_2(x)$<br>$x := x + 1$<br><br>$w_2(x)$<br>commit |

Valore iniziale: 2

Valore finale: 3 invece di 4 (la modifica di  $t_1$  viene persa)

- **lettura sporca**: una transazione legge il **risultato intermedio** di un'altra transazione che viene poi abortita (e la cui **modifica viene quindi annullata**)

| Transazione $t_1$                                        | Transazione $t_2$                                         |
|----------------------------------------------------------|-----------------------------------------------------------|
| bot<br>$r_1(x)$<br>$x := x + 1$<br>$w_1(x)$<br><br>abort | bot<br>$r_2(x)$<br>$x := x + 1$<br><br>$w_2(x)$<br>commit |

$t_2$  legge uno stato intermedio poi annullato

- **letture inconsistenti**: avviene quando una transazione legge oggetti che un'altra transazione sta modificando, con alcune letture che avvengono **prima** ed altre **dopo** le modifiche

| Transazione $t_1$              | Transazione $t_2$                                     |
|--------------------------------|-------------------------------------------------------|
| bot<br>$r_1(x)$<br>$w_1(y)$    | bot<br>$r_2(x)$<br>$x := x + 1$<br>$w_2(x)$<br>commit |
| $r_1(x)$<br>$w_1(z)$<br>commit |                                                       |

$t_1$  legge valori differenti per  $x$

- **aggiornamento fantasma**: avviene quando una transazione osserva solo parte degli effetti di un'altra transazione (osservando uno stato dei dati che non soddisfa i vincoli d'integrità)

| Transazione $t_1$                      | Transazione $t_2$                                                              |
|----------------------------------------|--------------------------------------------------------------------------------|
| bot<br>$r_1(x)$                        | bot<br>$r_2(y)$                                                                |
| $r_1(y)$                               | $y := y - 100$<br>$r_2(z)$<br>$z := z + 100$<br>$w_2(y)$<br>$w_2(z)$<br>commit |
| $r_1(z)$<br>$s := x + y + z$<br>commit |                                                                                |

Vincolo:  $x+y+z=1000$ ;  $s=1100$  ma vincolo è soddisfatto

- **inserimento fantasma**: avviene quando una transazione valuta due volte un valore aggregato relativo ad un insieme di elementi di un predicato di selezione e tra una valutazione e l'altra viene inserita una nuova tupla.  
ES: *select avg(voto) from esame where corso = 'basidati'*

**Lock di predicato** -> lock di predicato blocca un insieme di dati basato su una condizione logica, ovvero un predicato. Ad esempio, si può utilizzare un lock di predicato per bloccare tutte le righe di una tabella che soddisfano una specifica condizione, come "WHERE nome='Mario'" -> solo una T alla volta accede alle righe che soddisfano la condizione

## 6.3.2. Schedule e serializzabilità

### Teoria del controllo della concorrenza

Definisce formalmente una transazione come una sequenza di operazioni di lettura e scrittura che ha un identificatore della transazione assegnato al sistema ed inizia con `begin transaction` e finisce con `end transaction`

### Schedule

È una sequenza di operazioni di I/O presentate da transazioni concorrenti, con le seguenti proprietà:

- tutte le operazioni di ogni transazione che ha fatto un commit devono apparire nello schedule;
- per ogni transazione le operazioni devono apparire nello schedule con lo stesso ordine in cui appaiono nella transazione

Per il momento consideriamo la commit-proiezione ed ignoriamo le transazioni che abortiscono per semplificare (tutte le transazioni che guardiamo sono andate a buon fine)

### Controllo di concorrenza

Evita gli schedule che causano anomalie ed è gestito da un modulo che accetta o rifiuta le operazioni richieste dalle transazioni (chiamato scheduler). Il controllo è basato sull'identificazione di classi di schedule

### Transazioni

- $t_1$  :  $r_1(x)$   $w_1(x)$   $r_1(y)$   $w_1(y)$
- $t_2$  :  $r_2(y)$   $w_2(y)$

### schedule seriali:

- $S_1$  :  $r_1(x)$   $w_1(x)$   $r_1(y)$   $w_1(y)$   $r_2(y)$   $w_2(y)$
- $S_2$  :  $r_2(y)$   $w_2(y)$   $r_1(x)$   $w_1(x)$   $r_1(y)$   $w_1(y)$

### altri fra i possibili schedule:

- $S_3$  :  $r_1(x)$   $r_2(y)$   $w_1(x)$   $r_1(y)$   $w_2(y)$   $w_1(y)$
- $S_4$  :  $r_2(y)$   $r_1(x)$   $w_1(x)$   $r_1(y)$   $w_1(y)$   $w_2(y)$

accettabili sulla base di **definizioni di equivalenza**:

- schedule **seriale**: se per ogni transazione  $t_i$  nello schedule, tutte le operazioni di  $t_i$  sono **eseguite consecutivamente** (con  $n$  transazioni, esistono  $n!$  schedule seriali possibili);
- schedule **serializzabile**: è uno schedule **non seriale** che produce lo **stesso risultato di un qualche schedule seriale** delle **stesse transazioni** (richiede nozioni di **equivalenza** tra schedule e altri concetti quali: **view-equivalenza**, **conflict-equivalenza**, **two-phase locking**, **timestamp-based**)

### 6.3.3. View-serializzabilità

#### Definizioni

Alcune definizioni preliminari:

- si dice che  $r_i(x)$  **legge-da**  $w_j(x)$  in uno schedule  $S$  se:
  - $w_j(x)$  precede  $r_i(x)$  in  $S$ ;
  - **non c'è** alcun  $w_k(x)$  tra  $w_j(x)$  e  $r_i(x)$  in  $S$ ;
- si dice che  $w_i(x)$  è una **scrittura finale** su  $x$  in uno schedule  $S$  **se è l'ultima scrittura su  $x$**  in  $S$ . A

questo punto, si può definire:

- **view-equivalenza**: due schedule  $S_i$  e  $S_j$  (con  $i \neq j$ ) sono **view-equivalenti** ( $S_i \equiv_v S_j$ ) se hanno:
  - **le stesse relazioni legge-da**;
  - **le stesse scritture finali**;
- **view-serializzabilità**: uno schedule è view-serializzabile se è **view-equivalente a un qualche schedule seriale** (e indichiamo con **VSR** l'insieme degli schedule view-serializzabili)

#### View-serializzabilità: esempio (5)

$S: r_1(x) \ r_1(y) \ r_2(z) \ r_2(y) \ w_2(y) \ w_2(z) \ r_1(z)$

- legge-da:
  - $r_1(x) \leftarrow$
  - $r_1(y) \leftarrow$
  - $r_2(z) \leftarrow$
  - $r_2(y) \leftarrow$
  - $r_1(z) \leftarrow w_2(z)$
- scritture finali:
  - $y: w_2(y)$
  - $z: w_2(z)$

⇒ **non è view-serializzabile**  
(aggiornamento fantasma)

#### View-serializzabilità: esempio (3)

$S: r_1(x) \ r_2(x) \ w_2(x) \ w_1(x)$

- legge-da:
  - $r_1(x) \leftarrow$
  - $r_2(x) \leftarrow$
- scritture finali:
  - $x: w_1(x)$

⇒ **non è view-serializzabile**  
(perdita di update)

#### View-serializzabilità: esempio (2)

$S: w_0(x) \ r_1(x) \ w_1(x) \ r_2(x) \ w_1(z)$

- legge-da:
  - $r_1(x) \leftarrow w_0(x)$
  - $r_2(x) \leftarrow w_1(x)$
- scritture finali:
  - $x: w_1(x)$
  - $z: w_1(z)$
- **view-equivalente** allo schedule seriale  
 $w_0(x) \ r_1(x) \ w_1(x) \ w_1(z) \ r_2(x)$

⇒ **view-serializzabile**

#### View-serializzabilità: esempio (4)

$S: r_1(x) \ r_2(x) \ w_2(x) \ r_1(x)$

- legge-da:
  - $r_1(x) \leftarrow$
  - $r_2(x) \leftarrow$
  - $r_1(x) \leftarrow w_2(x)$
- scritture finali:
  - $x: w_2(x)$

⇒ **non è view-serializzabile**  
(letture inconsistenti)

Stabilire se **due schedule sono view-equivalenti** ha un **costo polinomiale**, ma la view-serializzabilità di uno schedule generico è un problema **NP-arduo** (in quanto è necessario **confrontare lo schedule con tutti i possibili schedule seriali**).

Perciò, data la complessità, la **view-equivalenza è inutilizzabile in pratica**, ed è necessario definire **condizioni più restrittive ma pratiche**.

### 6.3.4. Conflict-serializzabilità

#### Conflitto tra operazioni

Due operazioni  $a_i$  e  $a_j$  (con  $i \neq j$ ) sono **in conflitto** se: **costruisco le comb CSR**

- appartengono a **due diverse transazioni**;
- accedono allo **stesso oggetto**;
- **almeno una è una scrittura**

Esistono due tipi di conflitti:

- conflitti **read-write**;
- conflitti **write-write**.

$t_1: r_1(x) \quad r_1(y) \quad w_1(x) \quad w_1(y)$   
 $t_2: r_2(y) \quad r_2(x) \quad w_2(y)$

Conflitti:

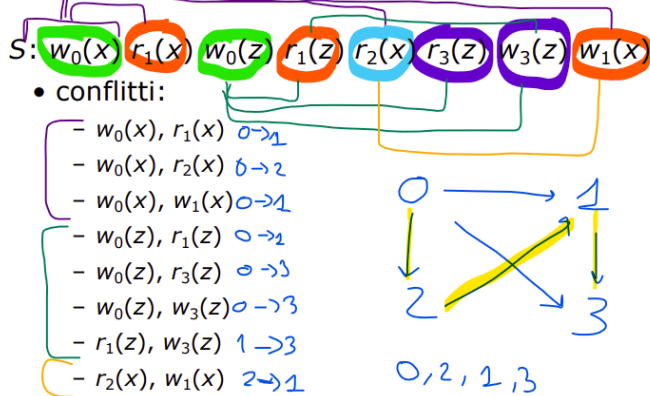
- $w_1(x), r_2(x)$
- $w_1(y), r_2(y)$
- $w_1(y), w_2(y)$
- $r_1(y), w_2(y)$

#### Definizioni

Si può definire:

- **conflict-equivalenza**: due schedule  $S_i$  e  $S_j$  (con  $i \neq j$ ) sono **conflict-equivalenti** ( $S_i \equiv_c S_j$ ) se:
  - contengono le **stesse operazioni**;
  - ogni coppia di **operazioni in conflitto** appare nello **stesso ordine** in entrambi gli schedule;
- **conflict-serializzabilità**: uno schedule è **conflict-serializzabile** se è **conflict-equivalente** ad un qualche **schedule seriale** (e indichiamo con **CSR** l'insieme degli schedule conflict-serializzabili)

#### Conflict-serializzabilità: esempio



Come fare un CSR?

segno tutte le possibili **combinazioni** per ogni variabile che mi possono **creare dei conflitti**. Mi scrivo i **vincoli che ne derivano**, disegno il **grafo dei conflitti** e controllo che sia **aciclico**. Se il grafo non contiene cicli allora è **CSR** e la sequenza è data dal **cammino che tocca tutte le T**.

Infine, scrivo le operazioni corrispondenti come **schedule seriale**.

**Conflict-equivalente** allo schedule seriale

$w_0(x) \quad w_0(z) \quad r_2(x) \quad r_1(x) \quad r_1(z) \quad w_1(x) \quad r_3(z) \quad w_3(z)$

#### CSR e VSR

L'insieme degli **schedule conflict-serializzabili** è un sottoinsieme dell'insieme degli **schedule view-serializzabili** ( $CSR \subseteq VSR$ ), perciò:

- $S \in CSR \rightarrow S \in VSR \rightarrow$  **Se è CSR allora è anche VSR**
- $S \notin VSR \rightarrow S \notin CSR \rightarrow$  **Se NON è VSR allora NON è CSR**

**CSR vs VSR: esempio** w2 e w3 sono chiamate “scritture cieche” appunto perché scrivono ma non leggono.

$t_1 : r_1(x) \ w_1(x)$

$t_2 : w_2(x)$  blind write

$t_3 : w_3(x)$  blind write

Questo schedule è VSR perché ho i vincoli: 1->2, 1->3 e 1->3, 2->3  
Quindi dal grafo trovo il cammino aciclico → 1,2,3

$S : r_1(x) \ w_2(x) \ w_1(x) \ w_3(x)$

- S è view-serializzabile  
-view-equivalente a  $t_1, t_2, t_3$

- S non è conflict-serializzabile

**MA NON È CSR** → se provo a scrivere le combinazioni vedo che mi crea dei cicli. Anche ad occhio vedo che ho  $r_1, w_2 \rightarrow 1 \rightarrow 2$  e  $w_2, w_1 \rightarrow 2 \rightarrow 1$  che è già di per lui un ciclo.

## Complessità

Stabilire se uno schedule è conflict-serializzabile ha un **costo lineare nella dimensione del grafo**, ma è comunque **molto costoso in pratica** (specialmente nel caso di dati distribuiti), perciò i **meccanismi utilizzati concretamente adottano delle condizioni ancora più restrittive ma efficienti**.

## ESERCIZIO SAM\_BD\_M6\_E1

Dato il seguente record di log:

DUMP, B(T1), U(T1,O1,B1,A1), B(T2), I(T2,O2,A2), B(T3), I(T1,O3,A3), C(T2), B(T4), I(T3,O4,A4), U(T4,O5,B5,A5), A(T4), D(T1,O6,B6), CK(...), B(T5), I(T5,O7,A7), A(T3), B(T6), U(T6,O9,B9,A9), C(T5), D(T6,O10,B10), C(T1), B(T7), GUASTO

Si richiede di:

1. scrivere, in corrispondenza di ogni record di checkpoint, le transazioni attive;
2. illustrare dettagliatamente i passi da compiere per effettuare la ripresa a caldo.

1. Nell'unico record di CK che abbiamo dobbiamo inserire le transazioni attive fino al CK quindi **T1 e T3** visto che T2 ha committato e T4 ha abortito.

| RECORD    | UNDO            | REDO         |
|-----------|-----------------|--------------|
| CK(T1,T3) | T1,T3           |              |
| B(T5)     | T1,T3,T5        |              |
| A(T3)     | T1,T3,T5        |              |
| B(T6)     | T1,T3,T5,T6     |              |
| C(T5)     | T1,T3,T6        | T5           |
| C(T1)     | T3,T6           | T1,T5        |
| B(T7)     | <b>T3,T6,T7</b> | <b>T1,T5</b> |

Quindi ora: prima disfo (**UNDO, in ordine inverso**) poi rifaccio (**REDO, in ordine giusto**)

|                       |                     |
|-----------------------|---------------------|
| <b>UNDO: T3,T6,T7</b> |                     |
| D(T6,O10,B10)         | insert O10, O10=B10 |
| U(T6,O9,B9,A9)        | O9=B9               |
| I(T3,O4,A4)           | delete O4           |
| <b>REDO: T1,T5</b>    |                     |
| U(T1,O1,B1,A1)        | O1=A1               |
| I(T1,O3,A3)           | insert O3, O3=A3    |
| D(T1,O6,B6)           | delete O6           |
| I(T5,O7,A7)           | insert O7, O7=A7    |

**Seguo l'ordine delle operazioni NON DELLE TRANSAZIONI, e mi raccomando ai versi!!!**

## ESERCIZIO SAM\_BD\_M6\_E2

Avendo le seguenti informazioni riguardo una **ripresa a caldo**, si indichi per ciascuna affermazione riportata nella tabella allegata se è sicuramente vera (*Vero*), sicuramente falsa (*Falso*), o se *non è possibile determinarlo* (potrebbe essere vera oppure falsa).

- record di checkpoint: CK(T2, T4, T6);
- insieme di UNDO: {T4, T5, T7};
- insieme di REDO: {T2, T6};
- operazioni di UNDO:
  - U(T4, O4, B4, A4) → O4 := B4
  - D(T7, O7, B7) → INSERT(O7), O7 := B7
  - D(T5, O5, B5) → INSERT(O5), O5 := B5
- operazioni di REDO:
  - I(T6, O6, A6) → INSERT(O6), O6 := A6
  - U(T2, O2, B2, A2) → O2 := A2
  - I(T6, O8, A8) → INSERT(O6), O8 := A8

1. la transazione T7 è iniziata (begin transaction) dopo che la transazione T2 è iniziata (begin transaction)  
**VERO** -> T2 era attiva nel CK mentre la T7 è comunque nell'UNDO; quindi, è iniziata dopo il CK e dopo 2
2. la transazione T4 è iniziata (begin transaction) dopo il checkpoint  
**FALSO** -> T4 era attiva nel CK, quindi, è iniziata per forza prima
3. la transazione T2 è iniziata (begin transaction) prima del checkpoint e ha fatto abort dopo il checkpoint  
**FALSO** -> T2 è iniziata prima del CK ma ha committato (è nel REDO) quindi non ha fatto abort
4. la transazione T5 ha fatto commit dopo il checkpoint  
**FALSO** -> T5 è nell'UNDO quindi non ha fatto ancora commit
5. la transazione T6 è iniziata (begin transaction) e ha fatto commit dopo che la transazione T4 è iniziata (begin transaction)  
**NPD** -> T4 e T6 sono entrambe iniziate prima del CK ma non sappiamo in che ordine
6. l'operazione di insert di O8 da parte di T6 precede nel log l'operazione di update di O2 da parte di T2  
**FALSO** -> nel REDO seguiamo l'ordine giusto
7. il log contiene un record di abort per la transazione T5  
**NPD** -> T5 è nell'UNDO ma non sappiamo se abbia fatto abort
8. l'operazione di delete di O7 da parte di T7 segue nel log l'operazione di insert di O6 da parte di T6  
**NPD** -> perché la prima è nell'UNDO e l'altra è nel REDO
9. l'operazione di delete di O5 da parte di T5 segue nel log l'operazione di update di O4 da parte di T4  
**FALSO** -> precede, l'UNDO legge in modo inverso
10. nel log esistono esattamente due operazioni da parte della transazione T6  
**VERO**

## ESERCIZIO SAM\_BD\_M6\_E3 → ad occhio t1 “ingloba” t2 quindi non è VSR

Dato il seguente schedule

$$r_1(x) \ w_2(y) \ w_2(x) \ w_1(y) \ r_3(x) \ r_3(y) \ w_3(x) \ w_3(y)$$

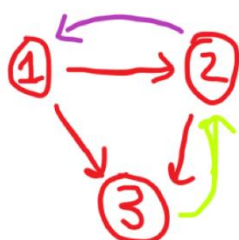
Si dica se è VSR e/o CSR, indicando (qualora esistano) tutti gli schedule seriali equivalenti. Si svolga l'esercizio illustrando dettagliatamente il processo/ragionamento seguito.

X -> r1,w2,r3,w3

Y -> w2,w1,r3,w3

|   | LEGGE DA   | VINCOLI | ALTRI VINCOLI                                                                                     |
|---|------------|---------|---------------------------------------------------------------------------------------------------|
| X | $r1 <- /$  | ----    | 1->2, 1->3 (solo perché x è scritto da w3 e w2)                                                   |
|   | $r3 <- w2$ | 2->3    | ----                                                                                              |
| Y | $r3 <- w1$ | 1->3    | 2->1 V 3->2 (la 2 non può mettersi in mezzo perché la 3 vuole leggere quello che ha scritto la 1) |

|   | SCRITTURE FINALI | ALTRE SCRITTURE | VINCOLI    |
|---|------------------|-----------------|------------|
| X | w3               | w2              | 2->3       |
| Y | w3               | w2,w1           | 2->3, 1->3 |



Negli altri vincoli di Y vedo che c'è un OR. Ma nessuno dei due può essere soddisfatto perché entrambi mi creano dei cicli. Di conseguenza lo schedule **NON è VSR**, quindi anche **NON CSR**.

Di solito se non è VSR allora non è neanche CSR. Però è sempre meglio verificare

### ESERCIZIO SAM\_BD\_M6\_E4

Dato il seguente schedule

$w_2(x) \ r_2(y) \ r_1(x) \ r_3(y) \ r_2(z) \ w_3(x) \ w_4(y) \ w_1(z) \ w_4(z)$

Si dica se è VSR e/o CSR, indicando (qualora esistano) tutti gli schedule seriali equivalenti. Si svolga l'esercizio illustrando dettagliatamente il processo/ragionamento seguito.

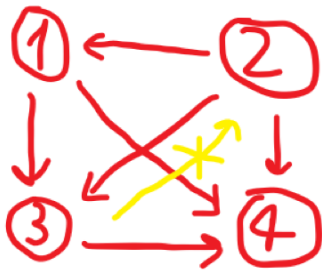
X: w2,r1,w3

Y: r2,r3,w4

Z: r2,w1,w4 → scrivo le operazioni che influiscono sui miei oggetti

|   | LEGGE DA   | VINCOLI | ALTRI VINCOLI                                                                                                                                                            |
|---|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| X | $r1 <- w2$ | 2->1    | 3->2 (questa ipotesi <b>so già che non va bene</b> ) V 1->3                                                                                                              |
| Y | $r2 <- /$  | ----    | 2->4                                                                                                                                                                     |
|   | $r3 <- /$  | ----    | 3->4 (legge la basi di dati iniziale però la 4 modifica quell'oggetto e quindi viene prima)                                                                              |
| Z | $r2 <- /$  | ----    | 2->4, 2->1 (R2 vuol leggere lo stato iniziale della basi di dati però deve venire prima di w4 w1 perché scrivono entrambe su z e quindi andrebbero a modificare il dato) |

|   | SCRITTURE FINALI | ALTRE SCRITTURE | VINCOLI        |
|---|------------------|-----------------|----------------|
| X | w3               | w2              | <b>2-&gt;3</b> |
| Y | w4               | ----            | ----           |
| Z | w4               | w1              | 1->4           |



Scrivo **sempre prima i vincoli e poi gli "altri vincoli"**. Se gli altri vincoli hanno un OR e una delle due relazioni va in conflitto con i vincoli la scarto (nel nostro esempio 3->2)

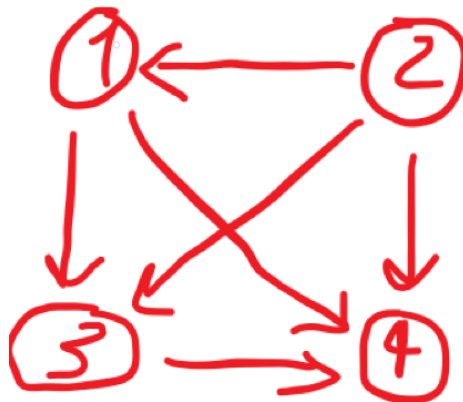
VSR → 2,1,3,4 → **t2,t1,t3,t4**

Guardiamo ora il conflict serializzabile CSR → teniamo buono le info sulle var che vengono "intaccate"

X:  
w2,r1 2->1  
w2,w3 2->3  
r1,w3 1->3

y:  
r2,w4 2->4  
r3,w4 3->4

z:  
r2,w1 2->1  
r2,w4 2->4  
w1,w4 1->4



CSR → 2,1,3,4 → **t2,t1,t3,t4**

### ESERCIZIO SAM\_BD\_M6\_E5

$r_1(x) \ w_1(t) \ r_3(y) \ w_3(y) \ w_1(x) \ w_1(y) \ r_2(t) \ r_3(z) \ w_3(t) \ r_2(x) \ w_2(z) \ w_4(t)$

Si dica se è VSR e/o CSR, indicando (qualora esistano) tutti gli schedule seriali equivalenti. Si svolga l'esercizio illustrando dettagliatamente il processo/ragionamento seguito.

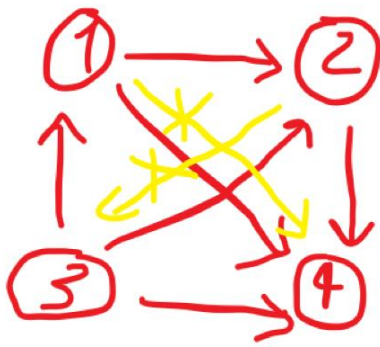
X: R1,W1,R2

Y: R3,W3,W1

Z: r3,w2

T: w1,r2,w3,w4 **so già che non è CSR perché su y ho w3 prima di w1 mentre su t ho w1 prima di w3 e quindi se leggo il grafico vedo un ciclo**

|   | LEGGE DA         | VINCOLI         | ALTRI VINCOLI                                                                     |
|---|------------------|-----------------|-----------------------------------------------------------------------------------|
| X | r1 <- /          | ----            | ----                                                                              |
|   | r2 <- w1         | 1->2            | ----                                                                              |
| Y | r3 <- /          | ----            | 3->1 (c'è w1 che scrive)                                                          |
| Z | r3 <- /          | ----            | 3->2 (c'è w2 che scrive)                                                          |
| T | r2 <- w1         | 1->2            | 3->1 ∨ 2->3<br>4->1 ∨ 2->4<br>(la 3 e la 4 non posson mettersi in mezz tra 1 e 2) |
|   | SCRITTURE FINALI | ALTRE SCRITTURE | VINCOLI                                                                           |
| X | w1               | ----            | ----                                                                              |
| Y | w1               | w3              | 3->1                                                                              |
| Z | w2               | ----            | ----                                                                              |
| T | w4               | w1,w3           | 1->4, 3->4                                                                        |



VSR 3,1,2,4 → t3,t1,t2,t4

Ora faccio il CSR ma so già che non lo è ad occhio:

X: w1,r2 --- 1->2

Y:r3,w1 --- 3->1

w3,w1 --- 3->1

w1,w3 --- 1->3 → già da qui posso vedere che non è CSR perché mi crea un ciclo

### ESERCIZIO SAM\_BD\_M6\_E6

Dato il seguente schedule

$w_2(y) \ r_3(z) \ w_2(x) \ r_1(y) \ w_3(z) \ r_4(x) \ w_1(z) \ w_3(x) \ w_1(x)$

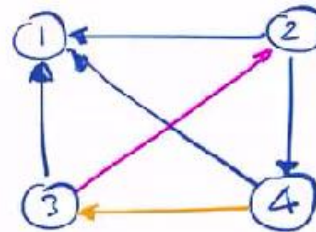
Si dica se è VSR e/o CSR, indicando (qualora esistano) tutti gli schedule seriali equivalenti. Si svolga l'esercizio illustrando dettagliatamente il processo/ragionamento seguito.

$w_2(y) \ r_3(z) \ w_2(x) \ r_1(y) \ w_3(z) \ r_4(x) \ w_1(z) \ w_3(x) \ w_1(x)$

Si dica se è VSR e/o CSR, indicando (qualora esistano) tutti gli schedule seriali equivalenti. Si svolga l'esercizio illustrando dettagliatamente il processo/ragionamento seguito.

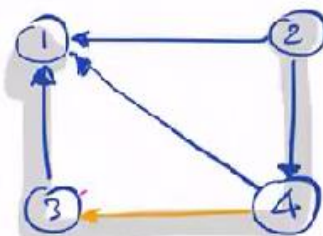
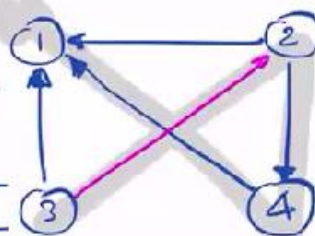
X:  $w_2 \ r_4 \ w_3 \ w_1$   
Y:  $w_2 \ r_1$   
Z:  $r_3 \ w_3 \ w_1$

$t_3 \ t_2 \ t_4 \ t_1$   
 $t_2 \ t_4 \ t_3 \ t_1$



| LESSON | DATA                 | VINCA             | ALTRA VINCA                                                                      |
|--------|----------------------|-------------------|----------------------------------------------------------------------------------|
| X      | $r_4 \leftarrow w_2$ | $2 \rightarrow 4$ | $(1 \rightarrow 2 \vee 4 \rightarrow 1), (3 \rightarrow 2 \vee 4 \rightarrow 3)$ |
| Y      | $r_1 \leftarrow w_2$ | $2 \rightarrow 1$ | —                                                                                |
| Z      | $r_3 \leftarrow r_1$ | —                 | $3 \rightarrow 1$                                                                |

| SCHEDULE | FIND  | AVRE        | VINCA                              |
|----------|-------|-------------|------------------------------------|
| X        | $w_1$ | $w_2 \ w_3$ | $2 \rightarrow 1, 3 \rightarrow 1$ |
| Y        | $w_2$ | —           | —                                  |
| Z        | $w_3$ | $w_3$       | $3 \rightarrow 1$                  |



3 2 4 1

2 4 3 1

**Esercizi appena fatti:** il primo che non era vsr quindi manco csr, il secondo sia vsr che csr sullo stesso schedule, il terzo era vsr ma non csr, il quarto era vsr rispetto a 2 schedule ma csr rispetto a uno schedule

## ESERCIZIO SAM\_BD\_M6\_E7

$r_1(y) \ r_2(t) \ r_3(x) \ w_3(x) \ w_1(y) \ w_1(x) \ w_2(t) \ r_4(z) \ w_4(t) \ r_2(y) \ w_2(z)$

Dire se può essere stato generato da uno scheduler basato su 2PL base, motivando opportunamente la risposta. Si consideri, per la soluzione, un lock a due stati.

Nel caso lo schedule non possa essere stato generato da uno scheduler basato su 2PL base, indicare il minimo numero di operazioni da rimuovere dallo schedule per renderlo 2PL e illustrare quali sono.

Non è 2PL perché t2 prende il lock su t con r2(t) poi lo lascia a t4 che fa w4(t) però poi dovrebbe riprendere il lock su y. Quindi **dovrebbe prendere tutto prima!!!!**

| OPERAZIONE | X | Y | Z | T |
|------------|---|---|---|---|
| r1(y)      |   | 1 |   |   |
| r2(t)      |   |   |   | 2 |
| r3(x)      | 3 |   |   |   |
| w3(x)      | 3 |   |   |   |
| w1(y)      |   | 1 |   |   |
| w1(x)      | 1 |   |   |   |
| w2(t)      |   |   |   | 2 |
| r4(z)      |   |   | 4 |   |
| w4(t)      |   |   |   | 4 |
| r2(y)      |   | 2 |   |   |
| w2(z)      |   |   | 2 |   |

La 3 può rilasciare perché ha finito e non ha altre operazioni dopo. Quando il 4 vuole T la 2 dovrebbe lasciare il lock (però ha delle operazioni dopo). Quindi la 2 dovrebbe portare a casa (anticipare) quello che le serve ovvero la Y e la Z → però la Z attualmente ce l'ha in mano la 4 quindi non può essere 2PL

Per farlo diventare 2PL possiamo togliere una delle operazioni evidenziate in rosso, quindi la risposta è num min: 1.

## ESERCIZIO SAM\_BD\_M6\_E8

$r_1(x) \ r_4(x) \ w_4(x) \ r_1(y) \ r_4(z) \ w_4(z) \ w_3(y) \ w_3(z) \ w_1(t) \ w_2(z) \ w_2(t)$

Dire se può essere stato generato da uno scheduler basato su 2PL base, motivando opportunamente la risposta. Si consideri, per la soluzione, un lock a due stati.

| OPERAZIONE | X | Y | Z | T |
|------------|---|---|---|---|
| r1(x)      | 1 |   |   |   |
| r4(x)      | 4 |   |   |   |
| w4(x)      | 4 |   |   |   |
| r1(y)      |   | 1 |   |   |
| r4(z)      |   |   | 4 |   |
| w4(z)      |   |   | 4 |   |
| w3(y)      |   | 3 |   |   |
| w3(z)      |   |   | 3 |   |
| w1(t)      |   |   |   | 1 |
| w2(z)      |   |   | 2 |   |
| w2(t)      |   |   |   | 2 |

Quando lo scheduler è 2PL → faccio anche l'altra tabella con "prendi il lock e portalo a casa" che consente di essere 2PL. Già all'inizio t1 deve portare a casa tutti i lock necessari prima (x,y,t) quindi così al secondo passaggio può lasciare x così lo prende la t4. Poi w4, quando prende il lock per x prende anche quello di Z che poi rilascerà.

Si consideri lo schedule:

Dire se può essere stato generato da uno scheduler basato su 2PL base, motivando opportunamente la risposta.

Si consideri, per la soluzione, un lock a due stati.

| OPER  | lock/unlock | x | y | z | t |
|-------|-------------|---|---|---|---|
| r1(x) | lock(t,x)   | 1 |   |   |   |
| r4(x) | lock(t,y)   |   | 1 |   |   |
| w4(x) | lock(t,t)   |   |   |   | 1 |
| r1(y) | unlock(t,x) |   |   |   |   |
| r4(z) | lock(t,z)   |   |   |   |   |
| w4(z) |             |   |   |   |   |
| w3(y) |             |   | 3 |   |   |
| w3(z) |             |   |   | 3 |   |
| w1(t) |             |   |   |   | 1 |
| w2(z) |             |   |   | 2 |   |
| w2(t) |             |   |   |   | 2 |

|       |             |  |  |  |   |
|-------|-------------|--|--|--|---|
| w2(z) | unlock(t,z) |  |  |  | 2 |
| w2(t) | unlock(t,t) |  |  |  | 2 |

### ESERCIZIO SAM\_BD\_M6\_E9

Si consideri un controllo di concorrenza *monoversione* basato su timestamp e un oggetto  $x$  con timestamp  $RTM(x)=2$  e  $WTM(x)=2$ . Si richiede di compilare la tabella allegata indicando, per ogni operazione riportata nella tabella:

1. se l'operazione viene accordata o meno;
2. l'eventuale transazione uccisa;
3. i nuovi valori di  $RTM(x)$  e  $WTM(x)$ .

| Richiesta   | Risposta | RTM(x) | WTM(x) | Trans. uccisa    |
|-------------|----------|--------|--------|------------------|
|             |          | 2      | 2      |                  |
| read(x,5)   | OK       | 5      | 2      |                  |
| write(x,7)  | OK       | 5      | 7      |                  |
| write(x,3)  | NO       |        |        | T3               |
| read(x,8)   | OK       | 8      | 7      |                  |
| read(x,14)  | OK       | 14     | 7      |                  |
| read(x,10)  | OK       | 14     | 7      | tengo il max RTM |
| write(x,17) | OK       | 14     | 17     |                  |
| read(x,15)  | NO       |        |        | T15              |
| write(x,16) | NO       |        | 16<WTM | T16              |

### ESERCIZIO SAM\_BD\_M6\_E10

Si consideri un controllo di concorrenza *multiversione* basato su timestamp e un oggetto  $x$  con timestamp  $RTM(x)=5$  e  $WTM(x)=2$ . Si richiede di compilare la tabella allegata indicando, per ogni operazione riportata nella tabella:

1. se l'operazione viene accordata o meno;
2. la versione dell'oggetto utilizzata;
3. i nuovi valori di  $RTM(x)$  e  $WTM(x)$ ;
4. l'eventuale transazione uccisa.

Indicare inoltre quali transazione sarebbero state uccise da un controllo di concorrenza *monoversione* basato su timestamp.

| Richiesta   | Risposta | Versione | RTM(x) | WTM(x) | Trans. uccisa                                                     |
|-------------|----------|----------|--------|--------|-------------------------------------------------------------------|
|             |          | $x_2$    | 5      | 2      |                                                                   |
| write(x,7)  | OK       | X7       | 7      | 7      |                                                                   |
| read(x,15)  | OK       | X7       | 15     | 7      |                                                                   |
| write(x,10) | NO       |          | TS<RTM |        | T10                                                               |
| read(x,8)   | OK       | X7       | 15     | 7      |                                                                   |
| write(x,18) | OK       | X18      | 18     | 18     |                                                                   |
| read(x,6)   | OK       | X2       | 6      | 2      | prendo WTM di X2                                                  |
| write(x,16) | OK       | X16      | 16     | 16     | La accetto perchè nessuno ha letto una versione tra la 10 e la 16 |

# **ESERCIZIO SAM BD\_M6\_E11 → T1,T2 mono , T3,T4 multi**

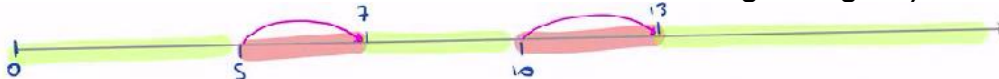
| Operazione   | Risposta | Operazione   | Risposta | Operazione   | Risposta | Operazione   | Risposta |
|--------------|----------|--------------|----------|--------------|----------|--------------|----------|
| write(x, 9)  | OK       | write(y, 6)  | OK       | write(z,10)  | OK       | write(t, 5)  | OK       |
| read(x, 13)  | OK       | read(y, 13)  | OK       | read(z, 6)   | OK       | read(t, 7)   | OK       |
| write(x, 14) | OK       | read(y, 17)  | OK       | read(z, 18)  | OK       | write(t, 10) | OK       |
| read(x, T1)  | OK       | write(y, T2) | NO       | write(z, T3) | NO       | read(t, 13)  | OK       |
| write(x, 11) | NO       | read(y, 10)  | OK       |              |          | write(t, T4) | OK       |
| write(x, 24) | OK       |              |          |              |          |              |          |

- T1 può essere da 14 a 24 → 14 c'è stata una scrittura (quindi non può andare indietro) e c'è 24 dopo che viene accettata → ricorda che una write viene rifiutata solo se il  $ts < wtm$  o  $ts < rtm$  quindi se è uguale va bene
- T2 può essere da 0 a 16 perché sto scrivendo dove qualcuno ha già letto ( read(x,17) ) mettiamo a 16 → perché nella tabella ci dice che è rifiutata) → la mia scrittura è arrivata troppo tardi (c'è qualcuno dopo di lei che ha già letto o scritto) **ATTENZIONE:** deve anche essere  $\neq 10$  perché non può essere ancora viva dopo nella read dato che è stata uccisa.
- T3 può essere  $0 \leq t3 < 6$  e  $10 \leq t3 < 18$  → arriviamo a dire no ad una scrittura (write(z,T3)) quando arriviamo a creare una versione che avrebbe dovuto essere letta da una operazione (read(z,6) --- read(z,18)) che però ne ha letta un'altra (X6---X10).



Se T3 finisce in mezzo tra 0 e 6 gli diciamo NO! Se T3 finisce in mezzo tra 10 e 18 gli diciamo di NO!

- T4 può essere  $0 \leq t4 < 5$ ,  $7 \leq t4 < 10$  e  $13 \leq t4$  → stesso ragionamento di prima ma qui t4 viene accettata quindi deve stare fuori dagli intervalli in cui l'operazione verrebbe rifiutata (quando si "intrufola" tra una lettura e una versione di scrittura già eseguite)



### 6.3.5. Locking a due fasi

#### Controllo della concorrenza

I sistemi utilizzati in pratica **non valutano la serializzabilità a posteriori**, ma operano in modo da garantirla, utilizzando una delle possibili tecniche, tra cui:

- **locking a due fasi**;
- **timestamp**: (esecuzione seriale ma rispettando l'ordine in cui le operazioni sono nate)

- **monoversione**

- **multiversione**

#### Lock

Il lock, la variabile su cui si basa il locking a due fasi, descrive lo stato di un dato rispetto alle operazioni che possono essere eseguite sul dato, e può essere

- lock a **due stati** (binario): (non fa la distinzione tra scrittura e lettura)
  - unlocked (0);
  - locked (1);
- lock a **tre stati**: (fa distinzione fra risorsa occupata in scrittura o lettura, se lettura più di uno può leggere)
  - unlocked;
  - r\_locked (in lettura, condiviso);
  - w\_locked (in scrittura, esclusivo).

#### Lock a due stati

Le **regole** di utilizzo del lock a due stati sono:

- per **accedere ad un dato**, una transazione deve chiedere un lock sul dato;
- la **transazione rilascia il lock** quando il **dato non le serve più**;
- una **transazione può accedere ad un dato solo se ha ottenuto il relativo lock**. Una

transazione che segue queste regole è detta ben formata rispetto al locking.

#### Lock a tre stati

Le **regole** di utilizzo del lock a tre stati sono:

- per **accedere in lettura** ad un dato, una transazione deve **chiedere un r\_lock**;
- per **accedere in scrittura** ad un dato una transazione deve **chiedere un w\_lock**;
- la transazione **rilascia il lock quando il dato non le serve più**;
- una transazione **può accedere ad un dato solo se ha ottenuto il relativo lock**. Una

transazione che segue queste regole è detta ben formata rispetto al locking.

#### Gestione dei lock

Il gestore dei lock è un modulo che:

- **riceve le richieste di lock**;
- le **concede o meno** a seconda delle tabelle dei conflitti (se la richiesta non è concessa, la transazione **richiedente è posta in attesa** fino a che il lock le viene concesso);
- **modifica** opportunamente le **tabelle di lock** (stato ed eventuale contatore)

## Tabella dei conflitti per lock a 2 e 3 stati

| Richiesta | Stato        |                |
|-----------|--------------|----------------|
|           | unlocked     | locked         |
| lock      | OK<br>locked | no<br>locked   |
| unlock    | errore       | OK<br>unlocked |

| Richiesta | Stato          |                                              |                |
|-----------|----------------|----------------------------------------------|----------------|
|           | unlocked       | r_locked                                     | w_locked       |
| r_lock    | OK<br>r_locked | OK (c=c+1)<br>r_locked                       | no<br>w_locked |
| w_lock    | OK<br>w_locked | no*<br>r_locked                              | no<br>w_locked |
| unlock    | errore         | OK (c=c-1)<br>c=0: unlocked<br>c>0: r_locked | OK<br>unlocked |

\*: a meno di upgrade di un r\_lock tenuto in modo esclusivo

### Locking a due fasi

Questa tecnica prevede che una transazione, dopo aver rilasciato un lock, non può acquisire altri lock, in modo tale che ogni transazione abbia due fasi:

- crescente (in cui acquisisce i lock)
- decrescente (in cui rilascia i lock)

Questa tecnica garantisce la serializzabilità

### Schedule 2PL

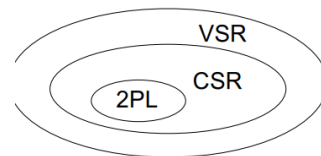
Gli schedule appartengono alla classe **2PL** se sono prodotti da uno scheduler che usa:

- transazioni ben formate rispetto al locking
- gestione dei lock basata sui conflitti
- locking a due fasi

### 2PL, CSR e VSR

L'insieme degli schedule **2PL** è un sottoinsieme dell'insieme degli schedule **conflict-serializzabili** ( $2PL \subseteq CSR \subseteq VSR$ ), perciò:

- $S \in 2PL \rightarrow S \in CSR \rightarrow$  se è 2PL allora è anche CSR
- $S \notin CSR \rightarrow S \notin 2PL \rightarrow$  se NON È CSR allora NON È 2PL



### 2PL vs CSR: esempio

S:  $r_1(x) \ w_1(x) \ r_2(x) \ w_2(x) \ r_3(y) \ w_1(y)$   
                                                                                   
                      $t_2$                      $t_3$

- è conflict-serializzabile  
 - conflict-equivalente a  $t_3, t_1, t_2$

- non è 2PL **X**

È CSR perché abbiamo:

$x \rightarrow r_1, w_2: 1 \rightarrow 2$   
 $w_1, r_2: 1 \rightarrow 2$   
 $w_1, w_2: 1 \rightarrow 2$

$y \rightarrow r_3, w_1: 3 \rightarrow 1$

Se faccio il grafo posso trovare un CSR  $\rightarrow 3, 1, 2$

MA NON È 2PL perché: T1 prende il lock su x, lo rilascia ma alla fine lo prende ancora su y, violando la proprietà 2PL.

*Se non ci fosse stata  $r_3(y)$  sarebbe stata ok solo se  $t_1$ , nel momento in cui rilasciava il lock su x, prendeva quello di y così da utilizzarlo dopo.*

### Locking a due fasi stretto

In realtà lo scheduler opera senza sapere come finiranno le transazioni; perciò, non si può più supporre di utilizzare una commit-proiezione e occorre aggiungere una restrizione al 2PL, ottenendo così il **2PL stretto**. Il 2PL stretto, utilizzato dai DBMS commerciali, oltre a vietare ad una transazione di acquisire lock dopo averne rilasciati, prevede che i lock di una transazione possono essere rilasciati solo dopo le operazioni di commit o abort.

Variazioni del 2PL

Esistono in realtà tre variazioni del locking a due fasi:

- **2PL base** (fase crescente e fase calante, se rilascio il lock non posso riprenderlo)
- **2PL stretto** (lock rilasciati solo dopo commit/abort) → non letture sporche
- **2PL conservativo** (garantisce l'assenza di deadlock): prevede, in aggiunta, che una transazione acquisisca tutti i lock prima di iniziare ad eseguire le operazioni

### 6.3.6. Timestamp

Un timestamp è un identificatore che definisce un ordinamento totale tra gli eventi temporali del sistema:

- un evento ha un timestamp maggiore rispetto a quello degli eventi che lo hanno preceduto;
- non esistono due eventi con timestamp uguale. Due

possibili implementazioni del timestamp si basano su:

- contatore
- clock di sistema

#### Transazioni e timestamp

Ogni transazione ha associato un timestamp  $ts$  assegnato dal sistema all'inizio della transazione, che rappresenta il tempo al quale la transazione è iniziata. Quindi uno schedule è accettato solo se riflette l'ordine seriale delle transazioni basato sul valore del timestamp delle transazioni.

#### Timestamp monoversione

Ogni oggetto  $x$  ha associato due indicatori:

- **$RTM(x)$**  : timestamp maggiore tra le transazioni che hanno letto  $x$  ;
- **$WTM(x)$**  : timestamp maggiore tra le transazioni che hanno scritto  $x$  .Lo

scheduler valuta e risponde alle richieste nel seguente modo:

- **$read(x, ts)$**  :
  - se  $ts < WTM(x)$  allora la richiesta è rifiutata e la transazione abortita;
  - altrimenti la richiesta è accettata e  $RTM(x) := \max(RTM(x), ts)$  ;
- **$write(x, ts)$**  :
  - se  $ts < WTM(x)$  oppure  $ts < RTM(x)$  allora la richiesta è rifiutata e la transazione abortita;
  - altrimenti la richiesta è accettata e  $WTM(x) := ts$

#### Timestamp monoversione: esempio

| Richiesta     | Risposta                | $RTM(x)$ | $WTM(x)$ |
|---------------|-------------------------|----------|----------|
|               |                         | 2        | 2        |
| $read(x,6)$   | ok                      | 6        | =2       |
| $read(x,8)$   | ok                      | 8        | =2       |
| $read(x,9)$   | ok                      | 9        | =2       |
| $write(x,8)$  | no ( $t_8$ è uccisa)    | =9       | =2       |
| $write(x,11)$ | ok                      | =9       | 11       |
| $read(x,10)$  | no ( $t_{10}$ è uccisa) | =9       | =11      |

nella prima linea provo un operazione di lettura al timestamp 6. 6 è  $\geq$  di 2 ( $WTM$ ) quindi è ok e aggiorno l' $RTM$  se il ts lo supera (in questo caso si). Seconda riga uguale, 8 è  $\geq$  2 quindi l'operazione è ok e aggiorno l' $RTM$  se il ts lo supera. Per la terza riga uguale. Alla quarta riga c'è una scrittura al ts 8.  $8 \geq 9$  e  $8 \geq 2$ ? **NO** quindi uccido l'operazione. Alla quinta riga c'è una scrittura su  $x$  al ts 11.  $11 \geq 2$ ? SI,  $11 \geq 9$ ? SI → quindi l'operazione è ok e aggiorno il  $WTM$  con il ts (resta uguale) l'ultima operazione di lettura al ts 10.  $10 \geq 9$  e  $10 \geq 11$ ? No, uccido l'operazione (posso anche vederlo perché to leggendo su  $x$  dopo aver scritto al ts 11)

## Vantaggi e svantaggi

Il controllo basato sui timestamp ha il vantaggio di **non richiedere alcun lock sulle risorse**, ed è quindi **immune ai deadlock**. Gli svantaggi di questa tecnica, però, consistono **nell'uccisione di molte transazioni** e nel fatto che **si comporta correttamente solo con l'assunzione di commit-proiezione (tutte le transazioni andranno a buon fine)** (per rimuoverla, occorre bufferizzare le scritture, mentre le letture di dati bufferizzati vanno messi in attesa del commit della transazione scrivente).

## Timestamp multiversione

**Ogni scrittura di un oggetto genera una nuova copia (o versione) dell'oggetto**; perciò, le letture richieste da transazioni più "vecchie" vengono eseguite su vecchie versioni (**anziché essere rifiutate e le transazioni abortite** → **risolvo uno svantaggio del monoversione, cerco di uccidere meno transazioni possibili**). Quindi, per ogni oggetto  $x$ , **esistono diverse versioni  $x_i$ , ognuna con i propri RTM e WTM**.

Lo scheduler valuta e risponde alle richieste nel seguente modo:

- **read( $x, ts$ )** :
  - **sempre accettata**, va letta la versione  $x_k$  tale che  $WTM(x_k) = \max(\{WTM(x_i) \mid WTM(x_i) \leq ts\})$  ;
  - $RTM(x_k) := \max(RTM(x_k), ts)$  ;
- **write( $x, ts$ )** :
  - trova la versione  $x_j$  tale che  $WTM(x_j) = \max(\{WTM(x_i) \mid WTM(x_i) \leq ts\})$  ;
  - se  $ts < RTM(x_j)$  la **richiesta è rifiutata e la transazione abortita**;
  - altrimenti:
    - la **richiesta è accettata**;
    - viene **creata una nuova versione  $x_k$**  ;
    - $RTM(x_k) := WTM(x_k) := ts$

## Timestamp e multiversione: esempio

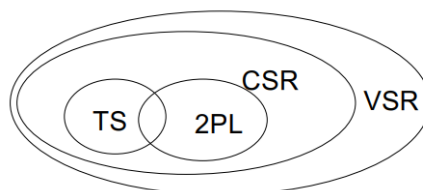
| Richiesta        | Risposta                | $x_k$    | $RTM(x_k)$ | $WTM(x_k)$ |
|------------------|-------------------------|----------|------------|------------|
|                  |                         | $x_2$    | 2          | 2          |
| read( $x, 5$ )   | ok                      | $x_2$    | 5          | =2         |
| read( $x, 8$ )   | ok                      | $x_2$    | 8          | =2         |
| write( $x, 7$ )  | no ( $t_7$ è uccisa)    |          |            |            |
| write( $x, 12$ ) | ok                      | $x_{12}$ | 12         | 12         |
| read( $x, 9$ )   | ok                      | $x_2$    | 9          | =2         |
| read( $x, 14$ )  | ok                      | $x_{12}$ | 14         | =12        |
| write( $x, 13$ ) | no ( $t_{13}$ è uccisa) |          |            |            |
| write( $x, 10$ ) | ok                      | $x_{10}$ | 10         | 10         |
| read( $x, 11$ )  | ok                      | $x_{10}$ | 11         | =10        |

Iniziamo con un oggetto  $x$  versione 2 con  $RTM$  e  $WTM = 2$  (nata al tempo 2). Arriva una read con  $RTM$  5 (alle read dico sempre di sì). Quindi aggiorno l' $RTM$  e legge sempre la versione 2. stessa cosa per la read 8, legge ver 2 e aggiorno  $RTM$ . Poi arriva una write 7 che viene uccisa perché è < del  $RTM$ . Arriva la write 12, creo una nuova versione  $x_{12}$  e aggiorno sia  $RTM$  che  $WTM$ . Arriva una read 9, ok. Ma leggo la versione 2 ( $9 < 12$ ). Poi aggiorno  $RTM$  a 12 e  $WTM$  con il timestamp della ver che ho letto quindi 2. Poi mi arriva read 14, ok. Qui leggo la ver 12 ( $14 < 12$ ). Metto  $RTM$  a 14 e  $WTM$  12 (ver che ho letto). Arriva una write 13 che viene uccisa perché è minore del  $RTM$ . Poi una write 10 che mi va bene, creo una ver 10 e metto sia  $RTM$  che  $WTM$  a 10. Infine, arriva una read 11 che va bene e la faccio leggere dalla ver 10 (non 12). Aggiorno  $RTM$  a 11 e  $WTM$  con la ver che ho letto.

## TS, 2PL, CSR e VSR

L'insieme degli schedule TS è un sottoinsieme dell'insieme degli schedule conflict-serializzabili ( $TS \subseteq CSR \subseteq VSR$ ), perciò:

- $S \in TS \rightarrow S \in CSR \rightarrow$  **se è TS allora è CSR**
- $S \notin CSR \rightarrow S \notin TS \rightarrow$  **se NON È CSR allora NON È TS**



## 2PL e TS a confronto

Nonostante possano verificarsi deadlock, il **2PL è una scelta migliore del TS**, in quanto è **più costoso rieseguire completamente le transazioni con azioni rifiutate**, piuttosto che **metterle semplicemente in attesa**.

## 6.3.7. Meccanismi per la gestione dei lock

### Gestione del lock

Il **gestore del lock** è invocato da tutti i processi che devono accedere alla base di dati, con le seguenti richieste:

- $r\_lock(T, x, errcode, timeout)$  ;
- $w\_lock(T, x, errcode, timeout)$  ;
- $unlock(T, x)$  .

Se il **timeout scade**, *errcode* segnala un errore e , generalmente, la transazione esegue un rollback e riparte.

### Tabelle di lock

Le tabelle di lock memorizzano, per ogni oggetto, informazioni utili al gestore dei lock, ovvero:

- **due bit di stato** (per rappresentare i tre stati);
- **un contatore di transazioni** che condividono  $r\_lock$  ;
- **un contatore** che rappresenta il numero di **processi in attesa**.

Queste informazioni, essendo richieste di frequente, sono mantenute in **memoria centrale**.

### Lock gerarchico

In molti **sistemi reali**, i **lock possono essere richiesti su oggetti a granularità differente**, potendo così operare su una **gerarchia di oggetti** e permettendo quindi di bloccare gli oggetti per livelli. A questa tecnica corrispondono diverse primitive per la richiesta di lock:

- **XL (exclusive lock)**: corrisponde al write lock del protocollo normale;
- **SL (shared lock)**: corrisponde al read lock del protocollo normale;
- **ISL (intentional shared lock)**: esprime l'intenzione di bloccare in modo condiviso uno dei nodi che discende dal nodo corrente;
- **IXL (intentional exclusive lock)**: esprime l'intenzione di bloccare in modo esclusivo uno dei nodi che discende dal nodo corrente;
- **SIXL (shared intentional-esclusive lock)**: chiede lock condiviso sul nodo corrente ed esprime l'intenzione di bloccare in modo esclusivo uno dei nodi che discende dal nodo corrente.

### Protocollo del lock gerarchico

Quando si utilizza un lock gerarchico, le transazioni devono **operare con un preciso protocollo**:

- i lock si richiedono dalla radice e scendendo lungo l'albero;
- i lock si rilasciano dal nodo bloccato e salendo lungo l'albero;
- una transazione può richiedere un lock SL o ISL su un nodo, solo se ha un lock ISL o IXL sul genitore;
- una transazione può chiedere un lock IXL, XL o SIXL su un nodo, solo se ha già un lock SIXL o IXL sul genitore.

### Granularità del lock gerarchico

La scelta della **granularità alla quale richiedere un lock** è lasciata agli sviluppatori delle applicazioni, i quali devono tenere in considerazione che:

- se la granularità è **troppo grande**:
  - molte risorse sono bloccate;
  - limita il parallelismo;
- se la granularità è **troppo piccola**:
  - molti lock sono richiesti;
  - più lavoro per il gestore dei lock;
  - rischi di fallimento dopo aver acquisito molte risorse.

### Lock in SQL (1999)

Le transazioni possono essere classificate come:

- **read only** (non possono modificare il contenuto della base di dati o il suo schema e perciò non possono richiedere lock esclusivi);
- **read write** (di default).

Inoltre, per ogni transazione, si può indicare un **livello di isolamento**, per ognuno dei quali è previsto **un approccio diverso per le letture**, garantendo così diverse proprietà:

- **read uncommitted**: non pone alcun vincolo (non chiede lock per leggere e non rispetta i lock esclusivi di altre transazioni, ma **può presentare tutte le anomalie delle transazioni concorrenti**);
- **read committed**: chiede lock condivisi per effettuare le letture (esclude così le letture di stati intermedi, evitando di leggere dati non committed e di fare letture sporche, ma **non garantisce serializzabilità**);
- **repeatable read**: richiede **2PL** stretto anche per le letture (utilizza lock a livello di tupla ed evita tutte le anomalie, ad **eccezione dell'inserimento fantasma** -> *inserimento cose nuove che nessuno ha lockato prima*);
- **serializable**: chiede **2PL** stretto anche per la lettura ed utilizza lock di predicato (è quello di default, in quanto **evita tutte le anomalie**).

Ogni livello richiede **2PL stretto per le scritture** (evitando così la perdita di aggiornamento).

### 6.3.8. Deadlock

Si è in presenza di un deadlock, quando **due transazioni concorrenti sono in attesa (diretta o indiretta) l'una dell'altra** (più in generale, il deadlock può interessare più transazioni):

- $t_i$  aspetta che  $t_j$  rilasci un lock;
- $t_j$  aspetta che  $t_i$  rilasci un lock.

#### Grafo di attesa

Si può definire il **grafo di attesa delle transazioni**, come un grafo in cui ogni nodo rappresenta una transazione, e ogni arco tra  $t_i$  e  $t_j$  rappresenta l'attesa da parte di  $t_i$  di una risorsa bloccata da  $t_j$ .

**Esiste un deadlock quando nel grafo di attesa è presente un ciclo.** *(nessuno lascia mai quello che ha in mano fino a quando non gli diamo quello che lui vuole)*

#### Timeout

Per evitare un deadlock infinito, le **transazioni rimangono in attesa solo per un tempo massimo prefissato (timeout)**: allo scadere del tempo, se la transazione è ancora in attesa, viene **restituita una risposta negativa e la transazione viene abortita**.

La **scelta del timeout** è molto importante, in quanto se **è troppo basso** potrebbero verificarsi **troppi abort non necessari**, mentre se **è troppo alto** potrebbe portare a dei **ritardi**.

Questa tecnica è utilizzata dalla maggior parte dei DBMS commerciali perché è molto semplice.

#### Rilevamento e risoluzione

In alternativa, questa tecnica rileva la **presenza di cicli** nel **grafo di attesa** controllando le tabelle di lock, in modo tale da **non porre vincoli al comportamento del sistema**. Utilizzando questa tecnica, occorre valutare **ogni quanto tempo effettuare il controllo** e, in caso si verificasse un **deadlock**, scegliere **quale transazione uccidere per rompere il ciclo**.

#### Prevenzione

Infine, questa tecnica **evita a priori che si verifichi un deadlock**, utilizzando tecniche preventive che assicurano che **non vi sarà mutua attesa tra le transazioni** (**uccidendo le transazioni che creerebbero cicli**, che anche in questo caso comporta una politica per la scelta della vittima). Le possibili tecniche preventive sono:

- **2PL conservativo** (non è sempre possibile);
- **ordinamento sugli oggetti**: stabilisce che gli oggetti vengano allocati, da ogni transazione, secondo un ordine stabilito.

#### Scelta della vittima

Nelle tecniche di risoluzione del deadlock che prevedono la scelta di una vittima, si possono adottare politiche:

- **preemptive** (interrompenti): uccidono la transazione che **possiede** la risorsa;
- **non-preemptive** (non interrompenti): uccidono la transazione che **richiede** la risorsa.

### Uccisione di transazioni con timestamp (**rispetto per gli anziani(samariti)**)

Nell'ipotesi che:

- $t_i$  richiede un lock su  $x$  ;
- $t_j$  ha un lock su  $x$  ;

le due possibili tecniche di risoluzione sono:

- non interrompente (**wait-die**):
  - se  $ts(t_i) < ts(t_j)$  , allora  $t_i$  aspetta;
  - altrimenti si esegue un abort di  $t_i$  , che viene poi rilanciato con lo stesso timestamp;
- interrompente (**wound-wait**):
  - se  $ts(t_i) > ts(t_j)$  , allora  $t_i$  aspetta;
  - altrimenti si esegue un abort di  $t_j$  , che viene poi rilanciato con lo stesso timestamp.

Il paragone dell'uccisione di transazioni con timestamp con la metafora "rispetto per gli anziani" consiste nel **considerare le transazioni con timestamp più vecchio come "anziani"** e **ucciderle solo se strettamente necessario**, come si farebbe con le persone anziane nella vita reale, in modo da **garantire il rispetto e la priorità di chi ha aspettato più a lungo**. In questo modo si **evitano inutili rollback** di transazioni che hanno **aspettato a lungo** per acquisire un lock.

Il motivo per cui le **transazioni uccise** vengono **eseguite nuovamente con lo stesso timestamp**, è che si vuole **evitare che queste vengano uccise in continuazione (starvation)**.

Questa tecnica in genere non è utilizzata dai DBMS commerciali, in quanto la probabilità di deadlock è di molto inferiore a quella di conflitto.

### Uccisione di transazioni senza timestamp

Nell'ipotesi che:

- $t_i$  richiede un lock su  $x$  ;
- $t_j$  ha un lock su  $x$  ;

le tre possibili tecniche di risoluzione sono:

- **no waiting**: si esegue l'abort di  $t_i$  , che viene poi rilanciato;
- **cautious waiting**:
  - se  $t_j$  non è in attesa,  $t_i$  aspetta;
  - altrimenti si esegue l'abort di  $t_j$  ;
- **uccisione** della **transazione** che ha **eseguito meno operazioni**.

### Livelock e starvation

Il **livelock** avviene quando una **transazione resta sempre in attesa di un lock che viene continuamente dato ad altri** (succede quando il gestore dei lock non gestisce bene l'allocazione).

La **starvation** avviene quando una **transazione viene continuamente uccisa** (succede se non vengono gestiti propriamente i timestamp).

## 7. Architetture distribuite

Sostanzialmente le basi di dati possono essere **distribuite anche su nodi remoti**:

Esistono diversi **paradigmi per la distribuzione dei dati**:

- **architetture client-server** (separazione del database server dal client);
- **database distribuiti** (più database server utilizzati dalla stessa applicazione);
- **database paralleli** (più dispositivi di memorizzazione e processori che operano in parallelo per migliorare le performance);
- **database replicati** (stessi dati fisicamente memorizzati su più server);
- **database warehouse** (server specializzati per la gestione di dati dedicati al supporto alle decisioni).

### Tipologie di architetture

Esistono due tipologie di sistemi:

- **OLTP** (On-Line Transaction Processing):
  - rivolti alla **gestione ottimizzata di transazioni affidabili** su **database server**;
  - specializzati per gestire centinaia o anche migliaia di transazioni al secondo;
- **OLAP** (On-Line Application Processing):
  - rivolti **all'analisi dei dati**;
  - operano su server di **data warehouse**, specializzati per la gestione di dati per sistemi di supporto alle decisioni.

I server, in genere, **contengono al loro interno sia funzioni OLTP che OLAP**.

### Proprietà dei sistemi distribuiti

I sistemi distribuiti garantiscono alcuni **vantaggi**:

- **portabilità**: possibilità di trasportare programmi da un ambiente all'altro (stabilita dal **tempo di compilazione** e facilitata dai linguaggi standard);
- **interoperabilità**: capacità di interagire tra sistemi eterogenei (stabilita dal **tempo di esecuzione** e facilitata da protocolli standard per l'accesso ai dati).

### 7.1. Architettura client-server

Questa architettura prevede che i **processi software che interagiscono siano divisi tra client** (che richiedono i servizi) e **server** (che forniscono servizi):

- richiede una definizione precisa di **interfaccia di servizi**, che elenca i servizi offerti dal server;
- il client ha un **ruolo attivo** (richiede);
- il server ha un **ruolo reattivo** (risponde);
- generalmente, un processo **client richiede pochi servizi** in sequenza ad uno o più processi server;
- generalmente, un processo **server risponde a più richieste** di molti processi client.

Il computer dedicato al **client** deve essere appropriato all'interazione con l'utente e deve **supportare strumenti di produttività**, mentre il computer che funge da **server** deve avere una **grande quantità di memoria** (per supportare la gestione del buffer) così come una **elevata capacità disco** (per memorizzare l'intera base di dati).

Questo **paradigma è abbastanza diffuso** per le basi di dati, in quanto:

- le funzioni di client e server sono ben definite;
- fornisce una conveniente separazione di attività di gestione;
- SQL stesso offre un paradigma di programmazione ideale per l'identificazione dell'interfaccia dei servizi, infatti:
  - le **interrogazioni SQL** sono **formulate dal client** e **inviate al server**;
  - i **risultati** delle interrogazioni sono **calcolati dal server** e **inviati al client**;

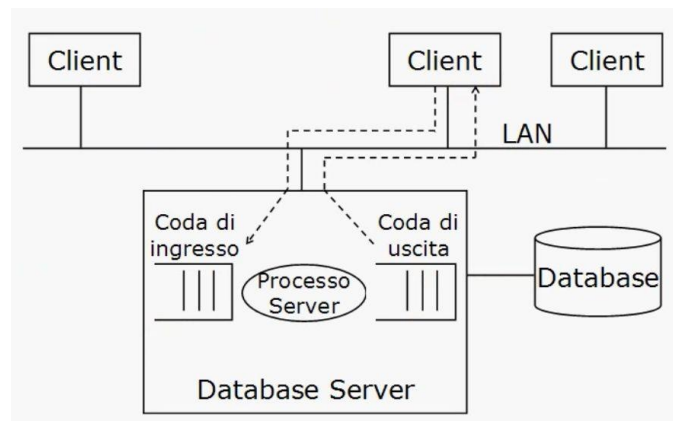
- la standardizzazione, la portabilità e l'interoperabilità di SQL permettono lo sviluppo di applicazioni client che coinvolgono sistemi server differenti.

Spesso il server è **multi-threaded**, ovvero si comporta come un processo singolo che lavora dinamicamente per conto di transazioni differenti, e ogni **unità di esecuzione** del processo server per una data transazione è chiamata **thread**.

I server, inoltre, sono processi permanentemente attivi, che **controllano**:

- una **coda di input** per le richieste dei client;
- una **coda di output** per i risultati delle interrogazioni.

Spesso, la distribuzione di richieste ai server e la restituzione di risposte ai client è eseguita da un processo detto **dispatcher**, e quando i dispatcher possono definire dinamicamente il numero di processi server attivi in funzione del numero di richieste ricevute, si dice che una **classe di server** è disponibile.



## Architetture two-tier a three-tiers

Esistono generalmente due tipologie di architetture client-server:

- **two-tier**: il client funziona sia da interfaccia utente sia da gestore di applicazioni (quindi supporta anchela logica delle applicazioni, ed è infatti detto **thick client**);
- **three-tier**: un secondo server, chiamato **application-server**, è responsabile della gestione delle applicazioni comuni a molti client (il client è quindi responsabile solo dell'interfaccia utente finale, è detto quindi **thin client** e può essere anche sviluppato attraverso un browser).

## 7.2. Basi di dati distribuite

### 7.2.1. Introduzione alle basi di dati distribuite

Una base di dati distribuita è un **sistema di base di dati** nel quale **almeno un client interagisce con più server per l'esecuzione di un'applicazione**.

### Vantaggi

Le basi di dati distribuite portano diversi vantaggi:

- **rispondono ai requisiti delle applicazioni**:
  - le organizzazioni sono strutturalmente distribuite;
  - la distribuzione dei dati permette la loro **gestione dove essi sono generati ed utilizzati**;
- **flessibilità e modularità** (possono essere configurate con aggiunte e modifiche progressive dei componenti);
- **affidabilità** (possono rispondere ai guasti con una riduzione delle prestazioni anziché con un blocco completo, nonostante siano più esposte ai dati).

## Classificazione

Le basi di dati distribuite possono essere classificate secondo due dimensioni:

- tipo di DBMS coinvolti:
  - **DDB omogenea**: tutti i server utilizzano lo stesso DBMS;
  - **DDB eterogenea**: i server utilizzano DBMS diversi;
- tipo di rete (**LAN** o **WAN**)

| Tipo di DBMS      | Tipo di rete                              |                                                         |
|-------------------|-------------------------------------------|---------------------------------------------------------|
|                   | LAN                                       | WAN                                                     |
| <b>Omogeneo</b>   | Applicazioni gestionali e finanziarie     | Sistemi di prenotazione e applicazioni finanziarie      |
| <b>Eterogeneo</b> | Applicazioni gestionali e interfunzionali | Sistemi di prenotazione integrati, sistemi interbancari |

## Autonomia locale e cooperazione

Da un punto di vista astratto, la base di dati distribuita può essere considerata come un'unica base di dati: deve essere progettata in modo da cercare di avere applicazioni che vengono eseguite su un singolo server, limitando la necessità di interazione e di trasporto dei dati.

## 7.2.2. Frammentazione e allocazione dei dati

### Frammentazione dei dati

La frammentazione applica delle operazioni algebriche su una relazione  $R$  per dividerla in frammenti  $R_1, \dots, R_n$ . Ne esistono di due tipi:

- **frammentazione orizzontale**:
  - ogni  $R_i$  ha come tuple un sottoinsieme delle tuple di  $R$ ;
  - ogni  $R_i$  può essere interpretato come il risultato di una **selezione** su  $R$ ;

esempio:

- IMPIEGATO ( Empnum, Nome, Cognome, Stipendio)
- IMPIEGATO1 =  $\sigma$  Empnum  $\leq 3$  IMPIEGATO
- IMPIEGATO2 =  $\sigma$  Empnum  $> 3$  IMPIEGATO
- **Ricostruisco** la relazione: **IMPIEGATO = IMPIEGATO1  $\cup$  IMPIEGATO 2**

- **frammentazione verticale**:
  - ogni  $R_i$  ha come schema un sottoinsieme degli attributi di  $R$ ;
  - ogni  $R_i$  può essere interpretato come il risultato di una **proiezione** su  $R$ .

esempio:

- IMPIEGATO ( Empnum, Nome, Cognome, Stipendio)
- IMPIEGATO1 =  $\pi$  Empnum , nome IMPIEGATO
- IMPIEGATO2 =  $\pi$  Empnum, Cognome, Stipendio IMPIEGATO
- **Ricostruisco** la relazione: **IMPIEGATO = IMPIEGATO1  $\bowtie$  IMPIEGATO 2 (per fare il join devo avere la chiave in comune)**

Nel processo di frammentazione dei dati, ci sono due importanti proprietà di correttezza che devono essere soddisfatte:

- **completezza**: ogni dato di  $R$  deve essere presente in un qualche suo frammento  $R_i$ ;
- **ricostruibilità**:  $R$  deve essere interamente ricostruibile a partire dai suoi frammenti  $R_i$ . Generalmente, i

frammenti orizzontali sono disgiunti (non hanno cioè tuple in comune) mentre i frammenti verticali includono la chiave primaria di  $R$  (garantendo la ricostruibilità).

### Schemi di allocazioni

Descrivono il **mapping delle relazioni** o dei **frammenti ai server** che li memorizzano. Ogni frammento corrisponde ad un file a livello fisico ed è allocato su uno specifico server (virtualmente, la relazione globale è una vista sui frammenti).

L'allocazione può essere:

- **non ridondante**: ogni frammento o relazione è allocato su **un solo server**;
- **ridondante**: almeno un frammento o relazione è allocato su **più di un server**.

### 7.2.3. Accesso ai dati distribuiti

#### Livelli di trasparenza

La **distinzione** tra la **frammentazione** e l'**allocazione** consente di **scrivere applicazioni a diversi livelli**: dal più **astratto** e indipendente dalla frammentazione dei dati, al più **concreto** e dipendente anche dalla loro allocazione fisica. Dal livello più astratto a quello più concreto, si distinguono quattro livelli di trasparenza:

- **frammentazione**, il programmatore:
  - **non** ha bisogno di **conoscere la frammentazione**;
  - **non** ha bisogno di **conoscere l'allocazione**;
- **allocazione**, il programmatore:
  - **deve conoscere** la struttura dei **frammenti**
  - **non** ha bisogno di **conoscere l'allocazione**
- **linguaggio**, il programmatore:
  - **deve conoscere** la struttura dei **frammenti**;
  - **deve conoscere l'allocazione**;
- **assenza di trasparenza**: ogni DBMS accetta il suo proprio "dialetto" SQL (il sistema è eterogeneo e i DBMS non supportano uno standard comune per l'interoperabilità).

FORNITORE(Fnum, Nome, Città)

- frammenti orizzontali:
  - FORNITORE1 =  $\sigma_{città='Milano'}$  (FORNITORE)
  - FORNITORE2 =  $\sigma_{città='Roma'}$  (FORNITORE)
- allocazione dei frammenti orizzontali (con replicazione):
  - FORNITORE1@ditta.milano.it
  - FORNITORE2@ditta.roma1.it
  - FORNITORE2@ditta.roma2.it

vogliamo scrivere una procedura che dato un numero di fornitore ne ritorna il nome ...

#### Trasparenza di frammentazione: es.

- il programmatore
  - **non** ha bisogno di **conoscere la frammentazione**
  - **non** ha bisogno di **conoscere l'allocazione**

```
procedure Query1(:fnum, :nome);  
  select Nome into :nome  
  from Fornitore  
  where Fnum = :fnum;  
end procedure;
```

#### Trasparenza di allocazione: es.

- il programmatore
  - **deve conoscere** la struttura dei **frammenti**
  - **non** ha bisogno di **conoscere l'allocazione**
  - in caso di ridondanze, non deve indicare quale copia è scelta per l'accesso (trasparenza di replicazione)

```
procedure Query2(:fnum, :nome);  
  select Nome into :nome  
  from Fornitore1  
  where Fnum = :fnum;  
if :empty then  
  select Nome into :nome  
  from Fornitore2  
  where Fnum = :fnum;  
end procedure;
```

#### Trasparenza di linguaggio: es.

- il programmatore
  - **deve conoscere** la struttura dei **frammenti**
  - **deve conoscere l'allocazione**
  - in caso di ridondanze, deve indicare quale copia è scelta per l'accesso

```
procedure Query3(:fnum, :nome);  
  select Nome into :nome  
  from Fornitore1@ditta.milano.it  
  where Fnum = :fnum;  
if :empty then  
  select Nome into :nome  
  from Fornitore2@ditta.roma1.it  
  where Fnum = :fnum;  
end procedure;
```

### Ottimizzazioni delle interrogazioni

L'applicazione può essere ottimizzata in due modi:

- **parallelismo**: si sottomettono richieste in parallelo anziché in sequenza, in modo tale da ridurre il tempo di risposta globale;
- **conoscenza delle proprietà logiche dei frammenti**: si eseguono *query al frammento dove si trovano i dati*, in modo tale da **aumentare l'efficienza** (anche se diminuisce la flessibilità)

#### Ottimizzazioni delle interrogazioni: es.

```
procedure Query4(:fnum,:nome,:città);
case :città of
  "Milano":
    select Nome into :nome
    from Fornitore1
    where Fnum = :fnum;
  "Roma":
    select Nome into :nome
    from Fornitore2
    where Fnum = :fnum;
end procedure;
```

Scelgo la tabella in base alla città. Se è Milano so che sarà nella tabella F1, se Roma so che sarà nella F2

### Classificazione di transazioni

Le transazioni distribuite possono essere classificate in base alla loro **complessità**:

- **richieste remote**: transazioni read-only composte da un numero arbitrario di query SQL, tutte indirizzate ad un singolo DBMS remoto che può essere solamente interrogato;
- **transazioni remote**: composte da un numero qualsiasi di comandi SQL, tutti indirizzati ad un singolo DBMS remoto; ogni transazione scrive su un singolo DBMS;
- **transazioni distribuite**: composte da un numero qualsiasi di comandi SQL indirizzati ad un numero arbitrario di DBMS remoti; ogni comando si riferisce ad un singolo DBMS ed ogni transazione può aggiornare più di un DBMS (richiede un protocollo di tipo **two-phase-commit**);
- **richieste distribuite**: transazioni arbitrarie composte da un numero qualsiasi di comandi SQL, indirizzati ad un numero arbitrario di DBMS remoti; ogni comando si può riferire ad un qualsiasi DBMS (richiede un **ottimizzatore di query distribuite**).

#### Transazione: esempio (1)

CC(Num,Nome,Saldo)

- CC1:  $\sigma_{Num \leq 1000}(CC)$
- CC2:  $\sigma_{Num > 1000}(CC)$
- assumiamo trasparenza di allocazione

Transazione distribuita

- trasferimento di 100 Euro dal conto 354 al conto 1487

#### Nota

è **necessario garantire atomicità**: o entrambe le modifiche sono eseguite o non lo è nessuna

#### Transazione: esempio (2)

```
begin transaction
```

```
update CC1
set Saldo = Saldo - 100
where CCNum = '354';
```

```
update CC2
set Saldo = Saldo + 100
where CCNum = '1487';
```

```
commit;
end transaction
```

## 7.3. Tecnologia della basi di dati distribuite

La **distribuzione** dei dati **non influenza**:

- **consistenza**: i vincoli di integrità descrivono solo proprietà locali (è un limite della corrente tecnologia DBMS, che infatti **non permette di definire vincoli su dati distribuiti**);
- **persistenza** (durabilità): ogni sistema garantisce persistenza ai dati localmente memorizzati (con soli meccanismi di recovery locali);

bensì **influenza** le proprietà di **isolamento e atomicità**, perciò occorre rivedere i meccanismi che le **garantiscono in un contesto distribuito** e non più centralizzato.

### 7.3.1. Ottimizzazione e atomicità

La distribuzione dei dati richiede modifiche a:

- **ottimizzazione** delle interrogazioni;
- controllo di **concorrenza** (isolamento);
- controllo di **affidabilità** (atomicità).

### Ottimizzazione di query distribuite

L'ottimizzazione è sotto la responsabilità del DBMS che riceve la query, che:

- decide la **suddivisione** della query in **sub-query**, ognuna indirizzata ad un **DBMS specifico**;
- costruisce una **strategia di esecuzione distribuita**:
  - esecuzione coordinata di vari programmi a vari DBMS;
  - scambio di dati tra DBMS (e relativo costo di trasmissione);
- assicura un'ottimizzazione **globale**.

Nel calcolo del costo delle interrogazioni distribuite ha particolare importanza la **quantità di dati trasmessi sulla rete**:

$$C_{totale} = C_{I/O} \times n_{I/O} + C_{CPU} \times n_{CPU} + C_{tr} \times n_{tr} \rightarrow \text{cacapupu} \text{ 😊}$$

### Atomicità

Questa proprietà può essere compromessa da guasti o malfunzionamenti:

- **guasti a nodi** (hardware o software);
- **perdite di messaggi**: lasciano l'esecuzione di un protocollo in uno stato non certo:
  - ogni messaggio del protocollo è seguito da un messaggio di conferma di ricezione (acknowledgement);
  - la perdita di uno dei messaggi lascia il mittente insicuro circa la sua ricezione;
- **guasti a link di comunicazione**: possono causare **partizionamenti** della rete (una trasmissione può essere simultaneamente attiva in più di una sottorete).

## 7.3.2. Controllo di concorrenza

In un sistema distribuito, una transazione  $t_i$  può eseguire più sottotransazioni a diversi nodi, la cui notazione  $t_{ij}$  indica l'esecuzione di  $t_i$  al nodo  $j$ .

### Serializzabilità globale

La **serializzabilità locale non garantisce quella globale**, che richiede invece l'esistenza di uno **schedule seriale S equivalente a tutti gli schedule locali  $S_i$  risultanti ad ogni nodo  $i$**  (la proiezione di S sul nodo  $i$  deve essere uguale a  $S_i$ ).

### Proprietà della serializzabilità globale

- Schedule globale **conflict-serializzabile**: garantita se ogni scheduler usa **2PL stretto** ed **esegue il commit in modo atomico** quando tutte le sottotransazioni ai vari nodi hanno tutte le risorse;
- schedule globale **seriale**: garantita se ogni transazione distribuita acquisisce un **singolo timestamp** e lo usa in tutte le richieste a tutti gli scheduler che fanno controllo della concorrenza basato su timestamp (richiede assegnamento di timestamp globale).

### Metodo di Lamport

Risolve il problema della sincronizzazione di eventi in un ambiente distribuito, in quanto **permette di assegnare timestamp che riflettono la precedenza tra eventi**, nel seguente modo:

- il timestamp ha **due gruppi di cifre**:
  - le **meno significative** identificano un **nodo**;
  - le **più significative** identificano gli **eventi** che **accadono al nodo** (contatore locale);
- ogni volta che due nodi si scambiano un messaggio, i **timestamp vengono sincronizzati** (l'evento **ricevente**

deve avere un **timestamp maggiore o uguale** dell'evento mittente, il che può richiedere l'incremento del contatore locale al nodo ricevente).

### Deadlock distribuiti

Anche in un contesto distribuito possono verificarsi situazioni di **attesa circolare** tra due o più nodi, le soluzioni possibili sono:

- **time-out** (utilizzata dalla maggior parte dei DBMS distribuiti);
- **rilevamento e risoluzione** (può essere eseguita con un protocollo asincrono e distribuito, attivato periodicamente ai vari DBMS).

### 7.3.3. Protocollo di commit a due fasi

I **protocolli per il commit distribuito** permettono ad una transazione di **raggiungere la decisione corretta di commit o abort su tutti i nodi che partecipano alla transazione**.

Il più diffuso è il **two-phase commit**, che prevede che la **decisione di commit o abort presa dalle parti sia registrata da una terza parte, che ratifica la decisione**, distinguendo:

- i server che partecipano alla decisione (**gestori di risorse**: RM);
- il processo coordinatore (**gestore della transazione**: TM).

Questo protocollo funziona tramite un **rapido scambio di messaggi** (broadcast o seriale) **tra TM e RM e le scritture dei record nei log, per assicurare la resistenza ai guasti**.

Il **log del gestore della transazione** contiene tre diversi tipi di record:

- **prepare** : contiene l'identità di tutti i processi RM (identificatori dei nodi e dei processi);
- **global commit o global abort** : descrive la decisione globale (la decisione del TM è finale al momento della scrittura nel log);
- **complete** : registra la fine del protocollo di commit a due fasi.

Mentre il **log dei gestori delle risorse** contiene, oltre ai record di **begin** , **insert** , **delete** , e **update** della transazione locale, anche il record di **ready** , che:

- indica la **disponibilità irrevocabile a partecipare al protocollo di commit a due fasi**, e contribuire quindi alla decisione finale;
- può essere scritto solo quando **RM si trova in uno stato affidabile**, cioè quando ha il **lock su tutte le risorse che devono essere scritte**;
- riposta l'identificatore, sia di processo che di nodo, del TM.

Quando un RM si dichiara pronto per una transazione, perde la propria autonomia e deve **sottostare alla decisione del TM** (perciò le risorse acquisite dalla transazione sono bloccate e rimane in una fase di incertezza). Invece, prima di dichiararsi pronto (o una volta dichiaratosi **non-ready**), il RM può abortire autonomamente facendo un **undo** dei suoi effetti, senza partecipare al protocollo.

### Funzionamento del protocollo

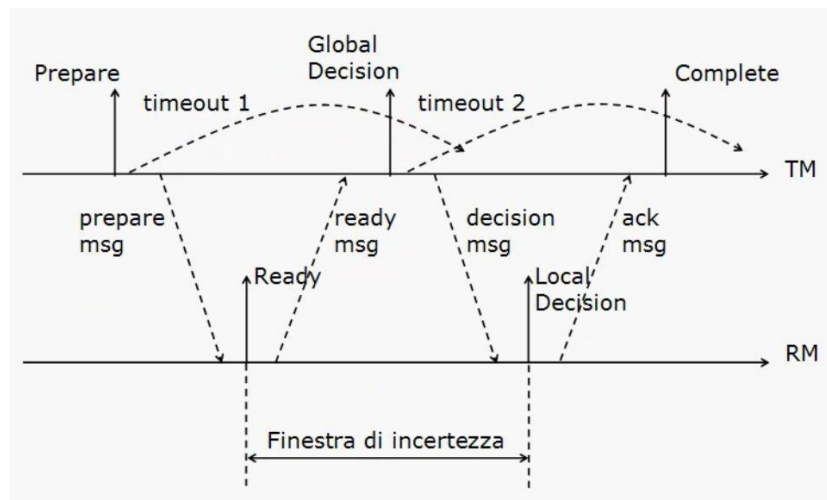
#### 1. **Prima fase:**

- il **TM** scrive il record **prepare** nel suo log e manda un messaggio **prepare** a tutti gli **RM**, impostando un **timeout**;
- ogni **RM**:
  - se è in uno **stato affidabile**, scrive nel log il record **ready** e trasmette al **TM** il messaggio **ready** ;
  - altrimenti, manda un messaggio **non-ready** e **termina la propria partecipazione al protocollo**;
- il **TM** raccoglie i messaggi di risposta e scrive nel log il record:

- **global commit**, se tutti gli **RM** hanno risposto **ready** ;
- **global abort**, se almeno un **RM** ha risposto **non-ready** o il **timeout** è scattato prima che tutti gli RM rispondessero;

## 2. **Seconda fase:**

- il **TM** trasmette la sua decisione globale a tutti gli **RM**, impostando un timeout;
- ogni **RM** in stato di **ready** :
  - scrive nel log il record relativo alla decisione globale e manda un messaggio di acknowledgement al **TM**;
  - esegue in loco la decisione globale;
- il **TM**:
  - raccoglie tutti gli acknowledgement dagli **RM** coinvolti nella seconda fase;
  - se il **timeout** *scade*, stabilisce un **altro timeout** e ripete il messaggio a tutti gli **RM** dai quali non ha ricevuto risposta;
  - quando tutte le risposte sono arrivate, scrive il record **complete** nel suo log.



## Protocolli di ripristino

Permettono di ripristinare la correttezza dello stato dei nodi a seguito di guasti avvenuti durante l'esecuzione del protocollo di commit a due fasi:

- **guasto di un RM:** si esegue una ripresa a caldo (durante la quale sono raccolti in un insieme gli identificatori delle transazioni in dubbio, per ognuna delle quali la decisione finale deve essere richiesta al TM), che dipende dall'ultimo record di log:
  - **abort** o azione: *undo* della transazione;
  - **commit** : *redo* della transazione;
  - **ready** : indica che il guasto è avvenuto durante il commit a due fasi; perciò, occorre interrogare il TM;
- **guasto del TM:** si esegue anche in questo caso una ripresa a caldo, che dipende sempre dall'ultimo record di log:
  - **prepare** : alcuni RM potrebbero essere in uno stato di blocco nella finestra di incertezza, perciò ci sono due opzioni:
    - scrivere **global abort** nel log ed eseguire nuovamente la seconda fase del protocollo;
    - ripetere la prima fase;
  - **global commit** o **global abort** : alcuni RM potrebbero essere stati correttamente informati mentre altri potrebbero essere ancora bloccati nella finestra di incertezza; perciò, il TM deve ripetere la seconda fase;
  - **complete** : non ha effetti sulla transazione;
- **perdita di messaggi:** la perdita di un messaggio **prepare** o **ready** non sono distinguibili dal TM; perciò, il timeout scadrà e verrà presa la decisione globale di abortire; analogamente, la perdita di un messaggio di

decisione globale o di `ack` non sono distinguibili dal TM; perciò, il timeout scadrà e verrà ripetuta la seconda fase;

- partizionamento della rete: in questo caso non causa ulteriori problemi, in quanto la transazione avrà successo solo se il TM e gli RM appartengono alla stessa partizione.

### Ottimizzazioni

Questo protocollo è abbastanza oneroso, soprattutto a causa delle scritture sincrone (force) richieste su ogni log; perciò, i sistemi utilizzano generalmente il protocollo con due ottimizzazioni:

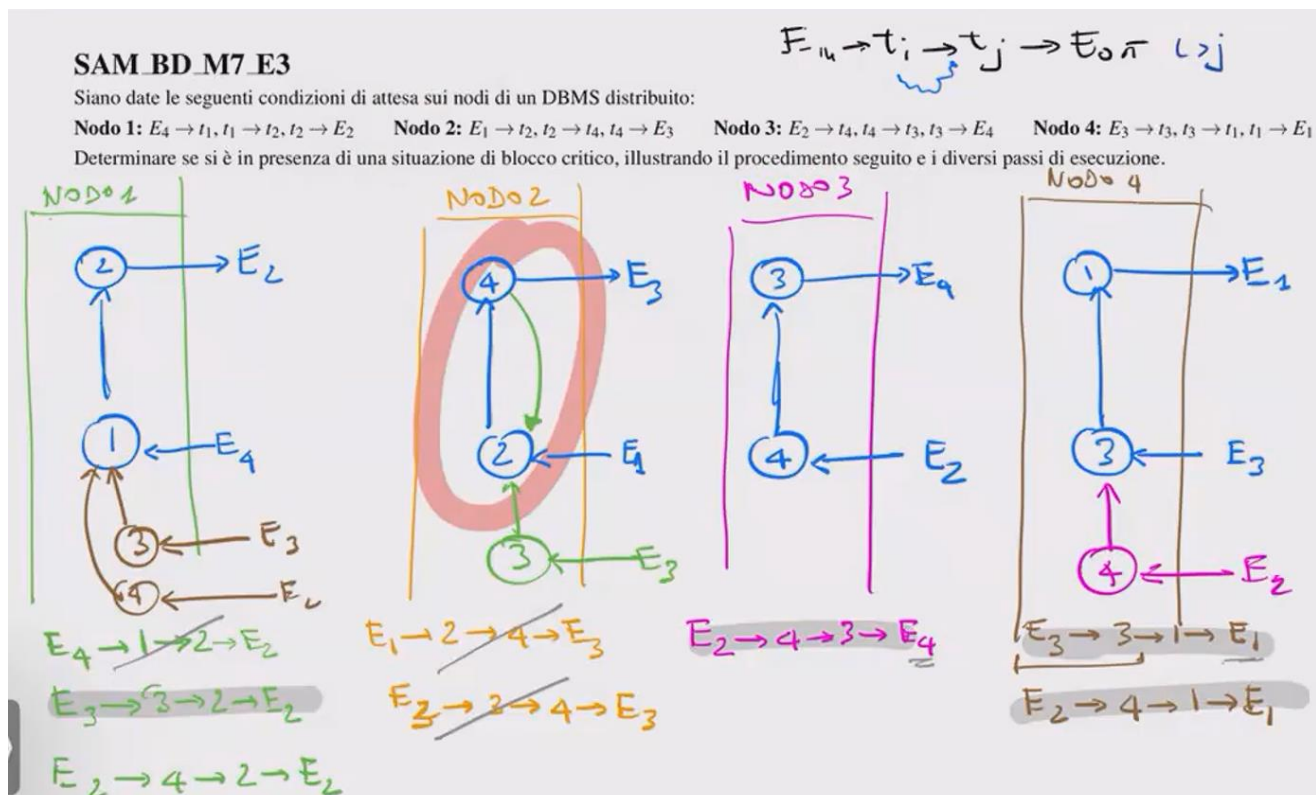
- abort presunto: un TM che riceve una richiesta di recovery remoto da un RM incerto e non ha alcuna informazione sulla transazione, restituisce un `global abort` di default, così:
  - si evita la scrittura sincrona dei record `prepare` e `global abort` (il cui ripristino equivale al default);
  - il record `complete` non è critico e può essere omesso (al più si ripete la seconda fase);
- read only: un partecipante che ha eseguito solo operazioni di lettura nella transazione può dichiararsi `read-only` e lasciare il protocollo, e verrà così ignorato dal coordinatore nella seconda fase del protocollo.

### Alcune varianti

Esistono due varianti a questo protocollo:

- commit a quattro fasi:
  - il TM è replicato da un processo di backup ad un nodo differente;
  - ad ogni fase del protocollo, il TM informa prima il backup della sua decisione e poi la comunica agli RM;
  - il backup può sostituire il TM in caso di guasti;
- commit a tre fasi:
  - introduce una terza fase di pre-commit;
  - se il TM cade, un partecipante può essere eletto come nuovo TM;
  - è inutilizzabile in pratica perché allunga la finestra di incertezza

### ESERCIZIO SAM\_BD\_M7\_E3



## SAM\_BD\_M7\_E1

Data la seguente base di dati:

ABITO(Codice, NomePersonaggio, **Categoria**, Taglia, Prezzo)  
ACCESSORIO(Codice, **Tipo**, NomePersonaggio, Prezzo)

- Progettare e scrivere la frammentazione *orizzontale* in base ai seguenti criteri:
  - ABITO diviso in 2 frammenti in base alla **Categoria** di personaggio a cui è ispirato il costume, che può essere "Principessa" oppure "Fatina" allocati nei 2 centri di gestione che si trovano rispettivamente a "Parma" e a "Ferrara".
  - ACCESSORIO diviso in 3 frammenti in base al **tipo** di accessorio, che può essere "tiara", "guanti" oppure "bacchetta magica", allocati nei 3 centri di gestione, uno per ciascuna *tipo*, che si trovano a Torino, Genova, e Bergamo rispettivamente.
- Scrivere poi una query che ritorni il prezzo massimo dei costumi da Fatina, considerando tutti i 3 diversi livelli di trasparenza (*frammentazione, allocazione e linguaggio*).

2

```

- abito_principessa = sel cat='principessa' ABITO
- abito_fatina = sel cat = 'fatina' ABITO
    
```

```

abito_principessa @ parma
abito_fatina @ Ferrara
    
```

3

```

- accessorio_tiarra = sel tipo='tiara' ACCESSORIO
- accessorio_guanti = sel tipo='guanti' ACCESSORIO
- accessorio_bacc_magica = sel tipo='bacchettamagica' ACCESSORIO
    
```

```

accessorio_tiarra @ torino
accessorio_guanti @ genova
accessorio_bacc_magica @ bergamo
    
```

**TRASPARENZA DI FRAMMENTAZIONE** --> query che ignorano che la base di dati sia stata distribuita

scrivo la query normale così come è

```

select MAX(Prezzo)
from ABITO
where categoria = 'Fatina'
    
```

**TRASPARENZA DI ALLOCAZIONE** --> query che lavorano a livello di frammento

```

select MAX(Prezzo)
from abito_fatina
    
```

**TRASPARENZA DI LINGUAGGIO** --> query che si interessano sia della frammentazione che dell'allocazione

```

select MAX(Prezzo)
from abito_fatina @ Ferrara
    
```

## SAM\_BD\_M7\_E2

Data la seguente base di dati:

CONTOCORRENTE(Numero, Tipologia, Saldo, PercInteresse, MaxScoperto, IDFiliale)  
FILIALEBANCA(ID, Città, Via, Provincia, Telefono, Direttore)

- Progettare e scrivere la frammentazione *orizzontale* in base ai seguenti criteri:
  - FILIALEBANCA** diviso in 4 frammenti in base alla provincia (che può essere "Milano", "Bergamo", "Cremona" e "Napoli") allocati nei 4 centri di calcolo allocati rispettivamente a "Milano", "Bergamo", "Cremona" e "Napoli".
  - CONTOCORRENTE** diviso in 4 frammenti in base alla filiale della banca cui ciascun conto corrente afferisce.
- Scrivere poi una query che ritorni il Saldo e il Numero di ContoCorrente di tutti i conti della filiale avente ID 96669 considerando tutti i 3 diversi livelli di trasparenza (*frammentazione, allocazione e linguaggio*).

```

FilialeBanca_M = sel provincia='milano' FilialeBanca
FilialeBanca_B = sel provincia='bergamo' FilialeBanca
FilialeBanca_C = sel provincia='cremona' FilialeBanca
FilialeBanca_N = sel provincia='napoli' FilialeBanca
    
```

```

FilialeBanca_M @ milano
FilialeBanca_B @ bergamo
FilialeBanca_C @ cremona
FilialeBanca_N @ napoli
    
```

```

ContoCorrente_M = riden idfiliale<- id (proiez_id FilialeBanca_M) join ContoCorrente
ContoCorrente_B = riden idfiliale<- id (proiez_id FilialeBanca_B) join ContoCorrente
ContoCorrente_C = riden idfiliale<- id (proiez_id FilialeBanca_C) join ContoCorrente
ContoCorrente_N = riden idfiliale<- id (proiez_id FilialeBanca_N) join ContoCorrente
    
```

```

ContoCorrente_M @ milano
ContoCorrente_B @ bergamo
ContoCorrente_C @ cremona
ContoCorrente_N @ napoli
    
```

**TRASPARENZA DI FRAMMENTAZIONE** --> query che ignorano che la base di dati sia distribuita

```

select saldo, Numero
from contocorrente join filialebanca on idfiliale=id
where id=96669
    
```

**TRASPARENZA DI ALLOCAZIONE** --> query che lavorano a livello di frammento

```

select saldo, Numero
from ContoCorrenteM
where id=96669
    
```

IF EMPTY --> stessa select di prima solo che cambio il nodo nel from (per tutti)

**TRASPARENZA DI LINGUAGGIO** --> query che si interessano sia dell'allocazione che della frammentazione

```

select saldo, Numero
from ContoCorrenteM @ milano
where id=96669
    
```

IF EMPTY --> stessa select di prima solo che cambio il nodo nel from (per tutti) e anche il @...

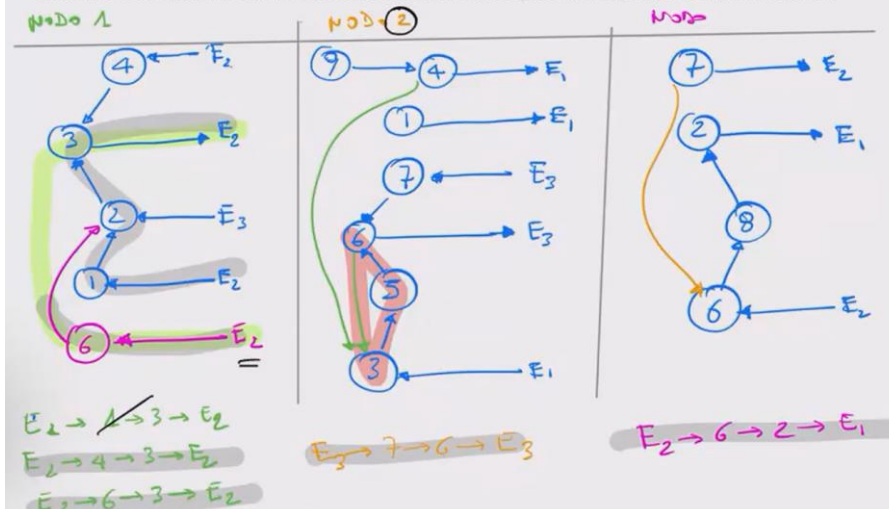
## ESERCIZIO SAM\_BD\_M7\_E4

### SAM BD M7 E4

Siano date le seguenti condizioni di attesa sui nodi di un DBMS distribuito:

- Nodo 1:**  $E_2 \rightarrow t_1, t_1 \rightarrow t_2, E_3 \rightarrow t_2, t_2 \rightarrow t_3, t_3 \rightarrow E_2, E_2 \rightarrow t_4, t_4 \rightarrow t_5$
- Nodo 2:**  $E_1 \rightarrow t_3, t_3 \rightarrow t_5, t_5 \rightarrow t_6, t_6 \rightarrow E_3, E_3 \rightarrow t_7, t_7 \rightarrow t_6, t_6 \rightarrow t_4, t_4 \rightarrow E_1, t_1 \rightarrow E_1$
- Nodo 3:**  $E_2 \rightarrow t_6, t_6 \rightarrow t_8, t_8 \rightarrow t_2, t_2 \rightarrow E_1, t_7 \rightarrow E_2$

Determinare se si è in presenza di una situazione di blocco critico, illustrando il procedimento seguito e i diversi passi di esecuzione.



## 8. Dati semistrutturati: XML

Nelle basi di dati relazionali, **tutti i dati devono rispettare lo schema**, ovvero la **struttura è definita**, perciò ci riferisce a questi dati come **dati strutturati**. In alcune applicazioni, però, **i dati possono essere raccolti prima ancora di sapere come verranno memorizzati e gestiti**, perciò possono non avere una struttura, o comunque **rispettare uno schema in modo parziale**, e si parla quindi di **dati semistrutturati**.

### 8.1. Introduzione a XML

XML (eXtended Markup Language) è un **formato di file** proposto dal W3C per **distribuire documenti elettronici sul web**, e serve a **rappresentare dati semistrutturati (o non strutturati)**. In origine è stato concepito come un'**estensione di SGML** e come **metalinguaggio per la specifica di linguaggi di markup**, ma **oggi è l'anello di congiunzione tra il linguaggio HTML e i sistemi informativi**.

#### XML, HTML e basi di dati

In XML, come in HTML:

- i **dati sono documenti**
- le **proprietà dei dati** sono espresse mediante **marcatore del contenuto**;e,

come nelle basi di dati:

- esiste un **modello dei dati** (DTD, XML Schema)
- esistono **linguaggi di query** e trasformazione
- i **dati sono neutri** rispetto alle modalità di **rappresentazione**

→ XML e HTML sono nati con **scopi diversi**: mentre **XML è progettato per descrivere i dati** ed evidenziare cosa rappresentano, **HTML è progettato per visualizzare i dati** ed evidenziare come farli apparire.

#### Uso di XML

L'utilizzo di XML serve a:

- **separare** i dati da come vengono visualizzati;
- **scambiare** i dati tra **sistemi** altrimenti **incompatibili**;
- scambiare informazioni in sistemi B2B;
- **condividere** dati;
- **memorizzare** dati;
- **supportare** documenti nei browser moderni.

#### Sintassi di XML

Di seguito le regole sintattiche dei documenti XML:

- **ogni documento deve iniziare** con la dichiarazione di:
  - **versione di XML** utilizzata;
  - tipo di **codifica dei caratteri** utilizzati nel file (opzionale);
- **ogni documento deve avere** un elemento **tag radice** (in cui devono essere **compresi tutti gli altri elementi**);
- tutti gli elementi hanno un **tag di apertura** e uno di **chiusura** (che **non può essere omissa**);
- i **tag sono case-sensitive**;
- i **tag** devono essere **annidati correttamente**

Un documento è detto **ben formato (well formed)** se rispetta le regole sintattiche proprie di XML.

## Elementi e attributi

Quindi, un documento XML è composto da **elementi** che:

- si possono estendere
- possono includere altri elementi (figli)
- hanno un contenuto (che può essere vuoto)
- possono avere degli attributi

Gli **attributi** degli elementi:

- consentono di associare un valore agli elementi **senza** essere considerati parte del loro contenuto;
- hanno valori racchiusi tra doppi apici;
- differiscono dagli elementi perché **non possono contenere elementi figli**;
- possono contenere **un solo valore**;
- **non** sono **facilmente estendibili**;
- **non** possono **descrivere delle strutture**;
- sono **difficilmente manipolabili** dalle applicazioni;
- sono **utili** per **memorizzare metadati**.

## Namespace

Dal momento che in XML i tag sono **personalizzabili**, e che in **documenti diversi** ci possono essere **elementi o attributi con lo stesso nome**, si utilizza un **prefisso** nei tag per **eliminare le ambiguità**.

Un **namespace** è una collezione di nomi (di attributi ed elementi) identificato da un **URI** (Uniform Resource Identifier, che, a differenza di un URL, **rappresenta un nome e non una locazione**), e ne va dichiarato l'utilizzo all'interno del documento XML.

Un namespace, **se associato ad un elemento**, si **applica anche ai figli dell'elemento**, e la sua **dichiarazione nel prologo** del documento si **applica a tutti gli elementi del documento**; tuttavia, **elementi fratelli possono avere namespace diversi**.

## 8.2. DTD e XML Schema

Permettono di **definire un modello** per i documenti, dettando il **tipo dei documenti**, ovvero:

- i **tag** (elementi) ammessi e le **loro proprietà** (cardinalità, attributi, domini e valori di default degli attributi, ecc.);
- le **regole di annidamento dei tag**.

Un **documento XML che rispetta il DTD o l'XML Schema** ad esso associato (che è cioè validato rispetto adesso) **è detto valido (valid)**

### Data Type Definition

Permette la dichiarazione di:

- **elementi**:
  - **definizione**:
    - contenenti altri elementi (figli);
    - con PCDATA (parsed character data, ovvero testo);
    - contenuto misto;
    - elementi vuoti;
  - **cardinalità**:
    - una volta;
    - una o più volte (+);
    - zero o più volte (\*);
    - zero o una volta (?);
- **attributi**:
  - **tipo** (pochi, predefiniti e solo testuali);
  - **vincoli** (obbligatorie (#REQUIRED), opzionali (IMPLIED), fissi (#FIXED) o di default).

```

<!ELEMENT ELENCO (PRODOTTO+)>
<!ELEMENT PRODOTTO (DESCRIZIONE, PREZZO?)>
<!ATTLIST PRODOTTO codice ID #REQUIRED>
<!ELEMENT DESCRIZIONE (#PCDATA)>
<!ELEMENT PREZZO (#PCDATA)>

<elenco>
  <prodotto codice="123">
    <descrizione> libro </descrizione>
    <prezzo> 50 </prezzo>
  </prodotto>
  <prodotto codice="456">
    <descrizione> penna stilo </descrizione>
  </prodotto>
</elenco>

```

#PCDATA → testo  
(prodotto) → figlio di elenco

## XML Schema

Noto anche come XSchema o XML Schema Definition (XSD, è **successivo al DTD**, ma lo **sta gradualmente sostituendo**, essendo diventato W3C recommendation. **Definisce gli elementi e la composizione di un documento XML** e, rispetto ai DTD, presenta i seguenti **vantaggi**:

- è **estendibile**;
- è un **file XML**;
- è più **ricco e completo** (contempla più tipi di dato, elementi, ecc.);
- gestisce il **namespace**.

Permette la dichiarazione di:

- **elementi semplici**: <xs:element ... ..>
  - **non possono contenere altri elementi o attributi**;
  - possono essere **solo tipi di dato standard** (string, date, boolean, ecc.) o **tipi di dato derivati** da questi;
  - possono avere valori di **default**;
  - possono avere **valori fissi**;
  - possono avere associate delle **restrizioni**:
    - lista di valori accettabili;
    - numero di decimali dopo la virgola;
    - numero esatto di caratteri, eventualmente alfanumerici;
    - limite numerico inferiore o superiore, compreso o escluso;
    - lunghezza minima o massima;
    - pattern (sequenza di caratteri);
    - tipo di gestione del whitespace

### Esempio

- **definizione in XSD**

```

<xs:element name="età" type="xs:integer" />
<xs:element name="cognome" type="xs:string" />

```
- **utilizzo in XML**

```

<età> 65 </età>
<cognome> Rossi </cognome>

```

- **elementi complessi**: <xs:complexType> ... ..</xs:complexType>
  - possono **contenere attributi o altri elementi**;
  - possono **utilizzare indicatori**;
- **indicatori**:
  - **ordinamento**:
    - può comparire qualsiasi elemento, in qualunque ordine (Any)
    - devono comparire tutti gli elementi, in qualunque ordine (All)
    - può comparire uno e un solo elemento (Choice)
    - devono comparire tutti gli elementi, nell'ordine specificato (Sequence)
  - **cardinalità** (numero massimo e minimo di occorrenze) (maxOccurs/minOccurs)
  - **raggruppamento** (Group name, Group reference)

## Elementi complessi: esempio

```
<xs:element name="persona">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="Cognome" type="xs:string" />
      <xs:element name="Nome" type="xs:string"
        maxOccurs="4" minOccurs="1" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

- **attributi:** `<xs:attribute ... .. >`
  - 0 possono essere solo tipi di dato standard (string, date, boolean, ecc.);
  - 0 possono avere valori di default (default)
  - 0 possono avere valori fissi (fixed)
  - 0 possono essere obbligatori o opzionali
  - 0 possono avere associate delle restrizioni (facet) (minimo e massimo, pattern di valori)

```
<xs:attribute name="nome" type="tipo" />
<xs:attribute name="nome" type="tipo"
  default|fixed="xyz"
  use="required|optional" />
```

### Esempio

- **definizione in XSD**

```
<xs:element name="cognome">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="lang"
          type="xs:string" use="required" />
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

- **utilizzo in XML**

```
<cognome lang="it"> Rossi </cognome>
```

```
<?xml version="1.0"?>
<schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="to" type="xs:string">
          <xs:element name="from" type="xs:string">
            <xs:element name="head" type="xs:string">
              <xs:element name="body" type="xs:string">
                </xs:sequence>
              </xs:complexType>
            <xs:attribute name="when" type="xs:date" use="required" />
          </xs:element>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </schema>

<?xml version="1.0"?>
<note when="15/07/2006">
  <to> Rossi Andrea </to>
  <from> Bianchi Riccardo </from>
  <head> Promemoria </head>
  <body> Compleanno Michela </body>
</note>
```

Proposto dal W3C, permette di esprimere interrogazioni su documenti XML ed è basato su XPath ed espressioni FLWOR (che si basa su clausole For, Let, Where, Order, Return).

## XPath

Permette di indirizzare parti di un documento, modellando il documento XML come un albero di nodi (con relazioni di contenimento padre-figlio). Una path expression viene valutata in base al nodo contesto e seleziona oggetti del documento che corrispondono ad un insieme di cammini sull'albero e può restituire:

- insieme di nodi (non ordinato, privo di duplicati);
- valore booleano;
- numero (in virgola mobile);
- stringa alfanumerica.

|     |                                         |
|-----|-----------------------------------------|
| .   | nodo corrente                           |
| ..  | nodo padre del nodo corrente            |
| /   | nodo radice, o figlio del nodo corrente |
| //  | discendente del nodo corrente           |
| @   | attributo del nodo corrente             |
| *   | qualsiasi nodo                          |
| [ ] | predicato                               |
| [n] | posizione                               |

Es: seleziona l'attributo codice di tutti gli elementi libro figli di libreria → libreria/libro/@codice

Trova tutti i libri nell'elenco all'interno del documento libri.xml

```
document("libri.xml")/Elenco/Libro
```

```
<?xml version="1.0"?>
<Elenco>
  <Libro disponibilità="N">
    <Titolo>Il nome della rosa</Titolo>
    <Autore>Umberto Eco</Autore>
    <Data>1987</Data>
    <ISBN>55-344-2345-1</ISBN>
    <Editore>Bompiani</Editore>
  </Libro>
  <Libro disponibilità="S">
    <Titolo>Il sospetto</Titolo>
    <Autore>F. Dürrenmatt</Autore>
    <Data>1990</Data>
    <ISBN>88-07-81133-2</ISBN>
    <Editore>Feltrinelli</Editore>
  </Libro>
</Elenco>
```

```
<Libro disponibilità="N">
  <Titolo>Il nome della rosa</Titolo>
  <Autore>Umberto Eco</Autore>
  <Data>1987</Data>
  <ISBN>55-344-2345-1</ISBN>
  <Editore>Bompiani</Editore>
</Libro>
<Libro disponibilità="S">
  <Titolo>Il sospetto</Titolo>
  <Autore>F. Dürrenmatt</Autore>
  <Data>1990</Data>
  <ISBN>88-07-81133-2</ISBN>
  <Editore>Feltrinelli</Editore>
</Libro>
```

Trova tutti i titoli dei libri dell'editore Feltrinelli che sono disponibili e che si trovano nel documento libri.xml

```
document("libri.xml")/Elenco/Libro[Editore="Feltrinelli"
AND @disponibilità="S"]/Titolo
```

```
<?xml version="1.0"?>
<Elenco>
  <Libro disponibilità="N">
    <Titolo>Il nome della rosa</Titolo>
    <Autore>Umberto Eco</Autore>
    <Data>1987</Data>
    <ISBN>55-344-2345-1</ISBN>
    <Editore>Bompiani</Editore>
  </Libro>
  <Libro disponibilità="S">
    <Titolo>Il sospetto</Titolo>
    <Autore>F. Dürrenmatt</Autore>
    <Data>1990</Data>
    <ISBN>88-07-81133-2</ISBN>
    <Editore>Feltrinelli</Editore>
  </Libro>
</Elenco>
```

```
<Titolo>Il sospetto</Titolo>
```

Trova gli elementi contenuti nel primo libro del documento libri.xml

```
document("libri.xml")/Elenco/Libro[1]/*
```

Trova l'insieme di tutti gli autori che si trovano nel documento libri.xml annidati a qualunque livello

```
document("libri.xml")//Autore
```

```
<?xml version="1.0"?>
<Elenco>
  <Libro disponibilità="N">
    <Titolo>Il nome della rosa</Titolo>
    <Autore>Umberto Eco</Autore>
    <Data>1987</Data>
    <ISBN>55-344-2345-1</ISBN>
    <Editore>Bompiani</Editore>
  </Libro>
  <Libro disponibilità="S">
    <Titolo>Il sospetto</Titolo>
    <Autore>F. Dürrenmatt</Autore>
    <Data>1990</Data>
    <ISBN>88-07-81133-2</ISBN>
    <Editore>Feltrinelli</Editore>
  </Libro>
</Elenco>
```

```
<Autore>Umberto Eco</Autore>
<Autore>F. Dürrenmatt</Autore>
```

```
<?xml version="1.0"?>
<Elenco>
  <Libro disponibilità="N">
    <Titolo>Il nome della rosa</Titolo>
    <Autore>Umberto Eco</Autore>
    <Data>1987</Data>
    <ISBN>55-344-2345-1</ISBN>
    <Editore>Bompiani</Editore>
  </Libro>
  <Libro disponibilità="S">
    <Titolo>Il sospetto</Titolo>
    <Autore>F. Dürrenmatt</Autore>
    <Data>1990</Data>
    <ISBN>88-07-81133-2</ISBN>
    <Editore>Feltrinelli</Editore>
  </Libro>
</Elenco>
```

```
<Titolo>Il nome della rosa</Titolo>
<Autore>Umberto Eco</Autore>
<Data>1987</Data>
<ISBN>55-344-2345-1</ISBN>
<Editore>Bompiani</Editore>
```

## Espressioni FLWOR

Le espressioni sono simili al costrutto SELECT - FROM - WHERE di SQL, ma anziché essere applicate a tabelle, associano ed utilizzano tali associazioni per produrre risultati. Le clausole permettono di:

- **FOR** : dichiarare variabili che permettono di iterare sugli elementi di un documento;
- **LET** : dichiarare variabili associate al risultato dell'espressione (eventualmente associandole a quelle introdotte con **FOR** );
- **WHERE** : esprimere condizioni (filtrando le tuple prodotte con i precedenti comandi);
- **ORDER** : ordinare le tuple ottenute;
- **RETURN** : generare il risultato.

## FOR

- **trova tutti i libri elencati nel documento libri.xml**

```
FOR $lib IN document("libri.xml")//Libro
RETURN $lib
```

- **trova tutti gli autori dei libri elencati nel documento libri.xml**

```
FOR $lib IN document("libri.xml")//Libro
FOR $auth IN $lib/Autore
RETURN $auth
```

## LET

- **trova tutti i libri elencati nel documento libri.xml**

```
LET $lib := document("libri.xml")//Libro
RETURN $lib
```

Nota: produce un singolo binding per la variabile, l'intero insieme viene assegnato alla variabile

## WHERE

- **trova tutti i libri pubblicati da Bompiani che sono disponibili**

```
FOR $lib IN document("libri.xml")//Libro
WHERE $lib/Editore="Bompiani"
AND $lib/@disponibilità="S"
RETURN $lib
```

## RETURN

- genera l'output di un'espressione FLWOR che può essere:

- un nodo
  - `<Autore>F. Dürrenmatt</Autore>`
- una foresta ordinata di nodi
  - `<Autore>Umberto Eco</Autore>`  
`<Autore>F. Dürrenmatt</Autore>`
- un valore
  - `F. Dürrenmatt`
- può contenere
  - costruttori di elementi
  - riferimenti a variabili definite da **for** e **let**
  - espressioni annidate
- un costruttore di elemento ha
  - tag iniziale e tag finale
  - lista opzionale di espressioni che determinano il contenuto dell'elemento

## **Esempio**

```
FOR $lib IN document("libri.xml")//Libro
WHERE $lib/Editore="Bompiani"
RETURN
    <EdB>
        $lib/Titolo
    </EdB>

<EdB><Titolo>Il nome della rosa</Titolo></EdB>
<EdB><Titolo>Il sospetto</Titolo></EdB>
```

## **ORDER BY**

Ordina il risultato della return

## **Esempio**

Trova titolo ed editore dei libri elencati nel documento libri.xml, ritorna il risultato ordinato per titolo

```
FOR $lib IN document("libri.xml")//Libro
RETURN
    <Libro>
        $lib/Titolo,
        $lib/Editore
    </Libro>
ORDER BY (Titolo ASCENDING)
```

## **XQuery: esempio (1)**

```
<costruttore>
  <nome-co>Toyota</nome-co>
  <anno>2005</anno>
  <modello>
    <nome-mo>Prius</nome-mo>
    <cilindrata>1497</cilindrata>
    <potenza>77</potenza>
    <aliment>benzina</aliment>
  </modello>
  ...
</costruttore>

<macchina>
  <venditore>Mario Rossi</venditore>
  <marca>Toyota</marca>
  <anno>2005</anno>
  <modello>Prius</modello>
  <colore>grigio metalizz</colore>
  <prezzo>25000</prezzo>
  ...
</macchina>
```

## **XQuery: esempio (2)**

```
document("costruttore.xml")
document("macchina.xml")
```

- realizzare un join a pari <nome-co>=<marca>, <nome-mo>=<modello> and <anno> = <anno>

```
FOR $c IN document("costruttore.xml")//costruttore,
    $m IN document("macchina.xml")//macchina,
    $mo IN $c/modello
WHERE $c/anno=$m/anno
    AND $c/nome-co=$m/marca
    AND $mo/nome-mo=$m/modello
RETURN
    <autovettura>
        $m/marca,
        $m/venditore,
        $m/modello,
        $mo/cilindrata,
        $m/prezzo
    </autovettura>
```

## **XQuery: esempio (3)**

- trova gli editori che nell'elenco hanno almeno 100 libri

```
FOR $e IN document("libri.xml")//Libro/Editore
LET $lib:=
    document("libri.xml")//Libro[Editore=$e]
WHERE count($lib) > 100
RETURN $e
```

## 9. Basi di dati attive

Sono delle **basi di dati che supportano la definizione e gestione di regole di produzione (dette regole attive)**, che **reagiscono agli eventi del sistema** seguendo il paradigma **Evento-Condizione-Azione**, secondo cui, a seguito dell'evento, se la condizione è soddisfatta, viene eseguita l'azione. Le regole attive permettono di raggiungere il livello di **indipendenza della conoscenza**, in quanto la conoscenza di tipo reattivo viene rimossa dai programmi applicativi e codificata sotto forma di regole attive. Quasi tutti i DBMS supportano delle **semplici regole attive** chiamate **trigger**.

### Trigger

La creazione dei trigger fa parte del **Data Definition Language**; i componenti dei trigger sono:

- **eventi**: primitive SQL per la manipolazione dei dati (*insert, delete, update*)
- **condizione**: predicato booleano espresso in SQL
- **azione**: sequenza di primitive SQL generiche o di procedura.

Durante la **specificazione di un trigger**, si può indicare il **livello di granularità** in cui il trigger opera:

- **tupla (row-level)**: l'attivazione avviene **per ogni tupla** coinvolta nell'evento
- **primitiva (statement-level)**: l'attivazione avviene **una sola volta per ogni evento** e si applica a tutte le tuple coinvolte nell'evento

e la sua **modalità di attivazione**:

- **immediata**: la valutazione avviene **immediatamente dopo** (opzione **after**) o **prima** (opzione **before**) dell'evento che lo ha attivato;
- **differita**: la valutazione avviene alla **fine (commit) della transazione**.

### Sintassi del trigger

```
create trigger NomeTrigger {before|after}
{insert|delete|update [of Colonne]} on Tabella
[referencing {[old_table [as] TabOld]|[new_table [as] TabNew]}]
{[old[_row] [as] TuplaOld]|[new[_row] [as] TuplaNew]}]
[for each {row|statement}] [when (PredicatoSQL)] ComandiSQL
```

dove:

- **statement-level** è di default;
- **referencing** permette di inserire variabili:
  - **old e new** si riferiscono allo stato precedente o successivo della tupla;
  - **old\_table e new\_table** si riferiscono allo stato precedente o successivo della tabella.

### Esecuzione dei trigger

L'esecuzione dei trigger segue il seguente ordine:

- esecuzione dei **before statement**;
- esecuzione dei trigger **before row**;
- esecuzione dei test di **integrità sulla tabella**;
- esecuzione dei trigger **after row**;
- esecuzione dei trigger **after statement**.

Nel caso in cui ci siano **più trigger della stessa categoria**, **l'ordine di esecuzione viene scelto dal sistema in un modo dipendente dall'implementazione**.

I trigger sono gestiti in un **Trigger Execution Context** (TEC) e la loro esecuzione può produrre eventi che ne fanno scattare altri, che sono valutati in un nuovo TEC interno. In ogni istante possono esserci più TEC per una transazione, uno dentro l'altro, ma solo uno può essere attivo (struttura a stack). Per i trigger row-level, il

TEC tiene conto di quali tuple sono già state considerate e quali sono ancora da considerare.

## Trigger in SQL: esempi

```
create trigger MonitoraConti
after update on Conto
for each row
when ( old.NomeConto = new.NomeConto
      and new.Totale > old.Totale)
begin
  insert into SingoliVersamenti
  values (new.NomeConto, new.Totale - old.Totale);
end;
```

```
create trigger ArchiviaFattureCanc
after delete on Fattura
referencing old_table as FattureVecchie
begin
  insert into FattureCancellate
  (select *
   from FattureVecchie);
end;
```

Viene creato un trigger "**MonitoraConti**" azionato dopo (after) un update sulla tabella conto. (Evento=update on conto). È un **row-level trigger (for each row)** → quindi **lavora per ogni tupla**. Quella dentro al **when** è la condizione. Quella dentro al **begin** è l'azione.

Viene creato un trigger **che archivia le fatture cancellate**. Viene azionato dopo (after) una delete sulla tabella fattura. E, dopo una delete su fattura chiedo al dbms di chiamarmi la "roba vecchia" come FattureVecchie

### Il trigger:

- Viene **attivato** dall'evento scatenante
- Valuta le **condizioni**
- Viene **eseguito** (vengono eseguite le azioni al suo interno)

## 9.1. Caratteristiche evolutive delle regole

Per quanto riguarda gli eventi, è possibile specificare:

- eventi **temporali** (anche periodici) (es ogni giorno alle 17)
- eventi **applicativi** (definiti dall'utente) (es "temp troppo alta")
- **combinazioni booleane** di eventi.

L'attivazione delle regole, invece, ha avuto le seguenti evoluzioni:

- regole **attivabili** o **disattivabili**;
- **gruppi di regole**;
- clausola **instead of** : quando la condizione è vera, **l'azione viene eseguita al posto dell'operazione che ha attivato la regola** (evento), perciò è una modalità alternativa a before e after;
- modalità **detached** : la regola viene **gestita da una transazione separata** e si può aggiungere a:
  - **immediate** : il trigger viene considerato ed eseguito **con l'evento** che lo ha attivato;
  - **deferred** : il trigger viene gestito **al termine della transazione**.

Inoltre è possibile impostare la **priorità** delle regole, regolandone l'**ordine di esecuzione** in caso ve ne siano diverse attivate contemporaneamente.

## 9.2. Proprietà delle regole

In caso ci siano più regole che vengono attivate a fronte di un determinato evento, è necessario avere delle **garanzie** che **l'interferenza tra le diverse regole e l'attivazione a catena non generi anomalie**. In questo modo, l'attivazione delle regole soddisfa le proprietà di:

- la **terminazione**: per qualunque stato iniziale e qualunque sequenza di modifiche, **le regole producono uno stato finale** (perciò **non devono verificarsi cicli interminabili**) (**IMPORTANTE**)

- la **confluenza**: per qualunque stato iniziale e qualunque sequenza di modifiche, **le regole producono uno e un unico stato finale, indipendente dall'ordine in cui i trigger vengono eseguiti**;
- il **determinismo delle osservazioni**: per qualunque stato iniziale e qualunque sequenza di modifiche, **le regole producono la stessa sequenza di azioni visibili**.

## Analisi di terminazione

La **proprietà di terminazione è la più importante**, e può essere analizzata utilizzando il **grafo di attivazione**, così composto:

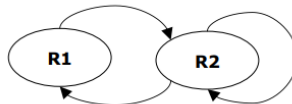
- un **nodo** per ogni **regola**;
- un **arco** da un **nodo  $R_i$**  a un **nodo  $R_j$**  se l'**esecuzione dell'azione di  $R_i$**  può **attivare** la **regola  $R_j$** .

Se il **grafo è aciclico**, si ha la garanzia che il **sistema è terminante** (l'aciclicità è una **condizione sufficiente ma non necessaria per la terminazione**).

## Regole non terminanti: esempio

**R1:**

```
create trigger AggiustaContributi
after update of Stipendio on Impiegato
referencing new_table as NuovoImp
begin
    update Impiegato
    set Contributi = Stipendio * 0.8
    where Matr in (
        select Matr
        from NuovoImp);
end;
```



**R2:**

```
create trigger ControllaSogliaBudget
after update on Impiegato
when 50000 > (
    select sum(Stipendio+Contributi)
    from Impiegato)
begin
    update Impiegato
    set Stipendio = 0.9*Stipendio;
end;
```

R1 può causare l'attivazione di R2 perché all'interno del begin fa un update (che è la condizione che fa partire R2).

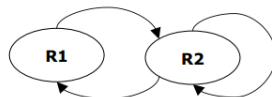
Ma anche R2 può attivare R1 perché nel suo begin ha un update su impiegato → Il problema è che oltre ad attivare R1 attiva anche se stesso (condizione → update on impiegato)

Quindi questo grafo di attivazione è ciclico e possiamo dire che è un **trigger NON terminante** anche perché abbassando i contributi in r1 e poi andando a controllare in r2 non smetteranno mai di chiamarsi a vicenda

## Regole terminanti: esempio

**R1:**

```
create trigger AggiustaContributi
after update of Stipendio on Impiegato
referencing new_table as NuovoImp
begin
    update Impiegato
    set Contributi = Stipendio * 0.8
    where Matr in (
        select Matr
        from NuovoImp);
end;
```



**R2:**

```
create trigger ControllaSogliaBudget
after update on Impiegato
when 50000 < (
    select sum(Stipendio+Contributi)
    from Impiegato)
begin
    update Impiegato
    set Stipendio = 0.9*Stipendio;
end;
```

Questi trigger sono uguali a quelli di prima ma in R2 la **condizione è 50000 < di ...**

Come abbiamo visto prima il "dialogo" tra R1 e R2 continua ad abbassare la somma stipendio + contributi quindi **prima o poi quella condizione non sarà più rispettata → R2 smetterà di attivare R1 (e anche sé stesso) → è un trigger terminante**

## 9.3. Applicazioni ed esempi

Le applicazioni delle regole attive possono essere classificate in:

- **regole interne alla base di dati:** generate dal sistema e **non visibili all'utente:**
  - **gestione dei vincoli d'integrità referenziale;**
  - derivazione o replicazione dei dati;
  - gestione di **versioni, privacy, sicurezza**, logging delle azioni, registrazione degli eventi, ecc.;
- **regole esterne** (o regole **aziendali**): esprimono **conoscenza di tipo applicativo** e sono **note all'utente** (esprimono vincoli o controlli non esprimibili con costrutti del DBMS, come **integrità** e **derivazione**)

### Gestione dell'integrità referenziale: es. (1)      Gestione dell'integrità referenziale: es. (2)

```
create table Impiegato(... ..
  foreign key(NDip) references
    Dipartimento(NroDip)
  on delete set null,
  on update cascade ... ..);
```

Operazioni che possono violare questo vincolo:

- insert into Impiegato
- update Impiegato.NDip
- delete from Dipartimento
- update Dipartimento.NroDip

- evento:
  - inserimento in Impiegato
- condizione:
  - il nuovo valore di NDip non è nella tabella Dipartimento
- azione:
  - proibisci inserimento e segnala errore

```
create trigger ControllaInsDipImpiegato
before insert on Impiegato
for each row
when (not exists (select * from Dipartimento
  where NroDip = new.NDip))
begin
  signal sqlstate '70006' ('Dip non valido');
end;
```

### Gestione dell'integrità referenziale: es. (3)      Gestione dell'integrità referenziale: es. (4)

- evento:
  - modifica dell'attributo NDip in Impiegato
- condizione:
  - il nuovo valore di NDip non è nella tabella Dipartimento
- azione:
  - proibisci la modifica e segnala errore

```
create trigger ControllaModDipImpiegato
before update of NDip on Impiegato
for each row
when (not exists (select * from Dipartimento
  where NroDip = new.NDip))
begin
  signal sqlstate '70006' ('Dip non valido');
end;
```

- evento:
  - cancellazione in Dipartimento
- condizione:
  - il valore di NroDip da cancellare è nella tabella Impiegato
- azione:
  - mette a null l'attributo NDip delle tuple coinvolte

```
create trigger ControllaCancDipartimento
after delete on Dipartimento
for each row
when (exists (select * from Impiegato
  where NDip = old.NroDip))
begin
  update Impiegato
  set NDip = null
  where NDip = old.NroDip;
end;
```

### Gestione dell'integrità referenziale: es. (5)

- evento:
  - modifica dell'attributo NroDip in Dipartimento
- condizione:
  - il vecchio valore di NroDip è usato nella tabella Impiegato
- azione:
  - si modifica anche NDip delle tuple coinvolte in Impiegato

```
create trigger ControllaModAttribNDip
after update of NroDip on Dipartimento
for each row
when (exists (select * from Impiegato
  where NDip = old.NroDip))
begin
  update Impiegato
  set NDip = new.NroDip
  where NDip = old.NroDip;
end;
```

### Regole aziendali (integrità): es.

Un impiegato deve avere uno stipendio minore del direttore del dipartimento al quale afferisce

IMPIEGATO(ImpNum, Mgr, Salario, DipNum)  
DIPARTIMENTO(DipNum, Direttore)

```
create trigger SalarioImpiegato
after update on Salario of Impiegato
for each row
when (new.Salario > select Salario
                    from Impiegato
                    where ImpNum = new.Mgr
                    and ImpNum in (select Direttore
                                   from Dipartimento
                                   where DipNum = new.DipNum))
begin
    signal sqlstate '70005' ('Salario troppo elevato');
end;
```

### Regole aziendali (derivazione): es. (2)

- evento:
  - cancellazione di un prodotto
- azione:
  - cancellare tutti i suoi sottoprodotti

```
create trigger CancellaProdotto
after delete on Prodotto
for each row
begin
    delete from Prodotto
    where SuperProdotto = old.Codice;
end;
```

### Regole aziendali (derivazione): es. (1)

PRODOTTO(Codice, Nome, SuperProdotto, Livello)

- gerarchia di prodotti
  - ogni prodotto ha un super-prodotto e un livello di profondità nella gerarchia
  - prodotti non contenuti in altri prodotti: SuperProdotto = NULL e Livello = 0

#### Regole

- se un prodotto è cancellato: cancellare tutti i sottoprodotti che lo compongono
- se un prodotto è inserito: calcolare il suo livello

### Regole aziendali (derivazione): es. (3)

- evento:
  - inserimento di un prodotto
- azione:
  - calcolare il suo livello

```
create trigger LivelloProdotto
after insert on Prodotto
for each row
begin
    if new.SuperProdotto is not null
        update Prodotto
        set Livello = 1 + (
            select Livello from Prodotto
            where Codice = new.SuperProdotto)
        where Codice = new.Codice
    else
        update Prodotto
        set Livello = 0
        where Codice = new.Codice;
    endif;
end;
```

## 10. Basi di dati per il supporto alle decisioni

### 10.1. OLTP e OLAP

#### Limiti delle basi di dati relazionali

Le basi di dati sono adatte per la **gestione efficiente di dati in linea** (OLTP), ma offrono **supporto limitato all'analisi dei dati** (OLAP), a causa della **rigidità** e della **complessità** delle applicazioni, che le rendono **difficili da ottimizzare** in modo tale da soddisfare sia le esigenze della **gestione in linea** sia quelle di **analisi**.

Occorre quindi trovare soluzioni che rendano **utilizzabili i dati prodotti** per la **gestione operativa** anche per la **gestione strategica**.

#### On Line Transaction Processing

Consiste nella tradizionale **elaborazione di transazioni**, realizzata dai processi operativi, ed è caratterizzata da:

- operazioni predefinite e relativamente **semplici**;
- operazioni che coinvolgono **“pochi” dati**;
- dati dettagliati, **aggiornati**;
- proprietà **ACID** delle transazioni essenziali.

#### On Line Analytical Processing

Consiste nell'elaborazione di **operazioni per il supporto alle decisioni**, caratterizzata da:

- operazioni **complesse** e **casuali** (non predefinite);
- operazioni che coinvolgono **molti dati**;
- dati aggregati, storici, anche **non aggiornati**;
- proprietà **ACID non rilevanti** (tipicamente si eseguono operazioni di lettura).

## OLTP e OLAP

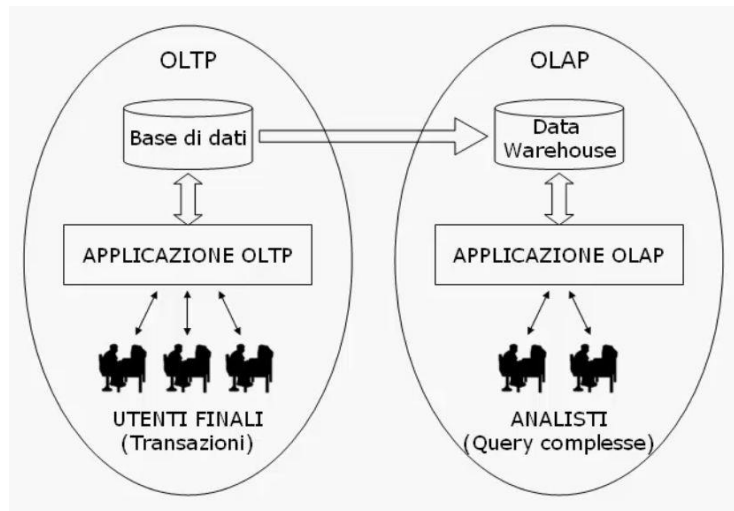
La configurazione di sistemi dedicati a **uno solo** dei due compiti è un **problema gestibile**, mentre è **estremamente difficile far convivere i due carichi di lavoro** (a causa della disomogeneità di utenti e requisiti ed delle differenze tecniche).

|                        | OLTP                                                     | OLAP                                              |
|------------------------|----------------------------------------------------------|---------------------------------------------------|
| <b>Utente</b>          | impiegato                                                | dirigente                                         |
| <b>Funzione</b>        | operazioni giornaliere                                   | supporto alle decisioni                           |
| <b>Progettazione</b>   | orientata all'applicazione                               | orientata ai dati                                 |
| <b>Dati</b>            | correnti, aggiornati, dettagliati, relazionali, omogenei | storici, aggregati, multidimensionali, eterogenei |
| <b>Uso</b>             | ripetitivo                                               | casuale                                           |
| <b>Accesso</b>         | read-write, indicizzato                                  | read, sequenziale                                 |
| <b>Unità di lavoro</b> | transazione breve                                        | interrogazione complessa                          |
| <b>Accesso record</b>  | decine                                                   | milioni                                           |
| <b>N. utenti</b>       | migliaia                                                 | centinaia                                         |
| <b>Dimensione</b>      | 100MB - 1GB                                              | 100GB - 1TB                                       |
| <b>Metrica</b>         | throughput                                               | tempo di risposta                                 |

Altre **differenze** tra **OLTP** e **OLAP** sono:

- **conflitto di lock:**
  - OLTP: tante transazioni rapide con lock esclusivi;
  - OLAP: poche transazioni lunghe con lock condivisi;
  - OLTP e OLAP: o le transazioni OLTP sono molto rallentate, o le query OLAP non riescono ad essere eseguite;
- uso degli **indici**:
  - OLTP: pochi e solo se necessari;
  - OLAP: tanti per coprire ogni esigenza;
  - OLTP e OLAP: o le transazioni OLTP rallentano l'aggiornamento di molti indici, o le query OLAP non hanno a disposizione gli indici necessari.;
- **precomputazione di query:**
  - OLTP: avviene molto raramente, per problemi di consistenza e di carico;
  - OLAP: importante per abbassare i tempi di risposta;
- **modello logico:**
  - OLTP: elevata frammentazione e numero di tabelle, generalmente normalizzate (prive di ridondanze e quindi di anomalie di comportamento);
  - OLAP: poche tabelle e non normalizzate.

Dal momento che OLTP e OLAP hanno un **conflitto intrinseco** (che non sparisce con l'aumentare della potenza di calcolo del sistema, perché non dipende da quella) si opta per la **soluzione che separa i due ambienti in basi di dati (OLTP) e data warehouse (OLAP)**



## 10.2. Data warehouse

Una **data warehouse** è una **base di dati per il supporto alle decisioni**, caratterizzata da:

- **struttura integrata:**
  - riunisce i dati di diverse sorgenti informative preesistenti;
  - rappresenta i dati in modo univoco, riconciliando le eterogeneità delle diverse rappresentazioni (nomi, strutture, codifiche, ecc.);
- **dati in forma aggregata:**
  - i dati delle sorgenti sono aggregati sulla base di opportune coordinate (tempo, collocazione geografica, tipologia di prodotto, ecc.);
  - è orientata ai dati (non alle applicazioni): mentre le basi di dati operazionali sono costruite a supporto dei singoli processi operativi o applicazioni, la **data warehouse** è costruita attorno alle principali entità del patrimonio informativo aziendale;
- **dati storici e temporali:** mantiene l'evoluzione storica delle informazioni con un ampio orizzonte temporale e indicazione di elementi di tempo: mentre le basi di dati operazionali mantengono il valore corrente delle informazioni (con un orizzonte temporale di pochi mesi), per la **data warehouse** è di interesse l'evoluzione storica delle informazioni con un orizzonte temporale nell'ordine di anni;
- **fuori linea e non volatile:** tipicamente formata in modo asincrono e periodico rispetto alle sorgenti:
  - operazioni di accesso e interrogazioni "diurne";
  - operazioni di caricamento e aggiornamento dei dati "notturne";
  - le operazioni possono riguardare milioni di record;
- **autonoma:** fisicamente separata dalle sorgenti informative per ragioni tecniche:
  - **non esiste** un'unica base di dati operazionali che contiene tutti i dati di interesse (e la data warehouse deve essere integrata);
  - l'analisi dei dati richiede organizzazioni speciali dei dati e metodi di accesso specifici;
  - degrado generale delle prestazioni senza la separazione.

### OLAP e controllo di gestione

Le **applicazioni per il controllo di gestione**:

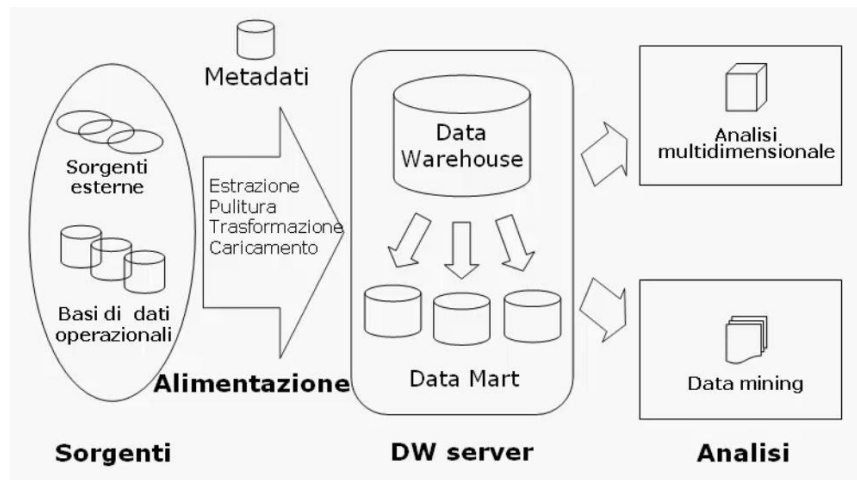
- hanno alcune caratteristiche in comune con le applicazioni OLAP:
  - principalmente operazioni di lettura di dati;
  - necessità di svolgere aggregazioni;
  - confronti con serie storiche;
- ma sono comunque differenti tra loro:
  - staticità delle query (non query ad hoc, ma repost fissi);

- necessità di mantenere il dettaglio dello schema;
- possono comunque condividere informazioni con l'OLAP, ma devono essere *progetti diversi*.

## Architettura della data warehouse

Le componenti della data warehouse sono:

- **sorgenti** dei dati;
- **server** (dedicato all'OLAP):
  - spesso gestisce anche viste materializzate (data mart);
  - può avere associati metadati e strumenti per l'assistenza allo sviluppo;
- sistema di **alimentazione**:
  - estrazione dei dati dalle sorgenti;
  - **pulizia** (eliminazione di errori e inconsistenze) e **trasformazione dei dati**;
  - caricamento nella data warehouse;
- **strumenti di analisi**:
  - analisi **multidimensionale** (operazioni interattive di aggregazione e disaggregazione di dati);
  - **data mining** (ricerche sofisticate per inserire nuovi dati e correlazioni tra dati).



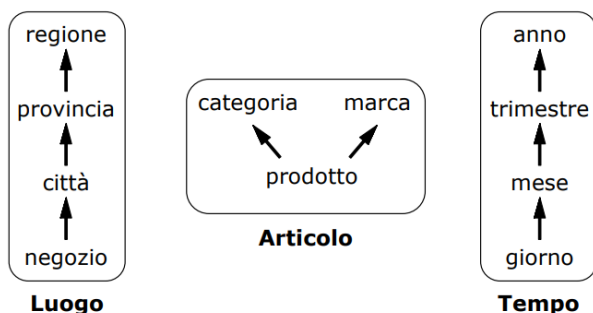
## 10.3. Rappresentazione multidimensionale dei dati

### Modello multidimensionale

Permette la rappresentazione dei dati ad alto livello ( a prescindere dai criteri di memorizzazione) che favorisce le operazioni di analisi. Questo modello è basato su tre concetti rilevanti: (*es. catena di negozi*)

- **fatto**: concetto sul quale centrare l'analisi (*vendita*)
- **misura**: proprietà atomica di un fatto da analizzare (*quantità venduta, incasso*)
- **dimensione**: descrive una prospettiva lungo la quale effettuare l'analisi (*articolo, tempo, luogo*)

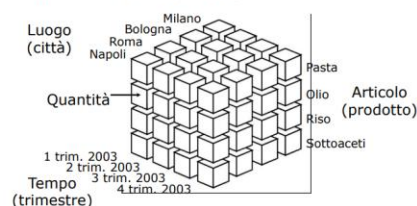
Ciascuna dimensione è organizzata in una **gerarchia** che rappresenta i possibili **livelli di aggregazione** per i dati della dimensione stessa.



**Modello multidimensionale: esempio (2)**

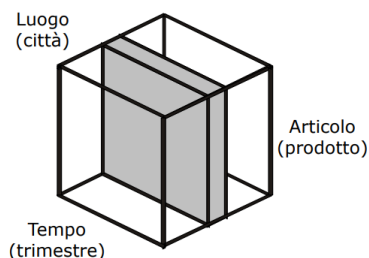
**Modello multidimensionale: esempio (1)**

- fatto: vendita
- misura: quantità
- dimensioni: articolo, tempo, luogo



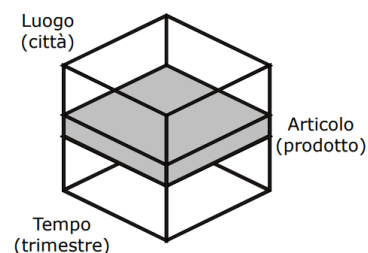
**Modello multidimensionale: esempio (3)**

Il manager di una città esamina la vendita dei prodotti in tutti i periodi relativamente alla città di sua competenza



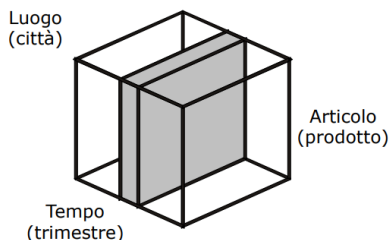
**Modello multidimensionale: esempio (4)**

Il manager di prodotto esamina la vendita di un prodotto in tutti i periodi e in tutte le città

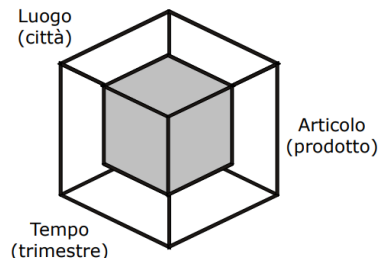


**Modello multidimensionale: esempio (5)**

Il manager finanziario esamina la vendita dei prodotti in tutte le città relativamente al periodo corrente e quello precedente



Il manager strategico si concentra su una categoria di prodotti, una area geografica e un orizzonte temporale medio



### Operazioni su dati multidimensionali

Quando si opera su dati multidimensionali, si vogliono eseguire operazioni di analisi su uno specifico sottoinsieme di dati; perciò, ci sono delle operazioni che possono essere effettuate sui dati per selezionarli gli insiemi di interesse:

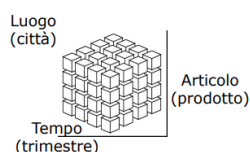
- **slice-and-dice**: seleziona e proietta;
- **roll-up**: aggrega i dati;
- **drill-down**: disaggrega i dati (operazione inversa del roll-up)

## Slice-and-dice, roll-up e drill-down

- **Slice-and-dice**: si riferisce all'abilità di **selezionare** parti della tabella pivot in base a uno o più criteri specifici. Ad esempio, si può "*slicare*" una tabella pivot per **visualizzare solo i dati relativi ad una particolare regione** o periodo di tempo.
- **Roll-up**: si riferisce alla capacità di **aggregare i dati** della tabella pivot in modo **gerarchico**. In pratica, si possono **sommare i dati di una colonna per livelli superiori di una gerarchia**. Ad esempio, si può sommare le vendite per prodotto, poi per categoria e infine per regione.
- **Drill-down**: si riferisce alla possibilità di **esplorare i dati della tabella pivot in modo dettagliato**. In pratica, si possono visualizzare i dati aggregati per livelli inferiori di una gerarchia. Ad esempio, si può esplorare la vendita di un particolare prodotto su una specifica regione.

### Drill-down: esempio

#### Slice-and-dice: esempio



seleziona le vendite di pasta per trimestre e città

| Pasta   | 1 trim. 2003 | 2 trim. 2003 | 3 trim. 2003 | 4 trim. 2003 |
|---------|--------------|--------------|--------------|--------------|
| Milano  | 130000       | 125000       | 110000       | 145000       |
| Bologna | 125000       | 125000       | 135000       | 110000       |
| Roma    | 80000        | 85000        | 90000        | 85000        |
| Napoli  | 70000        | 75000        | 70000        | 90000        |

| Pasta   | 1 trim. 2003 | 2 trim. 2003 | 3 trim. 2003 | 4 trim. 2003 |
|---------|--------------|--------------|--------------|--------------|
| Milano  | 130000       | 125000       | 110000       | 145000       |
| Bologna | 125000       | 125000       | 135000       | 110000       |
| Roma    | 80000        | 85000        | 90000        | 85000        |
| Napoli  | 70000        | 75000        | 70000        | 90000        |

#### Drill-down sul Luogo (da città a negozio)

| Pasta     | 1 trim. 2003 | 2 trim. 2003 | 3 trim. 2003 | 4 trim. 2003 |
|-----------|--------------|--------------|--------------|--------------|
| Milano-1  | 70000        | 65000        | 40000        | 75000        |
| Milano-2  | 60000        | 60000        | 70000        | 70000        |
| Bologna-1 | 60000        | 60000        | 55000        | 30000        |
| Bologna-2 | 30000        | 35000        | 35000        | 40000        |
| Bologna-3 | 35000        | 30000        | 45000        | 40000        |
| Roma-1    | 40000        | 40000        | 45000        | 35000        |
| ...       | ...          | ...          | ...          | ...          |

#### Roll-up: esempio

| Pasta   | 1 trim. 2003 | 2 trim. 2003 | 3 trim. 2003 | 4 trim. 2003 |
|---------|--------------|--------------|--------------|--------------|
| Milano  | 130000       | 125000       | 110000       | 145000       |
| Bologna | 125000       | 125000       | 135000       | 110000       |
| Roma    | 80000        | 85000        | 90000        | 85000        |
| Napoli  | 70000        | 75000        | 70000        | 90000        |

#### Roll-up sul Tempo (da trimestre a anno)

| Pasta   | 2003   |
|---------|--------|
| Milano  | 510000 |
| Bologna | 495000 |
| Roma    | 340000 |
| Napoli  | 305000 |

## 10.4. Realizzazione di data warehouse

Nella realizzazione di una data warehouse, ci sono due approcci possibili:

- **ROLAP** (Relational OLAP):
  - utilizza tecnologia relazionale, opportunamente adattata ed estesa;
  - i dati sono memorizzati in tabelle e le operazioni di analisi sono espresse con istruzioni SQL;
  - le strutture di accesso ai dati sono speciali per ottimizzare le operazioni;
- **MOLAP** (Multidimensional OLAP):
  - memorizza i dati direttamente in forma multidimensionale;
  - le strutture sono in genere proprietarie.

## Relational OLAP

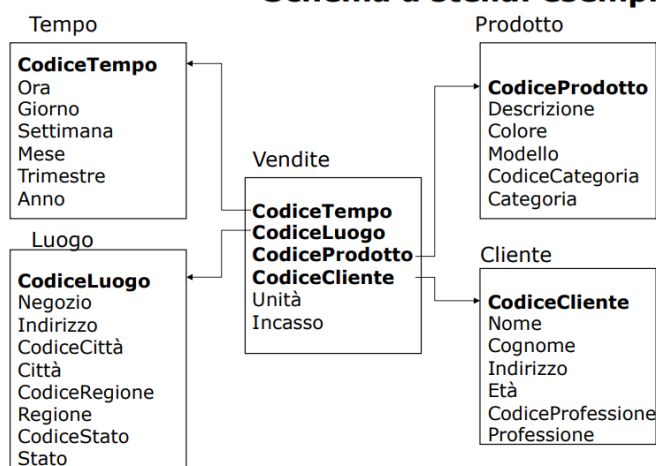
Generalmente costruito in modo incrementale, per collezione di data mart settoriali, utilizza uno schema **dimensionale** detto anche **a stella** (con restrizioni sulla struttura dello schema). In un sistema ROLAP i dati sono presentati all'utente con un modello di alto livello (multidimensionale) e le interrogazioni sullo schema sono trasformate dal sistema in istruzioni SQL.

### Schema a stella

I componenti dello schema a stella sono:

- **tabella dei fatti**: una tabella principale che memorizza i fatti del data mart;
- **tabelle dimensione**: relazioni ausiliarie che memorizzano i dati relativi alle dimensioni dell'analisi;
- **vincoli di integrità referenziale**: ognuno collega un attributo della tabella dei fatti ad una tabella dimensione.

#### Schema a stella: esempio

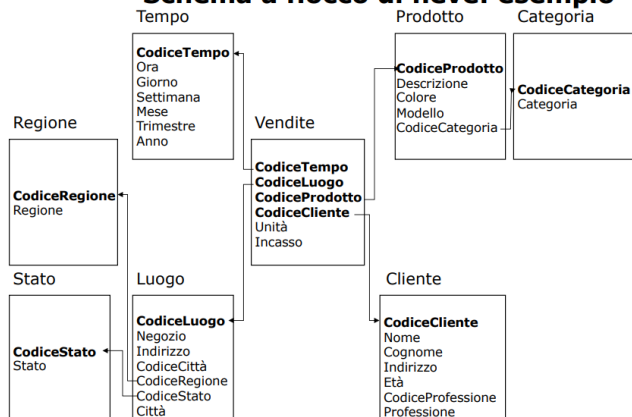


#### Lo schema a stella è caratterizzato da:

- per la tabella dei fatti:
  - la chiave è composta da attributi, che sono referimenti alle chiavi delle tabelle dimensione;
  - gli attributi non chiave rappresentano le misure del fatto e sono solitamente numerici;
  - è in forma normale (Boyce-Codd);
- per le tabelle dimensione:
  - chiave semplice (un solo attributo);
  - gli attributi non chiave rappresentano i livelli della dimensione o qualche loro proprietà e sono tipicamente testuali;
  - generalmente non in forma normale (comporta ridondanza ma maggiore efficienza).

Da una **normalizzazione** (anche parziale) di uno schema a stella risulta uno schema a fiocco di neve, che permette di **evitare ridondanze** eccessive nelle dimensioni. La normalizzazione dev'essere fatta con attenzione (altrimenti porterebbe al **degrado nelle prestazioni**) e dalla tabella dei fatti si raggiungono tutte le tabelle delle dimensioni (muovendosi lungo associazioni  $n:1$ ).

#### Schema a fiocco di neve: esempio



## Aggregazione SQL

Lo standard SQL offre degli operatori per le aggregazioni:

- esprime tutte le possibili aggregazioni delle tuple di una tabella;
- utilizza il valore polimorfo **ALL** :
  - presente in tutti i domini;

- corrisponde all'insieme di tutti i possibili valori del dominio;
- ha le seguenti opzioni:
  - **with data cube**: aggrega su tutte le possibili dimensioni;

#### With cube: esempio (1)

| Modello | Anno | Colore | Unità |
|---------|------|--------|-------|
| fiat    | 1994 | rosso  | 50    |
| fiat    | 1995 | rosso  | 85    |
| ford    | 1994 | rosso  | 80    |

```
select Modello, Anno, Colore, sum(Unità)
from Vendite as V, Prodotto as P, Tempo as T
where V.CodiceProdotto = P.CodiceProdotto
and V.CodiceTempo = T.CodiceTempo
and P.Modello in {'fiat','ford'}
and P.Colore = 'rosso'
and T.Anno between 1994 and 1995
group by Modello, Anno, Colore
with cube
```

#### With cube: esempio (2)

| Modello | Anno | Colore | sum(Unità) |
|---------|------|--------|------------|
| fiat    | 1994 | rosso  | 50         |
| fiat    | 1995 | rosso  | 85         |
| fiat    | 1994 | ALL    | 50         |
| fiat    | 1995 | ALL    | 85         |
| fiat    | ALL  | rosso  | 135        |
| fiat    | ALL  | ALL    | 135        |
| ford    | 1994 | rosso  | 80         |
| ford    | 1994 | ALL    | 80         |
| ford    | ALL  | rosso  | 80         |
| ford    | ALL  | ALL    | 80         |
| ALL     | 1994 | rosso  | 130        |
| ALL     | 1995 | rosso  | 85         |
| ALL     | ALL  | rosso  | 215        |
| ALL     | 1994 | ALL    | 130        |
| ALL     | 1995 | ALL    | 85         |
| ALL     | ALL  | ALL    | 215        |

- **with roll up**: aggrega in modo progressivo su una dimensione per volta (secondo un ordinespecificato)

#### With roll up: esempio (1)

| Modello | Anno | Colore | sum(Unità) |
|---------|------|--------|------------|
| fiat    | 1994 | rosso  | 50         |
| fiat    | 1995 | rosso  | 85         |
| ford    | 1994 | rosso  | 80         |

```
select Modello, Anno, Colore, sum(Unità) from
Vendite as V, Prodotto as P, Tempo as T
where V.CodiceProdotto = P.CodiceProdotto and
V.CodiceTempo = T.CodiceTempo
and P.Modello in {'fiat','ford'}
and P.Colore = 'rosso'
and T.Anno between 1994 and 1995
group by Modello, Anno, Colore
with roll up
```

#### With roll up: esempio (2)

| Modello | Anno | Colore | sum(Unità) |
|---------|------|--------|------------|
| fiat    | 1994 | rosso  | 50         |
| fiat    | 1995 | rosso  | 85         |
| ford    | 1994 | rosso  | 80         |
| fiat    | 1994 | ALL    | 50         |
| fiat    | 1995 | ALL    | 85         |
| ford    | 1994 | ALL    | 80         |
| fiat    | ALL  | ALL    | 135        |
| ford    | ALL  | ALL    | 80         |
| ALL     | ALL  | ALL    | 215        |

## Possibili ottimizzazioni

Alcune possibili ottimizzazioni ad una data warehouse implementata con un database relazionale sono:

- indici **bitmap**: permettono di valutare in modo efficiente disgiunzioni, congiunzioni e predicati diuguaglianza;
- indici **join**: precomputano il join tra tabelle dimensione e la tabella dei dati;
- **materializzazione delle viste**: vengono precalcolate le viste che servono per rispondere alle query piùfrequenti.

## 10.5. Data mining

Si riferisce a tecniche di analisi per estrarre informazioni non esplicite nei dati: è una materia interdisciplinareche coinvolge statistica, algoritmi, intelligenza artificiale e reti neurali.

Il processo di data mining è svolto in modo iterativo e adattivo con costruzione progressiva della conoscenza:

- **comprensione del dominio** applicativo;
- preparazione dell'**insieme di dati**:
  - selezione del sottoinsieme dei dati della data warehouse;
  - possibile **discretizzazione**;

- scoperta dei pattern;
- valutazione dei pattern;
- utilizzo dei risultati.

Due tipiche applicazioni del data mining servono per stabilire:

- **regole di associazione** (ricercano **regolarità nei dati**), nella seguente struttura:
  - **premessa** della regola (corpo);
  - **conseguenza** della regola (testa); e con le seguenti proprietà:
    - **supporto**: probabilità che siano **presenti in una transazione entrambi gli elementi di una regola quante volte sono insieme quelle due cose rispetto al totale delle transazioni**
    - **confidenza**: probabilità che sia **presente in una transazione la conseguenza di una regola, essendo presente la premessa**; quante volte sono insieme rispetto alla conseguenza?

SUPPORTO  $\rightarrow \frac{\#PC}{\#TOT}$  CONFIDENZA  $\rightarrow \frac{\#PC}{\#P}$  Nota:  $\text{sup}(x \rightarrow y) = \text{sup}(y \rightarrow x)$  e **se x è in tutte le transazioni allora  $\text{sup}(x \rightarrow y) = \text{conf}(x \rightarrow y)$  e  $\text{conf}(y \rightarrow x) = 1$**

Ad esempio, supponiamo di avere un gruppo di 1000 persone e che 200 di queste persone abbiano acquistato un determinato prodotto. In questo caso,  $\#PC/\#TOT$  sarebbe 0.2 (200 acquisti su 1000 persone totali).

Invece, se volessimo valutare la percentuale di acquisti all'interno del gruppo di persone che hanno visitato il sito web del prodotto (ad esempio, 300 persone), allora utilizzeremmo  $\#PC/\#P$ . Se 100 di queste persone hanno effettuato un acquisto, allora  $\#PC/\#P$  sarebbe 0.33 (100 acquisti su 300 persone che hanno visitato il sito).

### Regole di associazione: esempio

| No. | Prodotto | Premessa         | Conseguenza | Sup. | Conf. |
|-----|----------|------------------|-------------|------|-------|
| 1   | Pasta    | Pasta            | Ragù        | 0.33 | 0.33  |
| 1   | Ragù     | Pasta            | Acqua       | 0.66 | 0.66  |
| 2   | Pasta    | Pasta            | Passata     | 0.33 | 0.33  |
| 2   | Acqua    | Ragù             | Pasta       | 0.33 | 1     |
| 3   | Pasta    | Acqua            | Pasta       | 0.66 | 1     |
| 3   | Acqua    | Acqua            | Passata     | 0.33 | 0.5   |
| 3   | Passata  | Passata          | Pasta       | 0.33 | 1     |
|     |          | Passata          | Acqua       | 0.33 | 1     |
|     |          | {Pasta, Acqua}   | Passata     | 0.33 | 0.5   |
|     |          | {Pasta, Passata} | Acqua       | 0.33 | 1     |
|     |          | {Acqua, Passata} | Pasta       | 0.33 | 1     |

La tabella rappresenta un esempio di regole di associazione con il relativo supporto e confidenza. Ogni riga rappresenta una coppia di prodotti, dove il primo è la premessa e il secondo la conseguenza della regola.

Ad esempio, la prima riga indica che il 33% delle transazioni che contengono Pasta contengono anche Ragù, e che il 33% delle transazioni che contengono Pasta e Ragù

contengono anche Pasta. Il supporto è quindi 0.33 e la confidenza è 1.

La riga successiva indica che il 66% delle transazioni che contengono Pasta contengono anche Acqua, e che il 66% delle transazioni che contengono Pasta e Acqua contengono anche Pasta. Il supporto è quindi 0.66 e la confidenza è 1.

Le ultime tre righe rappresentano invece regole di associazione che coinvolgono tre prodotti, indicati tra parentesi graffe. Ad esempio, la penultima riga indica che il 33% delle transazioni che contengono Pasta e Passata contengono anche Acqua, e che il 50% delle transazioni che contengono Pasta, Acqua e Passata contengono anche Passata. Il supporto è quindi 0.33 e la confidenza è 0.5.

In generale, il supporto misura la frequenza con cui una determinata combinazione di prodotti appare nelle transazioni, mentre la confidenza misura la probabilità che la conseguenza si verifichi, dato che la premessa è stata osservata. Le regole di associazione sono utilizzate nell'ambito del data mining per individuare relazioni tra variabili, ad esempio per identificare prodotti che sono spesso acquistati insieme.

- regole di classificazione (catalogano nuovi fenomeni in classi predefinite), composta da:
  - fenomeno da classificare, presentato sotto forma di fatto elementare (tupla);
  - classificatore, algoritmo che svolge la classificazione:
    - costruito automaticamente tramite un insieme di dati di prova (training set);
    - applicato per la classificazione di fenomeni generici;
    - rappresentato come albero di decisione.

### Data mining per associazione

Permette di estrarre tutte le regole con supporto e confidenza superiori a valori prefissati.

Alcuni esempi di applicazioni:

- analisi di mercato (ad esempio prodotti acquistati insieme o in sequenza);
- analisi di comportamento (ad esempio l'individuazione di usi illeciti di carte di credito);
- previsione (ad esempio la previsione del costo delle cure mediche);
- controllo (errori di computazione).

### SAM\_BD\_M10\_E1 --- Esercizio query con CUBE

| GUSTO      | COLORE  | SCATOLE |
|------------|---------|---------|
| Mandorla   | Bianco  | 200     |
| Mandorla   | Rosso   | 100     |
| Mandorla   | Oro     | 50      |
| Cioccolato | Bianco  | 150     |
| Cioccolato | Argento | 200     |
| Cioccolato | Rosso   | 100     |
| Cocco      | Bianco  | 50      |
| Cocco      | Rosso   | 50      |

```
SELECT  Gusto, Colore, SUM(Scatole)
FROM    T
WHERE   Gusto = 'Mandorla' OR Gusto = 'Cocco'
GROUP BY Gusto, Colore WITH CUBE
```

| GUSTO                 | COLORE           | SCATOLE        |
|-----------------------|------------------|----------------|
| Mandorla              | Bianco           | 200            |
| <del>Mandorla</del>   | <del>Rosso</del> | <del>100</del> |
| <del>Mandorla</del>   | <del>Oro</del>   | <del>50</del>  |
| Cioccolato            | Bianco           | 150            |
| Cioccolato            | Argento          | 200            |
| <del>Cioccolato</del> | <del>Rosso</del> | <del>100</del> |
| Cocco                 | Bianco           | 50             |
| <del>Cocco</del>      | <del>Rosso</del> | <del>50</del>  |

```
SELECT  Gusto, Colore, SUM(Scatole)
FROM    T
WHERE   Colore = 'Bianco' OR Colore = 'Argento'
GROUP BY Gusto, Colore WITH ROLL UP
```

| GUSTO | COLORE | SCATOLE |
|-------|--------|---------|
| Man   | Bianco | 200     |
| Man   | Rosso  | 100     |
| Man   | Oro    | 50      |
| Cocco | Bianco | 50      |
| Cocco | Rosso  | 50      |
| Man   | ALL    | 350     |
| Cocco | ALL    | 100     |
| ALL   | Bianco | 250     |

|     |       |     |
|-----|-------|-----|
| ALL | Rosso | 150 |
| ALL | Oro   | 50  |
| ALL | ALL   | 450 |

| GUSTO  | COLORE  | SCATOLE |
|--------|---------|---------|
| Man    | Bianco  | 200     |
| Ciocco | Bianco  | 150     |
| Ciocco | Argento | 200     |
| Cocco  | Bianco  | 50      |
| Man    | ALL     | 200     |
| Ciocco | ALL     | 350     |
| Cocco  | ALL     | 50      |
| ALL    | ALL     | 600     |

### SAM\_BD\_M10\_E2 --- Esercizio supporto e confidenza

| Transazione | Data       | Prodotto | Qta |
|-------------|------------|----------|-----|
| 1           | 15/05/2012 | latte    | 80  |
| 1           | 15/05/2012 | pane     | 40  |
| 1           | 15/05/2012 | zucchero | 35  |
| 2           | 12/05/2012 | latte    | 90  |
| 2           | 12/05/2012 | biscotti | 70  |
| 3           | 18/05/2012 | latte    | 100 |
| 3           | 18/05/2012 | pane     | 45  |

Compilare la Tabella allegata, indicando le regole di associazione con supporto e confidenza superiori al 20%.

| Premessa       | Conseguenza | Supporto<br><b>#PC</b><br><b>#TOT</b>         | Confidenza<br><b>#PC</b><br><b>#P</b> |
|----------------|-------------|-----------------------------------------------|---------------------------------------|
| latte          | pane        | 2/3                                           | 2/3                                   |
| latte          | zucchero    | 1/3                                           | 1/3                                   |
| latte          | biscotti    | 1/3                                           | 1/3                                   |
| pane           | latte       | 2/3 (sarà uguale a latte pane data la regola) | 1                                     |
| pane           | zucchero    | 1/3                                           | 1/2                                   |
| zucchero       | latte       | 1/3                                           | 1                                     |
| zucchero       | pane        | 1/3                                           | 1                                     |
| biscotti       | latte       | 1/3                                           | 1                                     |
| latte,pane     | zucchero    | 1/3                                           | 1/2                                   |
| latte,zucchero | pane        | 1/3                                           | 1                                     |
| pane,zucchero  | latte       | 1/3                                           | 1                                     |

### SAM\_BD\_M10\_E3 --- Esercizio supporto e confidenza

1. Supponendo che:  $\text{supporto}(X \rightarrow Y) = \text{supporto}(Y \rightarrow X)$

- ☐ X compare in più transazioni di Y
- ☐ Y compare in più transazioni di X
- ☐ X e Y compaiono nello stesso numero di transazioni
- ☒ Non è possibile determinarlo
- ☐ Non è possibile

$$\frac{\#XY}{\#T} = \frac{\#YX}{\#T}$$

vale sempre, non possiamo determinarlo  
IDENTITA'

2. Supponendo che:  $\text{confidenza}(X \rightarrow Y) > \text{confidenza}(Y \rightarrow X)$

- ☐ X compare in più transazioni di Y
- ☒ Y compare in più transazioni di X
- ☐ X e Y compaiono nello stesso numero di transazioni
- ☐ Non è possibile determinarlo
- ☐ Non è possibile

$$\frac{\#XY}{\#X} > \frac{\#YX}{Y}$$

semplifica,  
 $y > x$

3. Supponendo che:  $\text{supporto}(X \rightarrow Y) > \text{supporto}(Y \rightarrow X)$

- ☐ X compare in più transazioni di Y
- ☐ Y compare in più transazioni di X
- ☐ X e Y compaiono nello stesso numero di transazioni
- ☐ Non è possibile determinarlo
- ☒ Non è possibile

perchè nella prima abbiamo detto che l'uguaglianza vale sempre quindi non è possibile la disuguaglianza

4. Supponendo che:  $\text{confidenza}(X \rightarrow Y) = \text{confidenza}(Y \rightarrow X)$

- ☐ X compare in più transazioni di Y
- ☐ Y compare in più transazioni di X
- ☒ X e Y compaiono nello stesso numero di transazioni
- ☐ Non è possibile determinarlo
- ☐ Non è possibile

$$\frac{\#XY}{\#X} = \frac{\#YX}{Y}$$

semplifica,  
 $y = x$

### SAM\_BD\_M10\_E4 --- Esercizio supporto e confidenza

1. Supponendo che  $\text{supporto}(A \rightarrow B) = \text{supporto}(A \rightarrow C)$ , si indichi se l'affermazione "B e C compaiono esattamente nello stesso numero di transazioni" è:

- ☐ Vera  
☐ Falsa

☒ Non è possibile determinarlo

$$\frac{|AB|}{|T|} = \frac{|AC|}{|T|}$$

B compare nello stesso num di trans. di C quando sono entrambi assieme ad A. Ma quando NON sono con A non sappiamo come si comportano.

2. Supponendo che  $\text{confidenza}(A \rightarrow B) = \text{supporto}(A \rightarrow B)$ , si indichi se l'affermazione "A compare in tutte le  $N_T$  transazioni" è:

- ☒ Vera  
☐ Falsa

☐ Non è possibile determinarlo

$$\frac{|AB|}{|A|} = \frac{|AB|}{|T|} \rightarrow |A| = |T|$$

3. Supponendo che  $\text{supporto}(A \rightarrow B) = \text{supporto}(A \rightarrow BC)$ , si indichi se l'affermazione " $\text{confidenza}(A \rightarrow B) = \text{confidenza}(A \rightarrow BC)$ " è:

- ☒ Vera  
☐ Falsa

☐ Non è possibile determinarlo

$$\frac{|AB|}{|T|} = \frac{|ABC|}{|T|} \rightarrow \frac{|AB|}{|A|} = \frac{|ABC|}{|A|} \quad \checkmark$$

4. Supponendo che  $\text{supporto}(A \rightarrow B) = \text{supporto}(B \rightarrow A)$ , si indichi se l'affermazione " $\text{confidenza}(A \rightarrow B) = \text{confidenza}(B \rightarrow A)$ " è:

- ☐ Vera  
☐ Falsa

☒ Non è possibile determinarlo

$$\frac{|AB|}{|T|} = \frac{|BA|}{|T|} \rightarrow \frac{|AB|}{|A|} = \frac{|BA|}{|B|}$$

5. Si indichi se l'affermazione " $\text{supporto}(A \rightarrow B) \geq \text{supporto}(A \rightarrow BC)$ " è:

- ☐ Vera  
☐ Falsa

☐ Non è possibile determinarlo

$$\frac{|AB|}{|T|} \geq \frac{|ABC|}{|T|}$$

## SAM\_BD\_M10\_E5 --- Esercizio supporto e confidenza

Sia data una collezione di numero  $N_T = 100$  transazioni e tre insiemi di oggetti X, Y e Z nella collezione. Supponendo che:

- $\text{supporto}(X \rightarrow Y) = 0.3$
- $\text{confidenza}(X \rightarrow Y) = 1$
- $\text{confidenza}(X \rightarrow Z) = 0.5$

Indicare, se possibile, i valori minimi e massimi assumibili da:

- $\text{supporto}(X \rightarrow Z)$   $\frac{xz}{T} = (0,15, 0,15)$
- $\text{supporto}(Y \rightarrow Z)$   $\frac{yz}{T} = (0,15, 0,35)$

$$\frac{xy}{T} = 0,3 \rightarrow \frac{30}{100}$$

x e y compaiono insieme in 30 transazioni

$$\frac{xy}{x} = 1$$

tutte le volte che compare x compare anche y (non so nulla sul viceversa)

$$\frac{xz}{x} = 0,5$$

in metà delle transazioni in cui compare x, c'è anche z (non so nulla sul viceversa)

## SAM\_BD\_M10\_E6 --- Esercizio supporto e confidenza

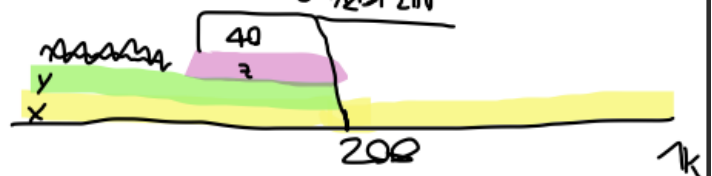
## SAM\_BD\_M10\_E6

Sia data una collezione di numero 1000 transazioni e tre insiemi di oggetti **X**, **Y** e **Z** nella collezione. Compilare la tabella allegata, supponendo che:  $\text{supporto}(X \rightarrow Y) = \text{confidenza}(X \rightarrow Y) = \text{confidenza}(Y \rightarrow Z) = 0.2$ .

quando  $\text{sup} = \text{conf}$  quello che ho nella premessa (x) c'è su tutto

|   | Minimo | Massimo |
|---|--------|---------|
| X | 1000   | 1000    |
| Y | 200    | 200     |
| Z | 40     | 80      |

$$\frac{xy}{x} = \frac{xy}{y} = \frac{yz}{y} = 0,2 \rightarrow 200$$



Indicare la relazione fra  $\text{supporto}(Y \rightarrow Z)$  e  $\text{supporto}(Z \rightarrow Y)$  scegliendo fra le seguenti opzioni:

- ☐  $\text{supporto}(Y \rightarrow Z) > \text{supporto}(Z \rightarrow Y)$
- ☐  $\text{supporto}(Y \rightarrow Z) < \text{supporto}(Z \rightarrow Y)$
- ☒  $\text{supporto}(Y \rightarrow Z) = \text{supporto}(Z \rightarrow Y)$
- ☐ non è possibile determinarlo

$$\frac{yz}{y} = \frac{zy}{x} \quad \checkmark$$

Indicare la relazione fra  $\text{confidenza}(Y \rightarrow Z)$  e  $\text{confidenza}(Z \rightarrow Y)$  scegliendo fra le seguenti opzioni:

- ☐  $\text{confidenza}(Y \rightarrow Z) > \text{confidenza}(Z \rightarrow Y)$
- ☐  $\text{confidenza}(Y \rightarrow Z) < \text{confidenza}(Z \rightarrow Y)$
- ☐  $\text{confidenza}(Y \rightarrow Z) = \text{confidenza}(Z \rightarrow Y)$
- ☒ non è possibile determinarlo

$$\frac{40}{200} < \frac{40}{80}$$

→ Non possiamo determinarlo